



TRƯỜNG ĐHCN TP HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÀI GIẢNG

KIẾN TRÚC MÁY TÍNH

(COMPUTER ARCHITECTURE)



Giới thiệu

Kiến trúc máy tính

(Computer Architecture)

phantrung.wordpress.com

phantrung595@gmail.com



Mục đích

- Lịch sử phát triển và hoạt động của máy tính
 - Các cấu trúc liên kết với nhau trong máy tính
 - **Biểu diễn dữ liệu và số học máy tính**
 - Cấu trúc và chức năng của CPU
 - **Bộ nhớ**
 - Hệ thống vào ra
 - Tập lệnh
 - Mạch logic số (thêm)
-



Tài liệu tham khảo

- William Stallings – **Computer Organization and Architecture** 8th Edition
- Giáo trình **Kiến trúc máy tính** của DH cần thơ



Nội dung môn học

1. Tổng quan về kiến trúc máy tính
2. Biểu diễn dữ liệu và số học máy tính
3. Mạch logic số (tham khảo)
4. Tập lệnh
5. Bộ xử lý trung tâm
6. Bộ nhớ máy tính
7. Hệ thống vào ra



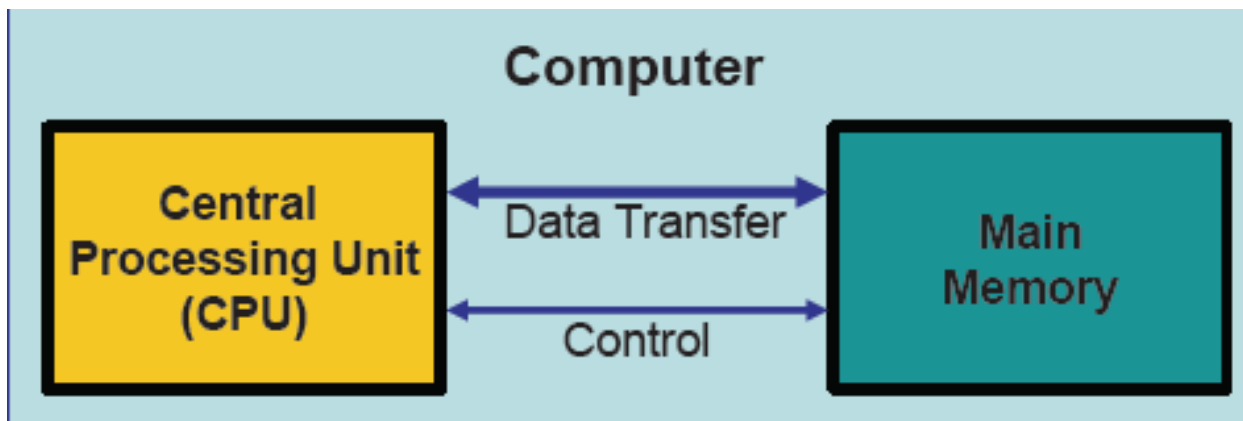
Chương 1 Tổng quan về KTMT

1. Một số khái niệm và công nghệ
2. Các thể hệ máy tính



1. Các khái niệm và công nghệ

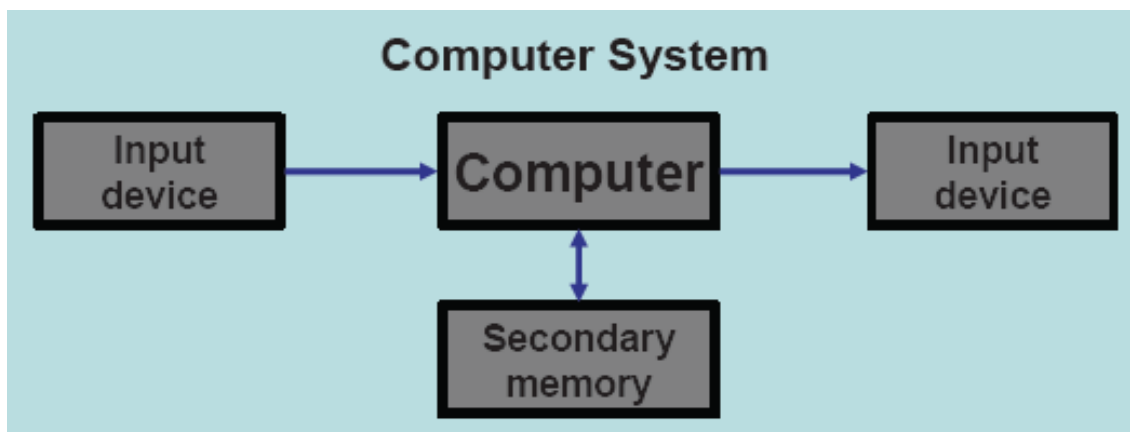
- **Máy tính** (Computer) là máy xử lý dữ liệu, hoạt động một cách tự động dưới sự điều khiển của một danh sách các lệnh (gọi là chương trình) được lưu trữ trong bộ nhớ chính của nó.





1. Các khái niệm và công nghệ

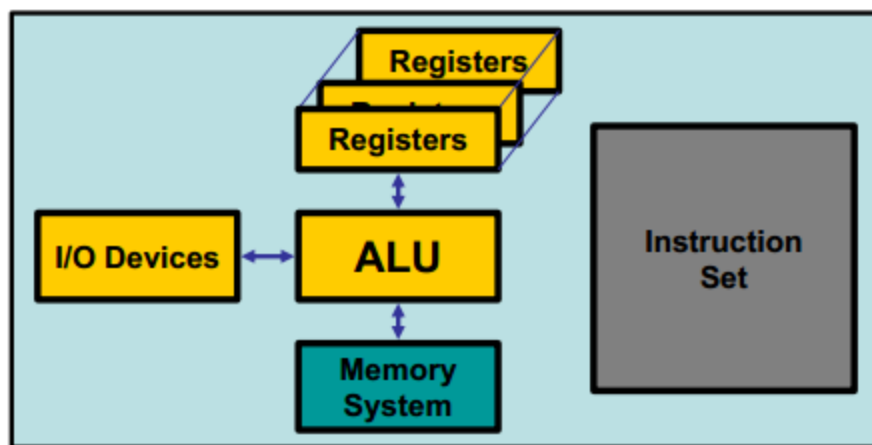
- Một **hệ thống máy tính** (Computer System) bao gồm một máy tính và các thiết bị ngoại vi.
- **Thiết bị ngoại vi** (Peripherals) bao gồm các thiết bị nhập, thiết bị xuất và bộ nhớ thứ cấp





1. Các khái niệm và công nghệ

- **Kiến trúc máy tính** (Architecture) liên quan đến các thuộc tính của hệ thống máy tính **có khả năng thấy được** đối với người lập trình, hoặc các thuộc tính **có ảnh hưởng trực tiếp** đến logic thực hiện chương trình





1. Các khái niệm và công nghệ

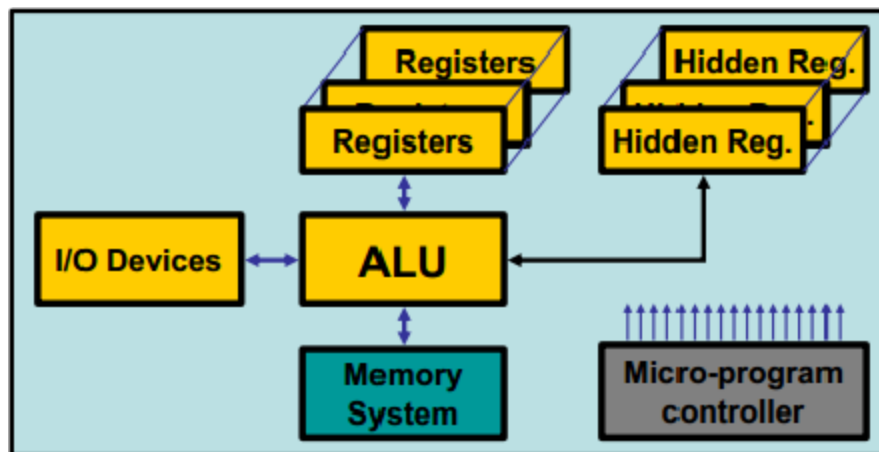
➤ Các thuộc tính KTMT:

- Tập lệnh
- Các phương pháp biểu diễn dữ liệu cơ bản
- Cơ chế xuất nhập
- Các khối cơ bản trong CPU
- Chức năng của các thành phần chính
- Sự thực hiện lệnh
- Tổ chức bộ nhớ(các kỹ thuật định vị bộ nhớ)
- Các cách mà các thành phần cơ bản kết nối với nhau
- ...



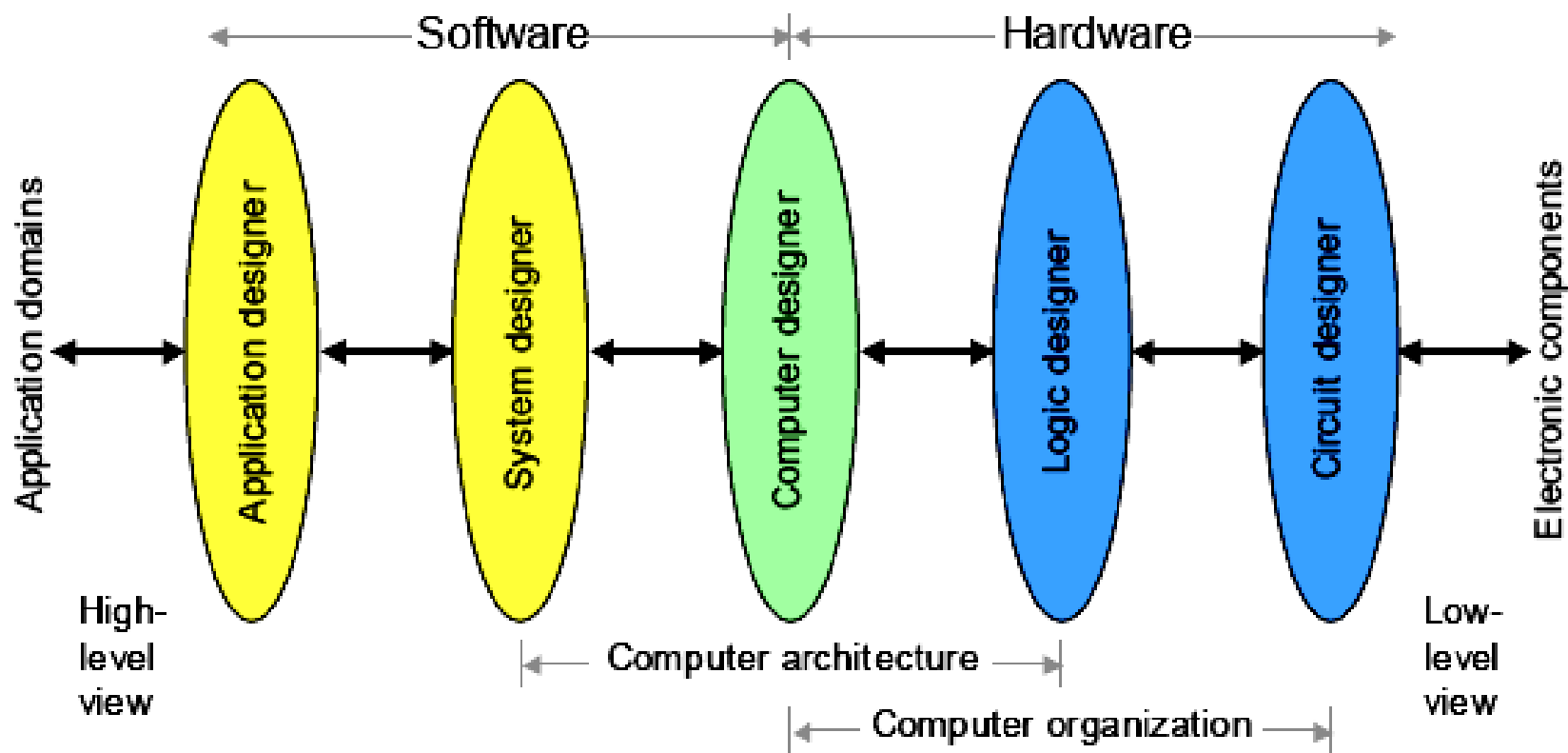
1. Các khái niệm và công nghệ

- **Tổ chức máy tính** (Computer Organization): đề cập đến các khối chức năng và sự kết nối giữa chúng để thực hiện các đặc tả kiến trúc (nghĩa là làm thế nào hiện thực các tính năng kiến trúc)
- Tín hiệu điều khiển, giao tiếp giữa máy tính và thiết bị ngoại vi, công nghệ bộ nhớ, ...





Vị trí KTMT và TCMT





1. Các khái niệm và công nghệ

➤ So sánh KTMT và TCMT

- Ví dụ chức năng “nhân”
 - Kiến trúc : có hay không có lệnh nhân.
 - Tổ chức : một đơn vị thực hiện chức năng “nhân” đặc biệt hay việc dùng nhiều đơn vị “cộng” để thực hiện chức năng “nhân”.
- Nhiều nhà sản xuất máy tính đưa ra dòng(họ)các mẫu máy tính. Các máy này có cùng kiến trúc nhưng khác nhau về mặt tổ chức
 - Tất cả tính họ x86 của Intel có cùng kiến trúc cơ bản
 - Họ System/370 của IBM có cùng kiến trúc cơ bản



1. Các khái niệm và công nghệ

- Điều này dẫn đến
 - Nhiều máy khác nhau trong cùng họ có giá thành và hiệu suất khác nhau.
 - Tổ chức sẽ thay đổi theo công nghệ
 - Tương thích về chương trình
 - Tối thiểu đối với máy thế hệ trước (backwatd)
- Trong máy vi tính, mối quan hệ giữa kiến trúc và tổ chức rất khăng khít với nhau
 - Thay đổi về công nghệ không chỉ ảnh hưởng đến tổ chức mà còn dẫn đến kiến trúc phức tạp hơn và hiệu quả hơn



Tại sao học KTMT

- Để trở thành chuyên nghiệp trong lĩnh vực máy tính ngày nay. Bạn không nên xem máy tính như một hộp đen (black box) thực hiện các chương trình bằng ma thuật.
- Bạn nên hiểu các thành phần chức năng của một hệ thống máy tính. Đặc tính hiệu suất và sự tương tác của chúng.
- Bạn cần hiểu rõ KTMT để có thể xây dựng các chương trình chạy hiệu quả trên máy tính.
- Khi chọn lựa để dùng một hệ thống, bạn phải có khả năng hiểu được ưu và nhược điểm của các thành phần khác nhau. Ví dụ tốc độ xung nhịp CPU so với kích thước bộ nhớ



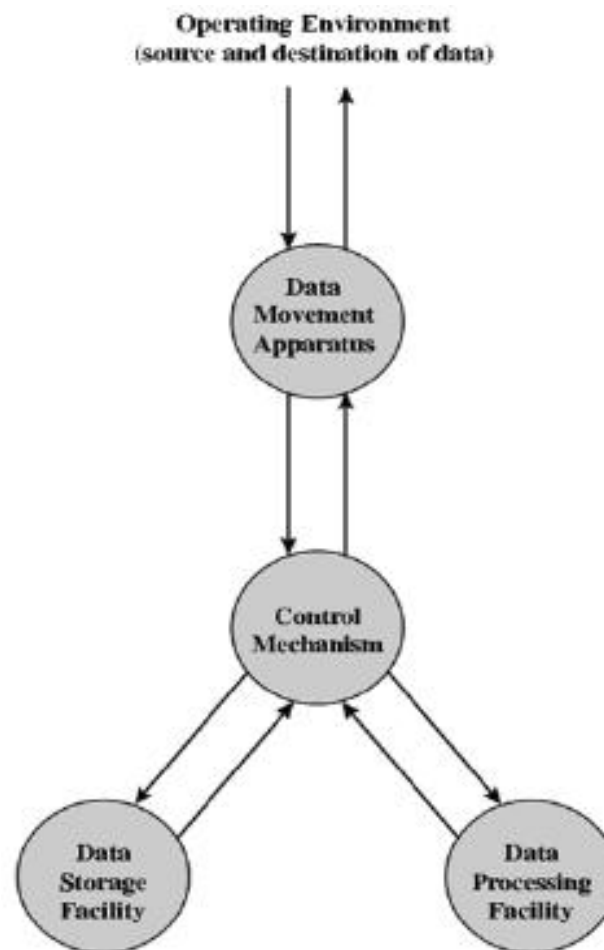
Cấu trúc và Chức năng

- Nhận biết bản chất phân cấp của các hệ thống phức tạp nhất
 - Hệ thống phân cấp là tập hợp các hệ thống con có quan hệ với nhau. Sao cho mỗi hệ thống con này lại có tính phân cấp về cấu trúc như vậy, cho đến khi chúng ta đạt đến hệ thống con nguyên tử thấp nhất.
- Cấu trúc là cách mà các thành phần quan hệ với các thành phần khác
- Chức năng là tác vụ của các thành phần, chức năng riêng biệt nằm trong cấu trúc
- Theo cách mô tả, có 2 cách tiếp cận
 - Bottom-up
 - Top-down



Chức năng máy tính

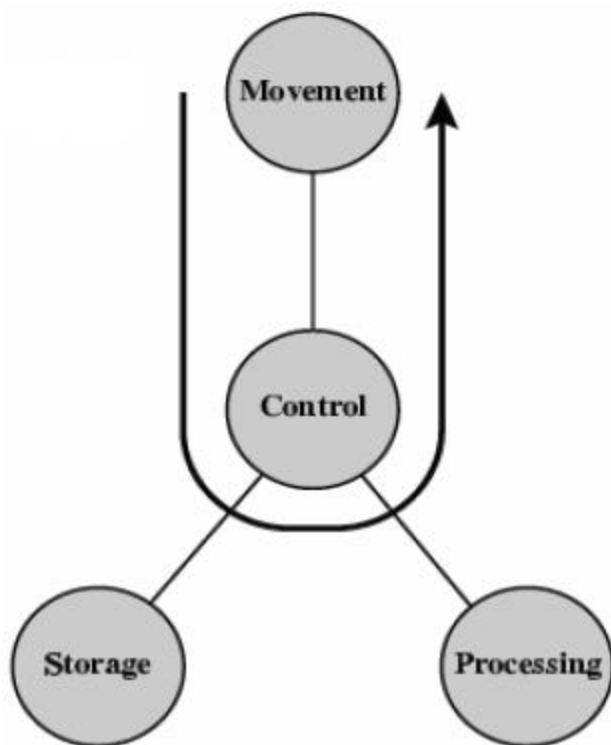
- Các chức năng của máy tính bao gồm:
- Xử lý dữ liệu (data processing)
 - Lưu trữ dữ liệu (data storage)
 - Dịch chuyển dữ liệu (data movement)
 - Điều khiển (control)



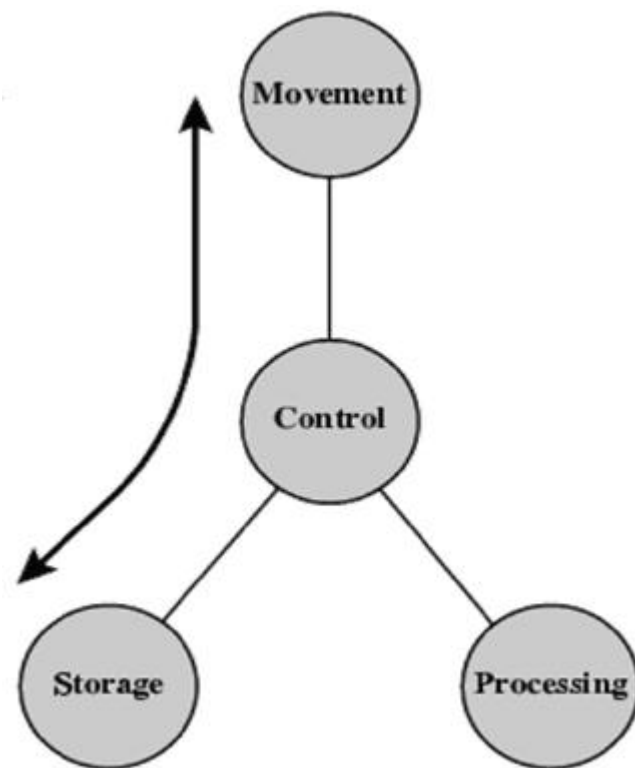


Các tác vụ

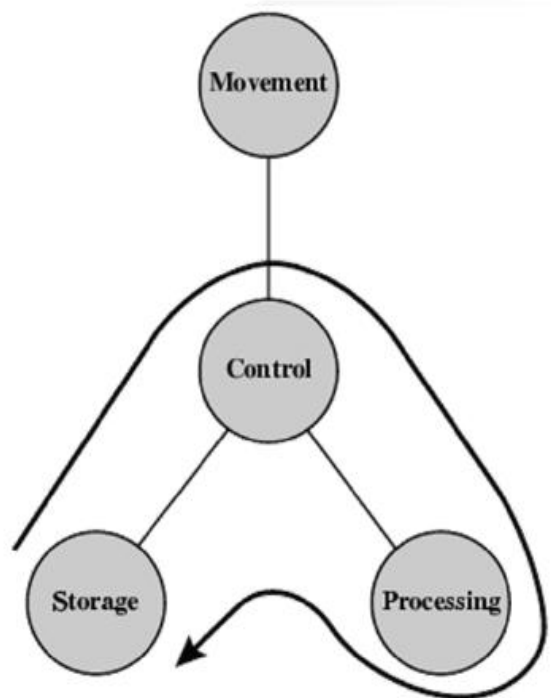
a. Dịch chuyển dữ liệu



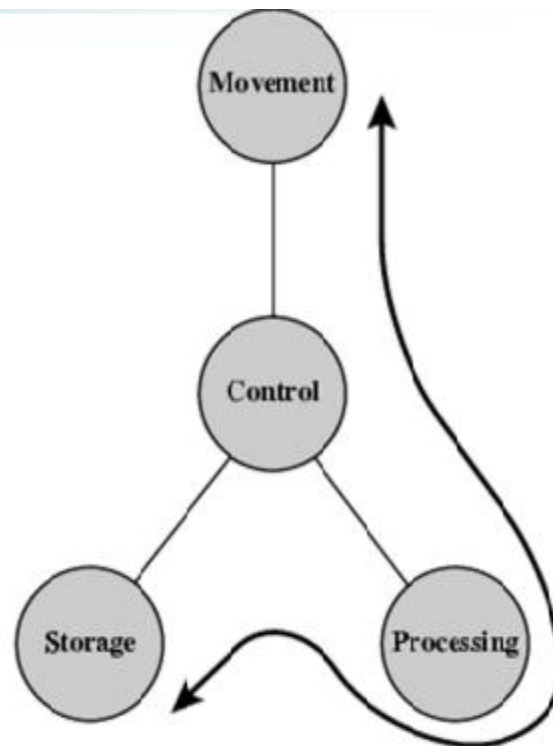
b. Lưu trữ dữ liệu



c. Xử lý dữ liệu từ bộ nhớ Và lưu trữ lại trong bộ nhớ)

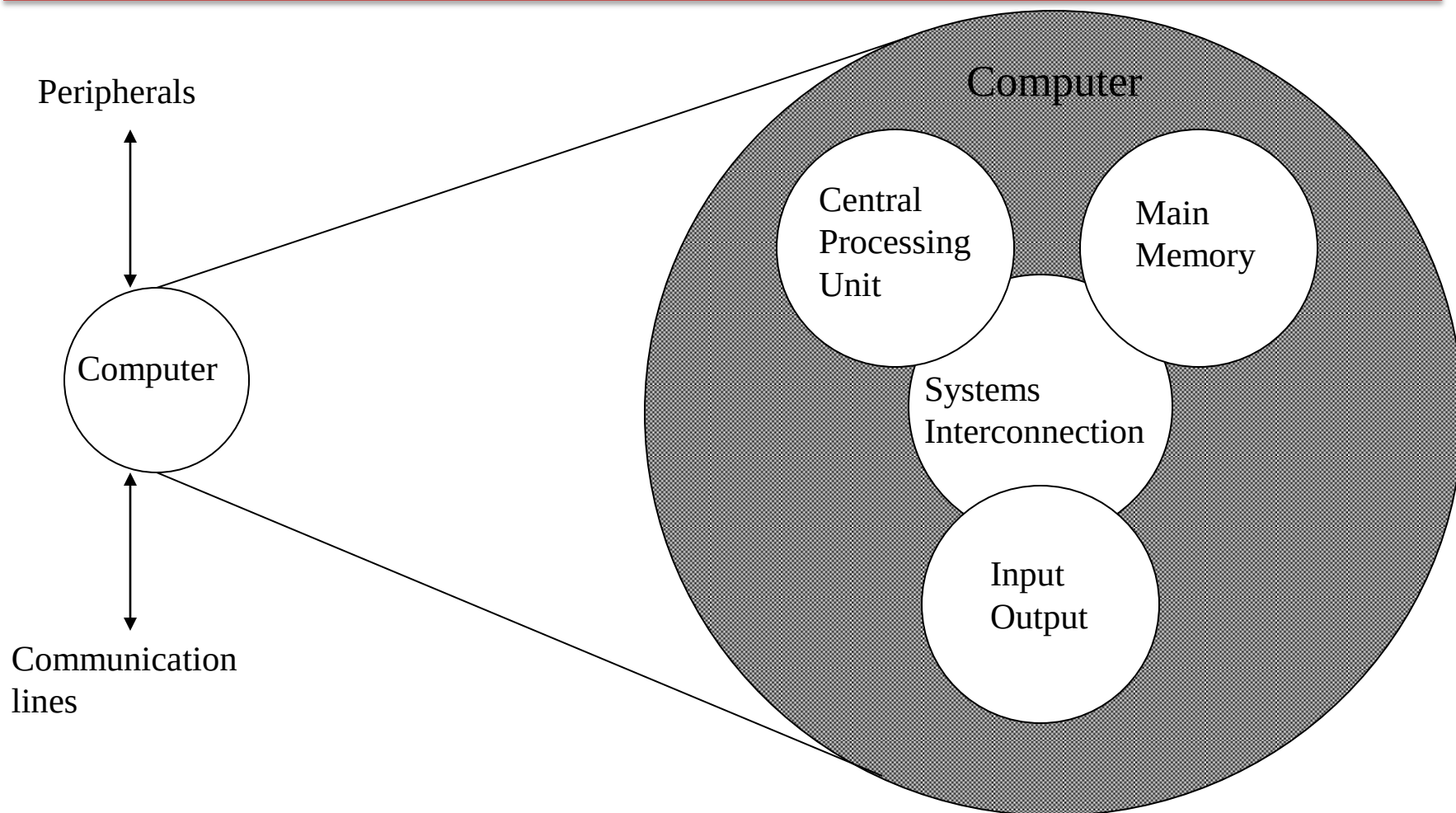


d. Xử lý dữ liệu từ bộ nhớ ra I/O



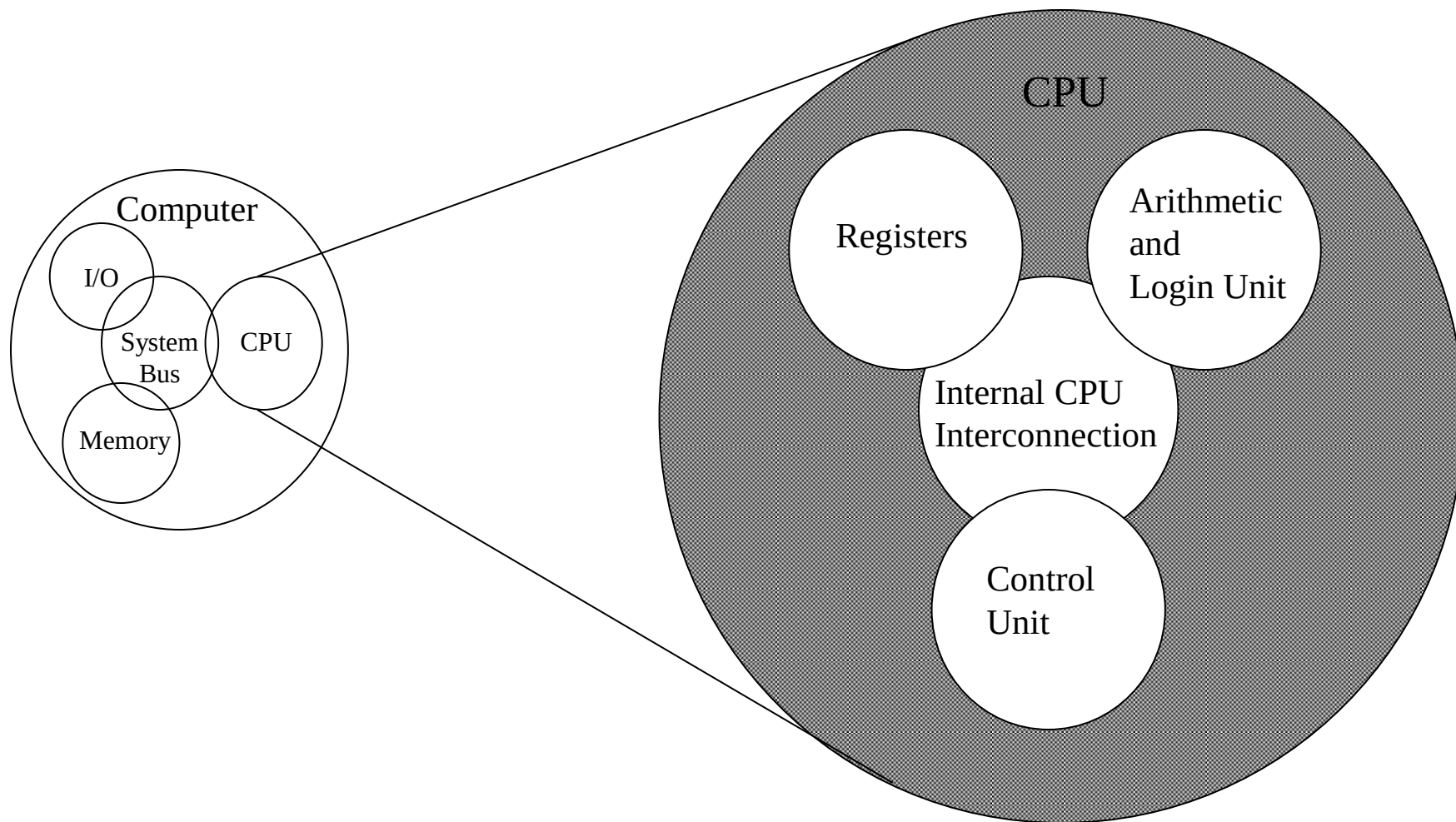


Cấu trúc – Top Level



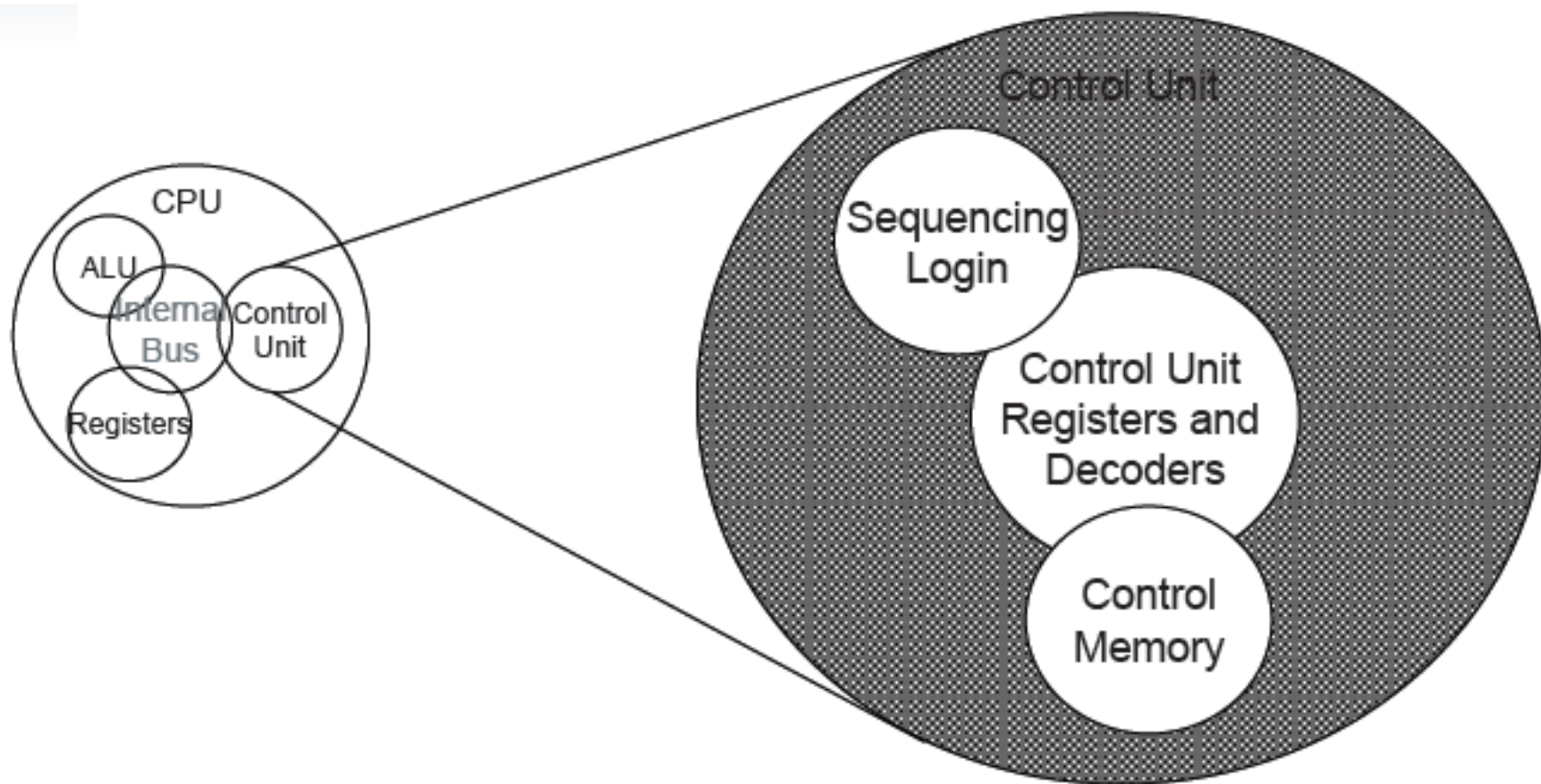


Cấu trúc - CPU





Cấu trúc – Bộ điều khiển





1. Các khái niệm và công nghệ

➤ Ngôn ngữ lập trình

- Ngôn ngữ tự nhiên (natural language):
 - Do con người sử dụng. Lệ thuộc ngữ cảnh, không có tính chính xác và nhất quán cần thiết cho máy tính.
 - Không sử dụng được cho máy tính
- Ngôn ngữ máy (machine language)
 - Là các ký hiệu nhị phân (0 và 1) mà các linh kiện điện tử trong máy tính hiểu và xử lý được.
 - Rất khó khăn khi con người sử dụng trực tiếp.
- Ngôn ngữ ký hiệu (Symbolic language/ Assembly language dạng ký hiệu/ gởi nhớ của tập lệnh CPU.
- Ngôn ngữ lập trình (Programming language)
 - Là trung gian giữa ngôn ngữ tự nhiên và ngôn ngữ máy.

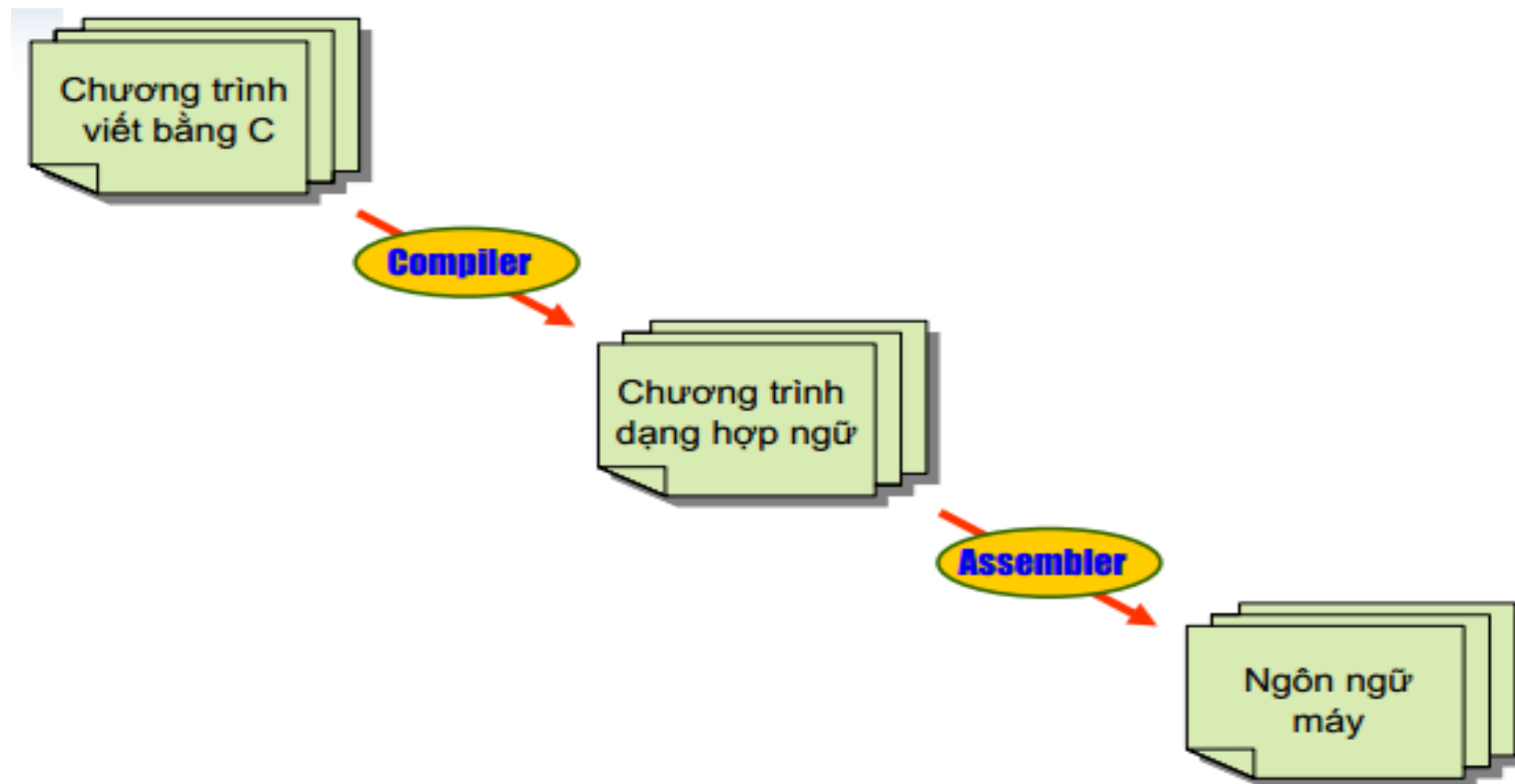


Bên dưới của chương trình

- Máy tính là bước phát triển kế tiếp của các mạch logic
- Thông tin trên máy tính được biểu diễn bởi các ký số nhị phân hay **bit**(binary digit)
- Máy tính hoạt động tuân theo các chỉ thị của chúng ta. Thuật ngữ dùng để gọi các chỉ thị riêng lẻ là **câu lệnh** (**instruction**)
- Mọi câu lệnh là một chuỗi xác định các bit, (giống như 1 số nhị phân) mà máy tính có thể hiểu được
 - Ví dụ **10001100100010** yêu cầu máy tính cộng 2 số nguyên
- Những nhà lập trình đầu tiên truyền đạt chỉ thị đến máy tính thông qua các con số nhị phân nói trên
 - Đây là công việc hết sức tẻ nhạt

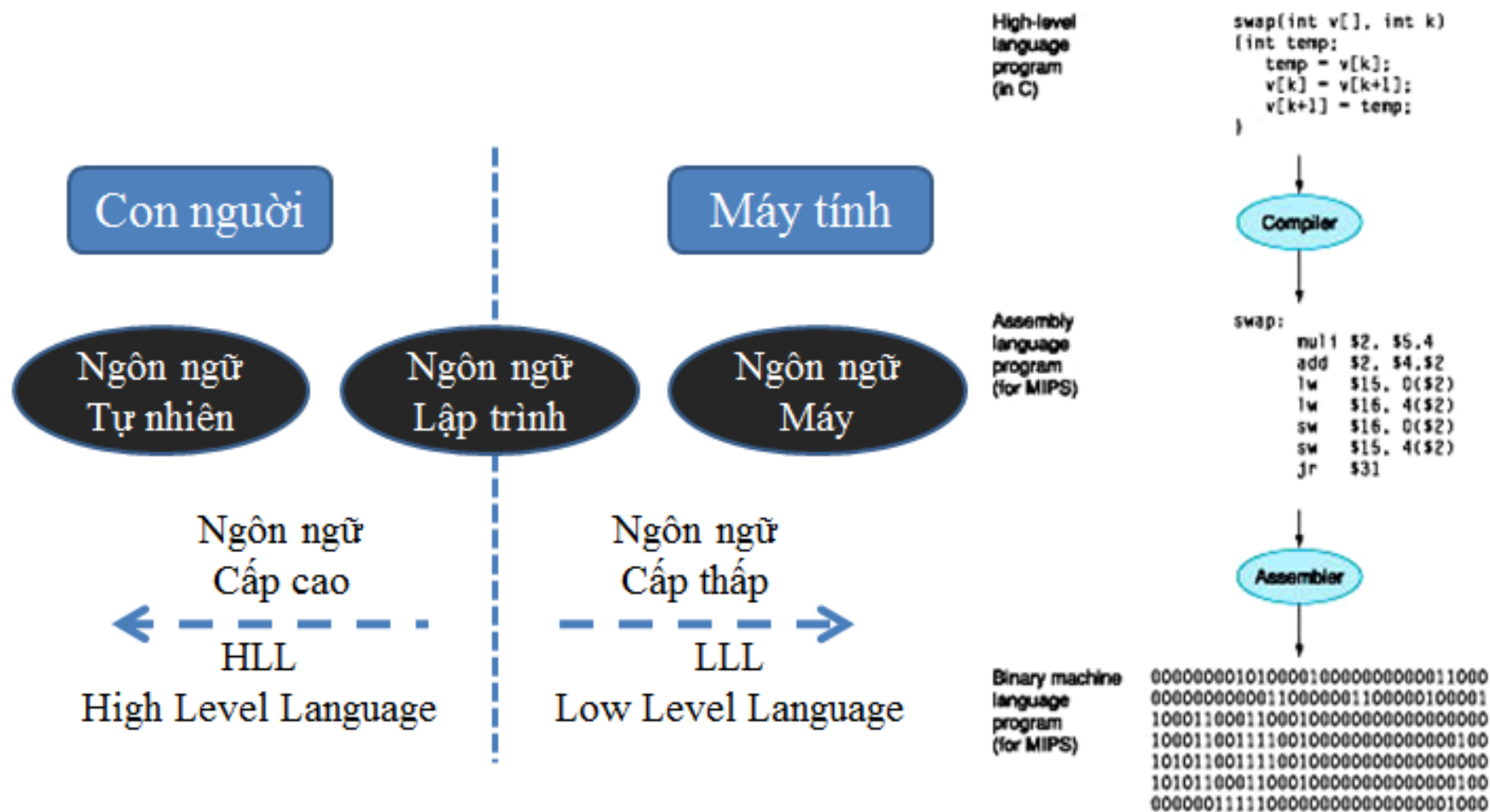
- Công cụ lập trình dùng các số nhị phân để viết ra các chỉ thị cho máy tính được gọi là **ngôn ngữ máy** (**machine language**).
- Con người nhanh chóng thay thế các số nhị phân bởi các ký hiệu gọi nhớ(symbolic), chúng là những ký hiệu gần với cách suy nghĩ của con người hơn
 - VD sử dụng **add A,B** thay thế cho **1001 1010 0001**
- Lúc đầu con người dùng tay để dịch các ký hiệu trên ra số nhị phân rồi đem thực hiện trên máy tính
- Sau đó, con người phát triển một chương trình trợ giúp việc dịch nói trên: **Assembler**
- Công cụ lập trình dùng các ký hiệu gọi nhớ nhằm viết ra các chỉ thị cho máy tính được gọi là **hợp ngữ** (**assembly language**)

- Mỗi dòng trong hợp ngữ là 1 câu lệnh để máy tính thực thi. Lập trình bằng hợp ngữ buộc người lập trình phải suy nghĩ hành động như một máy tính
 - Cấp hành động như máy tính gọi là **cấp thấp**(low level)
 - Ngôn ngữ máy và hợp ngữ là các **ngôn ngữ cấp thấp** (low level language)
- Theo hướng trên, người ta lại đưa ra các ký hiệu gần với suy nghĩa của con người và tạo nên các ngôn ngữ cấp cao (high level language)
 - VD **A + B** thay cho **add A, B**
- Sử dụng chương trình để dịch ngôn ngữ cấp cao sang hợp ngữ : **chương trình dịch** (compiler)





1. Các khái niệm và công nghệ



- Ngôn ngữ cấp cao mang lại nhiều lợi ích quan trọng
 - Cho phép người lập trình suy nghĩ dưới dạng ngôn ngữ tự nhiên(Anh ngữ, biểu thức toán,..) C, C#, LISP.
 - Tăng đáng kể hiệu năng lập trình; chương trình ngắn hơn, sáng sủa và dễ hiểu hơn.
 - Ngôn ngữ cấp cao độc lập đối với máy tính.
- Khả năng tái sử dụng chương trình mang lại hiệu quả cao hơn là viết toàn bộ chương trình từ đầu
 - ☞ Vd trình con, thư viện, thư viện các trình con xuất/nhập.
- Người ta nhận thấy việc thực thi các chương trình trên máy tính sẽ hiệu quả hơn nếu có 1 chương trình đặc biệt giám sát thực thi cho các chương trình trên
 - ☞ Hệ điều hành (operating system)

- Hệ điều hành là chương trình quản lý các tài nguyên của máy tính hỗ trợ tốt nhất cho việc thực thi của các chương trình khác nhau trên máy tính
- Phần mềm phân theo tính năng sử dụng:
 - Các chương trình cung cấp dịch vụ chung cho các chương trình khác được gọi là phần mềm hệ thống (system software)
 - ☞ Hệ điều hành, chương trình dịch, các driver phần cứng.
 - Phần mềm ứng dụng(applications software) là các phần mềm cung cấp dịch vụ cho các người sử dụng máy tính (Users)
 - ☞ Word, excel, photoshop,...



2. Các thế hệ máy tính

- Sự phát triển của máy tính được mô tả dựa trên sự tiến bộ của các công nghệ chế tạo các linh kiện cơ bản của máy tính như : bộ xử lý, bộ nhớ, các ngoại vi,... Việc chuyển từ thế hệ trước sang thế hệ sau được đặc trưng bằng một sự thay đổi cơ bản về công nghệ.



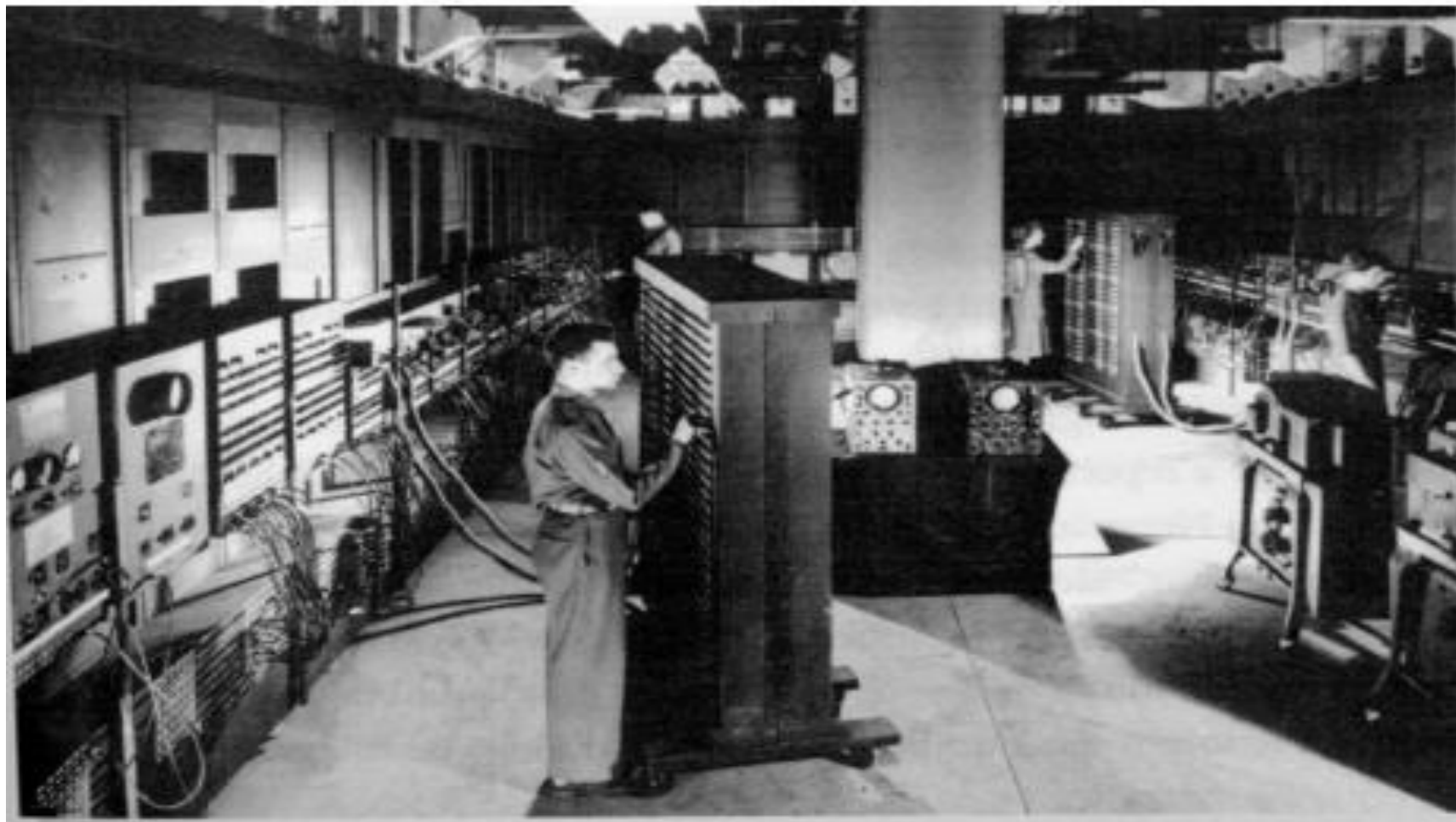
a. Thế hệ đầu tiên(1946-1957)

➤ ENIAC (Electronic Numerical Integrator And Computer)

- Gs Eckert và Mauchly ở ĐH Pennsylvania
- bắt đầu 1943 hoàn thành 1946 được dùng đến 1955
- 20 thanh ghi 10 bit (thập phân ko có nhị phân)
- Công việc lập trình bằng tay bởi nối hoặc ngắt điện
- 18 000 đèn điện tử (vacuum tubes)
- 1500 công tắc
- Cân nặng 30 tấn
- 140 KW
- Thực hiện 5 000 phép cộng trong 1 giây



Máy tính ENIAC

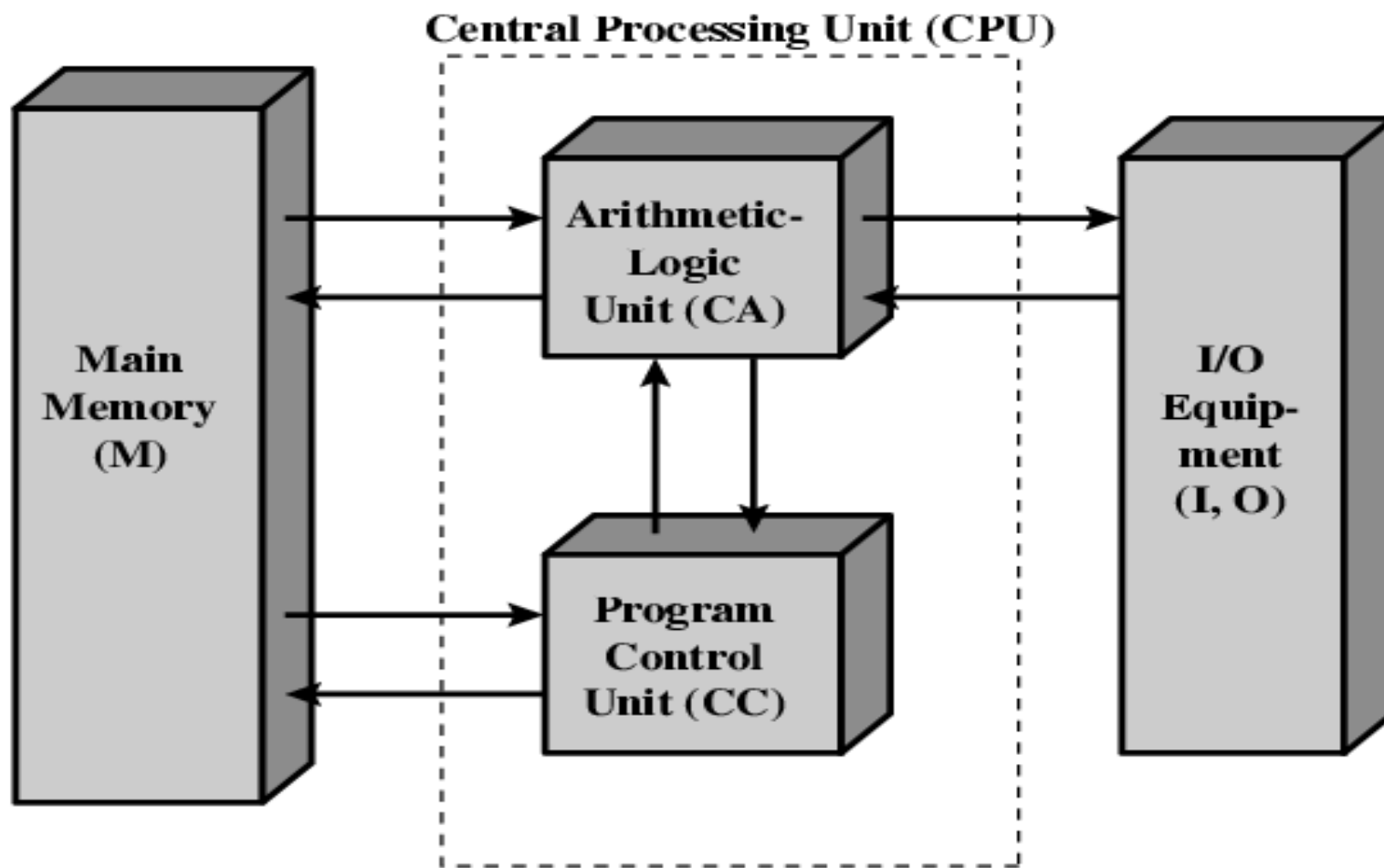


Máy tính Von Neumann/Turing

- Máy tính IAS (Institute for Advanced Studies)
 - Máy có mô hình cơ bản của máy tính ngày nay:
 - Bộ nhớ chính lưu chương trình và dữ liệu
 - Hoạt động của ALU trên dữ liệu nhị phân
 - CU thông dịch các lệnh từ bộ nhớ và thực thi
 - Thiết kế 1947 hoàn thành 1952
 - Xây dựng trên ý tưởng của Turing (Mỹ) và Von Neumann(Anh)
-



Cấu trúc của máy Von Neumann





Các máy tính thương mại

- Đầu 1950 có 48 máy hệ UNIVAC I (UNIVersal Automatic Computer) và 19 máy hệ IBM 701 được bán ra
- Cuối 1950 UNIVAC II có tốc độ nhanh hơn và bộ nhớ lớn hơn



b. Thế hệ thứ 2(1958 – 1964)

- Transistor thay thế các đèn điện tử
 - Kích thước máy tính giảm
 - Rẻ tiền hơn
 - Tốn ít năng lượng hơn
- Công ty Bell đã phát minh ra từ 1947 nhưng phải đến cuối thập niên 50 máy tính thương mại dùng transistor mới xuất hiện.
- Ngôn ngữ cấp cao xuất hiện và hệ điều hành kiểu tuần tự(Batch Processing) được dùng



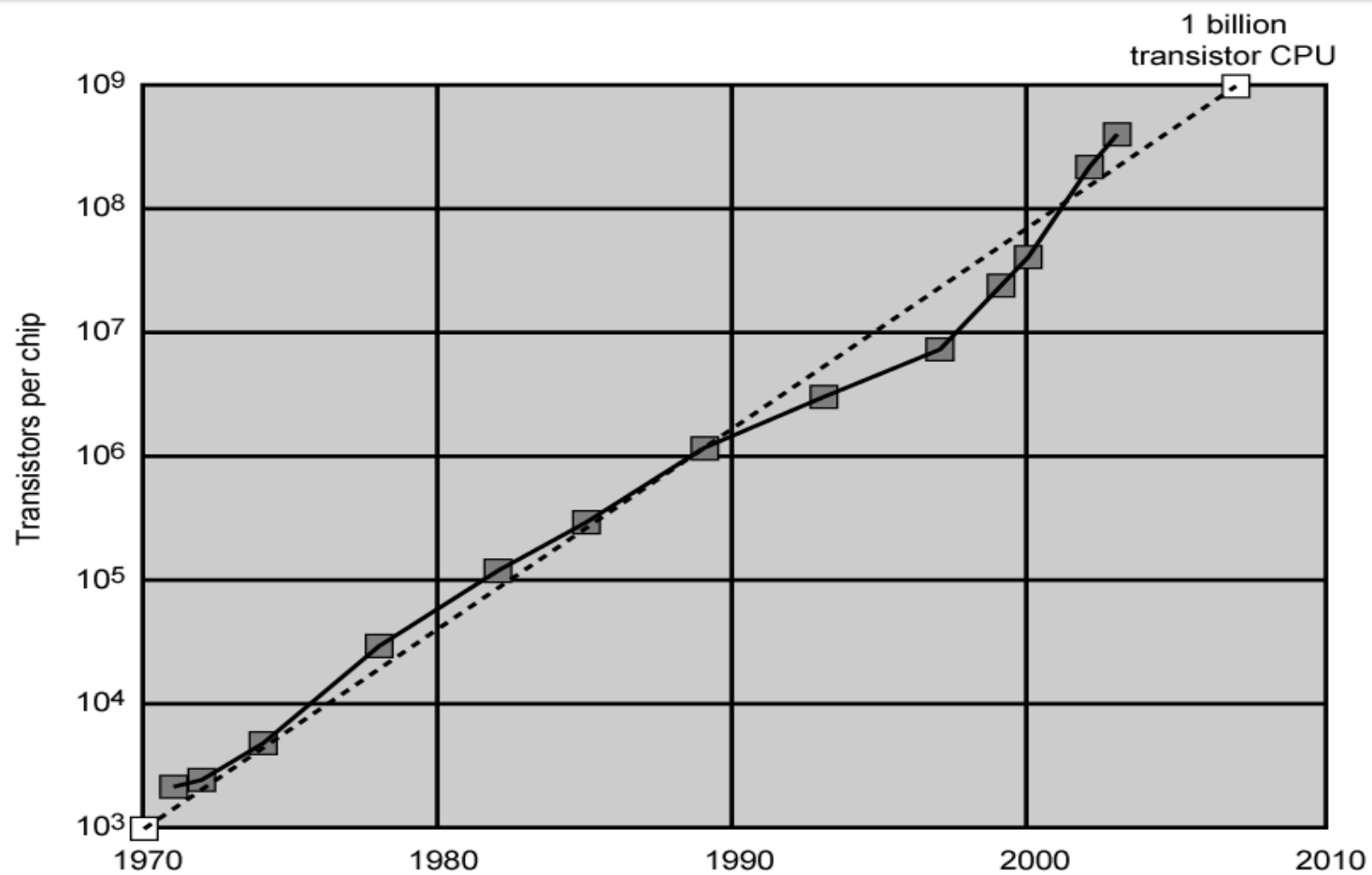
Các thế hệ của máy tính

- 1946 – 1957 bóng đèn điện tử
- 1958 – 1964 Transistor
- 1965 mạch tích hợp (IC : Integrated Circuit) tích hợp tỉ lệ thấp < 100 tbi trên 1 chip
- 1971 IC tích hợp tỉ lệ trung bình 100-3,000 tbi trên 1 chip
- 1971-1977 IC tích hợp tỉ lệ lớn 3,000-100,000 tbi trên 1 chip
- 1978-1991 tích hợp với tỉ lệ rất lớn 100,000 – 100,000,000 tbi trên 1 chip
- 1991 tích hợp với tỉ lệ quá lớn trên 100,000,000 tbi trên 1 chip



Qui luật Moore

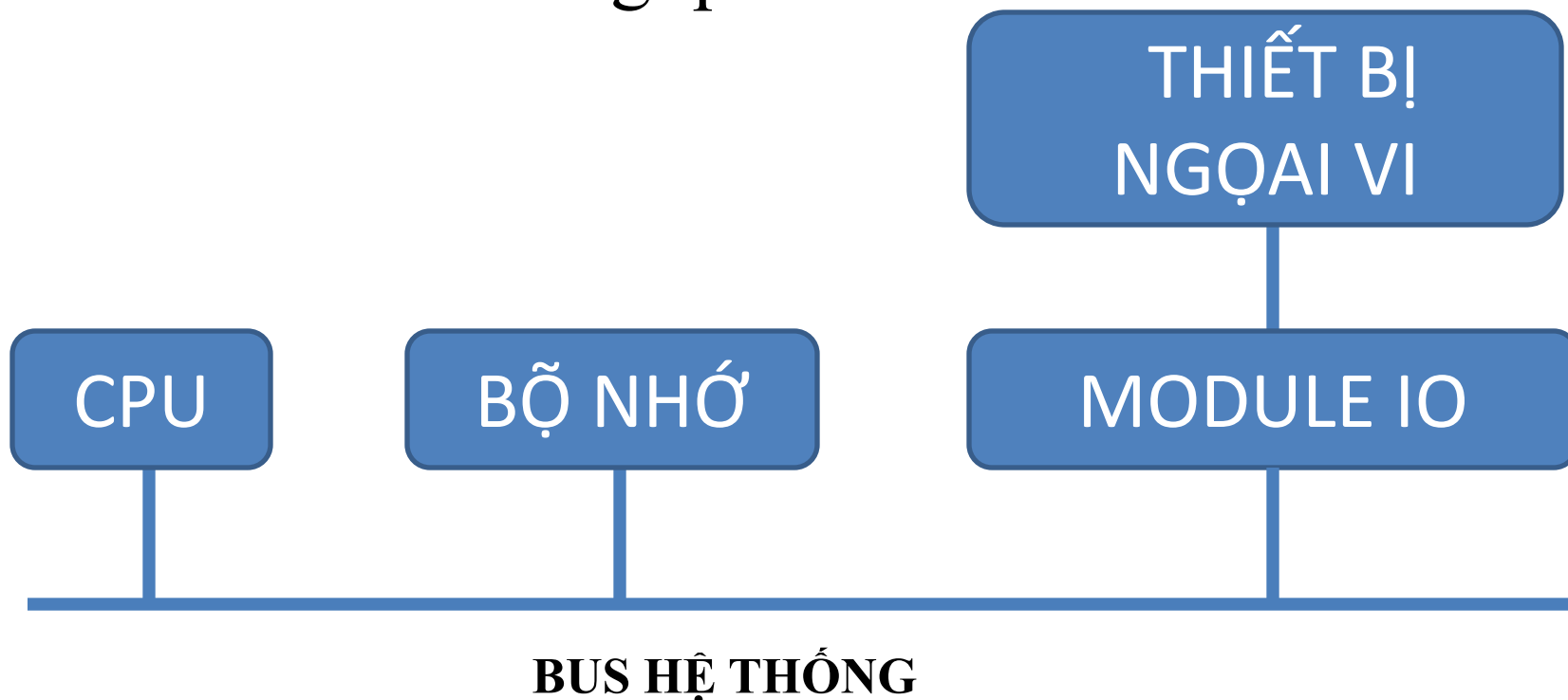
- 1965 Gordon Moore (đồng sáng lập công ty Intel) đã nhận thấy số transistor trong 1 chip sẽ tăng gấp đôi mỗi năm.
- Từ 1970 sự phát triển có chậm lại 1 chút nên ông đưa ra : **số transistor tăng gấp đôi sau 18 tháng**
 - Chi phí cho máy tính giảm.
 - Máy tính sẽ giảm kích thước
 - Hệ thống kết nối bên trong mạch ngắn: tăng độ tin cậy, tăng tốc độ.
 - Tiết kiệm năng lượng, cung cấp tỏa nhiệt thấp.
 - Các IC thay thế cho các linh kiện rời.





Tổ chức tổng quát máy tính

➤ Sơ đồ tổ chức tổng quát



- Chức năng: Điều khiển mọi hoạt động của máy tính và xử lý dữ liệu.
- Thành phần cơ bản:
 - CU (Control Unit) điều khiển hoạt động của máy tính theo chương trình đã định sẵn.
 - ALU (Arithmetic & Logic Unit) thực hiện các phép toán số học và logic trên các dữ liệu cụ thể.
 - RF (Register File) lưu trữ dữ liệu tạm thời phục vụ cho hoạt động của CPU
 - BIU (Bus Interface Unit) kết nối và trao đổi dữ liệu giữa Bus bên trong và Bus bên ngoài CPU

Bộ VXL hoạt động theo xung nhịp (clock) có tần số xác định.

Tốc độ VXL được đánh giá thông qua tần số xung nhịp

Gọi T_0 : chu kỳ xung nhịp, $f_0 = 1/T_0$ tần số xung nhịp mỗi thao tác của vxl cần k T_0 , T_0 càng nhỏ bộ xử lý chạy càng nhanh.

VD một máy tính Pentium 4 tốc độ 2GHz

$$\text{ta có } f_0 = 2 \text{ GHz} = 2 \cdot 10^9 \text{ Hz}$$

$$T_0 = 1/f_0 = 1/2 \cdot 10^9 = 0.5 \text{ ns}$$



Bộ nhớ

- Chức năng: lưu trữ chương trình và dữ liệu.
- Tổ chức : bộ nhớ được chia thành các ô nhớ có kích thước bằng nhau và được đánh địa chỉ. Mỗi ô nhớ có thể là 1 byte hoặc 1 từ máy (word). 1 word có thể là 1,2,4 hay 8 byte tùy theo nhà sản xuất máy tính.
- Thao tác cơ bản :
 - Đọc dữ liệu (Read)
 - Ghi dữ liệu(write)
- Các thành phần chính
 - Bộ nhớ trong (Internal Memory)
 - Bộ nhớ ngoài (External Memory)



Bộ nhớ trong(internal Memory)

- Chức năng và đặc điểm:
 - Chứa thông tin mà CPU có thể trao đổi trực tiếp.
 - Tốc độ rất nhanh.
 - Dung lượng không lớn.
 - Sử dụng bộ nhớ bán dẫn.
- Các loại bộ nhớ:
 - Bộ nhớ chính(Main Memory) :
 - RAM (Random Access Memory)
 - ROM (Read Only Memory)
 - Bộ nhớ Cache (Cache Memory) hay bộ nhớ đệm.

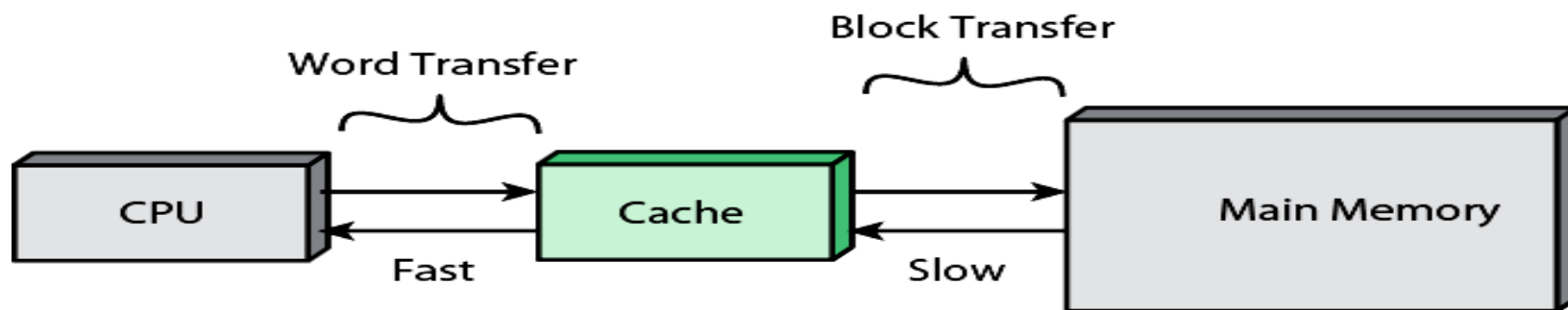


Cache Memory

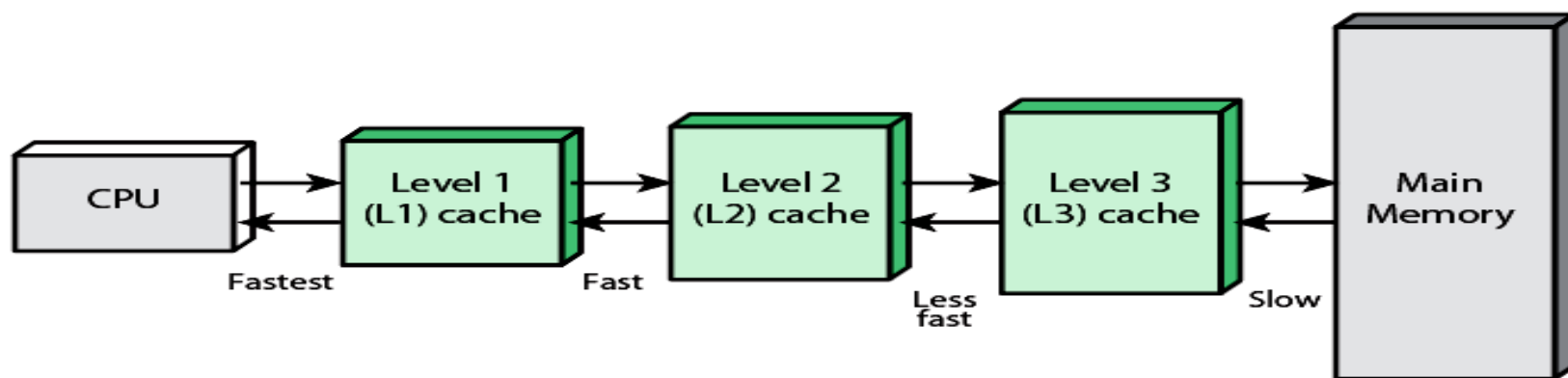
- Đây là bộ nhớ bán dẫn có tốc độ nhanh và chúng được đặt giữa CPU và bộ nhớ chính nhằm tăng tốc truy xuất của CPU tới bộ nhớ chính.
- Dung lượng nhỏ hơn nhiều so với bộ nhớ chính
- Tốc độ nhanh hơn rất nhiều lần.
- Ngày nay cache được tích hợp vào trong bộ VXL và nó trong suốt với người dùng.
- Cache có thể có hoặc không.



Các mức cache



(a) Single cache



(b) Three-level cache organization



External Memory

➤ Chức năng và đặc điểm:

- lưu trữ tài nguyên phần mềm máy tính.
- Được kết nối với hệ thống như thiết bị vào ra.
- Dung lượng rất lớn (vài trăm GB)
- Tốc độ chậm.

➤ Các loại bộ nhớ:

- Bộ nhớ từ : đĩa cứng, đĩa mềm,...
- Bộ nhớ quang: CD, VCD, DVD,...
- Bộ nhớ bán dẫn: flash disk,...

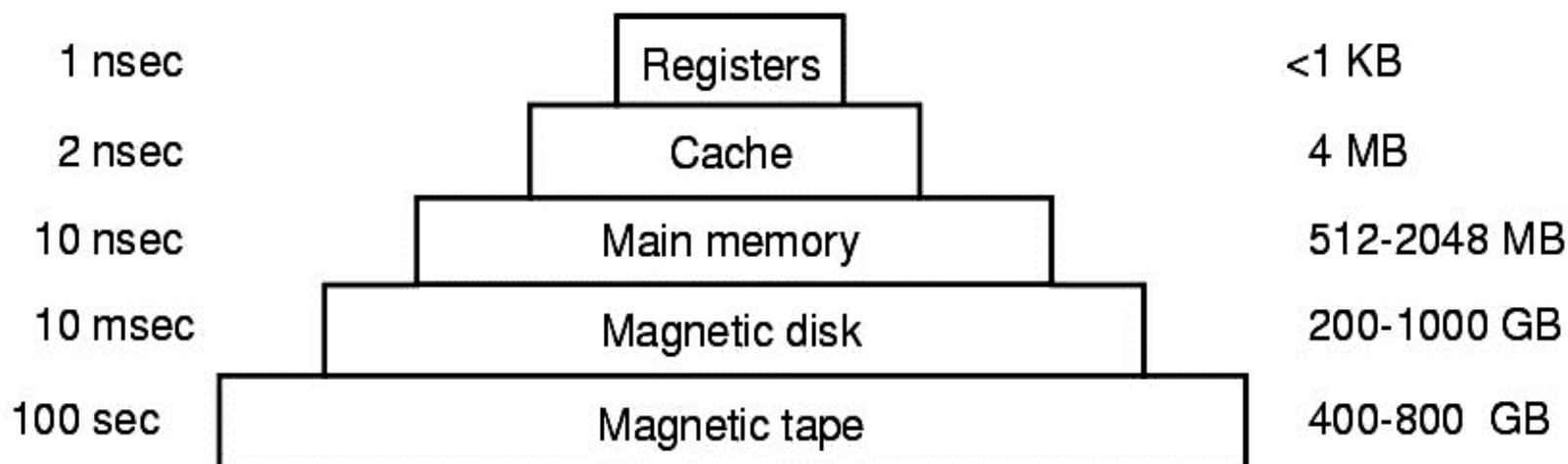


Phân cấp bộ nhớ

➤ Khác biệt : Dung lượng, tốc độ truy cập, giá thành.

Typical access time

Typical capacity





Đặc điểm các loại bộ nhớ

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape



Thiết bị ngoại vi(Peripherals)

- Chức năng: giao tiếp giữa máy tính với thế giới bên ngoài (con người).
 - Nhiệm vụ: chuyển đổi dạng dữ liệu từ bên ngoài (con người) thành dữ liệu máy tính và ngược lại.
 - Các thiết bị ngoại vi cơ bản
 - Thiết bị nhập(input devices): keyboard, mouse...
 - Thiết bị xuất (output devices): monitor, printer....
 - TB truyền thông (communication sevices) Modem,..
 - TB lưu trữ (storage devices)
-



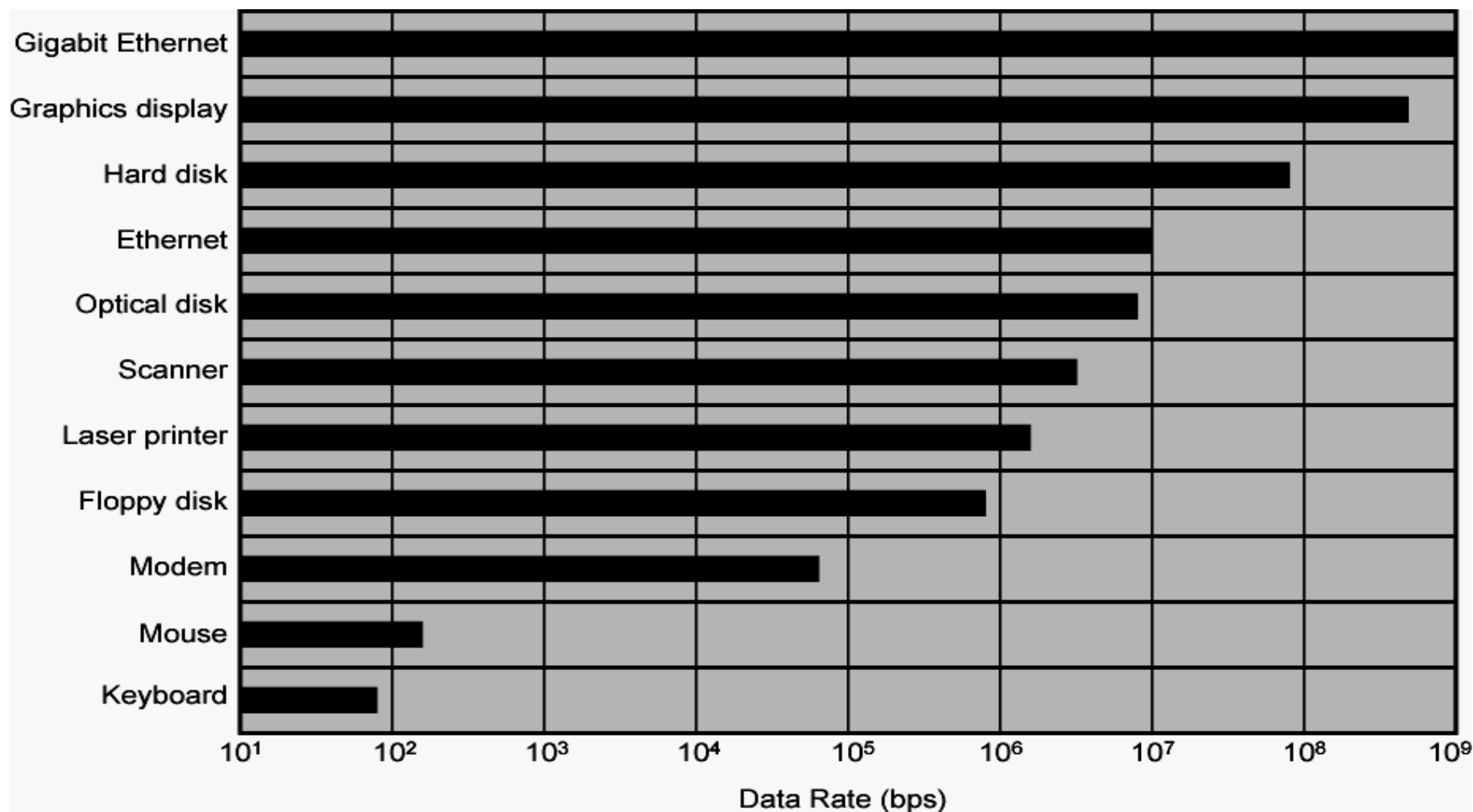
Các thiết bị lưu trữ (storage devices)

- Giấy
 - Băng giấy đục lỗ, Phiếu đục lỗ,
- Từ tính
 - Xuyên từ, Trống từ, Băng từ
 - Đĩa từ (Đĩa mềm, Đĩa cứng)
- Quang học
 - CD/ DVD
 - Blue-ray, HD-DVD
- Quang từ
 - MO disk
- Bán dẫn
 - USB Flash, SSD, thẻ nhớ, ...
- Khác
 - Bubble, Hologram, ...





Tốc độ truy cập thiết bị ngoại vi





Module IO

➤ Chức năng: nối ghép thiết bị ngoại vi với máy tính.

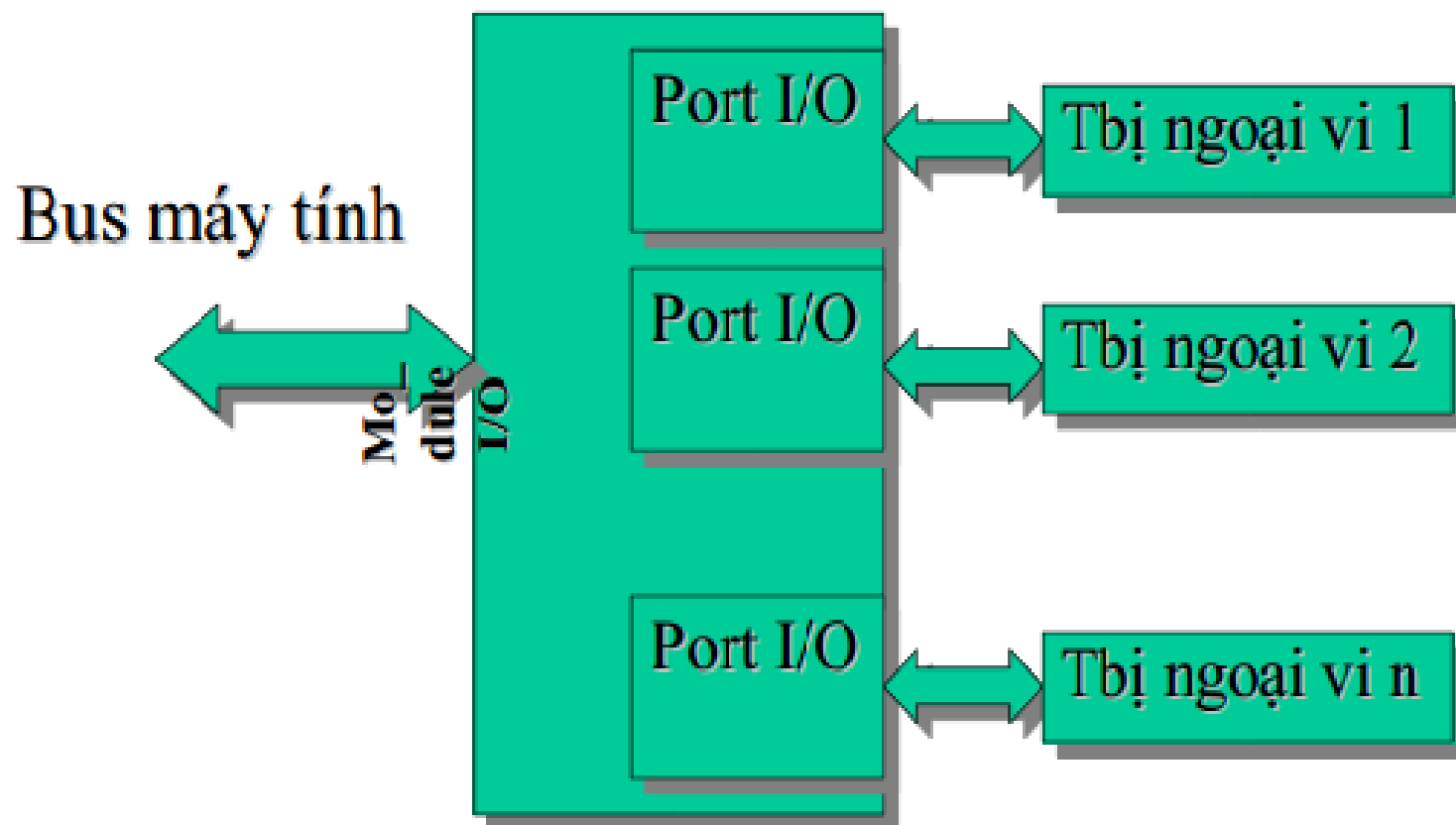
☞ mỗi Module có 1 hay nhiều cổng vào ra

☞ mỗi cổng được đánh địa chỉ xa1x định.

Các thiết bị ngoại vi được kết nối với máy tính thông qua các cổng vào ra(vd: COM. LPT. USB, VGA,...



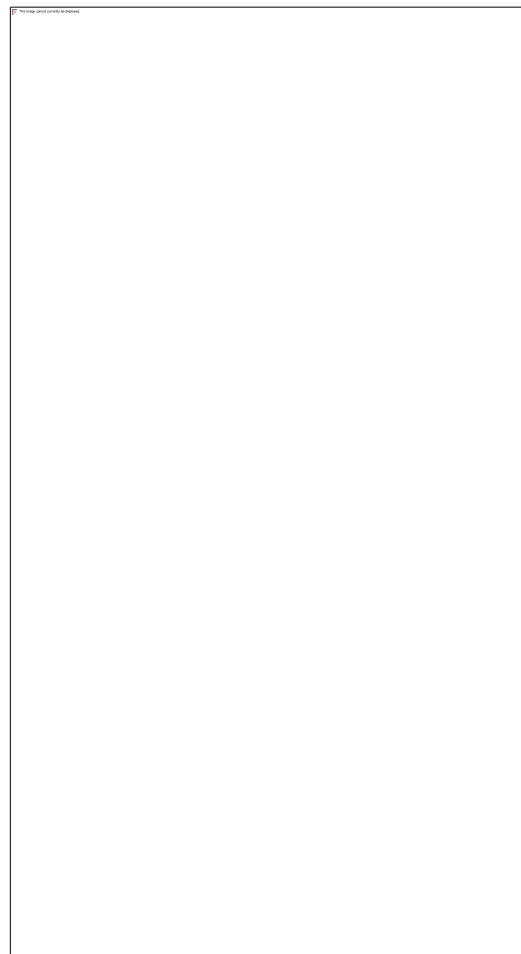
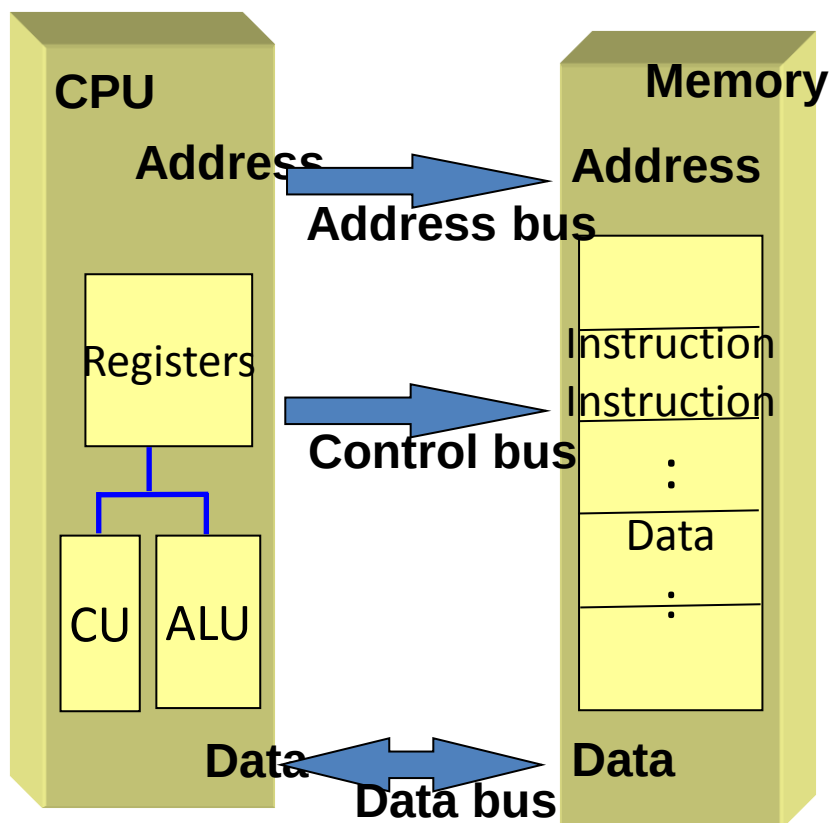
Cấu trúc vào ra cơ bản



- Khái niệm BUS : là tập hợp các đường dây dùng để vận chuyển dữ liệu từ thành phần này tới thành phần khác bên trong máy tính.
- Độ rộng của Bus : là số đường dây có khả năng vận chuyển các bit dữ liệu đồng thời.
- Phân loại bus: theo chức năng chia làm 3 loại:
 - Address bus
 - Không gian địa chỉ
 - Data bus
 - Control bus



Mô hình hệ thống 3 bus





Bus địa chỉ

- Chức năng: dùng để vận chuyển địa chỉ từ CPU đến các Module nhớ hay Module vào ra, nhằm xác định ngăn nhớ hay cổng vào ra nào cần truy xuất hay trao đổi thông tin. (đây là Bus một chiều).
- Độ rộng của Bus địa chỉ ($A_0, A_1, \dots, A_{n-1}$) Cho biết khả năng quản lý cực đại số các ngăn nhớ. Nếu sử dụng độ rộng Bus địa chỉ n đường thì dung lượng cực đại của bộ nhớ có thể quản lý là 2^n ngăn nhớ
- VD: Bus địa chỉ của 1 số bộ vi xử lý là
 - 8088/8086 $n=20$ 2^{20} (1MB)
 - 80286 $n=24$ 2^{24} (16MB)
 - 80386 $n=32$ 2^{32} (4GB)
 - Pentium II, III, IV $n=36$ 2^n (64GB)



BUS dữ liệu

- Chức năng: vận chuyển dữ liệu giữa CPU, Bộ nhớ và cổng vào ra.
- Độ rộng của Bus dữ liệu ($D_0, D_1, \dots, D_{m-1}$) cho biết số byte có khả năng trao đổi đồng thời.
 $M=8,16,32,64,128$ bit.
- VD
 - 8088 $\Rightarrow m=8$
 - 8086 $\Rightarrow m=16$
 - 80386 $\Rightarrow m=32$
 - Pentium $\Rightarrow m=64$



BUS điều khiển

- Chức năng : vận chuyển các tín hiệu điều khiển
 - Bao gồm:
 - Các tín hiệu từ CPU để điều khiển Module nhớ và Module IO.
 - Các tín hiệu từ Module nhớ, Module IO gửi yêu cầu đến CPU.
 - Là Bus cung cấp nguồn tín hiệu xung nhịp (clock) để đồng bộ các hoạt động của bus.
-

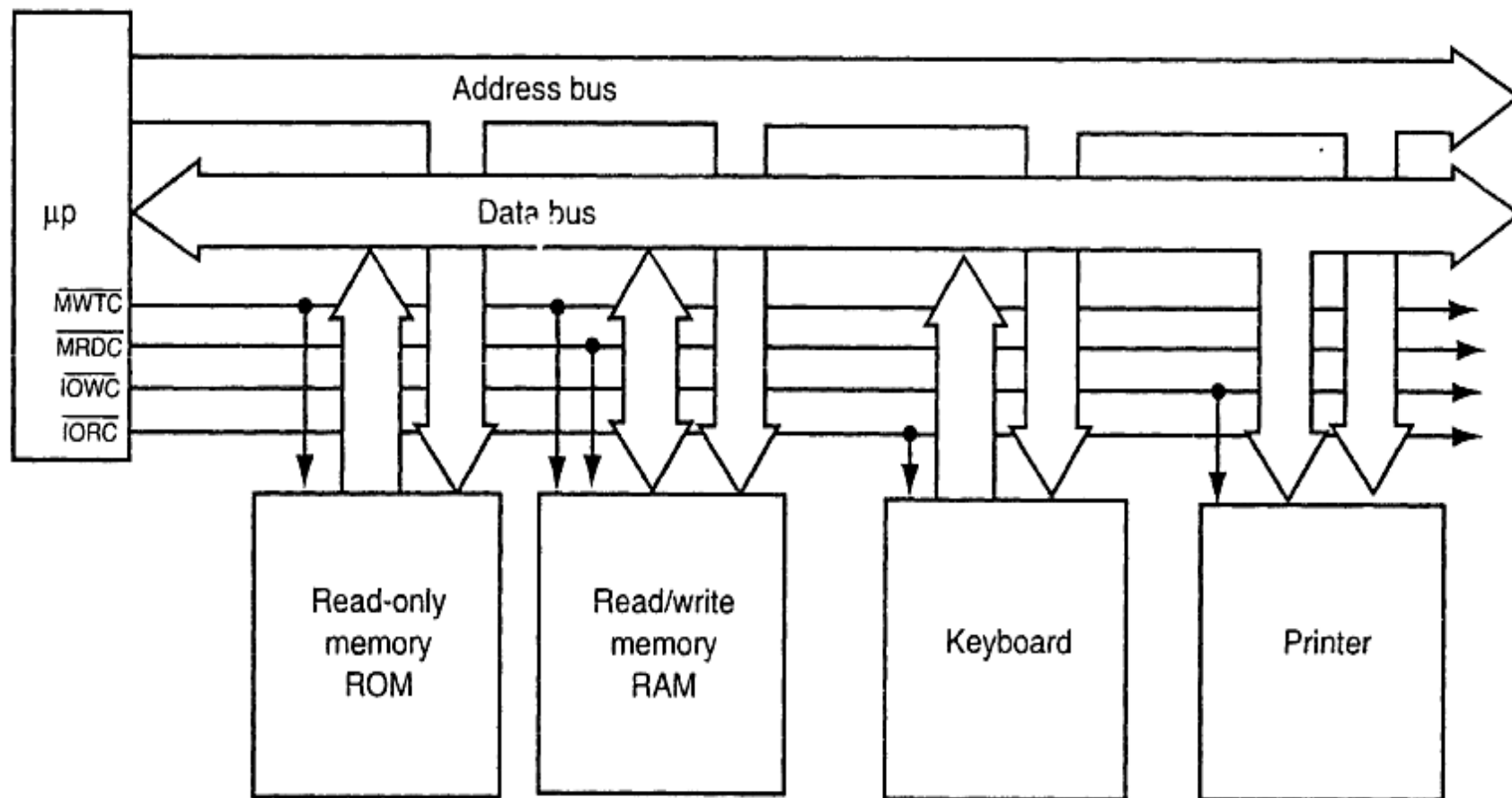


Một số tín hiệu điển hình

- MemR : đọc dữ liệu từ ngăn nhớ xác định trong bộ nhớ
- IOR : đọc dữ liệu từ một cổng vào ra.
- MemW: ghi dữ liệu có sẵn trên bus đến một ngăn nhớ xác định.
- IOW : ghi dữ liệu có sẵn ra cổng.
- Interrupt Request(INTR) yêu cầu ngắt từ thiết bị ngoại vi.
- Interrupt Acknowledge(INTA) chấp nhận ngắt phát ra từ CPU.
- Ngoài ra còn các tín hiệu khác như: t/h yêu cầu và chấp nhận CPU chuyển nhượng BUS (BRQ, BGT)...



Bus hệ thống



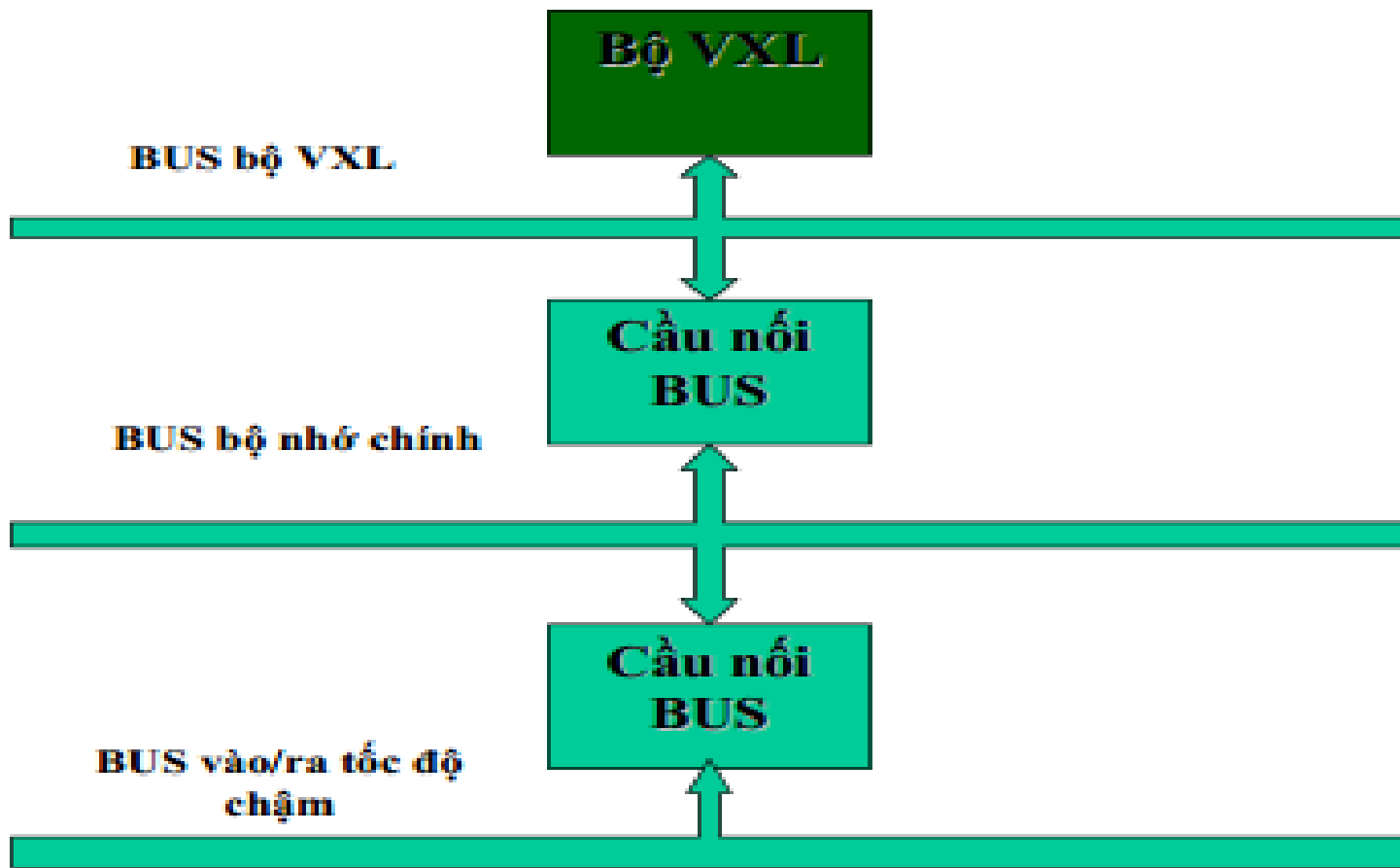


Đặc điểm của cấu trúc đơn BUS

- Có nhiều thành phần nối vào Bus chung.
- Tại một thời điểm chỉ phục vụ được một yêu cầu trao đổi dữ liệu.
- Các thành phần nối vào Bus có thể có tốc độ khác nhau.
- Các Module nhớ và Module IO phụ thuộc vào cấu trúc của CPU.
- ☞ Khắc phục:
 - ☞ Xây dựng cấu trúc đa Bus bao gồm các hệ thống Bus khác nhau về tốc độ.
 - ☞ Trong hầu hết các máy PC bus được phân thành 3 cấp và các bus nối với nhau thông qua cầu nối Bus.

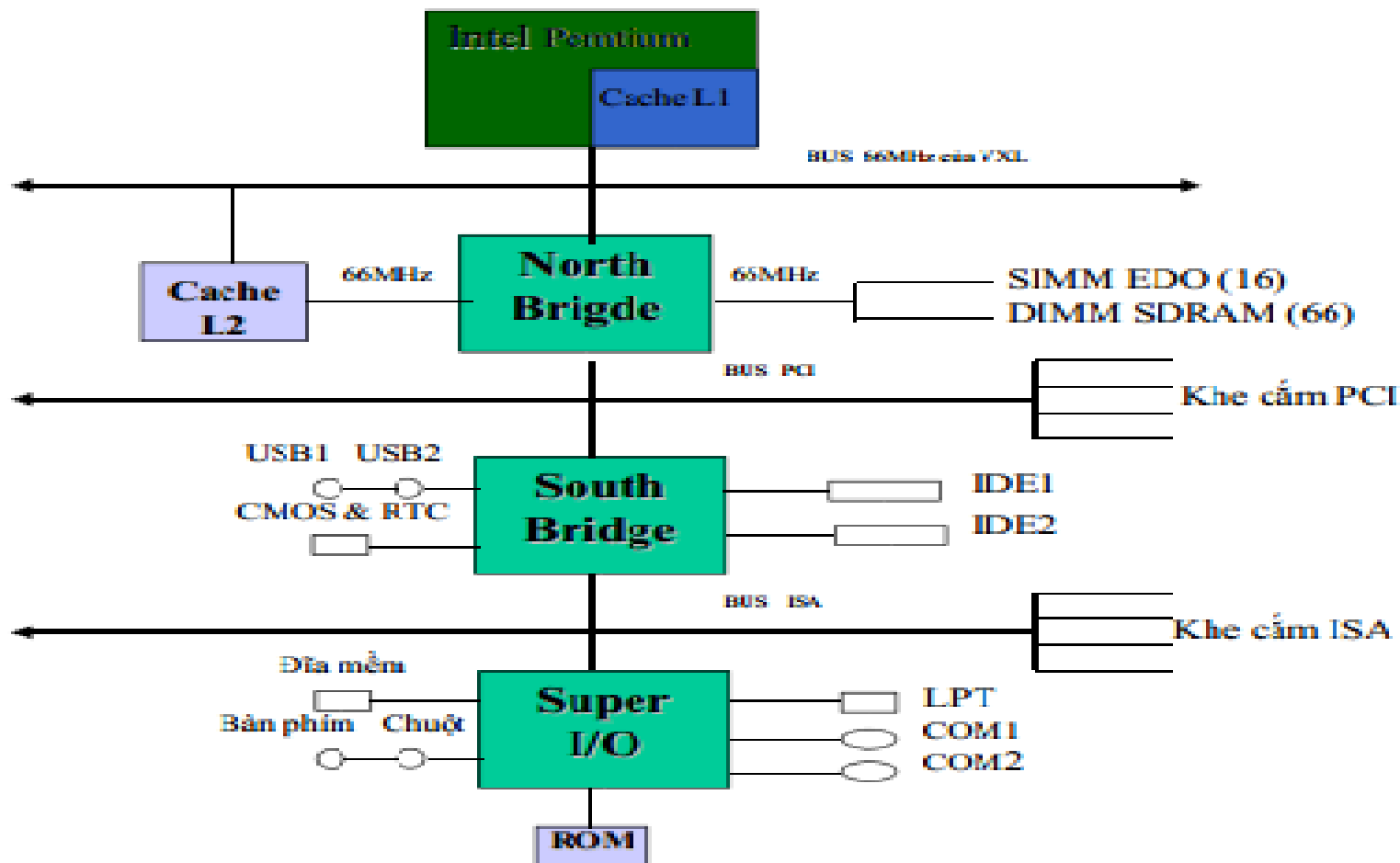


Bus phân 3 cấp





Cấu trúc Pentium II





Phần trao đổi và giải đáp

Chương 2

Biểu diễn dữ liệu và số học máy tính

2.1 Các hệ đếm cơ bản

2.3 Biểu diễn số nguyên

2.4 Số học nhị phân

2.5 Biểu diễn số dấu chấm động

2,6 Biểu diễn ký tự.



2.1 Các hệ đếm cơ bản

- Hệ thập phân (Decimal System) con người sử dụng
- Hệ nhị phân (Binary System) Máy tính sử dụng.
- Hệ thập lục phân (Hexadecimal System) dùng biểu diễn rút ngắn số nhị phân
- Cách chuyển đổi giữa các hệ số.



Hệ thập phân (Decimal)

Bộ ký tự cơ sở gồm 10 số : 0..9

Được sử dụng rộng rãi trong đời sống hàng ngày. Không hợp với MT

Dạng tổng quát: $a_{n-1} a_{n-2} a_{n-3} \dots a_1 a_0 a_{-1} a_{-2} \dots a_{-m}$

$$A = \sum_{i=-m}^{n-1} a_i * 10^i \text{ trong đó } a_i = 0..9$$

VD 123,45

Phần nguyên

$$123 : 10 = 12 \text{ dư } 3$$

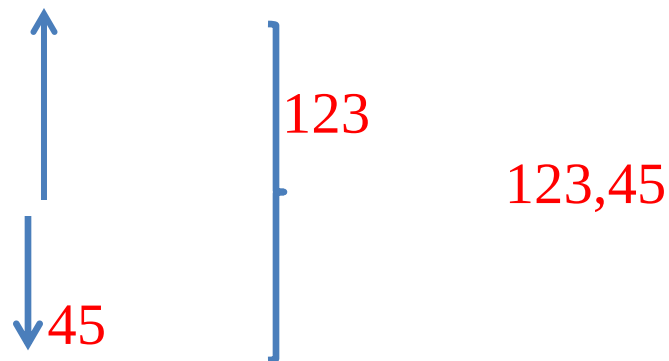
$$12 : 10 = 1 \text{ dư } 2$$

$$1 : 10 = 0 \text{ dư } 1$$

Phần phân

$$0,45 * 10 = 4,5$$

$$0,5 * 10 = 5$$





Hệ nhị phân (Binary)

Hệ nhị phân(Binary)

Bộ ký tự cơ sở gồm 2 số : 0,1

Mỗi ký số được gọi là bit(binay dig^{it}) đơn vị thông tin nhỏ nhất.

Các bội số : Byte(B), KB, MB. GB. TB, PB, EB.

Thích hợp với máy tính, khó sử dụng đối với người.

Dạng tổng quát: $a_{n-1} a_{n-2} a_{n-3} \dots a_1 a_0 a_{-1} a_{-2} \dots a_{-m}$

$$A = \sum_{i=-m}^{n-1} a_i * 2^i \quad a_i = 0,1$$

$$\text{VD : } 11011,011_2 = 2^4 + 2^3 + 2^1 + 2^0 + 2^{-2} + 2^{-3} = 27,375$$

Hệ Thập Lục Phân (Hexadecimal)

Thập lục phân (hexadecimal)

Bộ ký tự cơ sở 16 ký số : 0..9, A...F

Là một dạng viết gọn của số nhị phân($16 = 2^4$)

Hiện đang sử dụng rộng rãi trong máy tính.

Dạng tổng quát: $a_{n-1} a_{n-2} a_{n-3} \dots a_1 a_0 a_{-1} a_{-2} \dots a_{-m}$

$$A = \sum_{i=-m}^{n-1} a_i * 16^i \quad a_i = 0..9, A..F$$

VD : $89AB_H = 1000 \ 1001 \ 1010 \ 1011_B$



Dạng Tổng Quát

➤ Một số ở hệ cơ số B gồm n..-m ký số:

○ $a_{n-1} a_{n-2} a_{n-3} \dots a_1 a_0 a_{-1} a_{-2} \dots a_{-m}$

Dạng Tổng Quát :

$$A = \sum_{i=-m}^{n-1} a_i * B^i \quad a_i = 0, \dots, B - 1$$



Đổi chiều các hệ thống số

Thập phân	Nhị phân	Bát phân	Thập lục phân
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Chuyển đổi giữa các hệ thống số

- Đổi từ hệ thập phân sang hệ bất kỳ:
 - Quy tắc 1: Đổi phần nguyên riêng và đổi phần thập phân (lẻ) sau đó ghép lại.
 - Quy tắc 2: Đổi số nguyên hệ thập phân sang hệ B bằng cách chia liên tiếp số cần đổi cho B và giữ lại số dư cho đến khi thương số $= 0$. Số cần tìm là các số dư viết theo chiều ngược lại.
 - Quy tắc 3: Đổi phần thập phân sang hệ B bằng cách nhân liên tiếp phần thập phân cho B và giữ lại phần nguyên cho đến khi tích số $= 0$ (hoặc đủ độ chính xác). Số cần tìm là ký số nguyên viết theo chiều thuận.

- Ví dụ đổi $105.6875_{(10)}$ ra số nhị phân
 - Đổi phần nguyên 105 ra nhị phân
 - Đổi phần thập phân 0.6875 ra nhị phân
 - Ghép kết quả lại

○ Đổi $105_{(10)}$ ra nhị phân

$$105 : 2 = 52 \text{ dư } 1$$

$$52 : 2 = 26 \text{ dư } 0$$

$$26 : 2 = 13 \text{ dư } 0$$

$$13 : 2 = 6 \text{ dư } 1$$

$$6 : 2 = 3 \text{ dư } 0$$

$$3 : 2 = 1 \text{ dư } 1$$

$$1 : 2 = 0 \text{ dư } 1$$

Kết quả: $105_{(10)} = 1101001_{(2)}$

- Đổi $0.6875_{(10)}$ ra nhị phân
 - $0.6875 \times 2 = 1.375$ phần nguyên = 1
 - $0.375 \times 2 = 0.75$ phần nguyên = 0
 - $0.75 \times 2 = 1.5$ phần nguyên = 1
 - $0.5 \times 2 = 1.0$ phần nguyên = 1
- Kết quả : $0.6875_{(10)} = 0.1011_{(2)}$
- Ghép lại: $105.6875_{(10)} = 1101001.1011_{(2)}$
- Bài tập: Đổi $0.1_{(10)}$ ra nhị phân $\rightarrow$ Nhận xét?

- Đổi từ hệ nhị phân sang hệ bát phân và ngược lại
 - Qui tắc: ghép 3 ký số nhị phân đổi ra 1 ký số bát phân (hoặc ngược lại).
- Đổi từ hệ nhị phân sang hệ thập lục phân và ngược lại
 - Qui tắc: ghép 4 ký số nhị phân đổi ra 1 ký số thập lục phân (hoặc ngược lại).
- Ví dụ:
 - $B3_{(16)} = \underline{1011} \underline{0011}_{(2)}$
 - $\underline{10} \underline{110} \underline{011}_{(2)} = 263_{(8)}$
- Đổi giữa bát phân sang thập lục phân và ngược lại
 - Qui tắc: Đổi sang 1 hệ trung gian (thường là nhị phân như ví dụ trên)



2.2 Biểu diễn số nguyên

➤ Số nguyên không dấu

- Nguyên tắc tổng quát: Dùng n bit biểu diễn số nguyên không dấu A :

$$a_{n-1}a_{n-2}\dots a_2a_1a_0$$

- Giá trị của A được tính như biểu thức sau:

$$A = \sum_{i=0}^{n-1} a_i \cdot 2^i$$

- Dải biểu diễn của A : từ 0 đến $2^n - 1$

- Ví dụ 1. Biểu diễn các số nguyên không dấu sau đây bằng 8-bit: $A = 41$; $B = 150$
- Giải:
 - $A = 41 = 32 + 8 + 1 = 2^5 + 2^3 + 2^0$
 $41 = 0010\ 1001$
 - $B = 150 = 128 + 16 + 4 + 2 = 2^7 + 2^4 + 2^2 + 2^1$
 $150 = 1001\ 0110$

– Với $n = 8$ bit

$$0000\ 0000 = 0$$

$$0000\ 0001 = 1$$

$$0000\ 0010 = 2$$

$$0000\ 0011 = 3$$

...

$$1111\ 1111 = 255 \quad \text{Biểu diễn được các giá trị từ 0 đến 255}$$

– Chú ý:

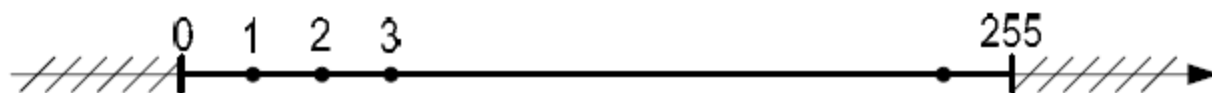
$$\begin{array}{r} 1111\ 1111 \\ + \underline{0000\ 0001} \end{array}$$

$$1\ 0000\ 0000 \quad \text{Vậy: } 255 + 1 = 0 ?$$

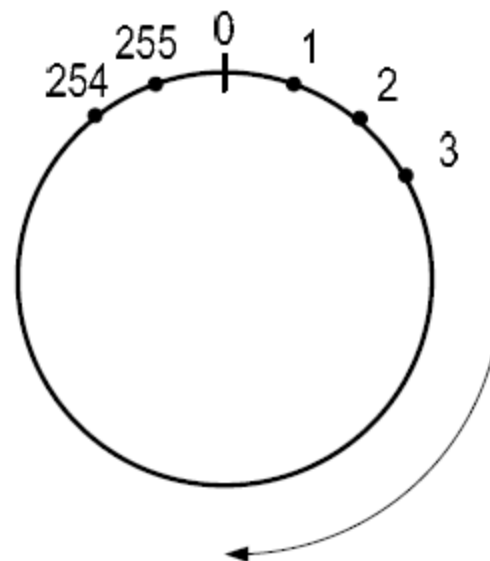
➔ do tràn nhớ ra ngoài

➤ Số nguyên không dấu (tiếp)

- Trục số học với $n=8$ bit



Trục số học máy tính:



- Với $n=16$ bit
 - Dải biểu diễn $0 \rightarrow 65.535$
- Với $n=32$ bit : $0 \rightarrow 2^{32}-1$
- Với $n=64$ bit : $0 \rightarrow 2^{64}-1$



Biểu diễn số nguyên có dấu

- Quy tắc 1: Dùng 1 bit cao nhất làm bit dấu, các bit còn lại biểu diễn như số không dấu
 - Bit dấu = 0 : số dương
 - Bit dấu = 1 : số âm
- Ví dụ số 4 bit
 - 1 bit dấu
 - 3 bit số nguyên
 - Dải biểu diễn $-7 \dots +7$

Thập Phân	Nhi Phân	Thập Phân	Nhi Phân
+0	0000	-0	1000
+1	0001	-1	1001
+2	0010	--2	1010
+3	0011	-3	1011
+4	0100	-4	1100
+5	0101	-5	1101
+6	0110	-6	1110
+7	0111	-7	1111

○ Nhược điểm

- Tồn tại 2 số 0: +0 và -0
- Kết quả tính toán không chính xác
- Ví dụ : tính $(+4) + (-2)$

+4 0100

-2 1010

+2 1110 (-6) → kết quả ra -6 chứ không phải +2

○ Cách khắc phục: Dùng số bù 2

○ Qui tắc 2: Dùng số bù 2 (two's-complement) để biểu diễn số âm.

- $[Số\ bù\ 2] = [Số\ bù\ 1] + 1$
- Số bù 1 tính bằng cách đảo các bit $1 \rightarrow 0$ và $0 \rightarrow 1$
- Ví dụ tìm số bù 2 của +2
 - Số +2 : 0010
 - Số bù 1 : 1101 (đảo các bit)
 - Công thêm 1: $\underline{\quad + \quad 1}$
 - Số bù 2 : 1110



– Ví dụ bảng số 4 bit dùng số bù 2 cho số âm

– Kết quả:

- Chỉ còn 1 số +0
- Bù 2 của bù 2 bằng chính nó
- Mở rộng thêm số -8
- Kết quả tính toán chính xác.
- Ví dụ : tính lại $(+4) + (-2)$

+4 0100

-2 1110

+2 1 0010 (+2)

(bỏ qua bit tràn số)

Thập Phân	Nhi Phân	Thập Phân	Nhi Phân
+0	0000	-	-
+1	0001	-1	1111
+2	0010	-2	1110
+3	0011	-3	1101
+4	0100	-4	1100
+5	0101	-5	1011
+6	0110	-6	1010
+7	0111	-7	1001
-	-	-8	1000

- Ví dụ biểu diễn các số nguyên có dấu sau đây bằng 8-bit:

$$A = +58 ; B = -80$$

- Giải:

$$A = +58 = 0011\ 1010_{(2)}$$

$$B = -80$$

$$\text{Ta có: } +80 = 0101\ 0000$$

$$\text{Số bù một} = 1010\ 1111$$

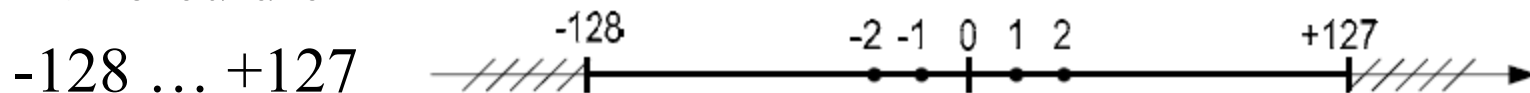
$$\begin{array}{r} \\ + 1 \\ \hline \end{array}$$

$$\text{Số bù hai} = 1011\ 0000$$

$$\text{Vậy: } B = -80 = 1011\ 0000_{(2)}$$

- Trục số học số nguyên có dấu với $n = 8$ bit

- Dải biểu diễn



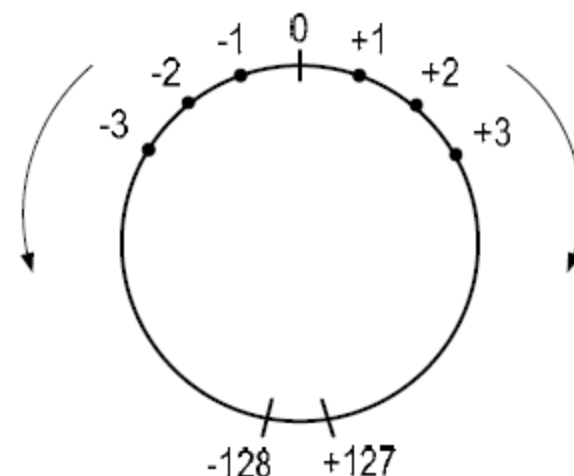
- Với $n=16$ bit

Trục số học máy tính:

- $-32768 \dots +32767$

- Với $n=32$ bit : $-2^{31} \dots +2^{31}-1$

- Với $n=64$ bit : $-2^{63} \dots +2^{63}-1$



– Nhược điểm: không thể chứa số lớn hơn dải giới hạn (tràn số)



Số BCD(Binary Coded Decimal)

- Số nhị phân có nhược điểm khó biểu diễn chính xác đối với số rất lớn hoặc rất nhỏ.
 - Trong 1 số trường hợp đòi hỏi tính toán chính xác từng ký số (ví dụ trong tài chính, ngân hàng,...)
 - Quy tắc : Mã hóa mỗi ký số thập phân 0...9 bằng 1 byte. Chỉ sử dụng 4 bit cuối, 4 bit đầu = 0 hoặc sử dụng cho các mục đích khác.
 - Để tiết kiệm bộ nhớ, có thể ghép 2 ký số vào 1 byte, 4 bit đầu 1 ký số, 4 bit cuối 1 ký số. Phương pháp này gọi là số BCD dạng dòn (packed-BCD)
 - Áp dụng cho số nguyên hoặc số thực dấu chấm tĩnh.
-



Bảng mã BCD

Ký số	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Ghi chú:

- Mỗi số BCD chính là mã nhị phân của các ký số.
- Có 6 mã không được sử dụng:
 - 1010
 - 1011
 - 1100
 - 1101
 - 1110
 - 1111
- Đơn vị 4 bit (=1/2 byte) được gọi là nibble



- Ví dụ biểu diễn số 35:
 - Unpacked-BCD: 0000 0011 0000 0101 (2 byte)
 - Packed-BCD: 0011 0101 (1 byte)

- Phép cộng trên số BCD

$$\begin{array}{r} 35 \quad 0011 \ 0101_{\text{BCD}} \\ + 61 \quad + 0110 \ 0001_{\text{BCD}} \\ \hline 96 \quad 1001 \ 0110_{\text{BCD}} \end{array}$$

kết quả đúng (không phải hiệu chỉnh)

$$\begin{array}{r} 87 \quad 1000 \ 0111_{\text{BCD}} \\ + 96 \quad + 1001 \ 0110_{\text{BCD}} \\ \hline 183 \quad 1 \ 0001 \ 1101 \rightarrow \text{kết quả sai} \\ \quad \quad + 0110 \ 0110 \quad \text{hiệu chỉnh} \end{array}$$

0001 1000 0011_{BCD} → kết quả đúng 1 8 3

Hiệu chỉnh: cộng thêm 6 ở những vị trí có nhớ (>9)



Số thực dấu chấm tĩnh (fixed-point decimal)

- Qui tắc: Qui ước 1 vị trí chứa dấu chấm thập phân. Số thực được lưu trữ bằng 2 số nguyên:
 - Số nguyên có dấu cho phần nguyên
 - Số nguyên không dấu cho phần thập phân
- Ví dụ: Một số thực 16 bit

12 bit integer part

4 bit fractional part

- Chọn vị trí dấu chấm sao cho phù hợp độ chính xác cần biểu diễn cho từng thành phần

- Nhược điểm: Khó biểu diễn các số quá nhỏ hoặc quá lớn
- Ví dụ:
 - Số 1,234,000,000,000,000,000.00 cần nhiều bit cho phần nguyên
 - Số 0.000 000 000 000 000 123 456 cần nhiều bit cho phần thập phân
- Cách khắc phục: Chuyển sang số dạng khoa học (dấu chấm động)



Số thực dấu chấm động (floating-point decimal)

- Qui tắc : Cho phép thay đổi vị trí dấu chấm thập phân cho phù hợp nhu cầu với độ chính xác vừa phải
- Ví dụ
 - $-123,000,000,000,000,000.00 = -123 \times 10^{15} (-123 \text{ E}+15)$
 - $+0.000\ 000\ 000\ 000\ 000\ 123 = 123 \times 10^{-18} (+123 \text{ E}-18)$
- Số thực được lưu trữ bằng 2 số nguyên
 - Số nguyên có dấu cho phần định trị
 - Số nguyên có dấu cho phần lũy thừa
- Nhược điểm: Độ chính xác giới hạn

- Tổng quát: một số thực X được biểu diễn theo kiểu số dấu chấm động như sau:

$$X = M * R^E$$

- M là phần định trị (Mantissa),
 - R là cơ số (Radix),
 - E là phần mũ (Exponent).
- Trước đây mỗi hãng sản xuất máy tính tự qui định các thành phần M , R và E riêng biệt dẫn đến khó trao đổi dữ liệu → cần chuẩn hóa



Chuẩn IEEE754

- Qui định về định dạng và sử dụng số dấu chấm động trong máy tính
- Do IEEE (Institute of Electrical and Electronic Engineers) ban hành lần đầu 1985, phiên bản mới nhất ban hành 2008
- Sử dụng cơ số nhị phân ($R=2$)
- Các định dạng
 - Chính xác đơn (single precision): 32 bit
 - Chính xác kép (double precision): 64 bit
 - Chính xác mở rộng (extended precision) trên CPU Intel: 80 bit
 - Phiên bản 2008 có thêm định dạng 128 bit

○ Định dạng số thực theo IEEE754



- **S** : là bit dấu của phần định trị **M**:
 - $S = 0$: số dương
 - $S = 1$: số âm
- **e** : là mã thừa n (excess- n) của phần mũ **E**, n là số bit biểu diễn số của **E** (do đó không cần lưu bit dấu cho **E**)
 - $e = E + (2^n - 1) \rightarrow E = e - (2^n - 1)$
- **m** : là phần lẻ của phần định trị **M** ở dạng chuẩn:
 - $M = 1.m$ (Chú ý: Không sử dụng số bù 2)
- Công thức xác định giá trị của số thực:
 - $X = (-1)^S \times 1.m \times 2^E$

Loại	Số bit của e	Số bit của m	Mã thừa n	Giải biểu diễn	Ký số chính xác
Chính xác đơn 32 bit	8	23	127	$\pm 1 E \pm 38$	7
Chính xác kép 64 bit	11	52	1023	$\pm 1 E \pm 308$	16
Mở rộng 80 bit	15	64	16383	$\pm 1 E \pm 4932$	19

- Cần chú ý khi sử dụng và so sánh số thực vì độ chính xác bị giới hạn.
- Ví dụ lưu số 123,456,789,012 bằng số thực 32 bit chỉ bảo đảm chính xác 7 ký số có nghĩa đầu tiên, các ký số còn lại không chính xác

- Ví dụ 1: Xác định giá trị của số thực được biểu diễn bằng 32-bit $X = C1560000_{(16)}$:
 - $X = \underline{1100\ 0001\ 0101\ 0110\ 0000\ 0000\ 0000\ 0000}_{(2)}$
 - $S = 1 \rightarrow$ số âm
 - $e = 1000\ 0010_{(2)} = 130_{(10)} \rightarrow E = 130 - 127 = 3$
- Vậy
 - $X = -1.10101100 * 2^3 = -1101.011_{(2)} = -13.375_{(10)}$

- Ví dụ 2: Biểu diễn số thực $X = 83.75$ về dạng số dấu chấm động IEEE754 dạng 32-bit

Giải:

- $X = 83.75_{(10)} = 1010011.11_{(2)} = 1.01001111 \times 2^6$

Ta có:

- $S = 0$ vì đây là số dương
- $E = e - 127 = 6 \rightarrow$
- $e = 127 + 6 = 133_{(10)} = 1000\ 0101_{(2)}$

Vậy:

- $X = \underline{0100\ 0010\ 1010\ 0111}\ 1000\ 0000\ 0000\ 0000_{(2)}$
- $X = 42A78000_{(16)}$

– Các quy ước đặc biệt

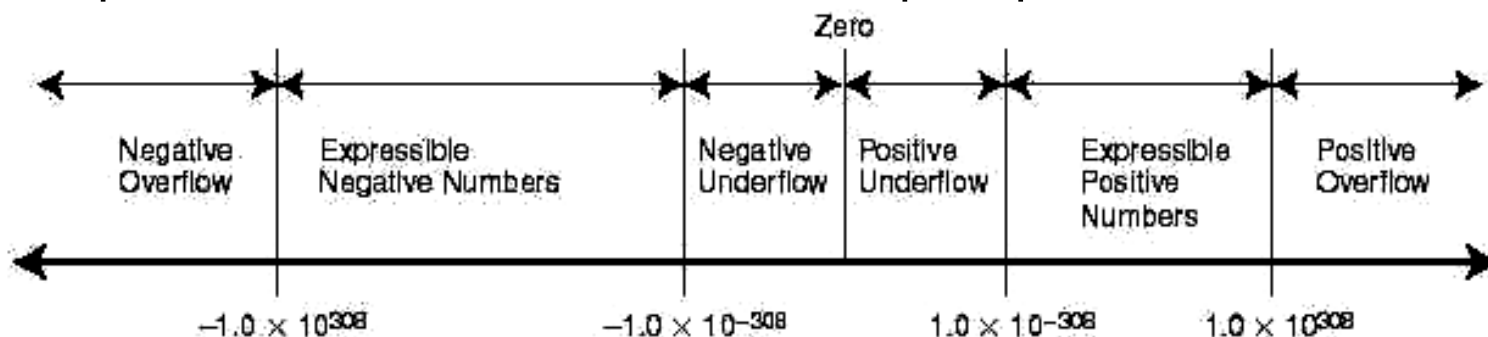
- Các bit của e bằng 0, các bit của m khác 0, thì phần định trị không ở dạng chuẩn: 0.m (dạng chuẩn là 1.m)
- Các bit của e bằng 0, các bit của m bằng 0, thì $X = \pm 0$
- Các bit của e bằng 1, các bit của m bằng 0, thì $X = \pm \infty$
- Các bit của e bằng 1, còn m có chứa ít nhất một bit bằng 1, thì nó không biểu diễn cho số nào cả (NaN - not a number)

Normalized	$\pm$	$0 < \text{Exp} < \text{Max}$	Any bit pattern
Denormalized	$\pm$	0	Any nonzero bit pattern
Zero	$\pm$	0	0
Infinity	$\pm$	1 1 1 ... 1	0
Not a number	$\pm$	1 1 1 ... 1	Any nonzero bit pattern

↖ Sign bit

Các khả năng tràn số

- Tràn trên số mũ (Exponent Overflow): mũ dương vượt ra khỏi giá trị cực đại của số mũ dương có thể. ($\rightarrow \infty$)
- Tràn dưới số mũ (Exponent Underflow): mũ âm vượt ra khỏi giá trị cực đại của số mũ âm có thể ($\rightarrow 0$).
- Tràn trên phần định trị (Mantissa Overflow): cộng hai phần định trị có cùng dấu, kết quả bị nhớ ra ngoài bit cao nhất.
- Tràn dưới phần định trị (Mantissa Underflow): Khi hiệu chỉnh phần định trị, các số bị mất ở bên phải phần định trị.





Tính toán trên số thực

- Thực hiện các phép tính phức tạp
- Đối với phép tính cộng & trừ
 - Kiểm tra = zéro ?
 - Hiệu chỉnh phần số mũ
 - Cộng hoặc trừ phần định trị
 - Chuẩn hoá kết quả
- Đối với phép tính nhân & chia
 - Kiểm tra = zéro ?
 - Cộng hoặc trừ phần số mũ
 - Nhân hoặc chia phần định trị, xác định dấu kết quả
 - Chuẩn hoá và làm tròn



Đơn vị đo tốc độ tính toán

- Tốc độ xung nhịp (Hertz)
 - Dựa trên đồng hồ xung nhịp
- Millions of instructions per second (MIPS)
 - Dựa trên phép tính số nguyên
- Millions of floating point operations per second (MFLOPS)
 - Dựa trên phép tính số thực
- Benchmarks
 - Dựa trên các phần mềm đặc trưng thông dụng viết bằng NNLT cấp cao (HLL)
 - Ví dụ: Hệ thống SPEC (System Performance Evaluation Corporation)



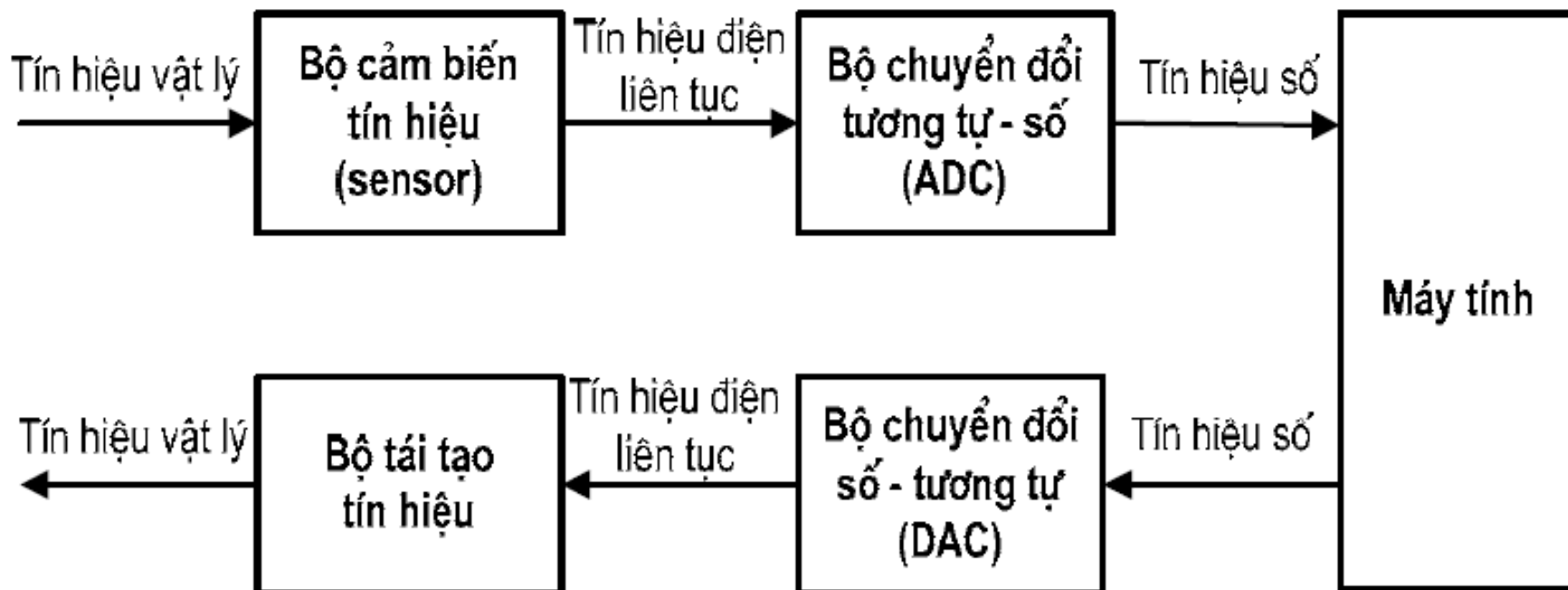
Biểu diễn ký tự

➤ Khái niệm

- Thông tin và dữ liệu do con người sử dụng ở dạng văn bản thường được sử dụng nhất
- Văn bản bao gồm các loại ký tự:
 - Chữ thường : a...z
 - Chữ hoa : A...Z
 - Số : 0...9
 - Các dấu : +, -, @, #, &, [,],...
- Ngoài ra còn cần biểu diễn các ký tự điều khiển khi trao đổi thông tin với thiết bị ngoại vi : xuống dòng, sang trang, xóa, ...

- Nguyên tắc chung về mã hóa dữ liệu
 - Mọi dữ liệu đưa vào máy tính đều phải được mã hóa thành số nhị phân, ánh xạ 1-1 và có chiều dài bit bằng nhau.
 - Các loại dữ liệu
 - Dữ liệu nhân tạo: do con người qui ước trong các bộ mã chuẩn riêng biệt
 - Dữ liệu tự nhiên: tồn tại khách quan với con người, thường ở dạng Analog → cần chuyển đổi ra dạng Digital
 - Độ dài từ dữ liệu là số bit được sử dụng để mã hóa loại dữ liệu tương ứng
 - Thường là bội của 8-bit
 - VD: 8, 16, 32, 64 bit
 - Các phần mềm điều khiển nhập/ xuất TBNX sẽ đảm nhiệm việc mã hóa

- Qui tắc chuyển đổi tín hiệu vật lý dạng analog sang dạng digital (ví dụ: âm thanh, hình ảnh, video,...)



- Mã hóa ký tự trong máy tính
 - Ban đầu mỗi công ty sản xuất đưa ra 1 bộ mã theo qui ước riêng → không trao đổi được thông tin giữa các loại máy tính
 - VD:

<u>Cty 1</u>	<u>Cty 2</u>
A 1	A 10
B 2	B 11
C 3 ...	C 12 ...
 - Bộ mã EBCDIC của IBM được sử dụng rộng rãi nhất
 - Cần có bộ mã chuẩn thống nhất cho mọi máy tính và mọi quốc gia: Mã ASCII và Unicode

- Mã hóa ký tự trong máy tính (tiếp)
 - Mã EBCDIC (Extended BCD Interchange Code) do công ty IBM ban hành 1963 để sử dụng cho hệ thống IBM/360 và sau đó được áp dụng cho nhiều hệ thống khác
 - Được mở rộng từ mã BCD
 - Sử dụng 8 bit có thể mã hóa tối đa 256 ký tự (thực tế EBCDIC không sử dụng hết)
 - 0-63: Các ký tự điều khiển, không in được
 - 64-127: Dấu
 - 129-169: Chữ thường
 - 193-233: Chữ hoa
 - 240-249: Số



Bảng mã EBCDIC

Digit

Zone	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	SOH	STX	ETX	PF	HT	LC	DEL		RLF	SMM	VT	FF	CR	SR	SI
0001	DLE	DC1	DC2	TM	RES	NL	BS	IL	CAN	EM	CC	CU1	IFS	IGS	IRS	IUS
0010	DS	SOS	FS		BYP	LF	ETB	ESC			SM	CU2		ENO	ACK	BEL
0011			SYN		PN	RS	UC	EOI				CU3	DC4	NAK		SUB
0100	SP										[	.	<	(	+	!
0101	&										I	\$	*	)	:	"
0110	-	/										_	%	-	>	?
0111										'	@	#	@	'	=	"
1000		a	b	c	d	e	f	g	h	i						
1001		j	k	l	m	n	o	p	q	r						
1010		-	s	t	u	v	w	x	y	z						
1011																
1100	{	A	B	C	D	E	F	G	H	I						
1101	}	J	K	L	M	N	O	P	Q	R						
1110	\		S	T	U	V	W	X	Y	Z						
1111	0	1	2	3	4	5	6	7	8	9						

- Mã hóa ký tự trong máy tính (tiếp)
 - Mã ASCII (American Standard Code for Information Interchange) do ANSI ban hành từ 1963
 - Sau này được CCITT (ITU) và ISO công nhận và được sử dụng rộng rãi trên thế giới
 - Sử dụng 7 bit để mã hóa được tối đa 128 ký tự. Mỗi ký tự lưu trong 1 byte dữ liệu. Bit thứ 8 sau này được sử dụng làm bit kiểm tra (parity bit) hoặc để mở rộng bộ mã (mã ASCII mở rộng 8 bit).
 - 0-31: Các ký tự điều khiển, không in được
 - 48-57: Số
 - 65-90: Chữ hoa
 - 97-122: Chữ thường
 - 32-48, 58-64, 91-96, 123-127: Dấu (xen kẽ giữa các vùng)



Bảng mã ASCII

	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
a	LF	SUB	*	:	J	Z	j	z
b	VT	ESC	+	;	K	[	k	{
c	FF	FS	,	<	L	\	l	
d	CR	GS	-	=	M	]	m	}
e	SO	RS	.	>	N	^	n	~
f	SI	US	/	?	O	_	o	DEL

Ghi chú:

Bảng trình bày theo
số thập lục phân

Theo số thập phân

- 0-31: Ký tự điều khiển
- 48-57: Số
- 65-90: Chữ hoa
- 97-122: Chữ thường



Mã tiếng Việt có dấu

- Ban đầu 1 số công ty đưa ra các bộ mã khác nhau nhưng đều mở rộng từ bộ mã ASCII chuẩn 7 bit lên 8 bit: VNI, ABC, ĐHBK, Vietware,...(khoảng 43 bộ mã)
- Do 128 vị trí mở rộng không đủ chứa các ký tự tiếng Việt có dấu nên mỗi bộ mã áp dụng các cách khắc phục khác nhau nhưng vẫn còn nhiều nhược điểm:
 - Dùng 2 bộ mã dựng sẵn khác nhau cho chữ thường và chữ hoa
 - Dùng 1 bộ mã, một số ký tự còn thiếu sẽ chèn vào vùng ASCII chuẩn
 - Dùng 1 byte ký tự không dấu và 1 byte dấu riêng biệt (mã tổ hợp)
- Năm 1993 VN ban hành bộ mã 8 bit TCVN 5712 và 1999 chỉnh sửa thêm nhưng vẫn còn nhiều tồn tại nên ít được sử dụng.
- Năm 2001 VN ban hành bộ mã 16 bit TCVN 6909 phù hợp với chuẩn Unicode và ISO/IEC 10646 khắc phục hầu hết các nhược điểm nên được sử dụng rộng rãi



Mã Unicode

— Đặc điểm

- Ban hành năm 1991, hiện nay đã đến phiên bản 6.2 (09/2012).
- Unicode cung cấp một mã số duy nhất cho mỗi ký tự, cho mọi hệ máy tính, cho mọi chương trình, cho mọi ngôn ngữ. Hiện nay có thể mã hóa trên 1 triệu ký tự.
- Chuẩn Unicode đã được những công ty công nghệ hàng đầu, như Apple, HP, IBM, Microsoft, ... chấp nhận
- Unicode tương thích với ISO/IEC 10646 và mã ASCII
- Hỗ trợ 3 kiểu định dạng UTF-8, UTF-16 và UTF-32
- Hiện được sử dụng rộng rãi trên toàn cầu, kể cả ở VN

Thứ tự lưu trữ các byte trong bộ nhớ

- Bộ nhớ chính thường tổ chức theo byte
 - Hai cách lưu trữ dữ liệu nhiều byte:
 - Đầu nhỏ (*Little-endian*): Byte có ý nghĩa thấp được lưu trữ ở ngăn nhớ có địa chỉ nhỏ, byte có ý nghĩa cao được lưu trữ ở ngăn nhớ có địa chỉ lớn.
 - Đầu to (*Big-endian*): Byte có ý nghĩa cao được lưu trữ ở ngăn nhớ có địa chỉ nhỏ, byte có ý nghĩa thấp được lưu trữ ở ngăn nhớ có địa chỉ lớn.
 - Áp dụng: Mã Unicode, số, chuỗi ký tự
-



- Ví dụ lưu trữ dữ liệu 32-bit
 - Intel 80x86 và các Pentium: little-endian
 - Motorola 680x0, SunSPARC: big-endian

0001 1010 0010 1011 0011 1100 0100 1101

1A	2B	3C	4D
----	----	----	----

4D	300
3C	301
2B	302
1A	303

little-endian

1A	300
2B	301
3C	302
4D	303

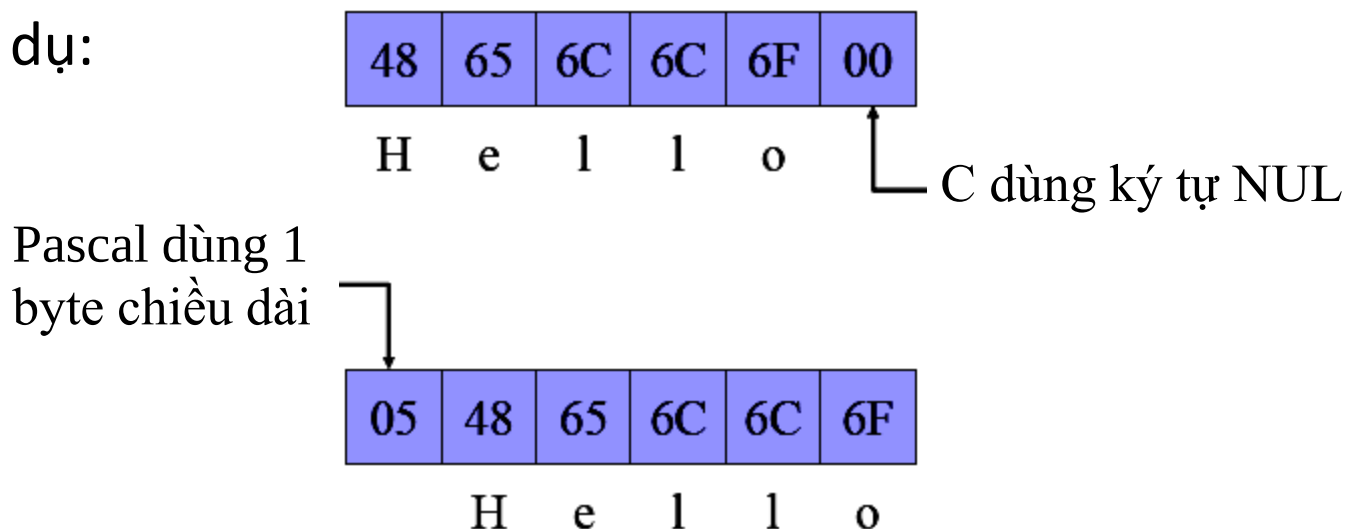
big-endian



Lưu trữ chuỗi ký tự

- Chuỗi ký tự gồm nhiều ký tự ghép lại, mỗi ký tự chiếm 1 byte bộ nhớ nếu là mã ASCII (2 byte nếu là Unicode)
- Cần xác định chiều dài chuỗi (số ký tự có trong chuỗi)
- Mỗi ngôn ngữ lập trình cấp cao qui định cách xác định khác nhau cho chuỗi ký tự khi lưu trữ.

– Ví dụ:





Biểu diễn các dạng thông tin khác

- Các chuẩn định dạng thông tin thông dụng:
 - Hình ảnh: BMP, TIFF, PNG, GIF, JPEG,...
 - Âm thanh: WAV, MIDI, MP3, AVI,...
 - Văn bản: PDF, HTML, XML,...
 - Video: MPEG-4, WMV, DivX,...
 - Animation: Flash, SVG, CSS, ...
 - Khác: Mã vạch, RFID, OCR



Biểu diễn chương trình

- Tập lệnh CPU cũng phải được mã hóa bằng số nhị phân → Chương trình ngôn ngữ máy ở dạng số nhị phân
- Hiện nay mỗi công ty sản xuất máy tính qui định bộ mã lệnh riêng cho CPU của mình sản xuất → Chương trình viết cho máy này không thể chạy trên máy khác vì khác mã lệnh

Câu hỏi: tại sao không đưa ra chuẩn thống nhất mã lệnh cho mọi loại CPU?



Câu hỏi





Chương 3 Mạch logic số

3.1 Transistor và các cổng logic

3.2 Đại số Boole

3.3 Mạch tổ hợp

3.4 Mạch tính toán

3.5 Mạch tuần tự

3.6 Mạch bộ nhớ



Transistor và các cổng logic

➤ Transistor

- Phần tử cơ bản nhất cấu tạo máy tính số ngày nay là transistor do John Bardeen và Walter Brattain phát minh năm 1947.
- Transistor thường được sử dụng như một thiết bị khuếch đại hoặc một khóa điện tử

➤ Mỗi transistor đều có ba cực:

- Cực gốc (*base*)
- Cực góp (*collector*)
- Cực phát (*emitter*)

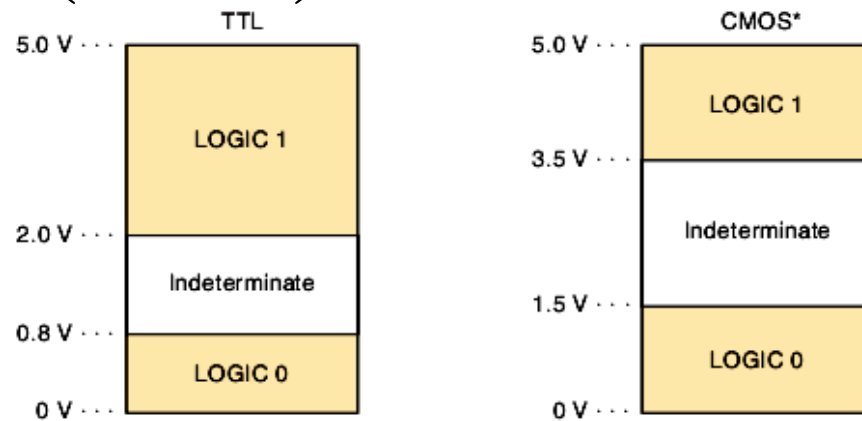


➤ Cổng logic (gate)

- Các transistor được ghép nối lại để tạo thành các cổng logic có thể thực hiện các phép toán logic cơ bản: NOT, AND, OR, NAND (NOT AND) và NOR (NOT OR)

- Giá trị logic

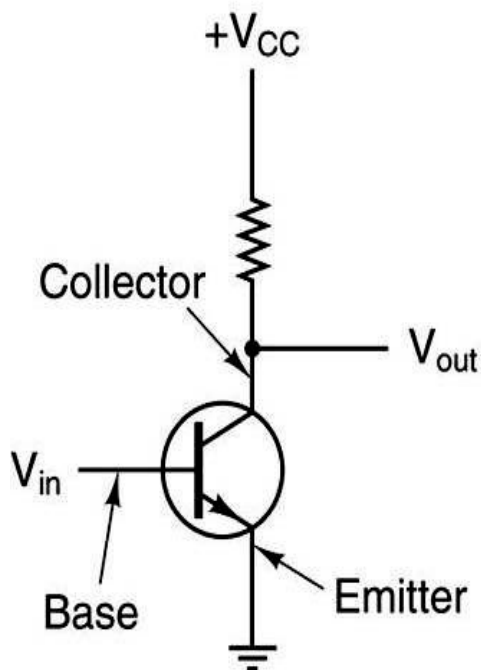
- 0 : mức điện áp 0...1,5 volt
- 1 : mức điện áp 2...5 volt



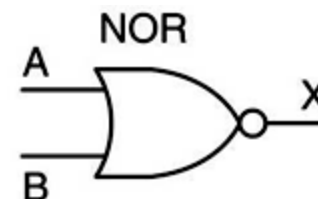
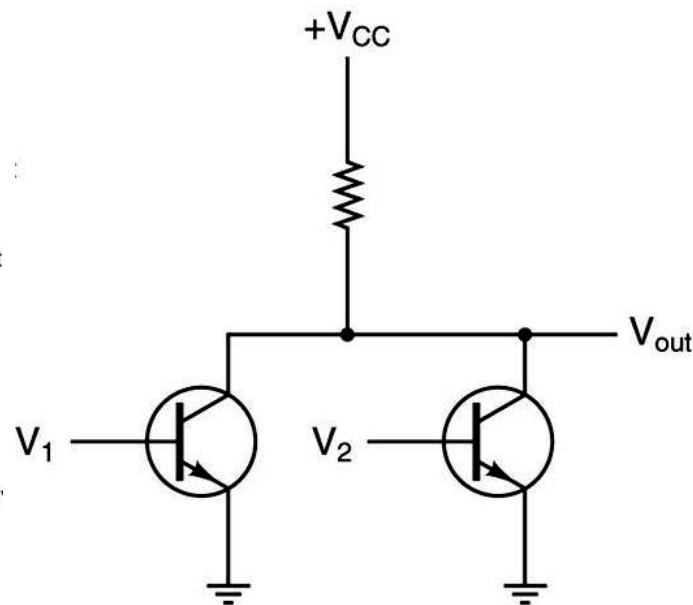
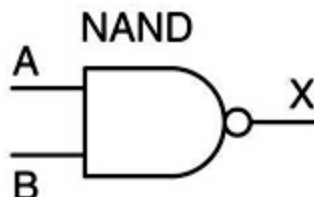
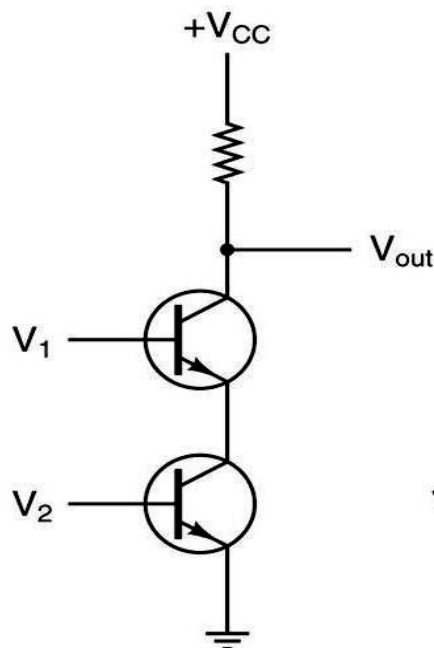
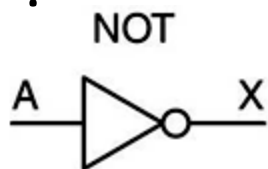
- Các cổng cơ bản này lại được lắp ghép thành các phần tử chức năng lớn hơn như mạch cộng 1 bit, nhớ 1 bit, v.v... từ đó tạo thành 1 máy tính hoàn chỉnh



Cấu tạo các cổng NOT, NAND và NOR

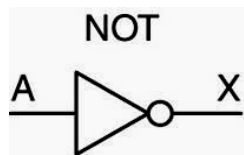


Ký hiệu

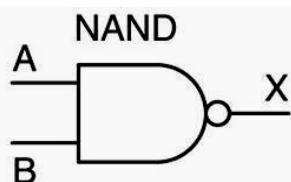




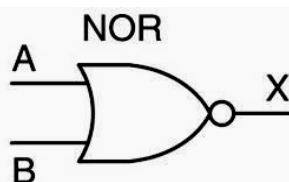
Bảng chân trị và ký hiệu các cổng



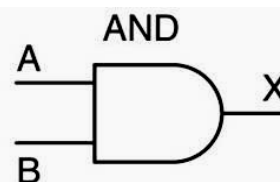
A	X
0	1
1	0



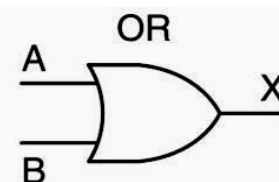
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0



A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

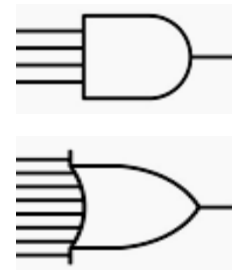


A	B	X
0	0	0
0	1	0
1	0	0
1	1	1



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

- Đối với các cổng nhiều ngõ vào, ngõ ra $X=1$ khi:
 - AND : mọi ngõ vào bằng 1
 - OR: ít nhất 1 ngõ vào bằng 1
 - NAND : ít nhất 1 ngõ vào bằng 0
 - NOR : mọi ngõ vào bằng 0



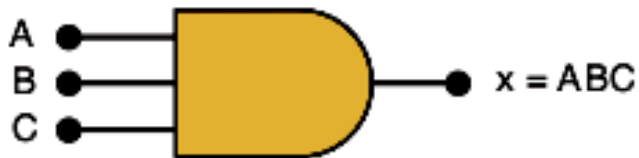


Bảng chân trị OR và AND 3 ngõ vào



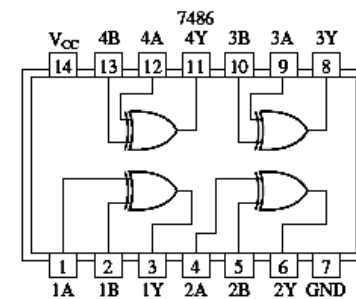
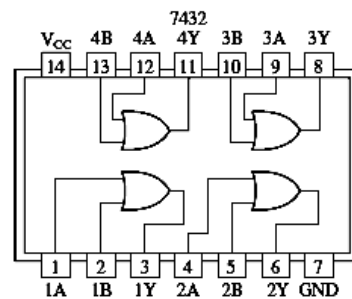
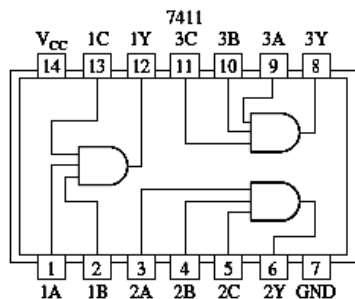
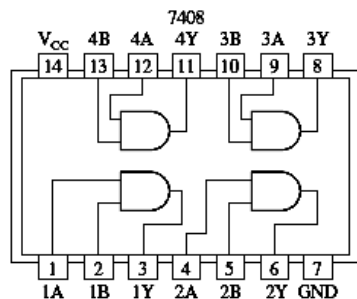
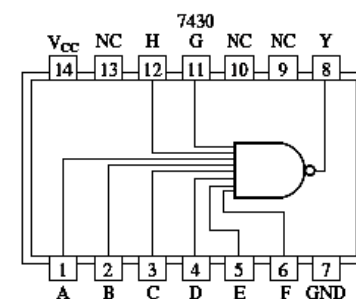
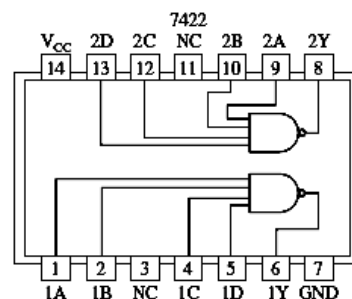
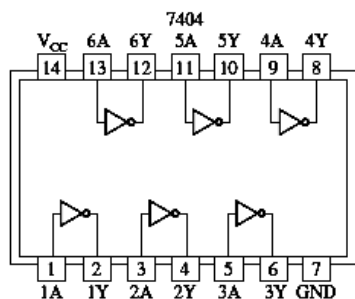
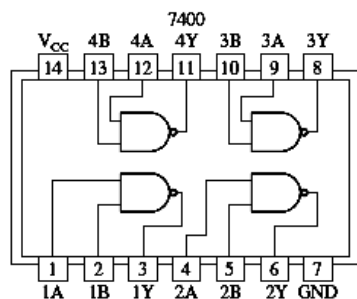
A	B	C	$x = A + B + C$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

A	B	C	$x = ABC$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1





Một số vi mạch 7400





Đại số Boole

➤ Giới thiệu

- Đại số Boole (Boolean algebra) do nhà toán học George Boole phát triển từ năm 1854 làm cơ sở cho phép toán logic.
- Năm 1938 Claude Shannon chứng minh có thể dùng đại số Boole để thiết kế mạch số trong máy tính
- Đại số Boole dựa trên các biến logic và các phép toán logic
 - Biến logic có thể nhận giá trị 1 (TRUE) hoặc 0 (FALSE)
 - Phép toán logic cơ bản là AND, OR và NOT
 - Hàm logic gồm tập các phép toán và biến logic

➤ Các phép toán logic cơ bản

- Phép toán logic cơ bản AND, OR và NOT với ký hiệu như sau:

- A AND B : $A \cdot B$
- A OR B : $A + B$
- NOT A : $\bar{A}$

- Các phép toán khác: NAND, NOR, XOR:

- $\bar{A} \text{ NAND } B : A \cdot B$
- $\bar{A} \text{ NOR } B : A + B$
- $\bar{A} \text{ XOR } B : A \oplus B = A \cdot B + \bar{A} \cdot \bar{B}$

EXCLUSIVE OR gate



- Thứ tự ưu tiên: NOT, AND và NAND, OR và NOR

- Bảng chân trị (Truth table)

P	Q	NOT P ($\bar{P}$)	P AND Q ($P \cdot Q$)	P OR Q ($P + Q$)	P NAND Q ($\overline{P \cdot Q}$)	P NOR Q ($\overline{P + Q}$)	P XOR Q ($P \oplus Q$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

- Ứng dụng đại số Boole
 - Phân tích chức năng mạch logic số
 - Thiết kế mạch logic số dựa trên hàm cho trước

- Ví dụ 1: Cài đặt 1 hàm logic $M=F(A, B, C)$ theo bảng chân trị cho trước
- Qui tắc: $M=0$ nếu mọi đầu vào là 0, $M=1$ nếu mọi đầu vào là 1 (tổng các tích).
- Bước 1: Xác định các dòng trong bảng chân trị có kết quả bằng 1
- Bước 2: Các biến đầu vào được AND với nhau nếu giá trị trong bảng bằng 1. Nếu giá trị biến bằng 0 cần NOT nó trước khi AND
- Bước 3: OR tất cả các kết quả từ bước 2.

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$M=\overline{A}BC+\overline{A}\overline{B}C+A\overline{B}\overline{C}+ABC$$

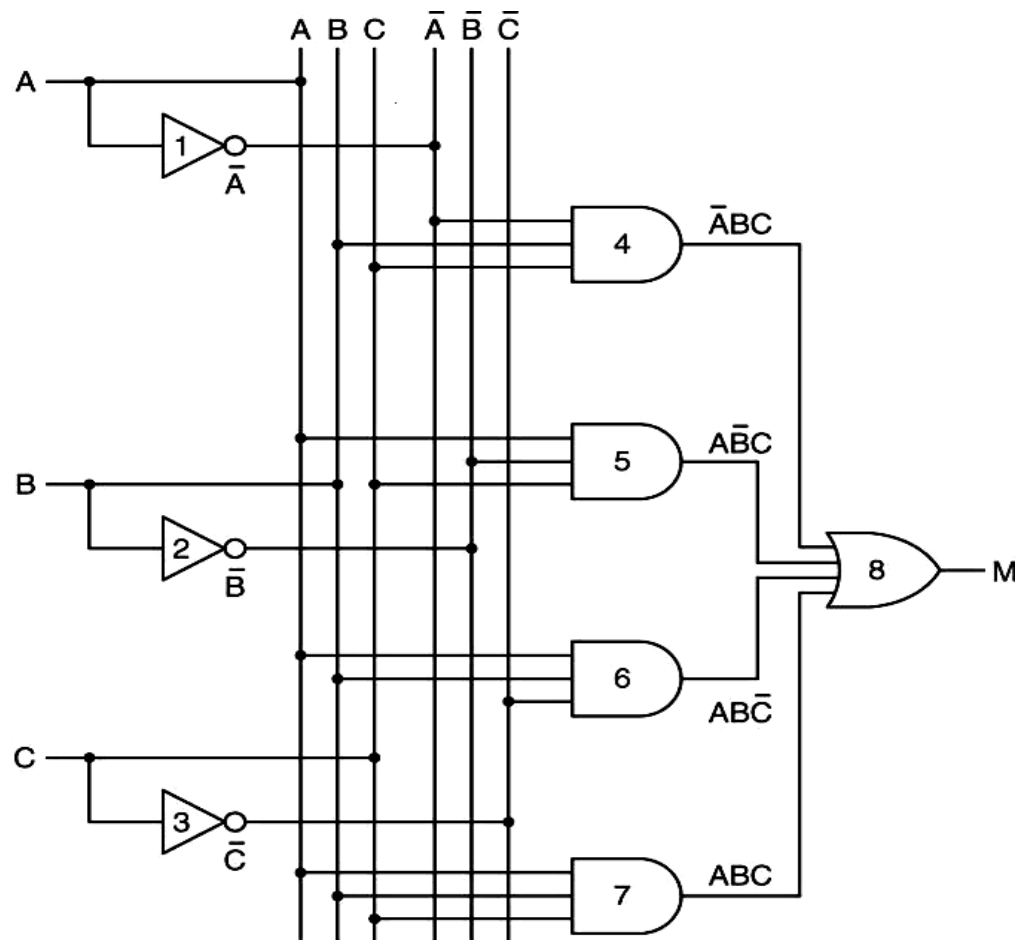
• Ví dụ 1 (tiếp)

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

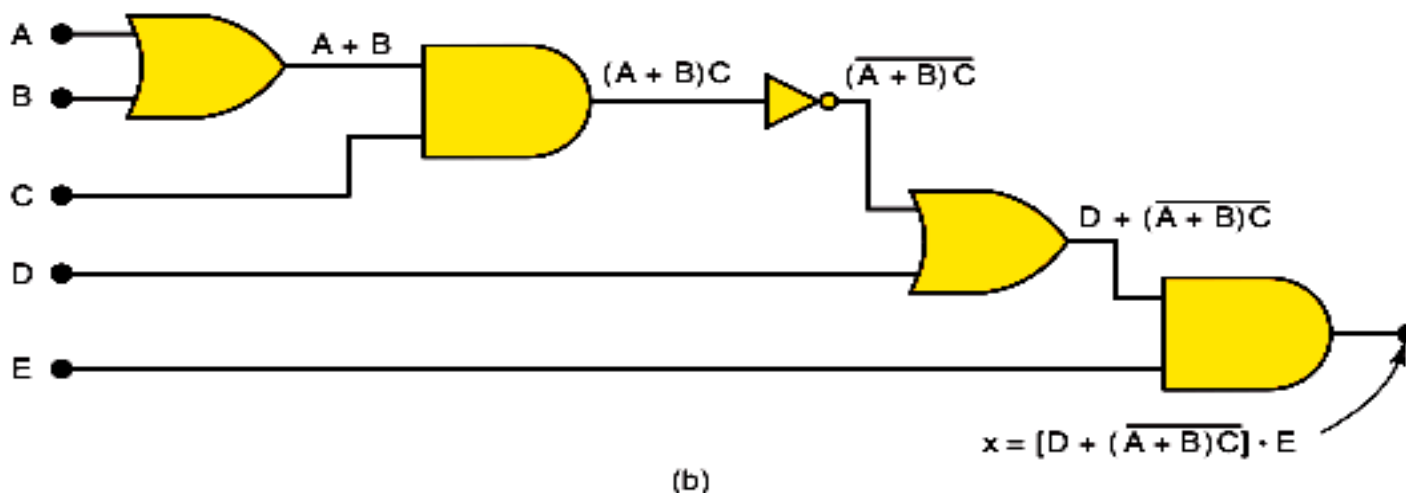
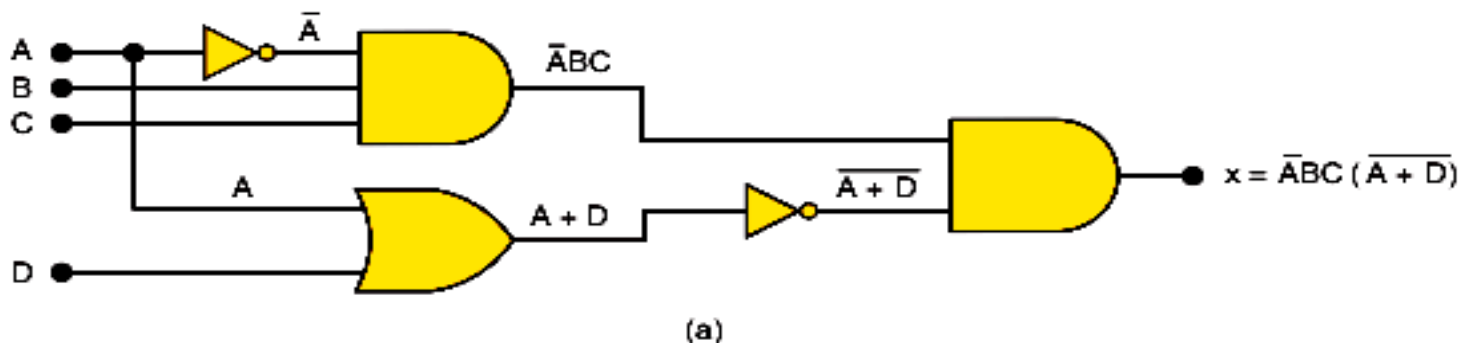
$$M = \bar{A}BC + A\bar{B}C + ABC + A\bar{B}\bar{C}$$

Chú ý:

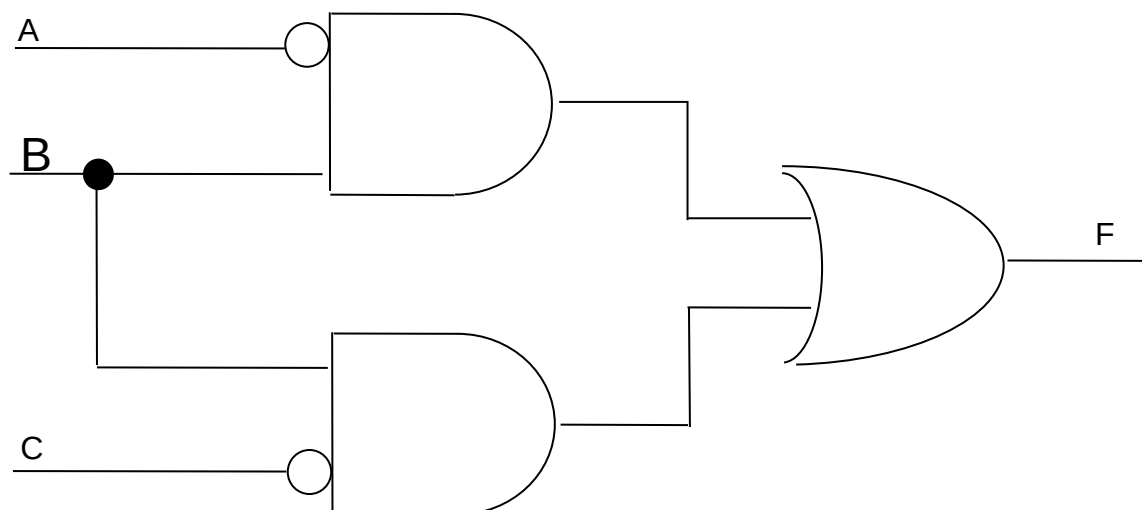
- Mạch thiết kế theo cách này chưa tối ưu.
- Có 3 cách biểu diễn 1 hàm logic



- Ví dụ 2: Xác định hàm logic từ mạch cho trước

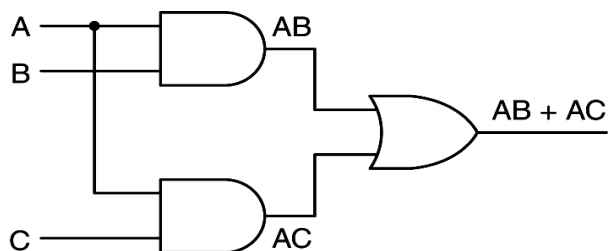


- Lập bảng chân trị cho mạch sau với F là đầu ra và a, b, c là đầu vào:



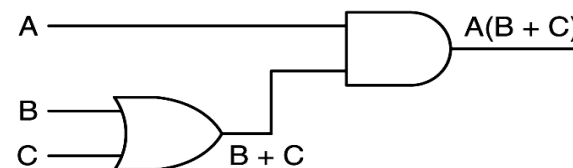
- Các mạch tương đương

– Ví dụ: $AB+AC$ và $A(B+C)$



A	B	C	AB	AC	AB + AC
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

(a)

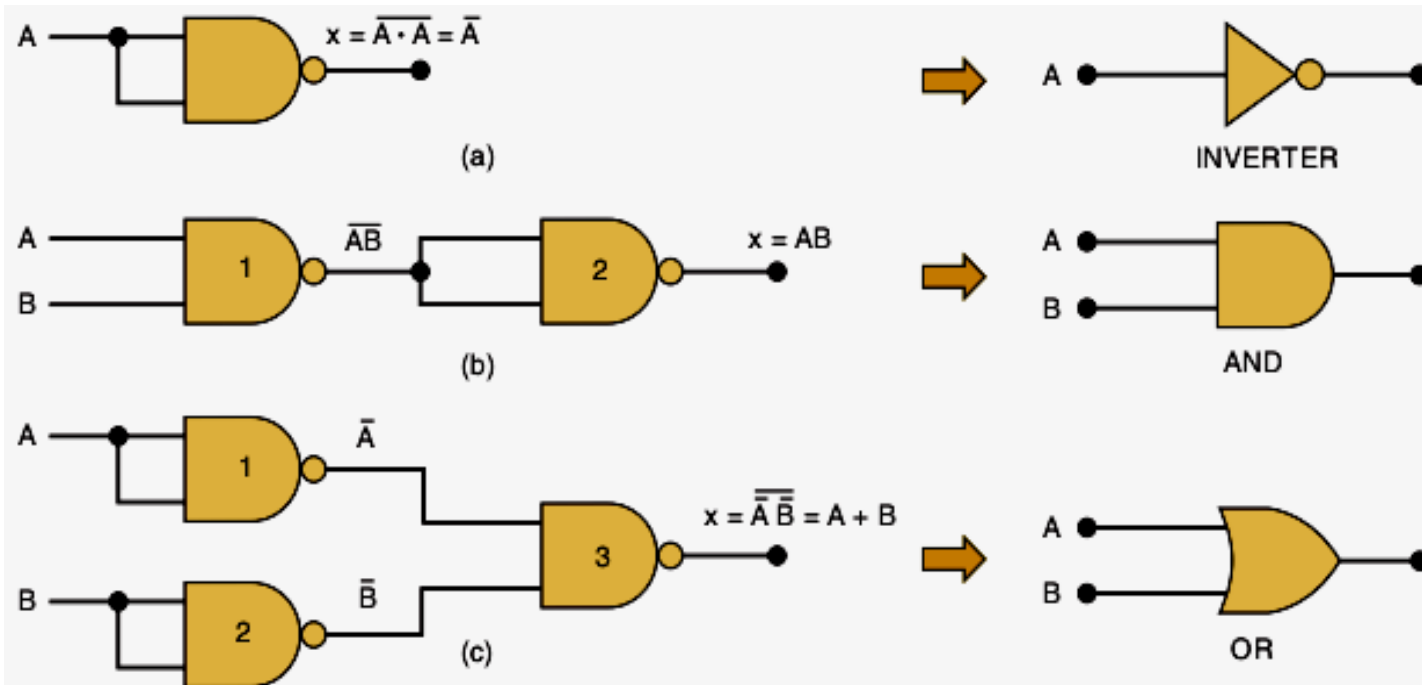


A	B	C	A	B + C	A(B + C)
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	1	0
1	0	0	1	0	0
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

(b)

• Các mạch tương đương (tiếp)

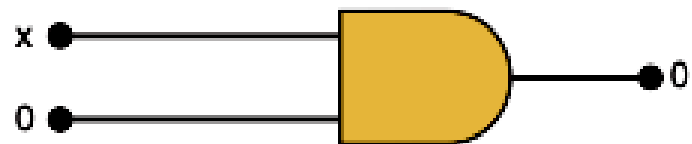
- Nhận xét: Nên sử dụng mạch tiết kiệm các cổng logic nhất
- Trong thực tế người ta dùng cổng NAND (hoặc NOR) để tạo ra mọi cổng khác



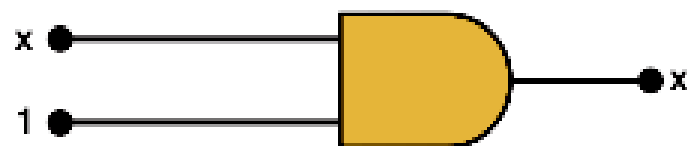


Các định luật của đại số Boole

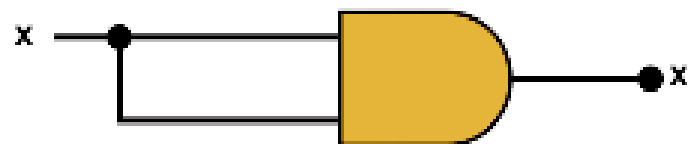
Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{\bar{A} + \bar{B}} = \bar{A}\bar{B}$



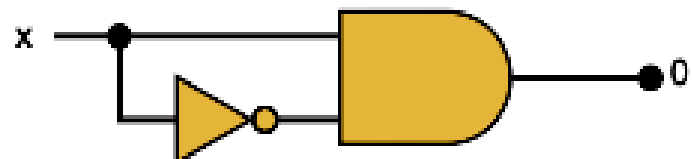
(1) $x \cdot 0 = 0$



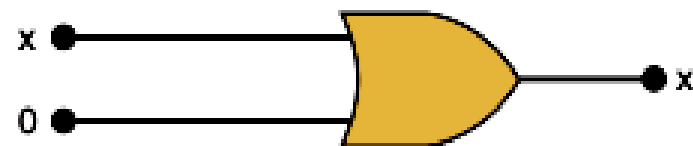
(2) $x \cdot 1 = x$



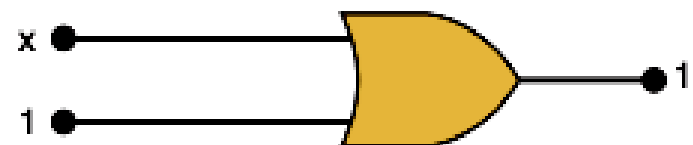
(3) $x \cdot x = x$



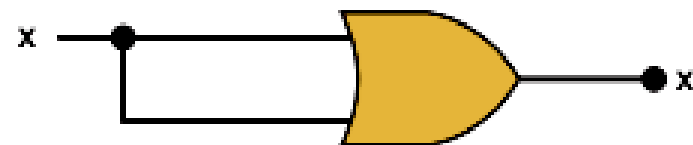
(4) $x \cdot \bar{x} = 0$



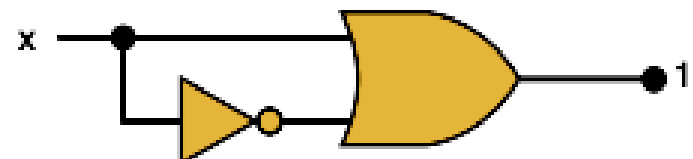
(5) $x + 0 = x$



(6) $x + 1 = 1$



(7) $x + x = x$



(8) $x + \bar{x} = 1$

➤ Ứng dụng các định luật

- Đơn giản biểu thức logic → Tiết kiệm cổng logic
- Ví dụ : Chứng minh $AB + \overline{A}C + BC = AB + \overline{A}C$

$$\begin{aligned} & AB + \overline{A}C + BC \\ &= AB + \overline{A}C + 1 \cdot BC \\ &= AB + \overline{A}C + (A + \overline{A}) \cdot BC \\ &= AB + \overline{A}C + ABC + \overline{A}BC \\ &= AB + ABC + \overline{A}C + \overline{A}BC \\ &= AB \cdot 1 + ABC + \overline{A}C \cdot 1 + \overline{A}C \cdot B \\ &= AB(1 + C) + \overline{A}C(1 + B) \\ &= AB \cdot 1 + \overline{A}C \cdot 1 = AB + \overline{A}C \end{aligned}$$

– Bài tập : Chứng minh $(\overline{X + Y})Z + X\overline{Y} = \overline{Y}(X + Z)$



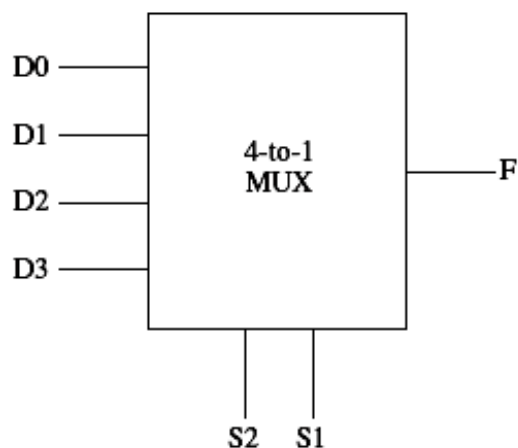
Mạch tổ hợp

➤ Khái niệm

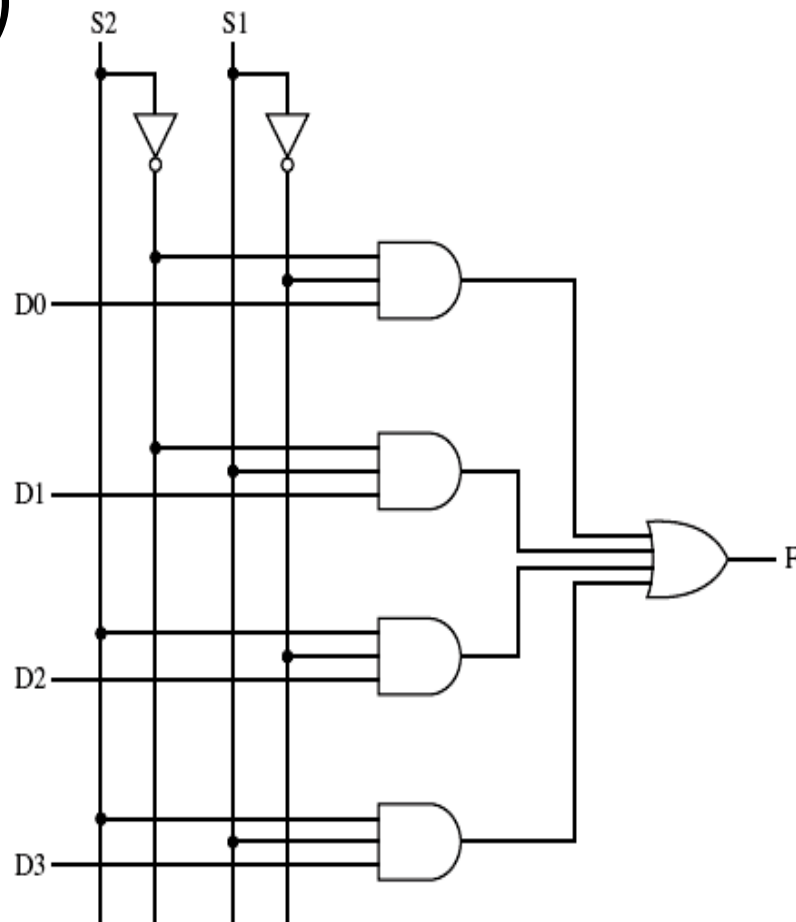
- Mạch tổ hợp (combinational circuit) là mạch logic trong đó tín hiệu ra chỉ phụ thuộc tín hiệu vào ở thời điểm hiện tại.
- Là mạch không nhớ (memoryless) và được thực hiện bằng các cổng logic cơ bản
- Mạch tổ hợp được cài đặt từ 1 hàm hoặc bảng chân trị cho trước
- Được ứng dụng nhiều trong thiết kế mạch máy tính

➤ Bộ dồn kênh (Multiplexer)

- 2^n đầu vào dữ liệu D
- n đầu vào lựa chọn S
- 1 đầu ra F
- (S) xác định đầu vào (D) nào sẽ được nối với đầu ra (F)

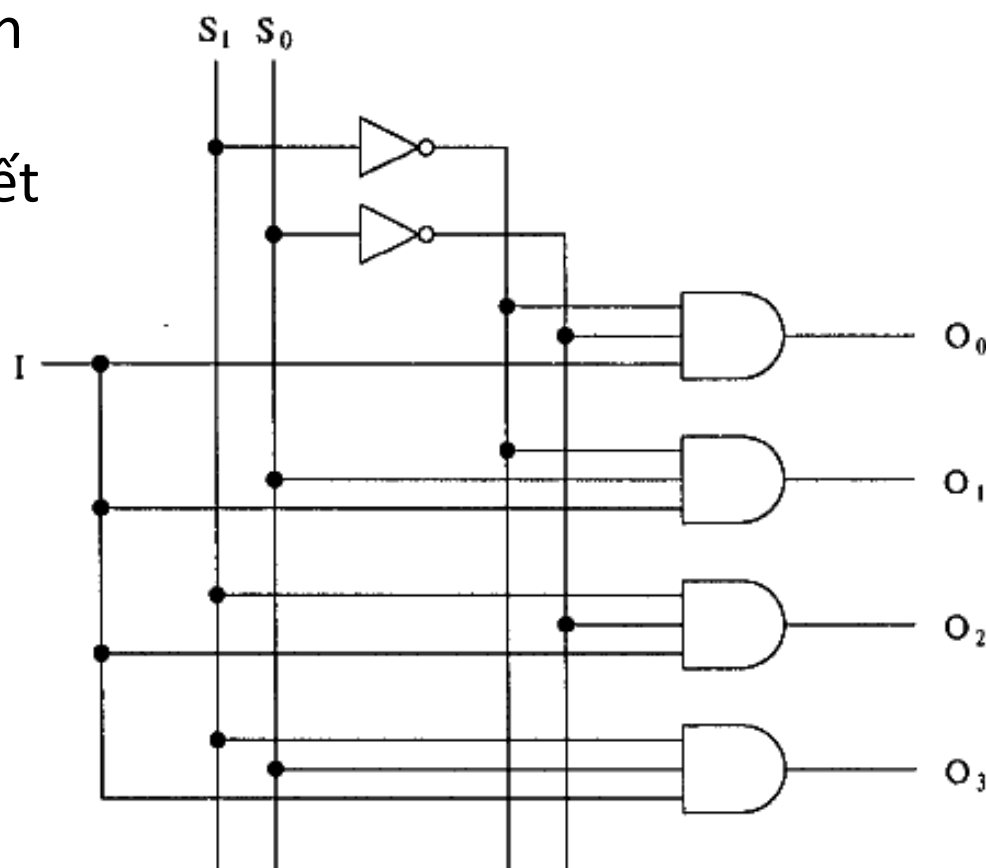
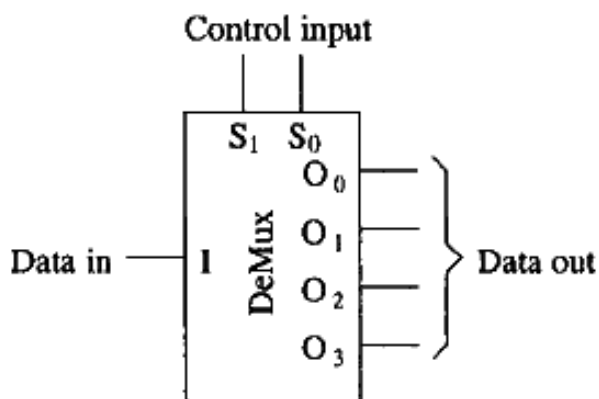


S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



➤ Bộ phân kênh (Demultiplexer)

- Ngược với bộ dồn kênh
- Tín hiệu điều khiển (S)
- sẽ chọn đầu ra nào kết
- nối với đầu vào (I)
- Ví dụ: Demux 1-to-4

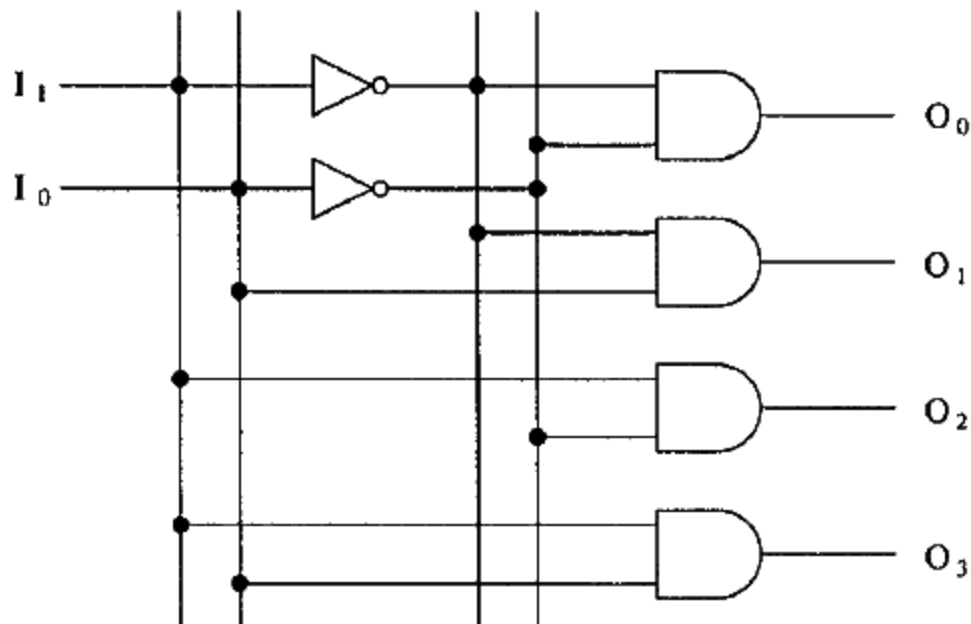
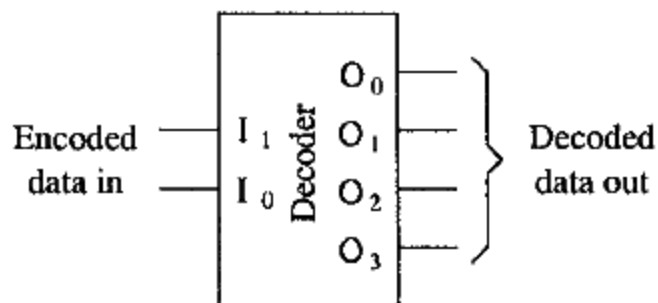


➤ Bộ giải mã (Decoder)

- Bộ giải mã chọn một trong 2^n đầu ra (O) tương ứng với một tổ hợp của n đầu vào (I)

- Ví dụ : Mạch giải mã 2 ra 4

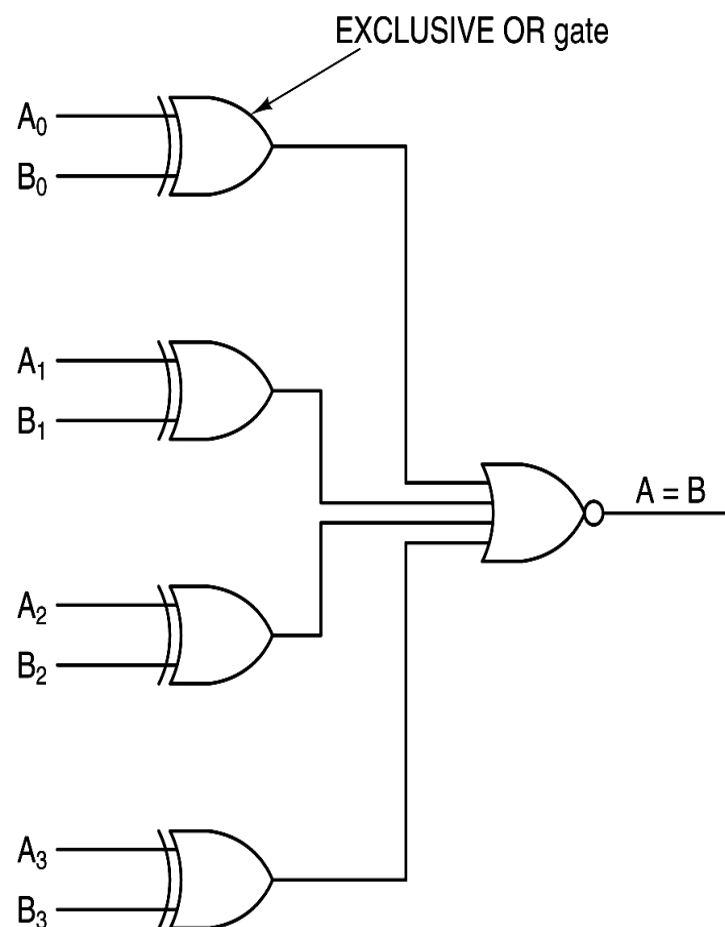
I_1	I_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



➤ Mạch so sánh (Comparator)

- So sánh các bit của 2 ngõ vào và xuất kết quả 1 nếu bằng nhau.
- Ví dụ : Mạch so sánh 4 bit dùng các cổng XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

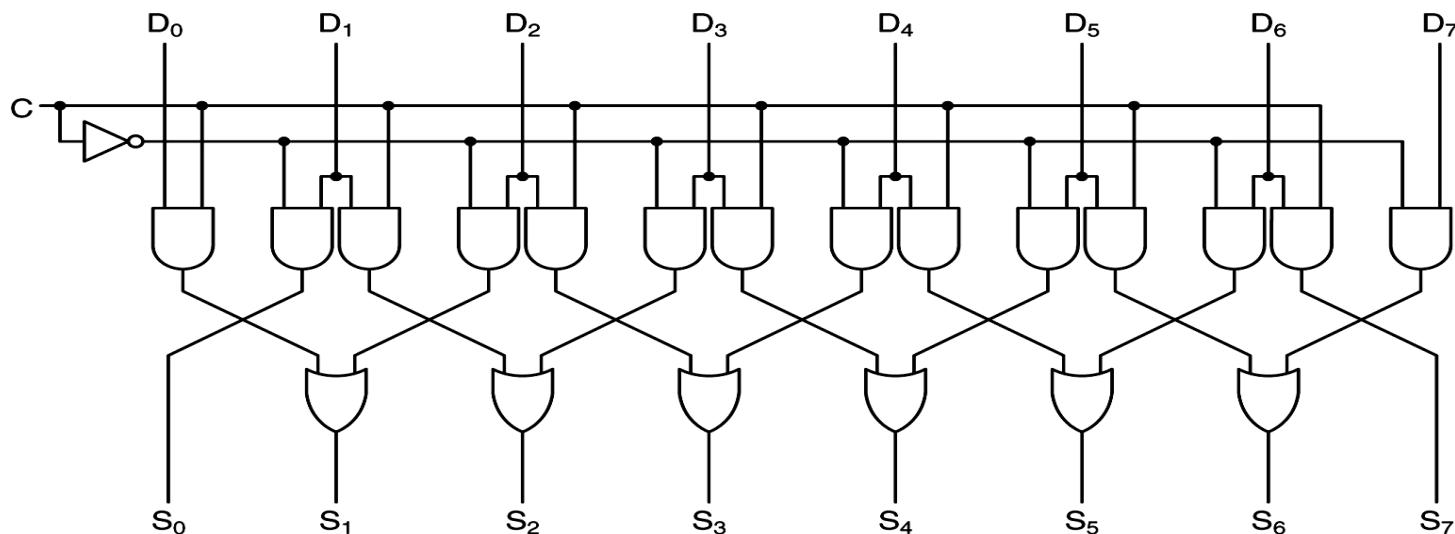




Mạch tính toán

➤ Mạch dịch (Shifter)

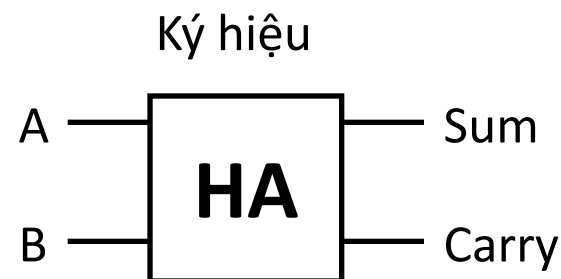
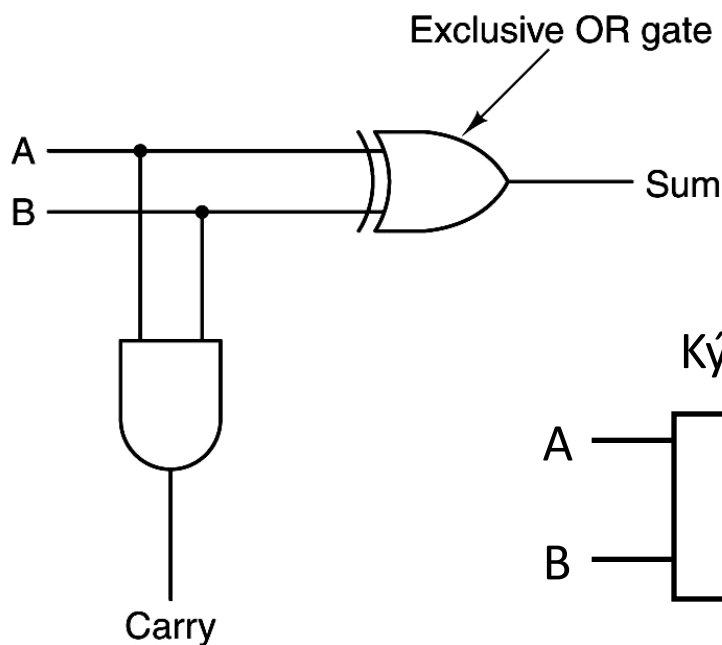
- Dịch các tín hiệu sang trái hoặc phải 1 vị trí. Ứng dụng cho phép nhân/chia cho 2.
- Ví dụ : mạch dịch 8 bit với tín hiệu điều khiển chiều dịch trái ($C=0$) hay phải ($C=1$)



➤ Mạch cộng bán phần (Half adder)

- Cộng 2 bit đầu vào thành 1 bit đầu ra và 1 bit nhớ

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

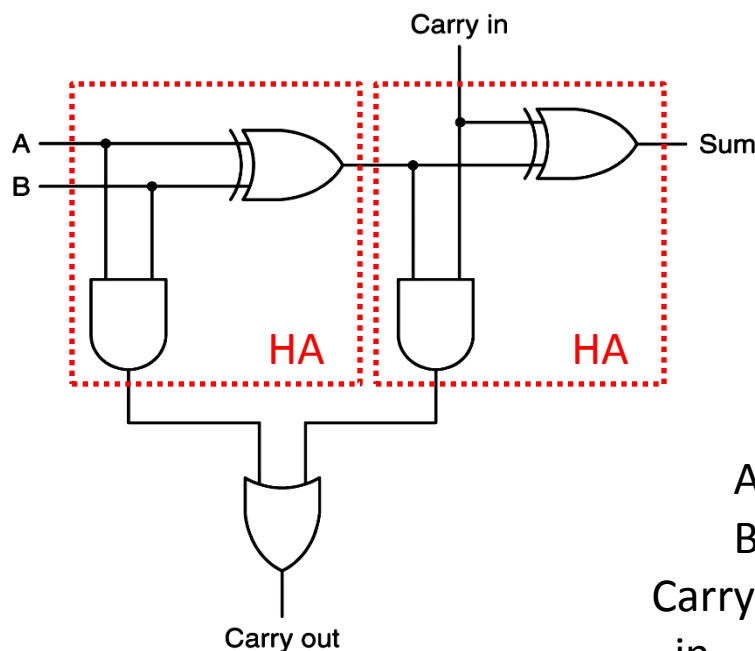


➤ Mạch cộng toàn phần (Full adder)

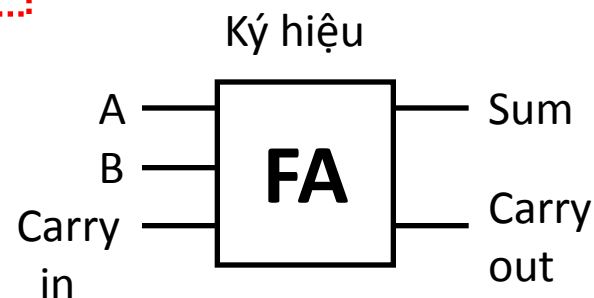
- Cộng 3 bit đầu vào thành 1 bit đầu ra và 1 bit nhớ
- Cho phép xây dựng bộ cộng nhiều bit

A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a)

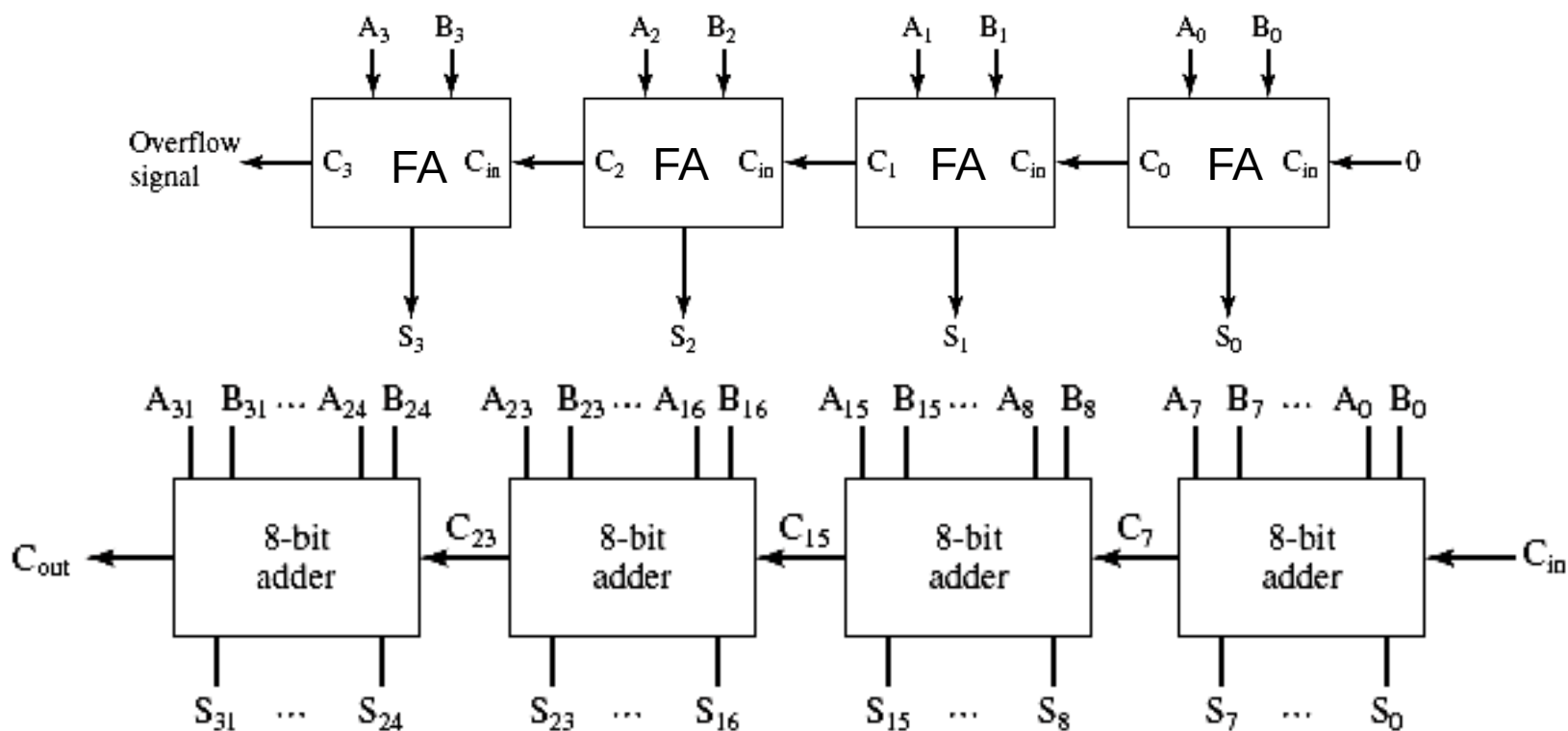


(b)



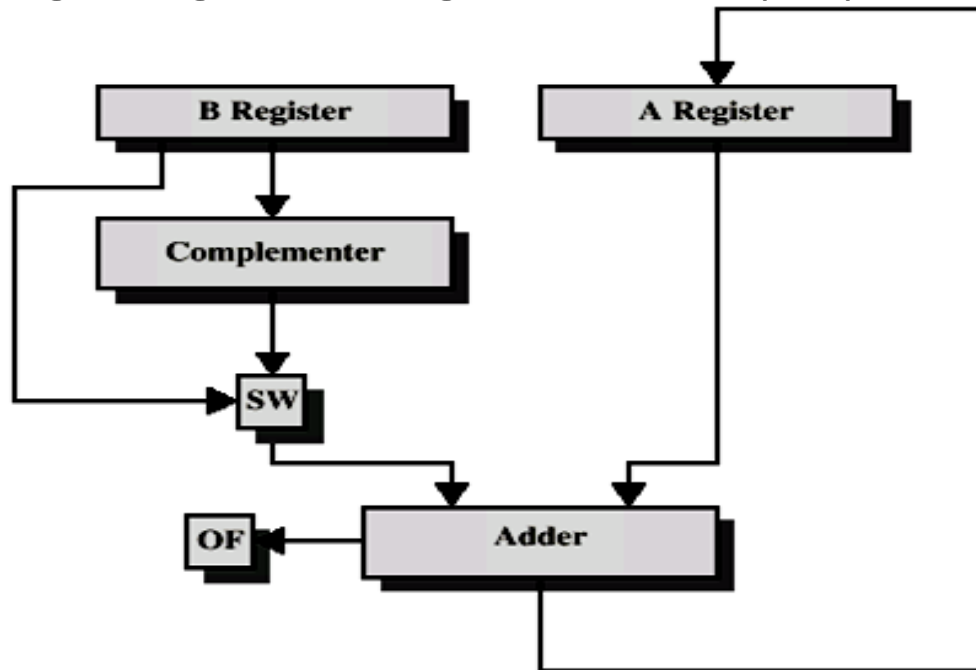
➤ Mạch cộng nhiều bit

- Ghép từ nhiều bộ cộng toàn phần



➤ Mạch cộng và trừ

- Mạch trừ: Đổi sang số bù 2 rồi cộng
- Có thể dùng chung mạch cộng để thực hiện phép trừ



OF = overflow bit
SW = Switch (select addition or subtraction)



- Ví dụ ALU 1 bit

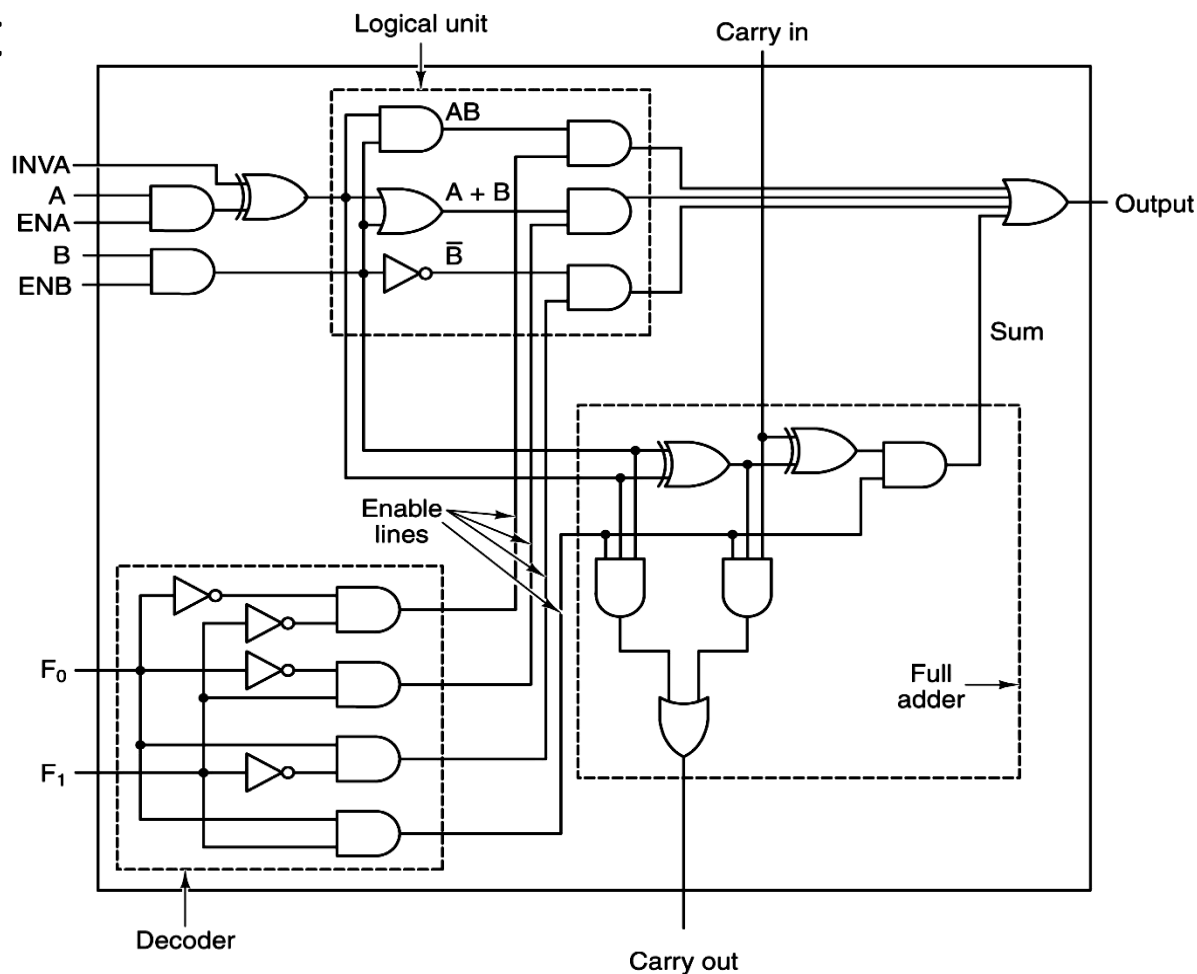
F_0F_1	Functions
00	A AND B
01	A OR B
10	
11	A + B

Điều kiện
bình thường

ENA=1

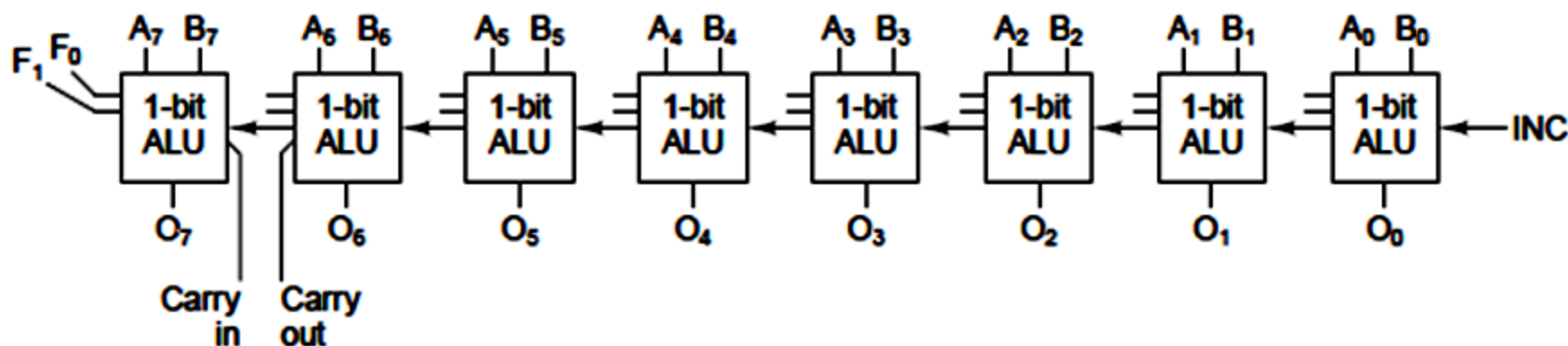
ENB=1

INVA=0



➤ ALU 8 bit

- Ví dụ tạo 1 mạch ALU 8 bit bằng cách ghép 8 bộ ALU 1 bit ở ví dụ trước





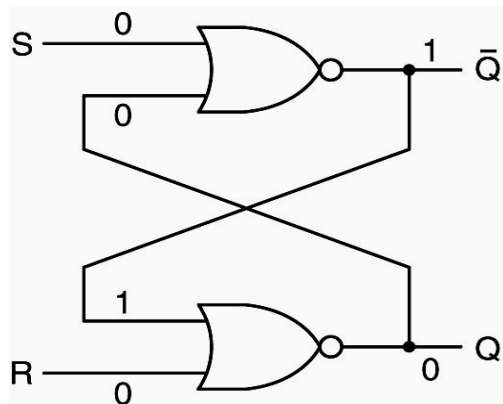
Mạch tuần tự

➤ Khái niệm

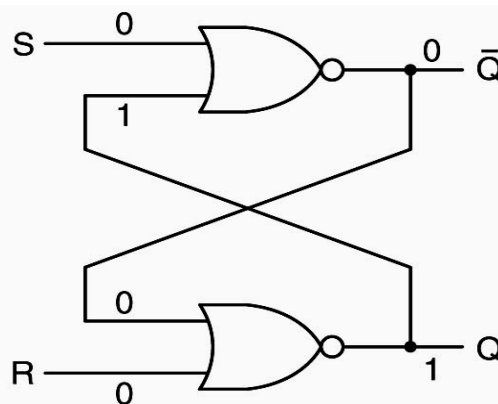
- Mạch tuần tự (sequential circuit) là mạch logic trong đó tín hiệu ra phụ thuộc tín hiệu vào ở hiện tại và quá khứ
- Là mạch có nhớ, được thực hiện bằng phần tử nhớ (Latch, Flip-Flop) và có thể kết hợp với các cổng logic cơ bản
- Ứng dụng làm bộ nhớ, thanh ghi, mạch đếm,... trong máy tính

➤ Mạch chốt (Latch)

- Dùng 2 cổng NOR mắc hồi tiếp với nhau. S, R là ngõ vào, Q và $\bar{Q}$ là ngõ ra.
- Đây là mạch chốt SR. Nó có thể ở 1 trong 2 trạng thái $Q=1$ hoặc $Q=0$ khi $S=R=0$.
- Khi $S=1 \rightarrow Q=1$ bất kể trạng thái trước đó (set)
- Khi $R=1 \rightarrow Q=0$ bất kể trạng thái trước đó (reset)



(a)



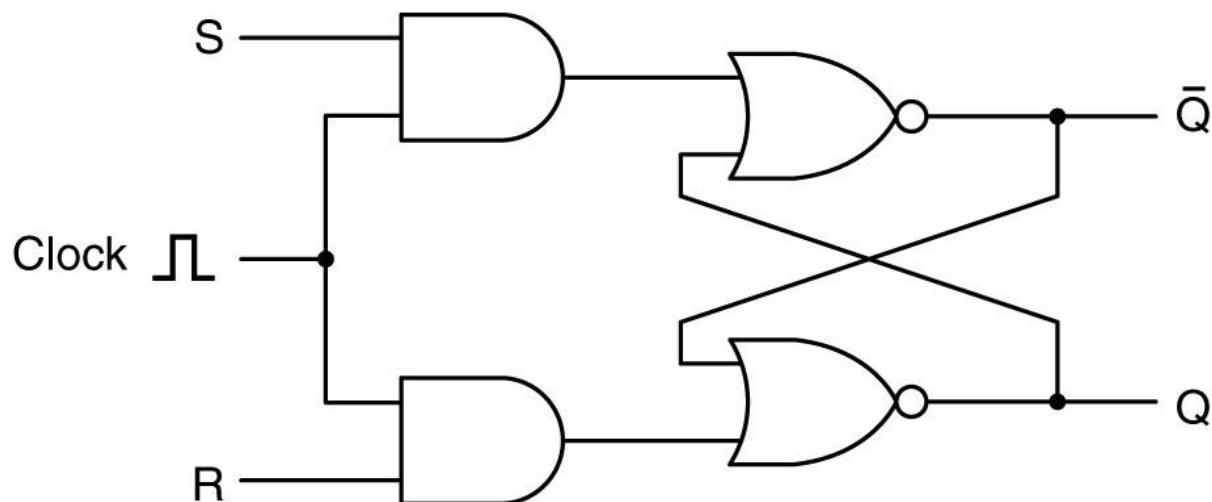
(b)

A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0

(c)

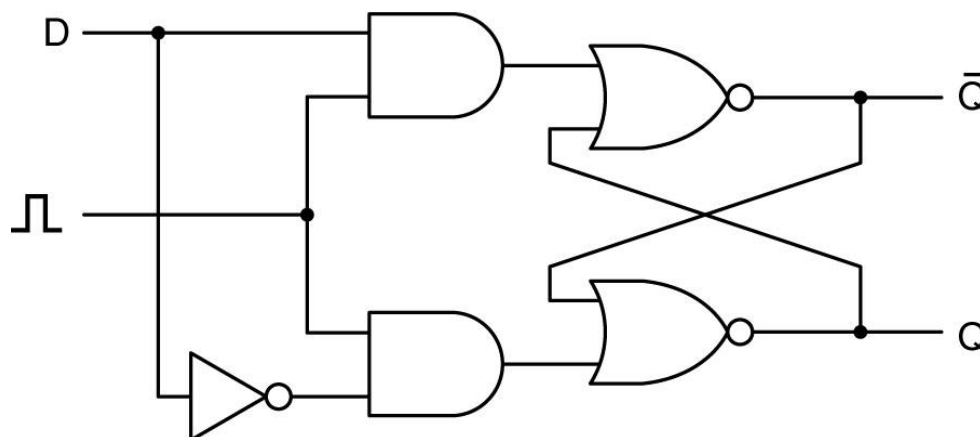
➤ Mạch chốt SR có xung Clock

- Thêm vào mạch chốt SR 2 cổng AND nối với xung đồng hồ để điều khiển trạng thái mạch chốt tại thời điểm xác định
- Tín hiệu vào chỉ có tác dụng khi xung clock=1 (mức cao)



➤ Mạch chốt D có xung Clock

- Mạch chốt SR sẽ ở trạng thái không xác định khi $S=R=1$
- Khắc phục bằng cách chỉ dùng 1 tín hiệu vào và đầu nối R với S qua cổng NOT
- Đây chính là mạch bộ nhớ 1 bit với D là ngõ vào, Q là ngõ ra

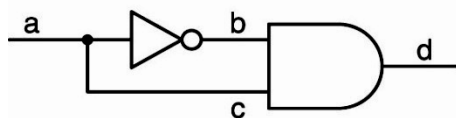




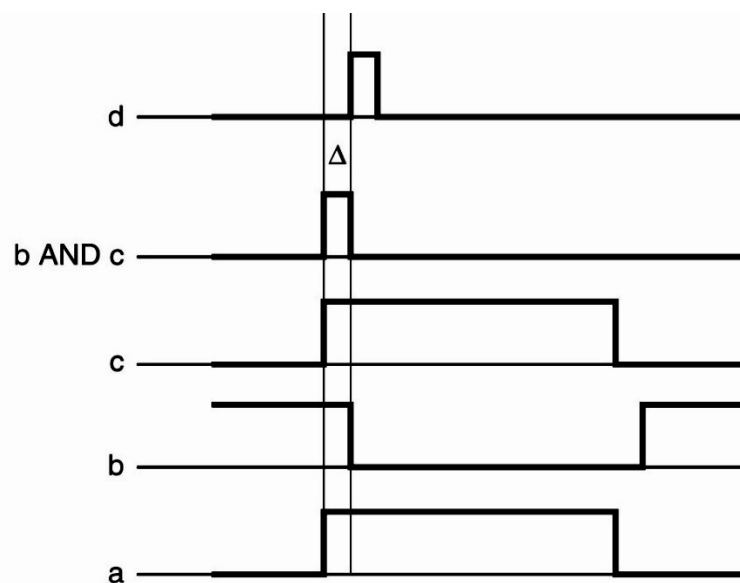
Mạch bộ nhớ

➤ Flip-Flop

- Trong thực tế ta muốn bộ nhớ chỉ được ghi trong 1 khoảng thời gian nhất định → cần thiết kế mạch xung Clock tác dụng theo cạnh (lên hoặc xuống)



(a)

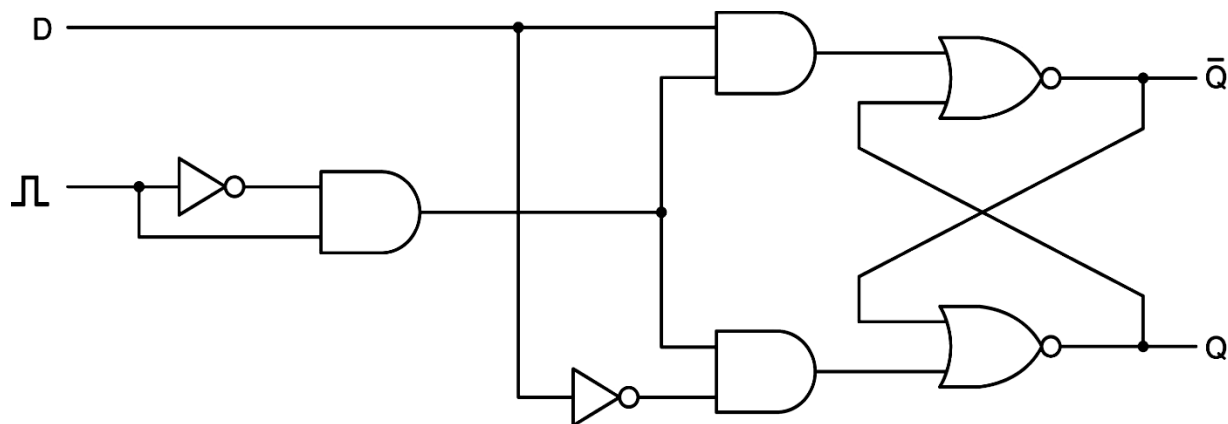


Time →

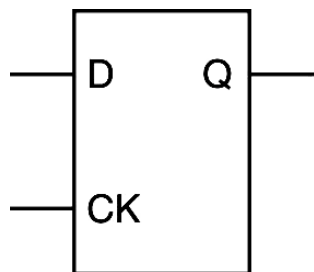
(b)

➤ D Flip-Flop

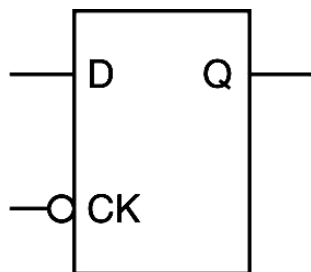
- Là mạch chốt D có xung Clock điều khiển bằng Flip-flop
- Phân biệt:
 - Flip-flop: edge triggered
 - Latch: level triggered



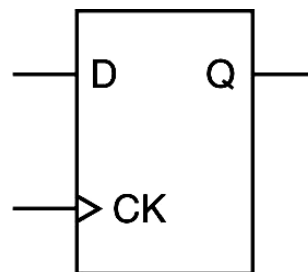
➤ Ký hiệu mạch chốt và Flip-Flop



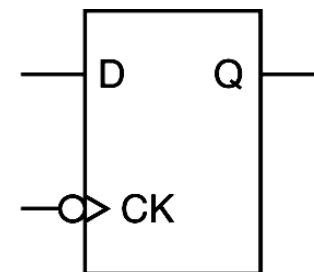
(a)



(b)



(c)

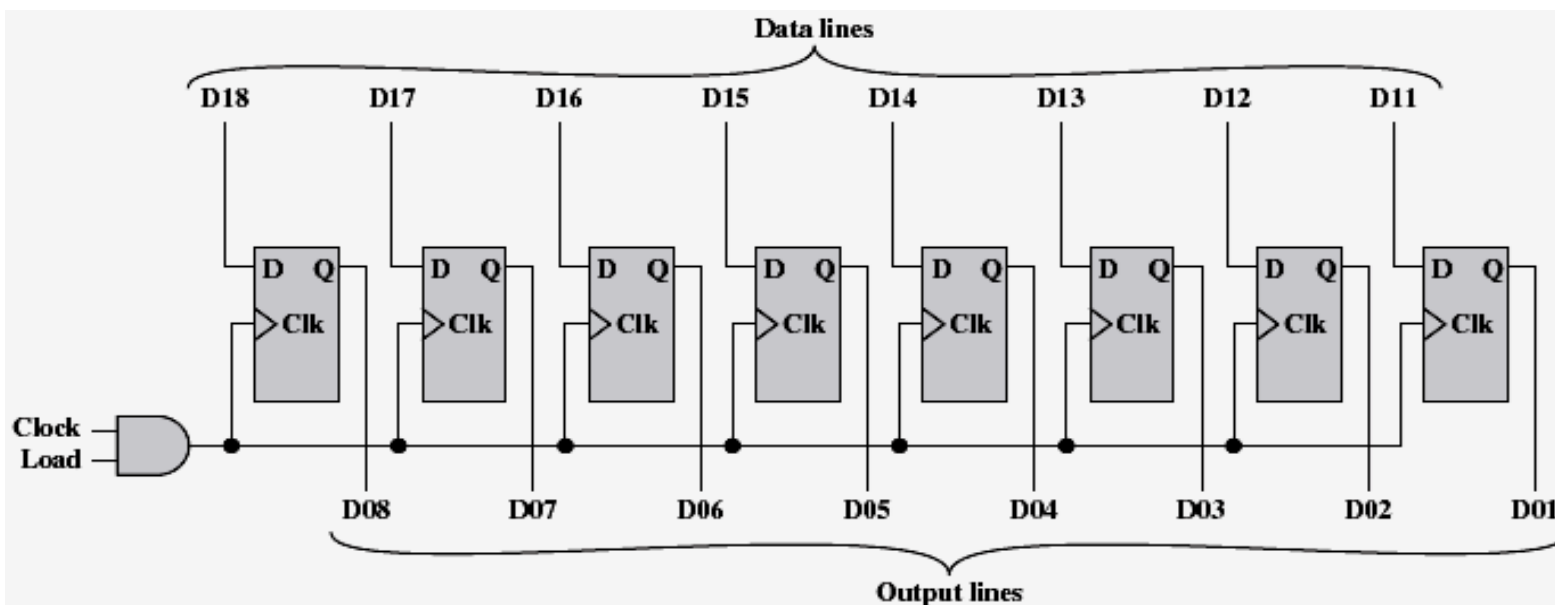
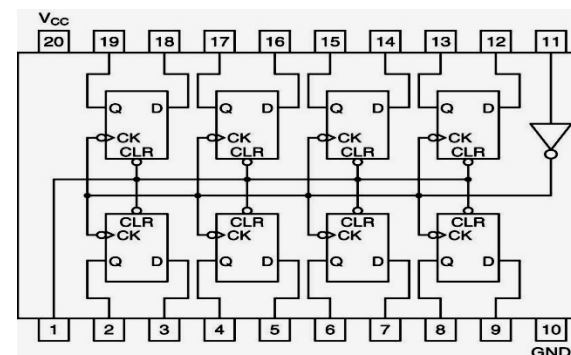


(d)

- a) Mạch chốt D tác động theo mức 1 (clock=1)
- b) Mạch chốt D tác động theo mức 0 (clock=0)
- c) Flip-flop D tác động theo cạnh lên (clock= $0 \rightarrow 1$)
- d) Flip-flop D tác động theo cạnh xuống (clock= $1 \rightarrow 0$)

➤ Thanh ghi (Register)

- Việc ghép nối nhiều ô nhớ 1 bit tạo thành các ô nhớ lớn hơn
- Ví dụ : Vi mạch 74273 gồm 8 D flip-flop ghép nối lại tạo thành 1 thanh ghi 8 bit





➤ Ví dụ : mạch bộ nhớ 4 ô x 3 bit

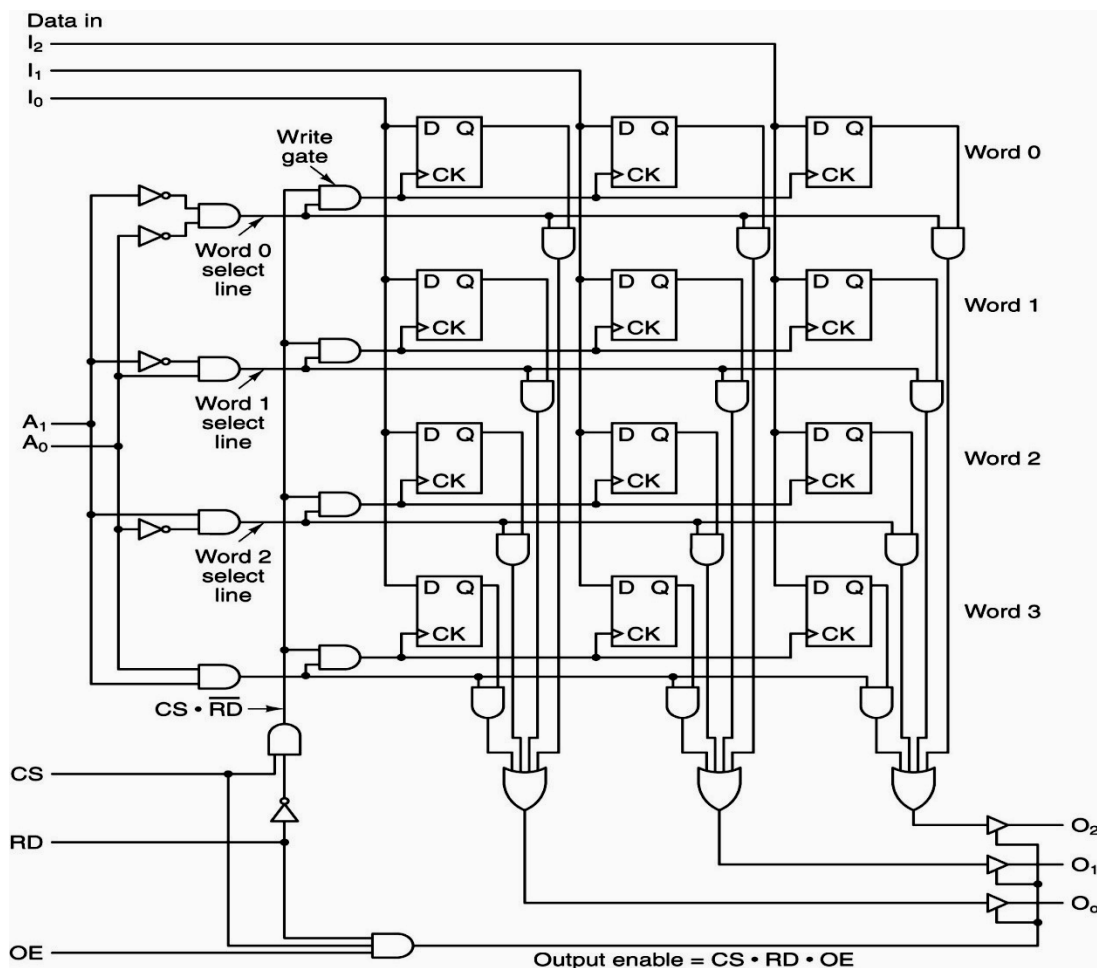
- A: Address
- I: Input data
- O: Output data
- CS: Chip select
- RD: Read/write
- OE: Output enable

Write:

CS=1, RD=0, OE=0

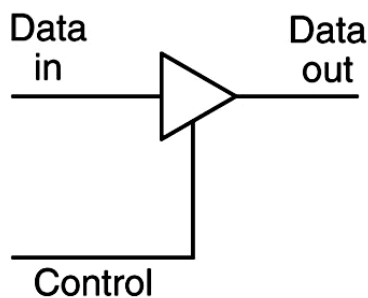
Read:

CS=1, RD=1, OE=1



➤ Mạch đệm (Buffer)

- Dùng để đọc dữ liệu đồng bộ trên nhiều đường tín hiệu bằng 1 đường điều khiển riêng.
- Sử dụng các cổng 3 trạng thái (tri-state devices)



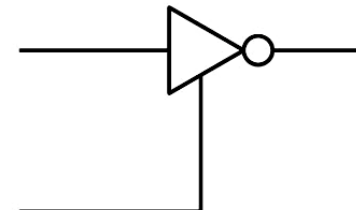
(a)



(b)



(c)

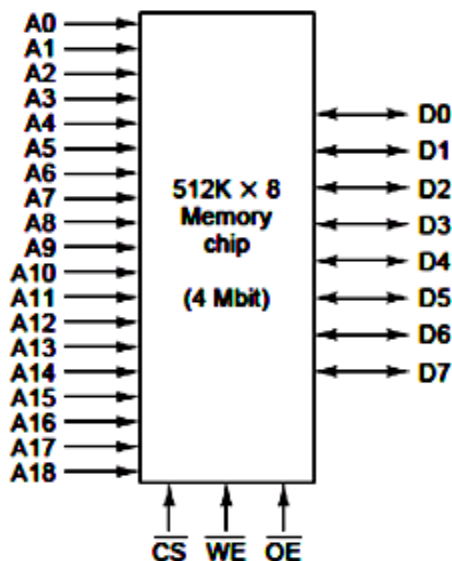


(d)

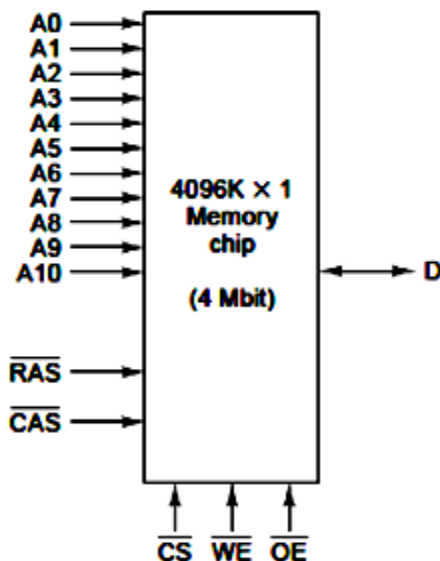
- a. Buffer không đảo.
- b. Khi control ở mức cao (=1).
- c. Khi control ở mức thấp (=0).
- d. Buffer đảo.

➤ Chip bộ nhớ

- Bộ nhớ thường gồm nhiều ô nhớ ghép lại
- Ví dụ 1: Chip bộ nhớ 4Mbit có thể tạo thành từ 512K ô 8 bit hoặc ma trận 2048x2048 ô 1 bit



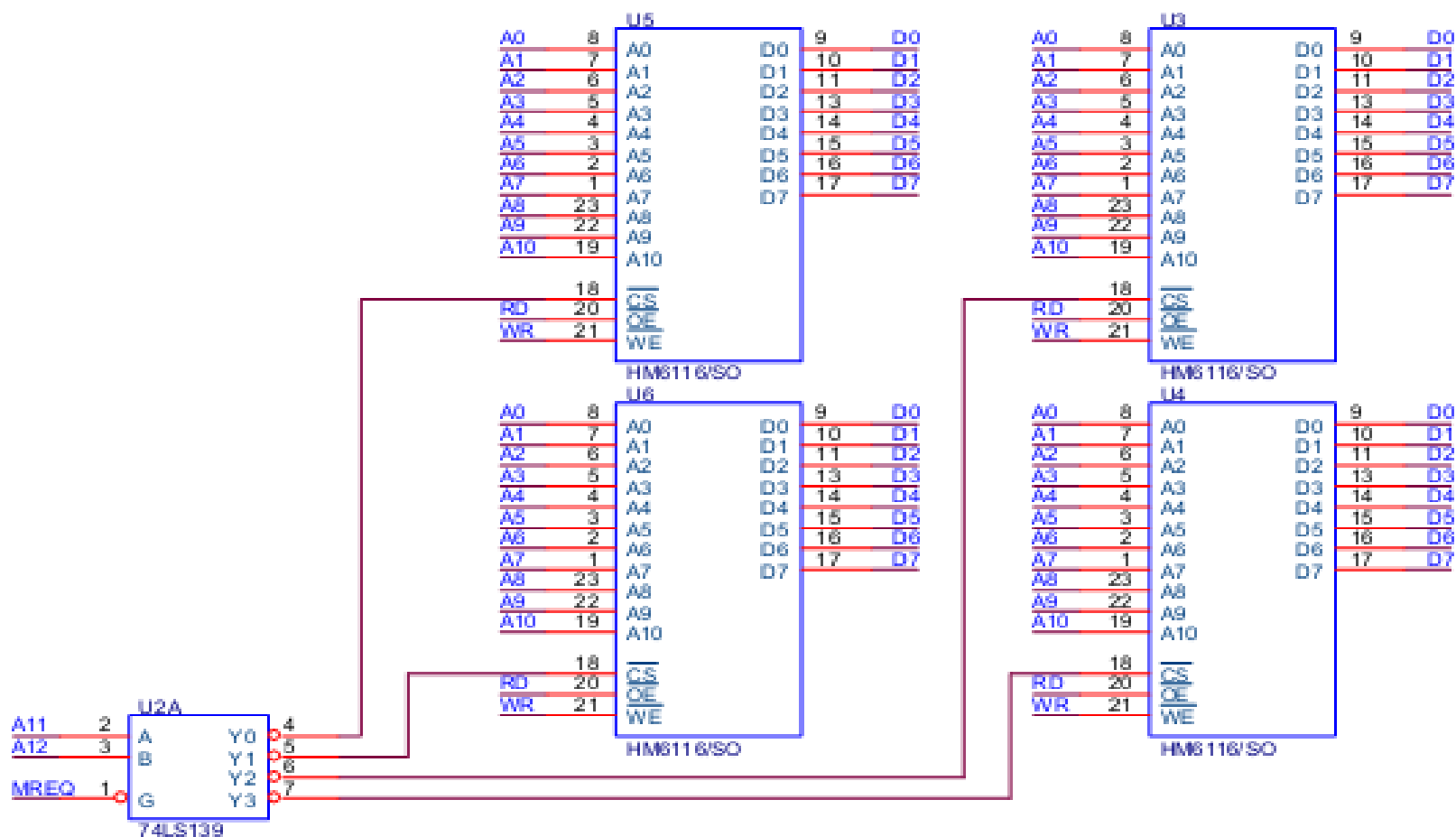
(a)



(b)

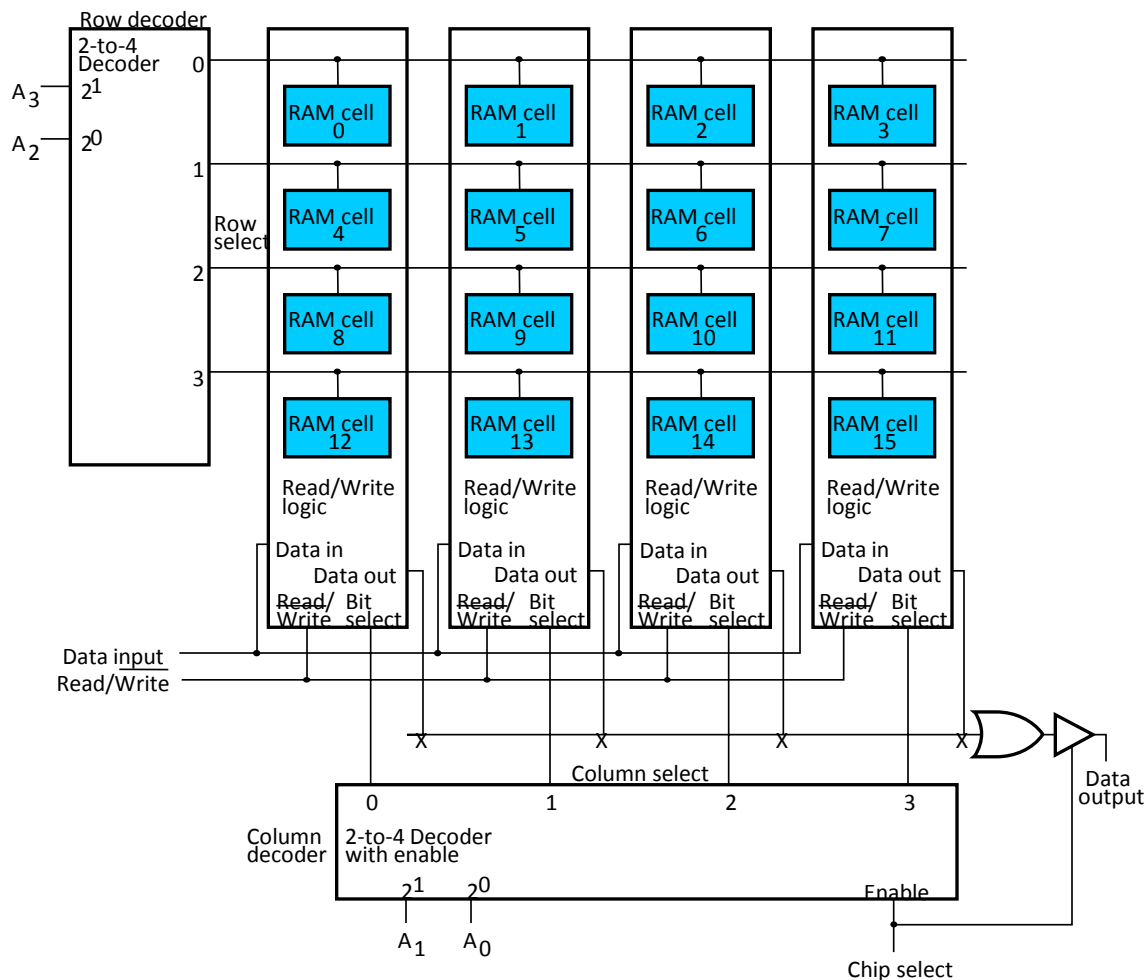
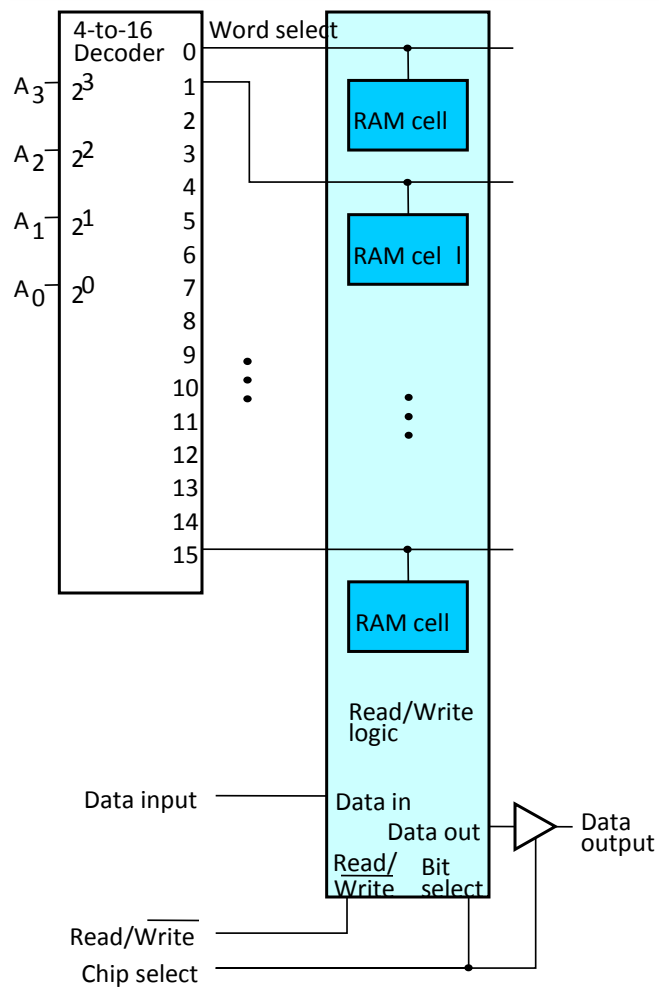
Ghi chú:

RAS: Row Address Strobe
 CAS: Column Address Strobe
 CS: Chip select
 WE: Write enable
 OE: Output enable
 D: Data
 A: Address



➤ Chip bộ nhớ (tiếp)

- Mạch giải mã địa chỉ n bit có thể giải mã cho 2^n ô nhớ → cần n chân tín hiệu địa chỉ
- Có thể giảm kích thước bộ giải mã còn bằng cách tổ chức thành ma trận các ô nhớ → sử dụng 2 bộ giải mã cho hàng và cột riêng
- Ví dụ: bộ nhớ 16 ô cần 4 bit địa chỉ có thể tổ chức thành ma trận 4×4 → chỉ cần giải mã 2 bit cho hàng và 2 bit cho cột.
- Có thể ghép địa chỉ hàng và cột chung 1 chân tín hiệu → giảm số chân kết nối bus địa chỉ
- Nhược điểm: cần gấp đôi thời gian truy cập bộ nhớ

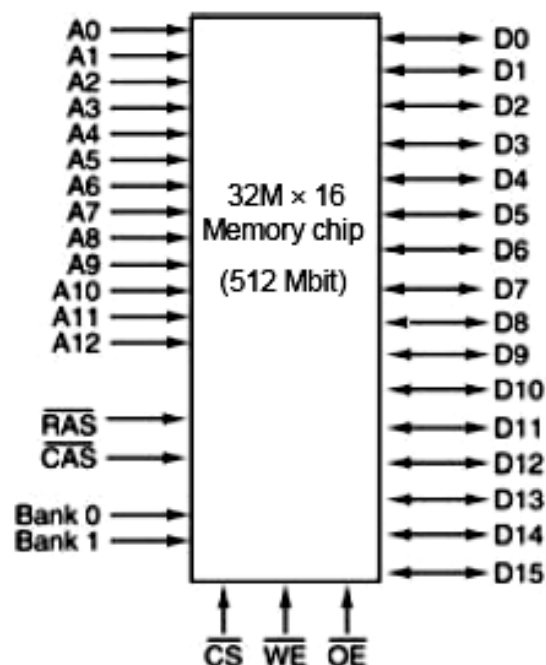
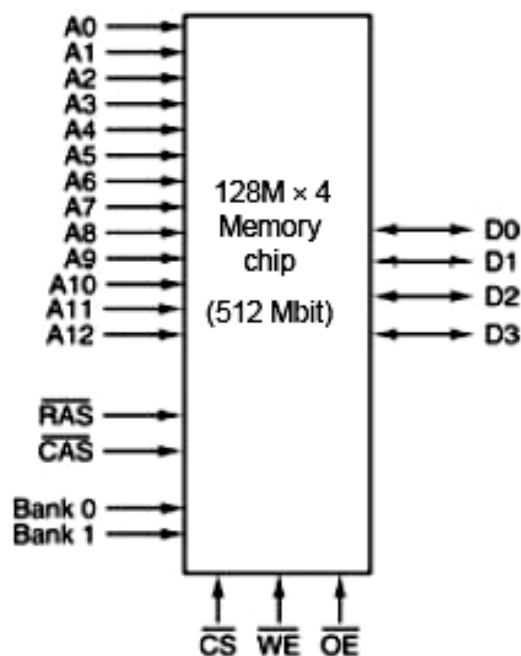




➤ Chip bộ nhớ (tiếp)

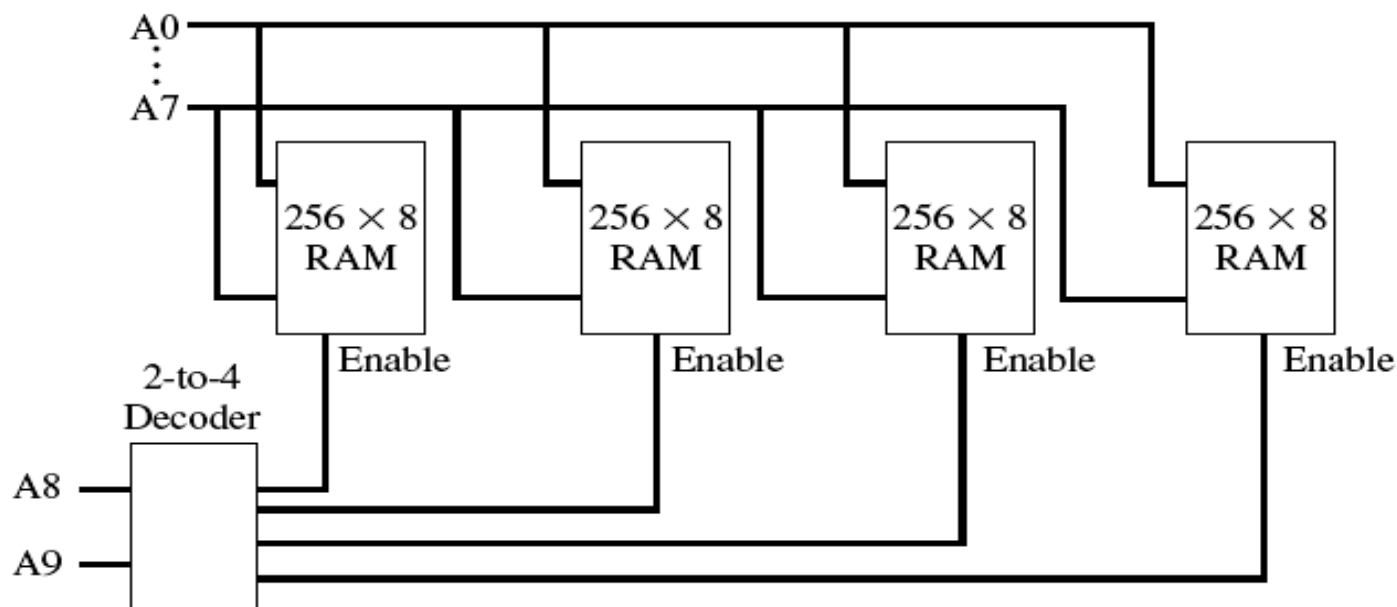
○ Ví dụ 2: Chip bộ nhớ 512Mbit = 4 bank 128Mbit

- Ma trận 13 bit hàng * 12 bit cột * ô nhớ 4 bit
- Ma trận 13 bit hàng * 10 bit cột * ô nhớ 16 bit



➤ Tổ chức bộ nhớ

- Bộ nhớ thường gồm nhiều chip nhớ dung lượng nhỏ ghép lại
- Dùng 1 mạch giải mã địa chỉ để chọn chip khi truy cập
- Ví dụ: Bộ nhớ 1KB gồm 4 chip 256B ghép lại





Câu hỏi





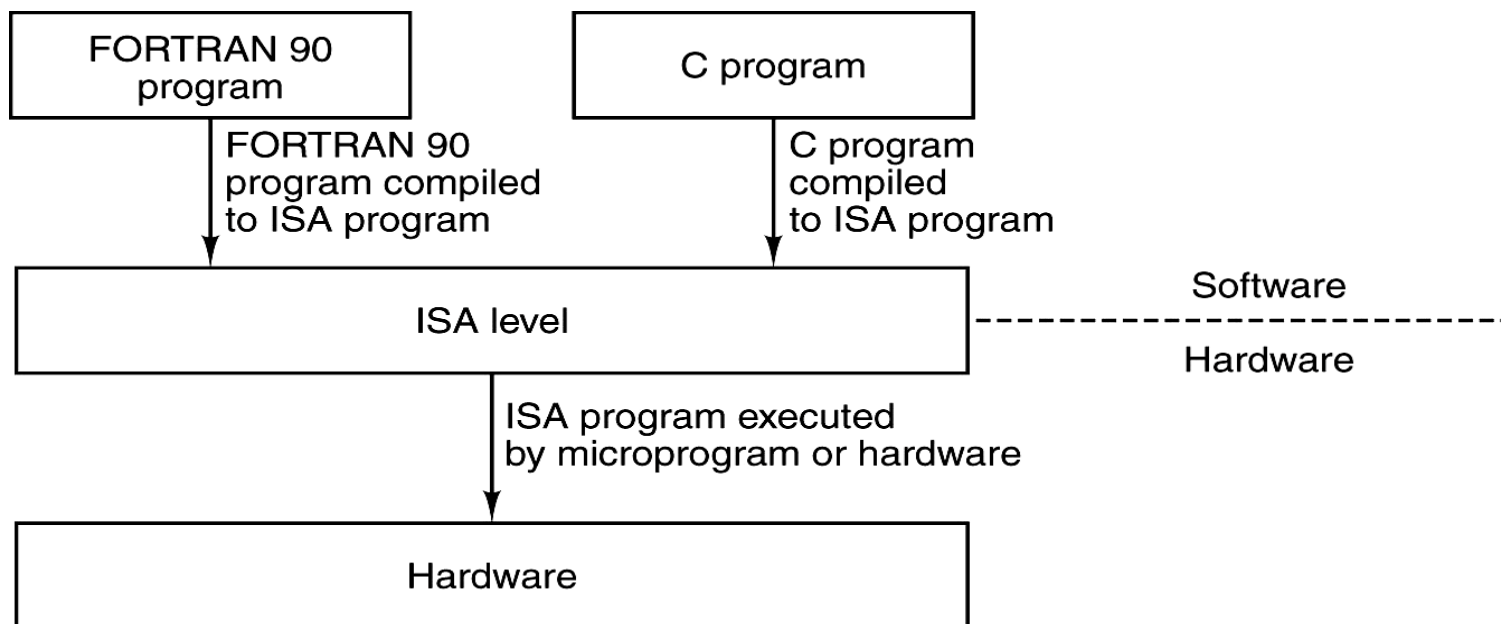
Chương 4 Kiến trúc tập lệnh (Instruction Set Architecture)

1. Mô hình lập trình của máy tính
2. Các đặc trưng của lệnh máy
3. Các kiểu thao tác của lệnh
4. Các phương pháp định địa chỉ
5. Phân loại tập lệnh
6. Kiến trúc tập lệnh Intel x86

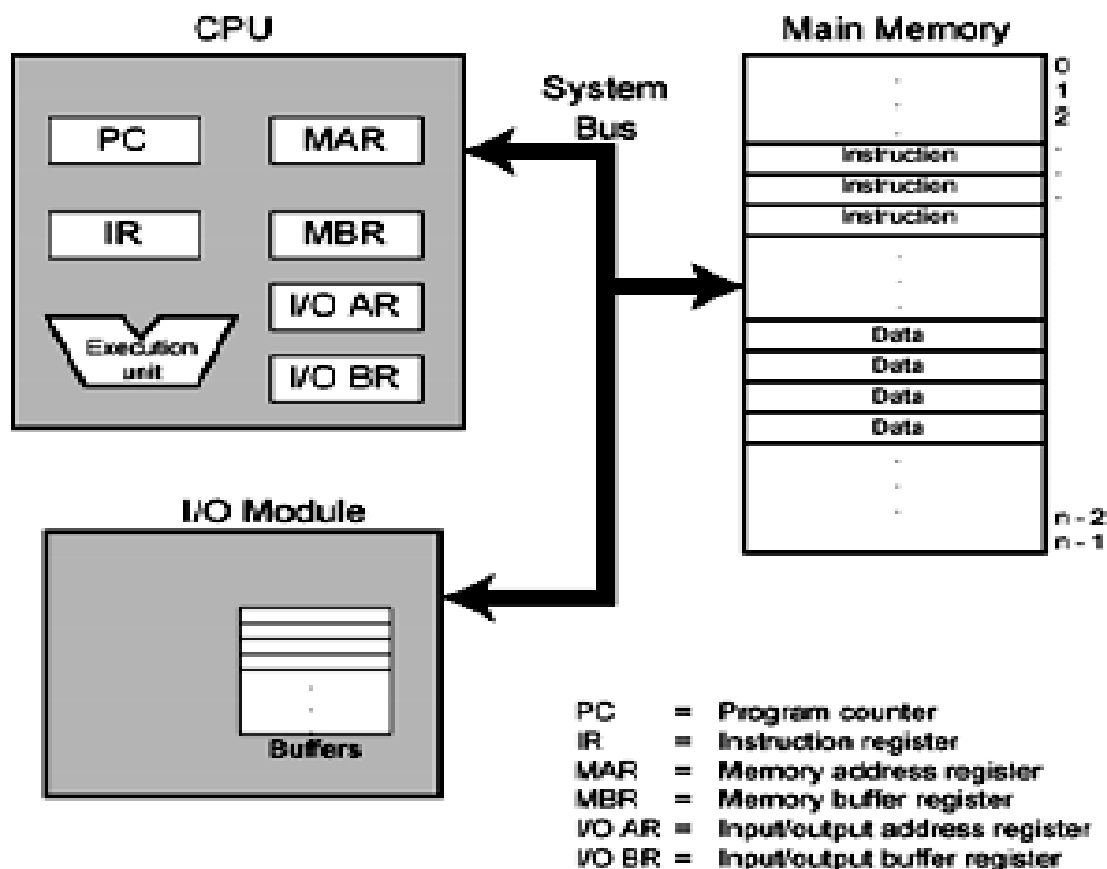


1. Mô hình lập trình của máy tính

- Vị trí kiến trúc tập lệnh ISA trong máy tính
 - Nằm giữa phần cứng và NNLT cấp cao HLL
 - Giúp phần mềm tương thích khi kiến trúc phần cứng thay đổi

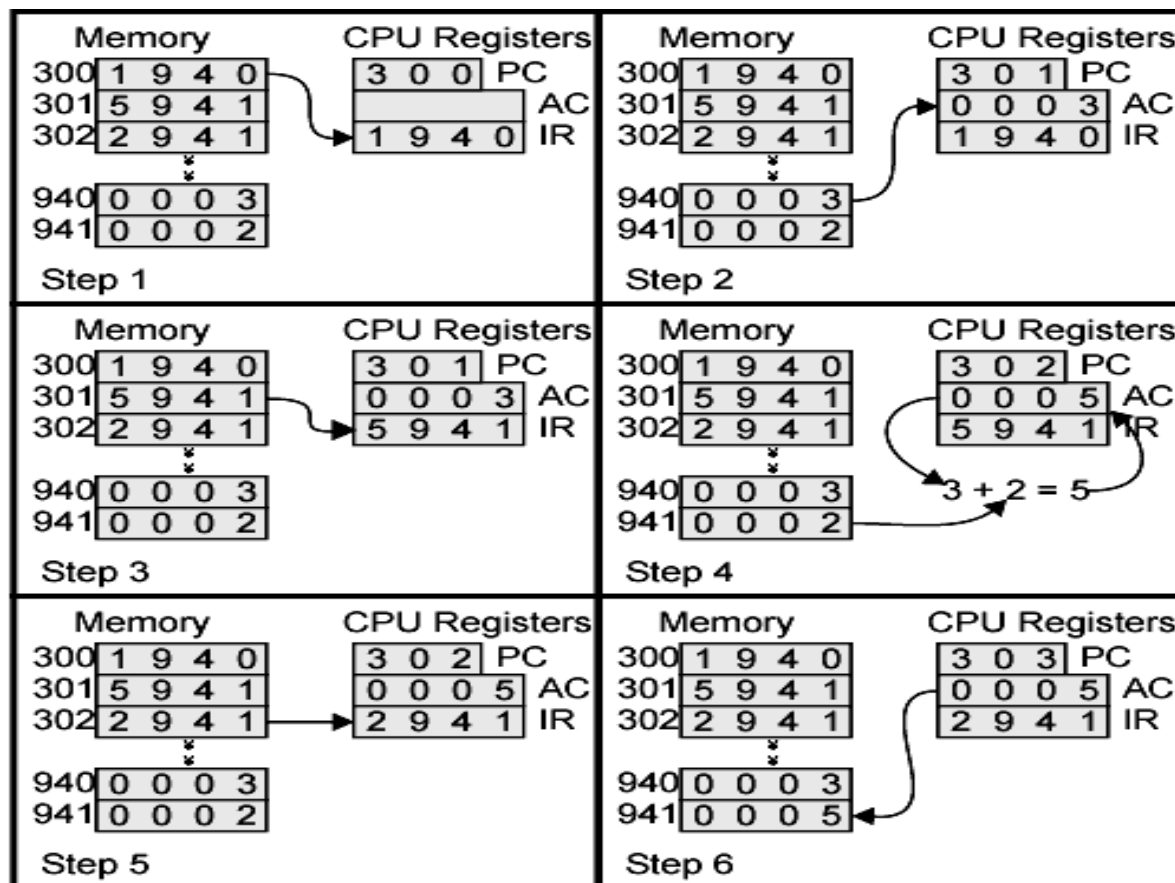


➤ Máy tính theo quan điểm lập trình





- Ví dụ về sự thi hành chương trình



➤ Tập thanh ghi (Registers)

- Chứa các thông tin tạm thời phục vụ cho hoạt động ở thời điểm hiện tại của CPU
- Được coi là mức đầu tiên của hệ thống bộ nhớ
- Số lượng thanh ghi nhiều → tăng hiệu năng của CPU
- Có hai loại thanh ghi:
 - Các thanh ghi lập trình được
 - Các thanh ghi không lập trình được

➤ Phân loại thanh ghi theo chức năng

- Thanh ghi địa chỉ: quản lý địa chỉ của bộ nhớ hay cổng IO.
- Thanh ghi dữ liệu: chứa tạm thời các dữ liệu.
- Thanh ghi đa năng: có thể chứa địa chỉ hoặc dữ liệu.
- Thanh ghi điều khiển/trạng thái: chứa các thông tin điều khiển và trạng thái của CPU.
- Thanh ghi lệnh: chứa lệnh đang được thực hiện.

➤ Một số thanh ghi điển hình

- Các thanh ghi địa chỉ (Address Register)
 - Bộ đếm chương trình PC (Program Counter)
 - Con trỏ dữ liệu DP (Data Pointer)
 - Con trỏ ngăn xếp SP (Stack Pointer)
 - Thanh ghi cơ sở và thanh ghi chỉ số (Base Register & Index Register)
- Các thanh ghi dữ liệu (Data Register)
- Thanh ghi trạng thái (Status Register)

➤ Bộ đếm chương trình PC

- Còn được gọi là con trỏ lệnh IP (Instruction Pointer)
- Giữ địa chỉ của lệnh tiếp theo sẽ được thi hành.
- Sau khi một lệnh được nhận vào, nội dung PC tự động tăng để trỏ sang lệnh kế tiếp.

➤ Thanh ghi con trỏ dữ liệu DP

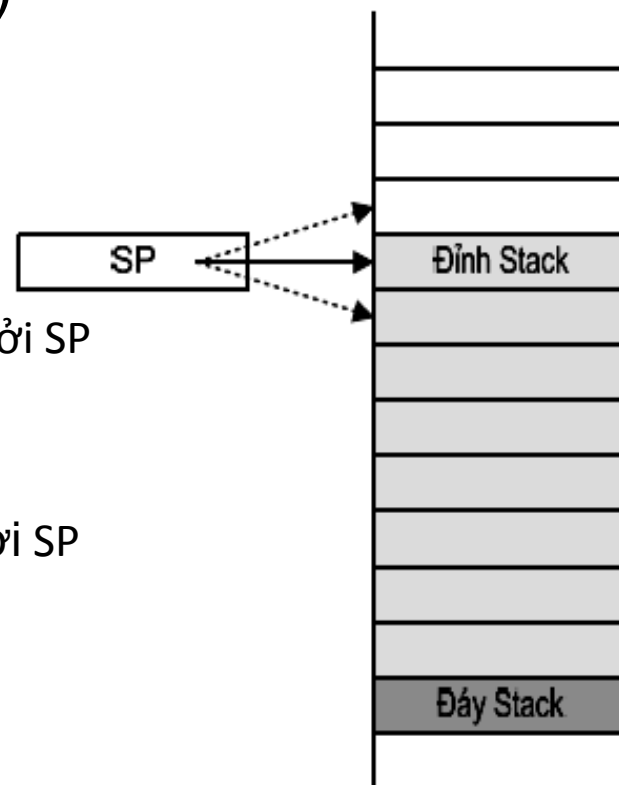
- Chứa địa chỉ của ô nhớ dữ liệu mà CPU muốn truy cập
- Thường có nhiều thanh ghi con trỏ dữ liệu cho phép chương trình có thể truy cập nhiều vùng nhớ đồng thời

➤ Ngăn xếp (Stack)

- Ngăn xếp là vùng nhớ có cấu trúc LIFO (Last In - First Out) hoặc FILO (First In - Last Out)
- Ngăn xếp thường dùng để phục vụ cho chương trình con
- Đáy ngăn xếp là một ô nhớ xác định
- Đỉnh ngăn xếp là thông tin nằm ở vị trí trên cùng trong ngăn xếp
- Đỉnh ngăn xếp có thể bị thay đổi

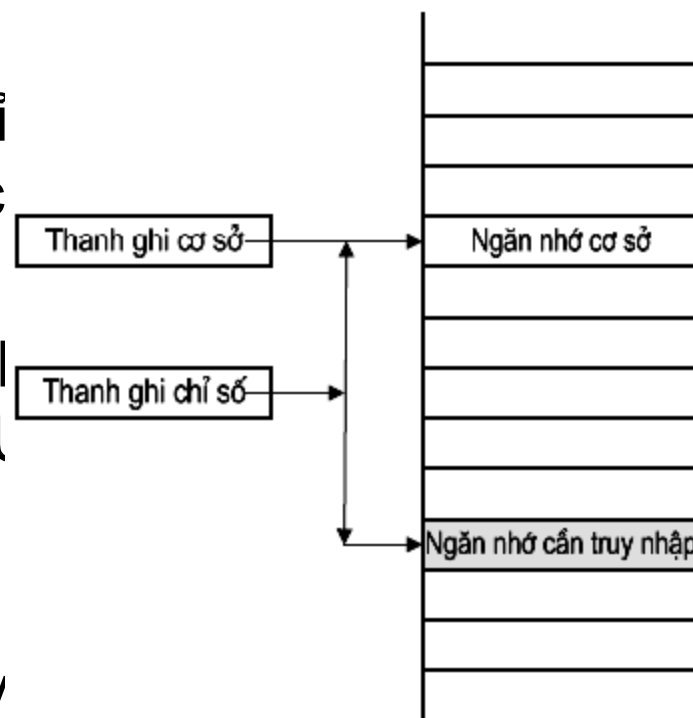
➤ Con trỏ ngăn xếp SP (Stack Pointer)

- Chứa địa chỉ của ô nhớ đỉnh ngăn xếp
- Khi cất một thông tin vào ngăn xếp:
 - Thao tác PUSH
 - Nội dung của SP tự động tăng
 - Thông tin được cất vào ô nhớ đang trỏ bởi SP
- Khi lấy một thông tin ra khỏi ngăn xếp:
 - Thao tác POP
 - Thông tin được đọc từ ô nhớ đang trỏ bởi SP
 - Nội dung của SP tự động giảm
- Khi ngăn xếp rỗng, SP trở vào đáy



➤ Thanh ghi cơ sở và thanh ghi chỉ số

- Thanh ghi cơ sở: chứa địa chỉ của ngăn nhớ cơ sở (địa chỉ cơ sở)
- Thanh ghi chỉ số: chứa độ lệch địa chỉ giữa ngăn nhớ mà CPU cần truy cập so với ngăn nhớ cơ sở (chỉ số)
- Địa chỉ của ngăn nhớ cần truy cập = địa chỉ cơ sở + chỉ số



➤ Thanh ghi dữ liệu (Data Register)

- Chứa các dữ liệu tạm thời hoặc các kết quả trung gian
- Cần có nhiều thanh ghi dữ liệu
- Các thanh ghi số nguyên: 8, 16, 32, 64 bit
- Các thanh ghi số dấu chấm động: 32, 64, 80 bit

➤ Thanh ghi trạng thái (Status Register)

- Còn gọi là thanh ghi cờ (Flags Register) hoặc từ trạng thái chương trình PSW (Program Status Word)
- Chứa các thông tin trạng thái của CPU
 - Các cờ phép toán: báo hiệu trạng thái của kết quả phép toán
 - Các cờ điều khiển: biểu thị trạng thái điều khiển của CPU

➤ Ví dụ cờ phép toán

- Zero Flag (cờ rỗng): được thiết lập lên 1 khi kết quả của phép toán bằng 0.
- Sign Flag (cờ dấu): được thiết lập lên 1 khi kết quả phép toán nhỏ hơn 0 (kết quả âm)
- Carry Flag (cờ nhớ): được thiết lập lên 1 nếu phép toán có nhớ ra ngoài bit cao nhất → cờ báo tràn với số không dấu.
- Overflow Flag (cờ tràn): được thiết lập lên 1 nếu cộng hai số nguyên cùng dấu mà kết quả có dấu ngược lại → cờ báo tràn với số có dấu .

➤ Ví dụ cờ điều khiển

○ Interrupt Flag (Cờ cho phép ngắt):

- Nếu $IF = 1 \rightarrow$ CPU ở trạng thái cho phép ngắt với tín hiệu yêu cầu ngắt từ bên ngoài gửi tới
- Nếu $IF = 0 \rightarrow$ CPU ở trạng thái cấm ngắt với tín hiệu yêu cầu ngắt từ bên ngoài gửi tới

○ Direction Flag (Cờ hướng):

- Nếu $DF=0 \rightarrow$ Truy cập bộ nhớ theo hướng tăng
- Nếu $DF=1 \rightarrow$ Truy cập bộ nhớ theo hướng giảm



➤ Ví dụ: Tập thanh ghi của một số bộ xử lý

Data Registers	
D0	
D1	
D2	
D3	
D4	
D5	
D6	
D7	

Address Registers	
A0	
A1	
A2	
A3	
A4	
A5	
A6	
A7	
A7'	

Program Status	
Program Counter	
Status Register	

(a) MC68000

General Registers	
AX	Accumulator
BX	Base
CX	Count
DX	Data

Pointer & Index	
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Dest Index

Segment	
CS	Code
DS	Data
SS	Stack
ES	Extra

Program Status	
Instr Ptr	
Flags	

(b) 8086

General Registers			
EAX		AX	
EBX		BX	
ECX		CX	
EDX		DX	
ESP		SP	
EBP		BP	
ESI		SI	
EDI		DI	

Program Status	
FLAGS Register	
Instruction Pointer	

(c) 80386 - Pentium II



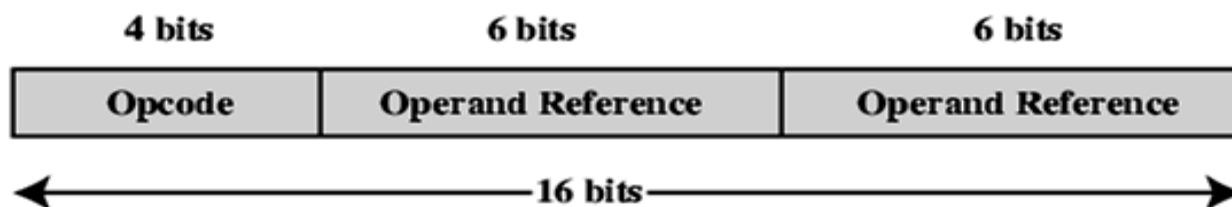
4.2 Các đặc trưng của lệnh máy

- Giới thiệu chung về tập lệnh
 - Mỗi bộ xử lý có một tập lệnh xác định
 - Tập lệnh thường có hàng chục đến hàng trăm lệnh
 - Mỗi lệnh là một chuỗi số nhị phân mà bộ xử lý hiểu được để thực hiện một thao tác xác định.
 - Các lệnh được mô tả bằng các ký hiệu gọi nhớ → chính là các lệnh của hợp ngữ (assembly), ví dụ: ADD, SUB, LOAD, STORE,...

➤ Các thành phần của lệnh máy

Opcode	Operand address
--------	-----------------

- Mã thao tác (operation code): mã hóa cho thao tác mà bộ xử lý phải thực hiện bằng số nhị phân (làm gì?)
- Địa chỉ toán hạng (operand address): chỉ ra nơi chứa các toán hạng mà thao tác sẽ tác động (làm ở đâu?)
 - Toán hạng nguồn: dữ liệu vào của thao tác
 - Toán hạng đích: dữ liệu ra của thao tác
 - Toán hạng: Thanh ghi, bộ nhớ, thiết bị ngoại vi,...
 - Ví dụ: 1 lệnh 16 bit có 2 toán hạng



➤ Số lượng địa chỉ toán hạng trong lệnh

○ Ba địa chỉ toán hạng:

- 2 toán hạng nguồn, 1 toán hạng đích
- Ví dụ : $a = b + c \rightarrow \text{ADD } A, B, C$
- Từ lệnh dài vì phải mã hoá địa chỉ cho cả ba toán hạng

○ Hai địa chỉ toán hạng:

- Một toán hạng vừa là toán hạng nguồn vừa là toán hạng đích; toán hạng còn lại là toán hạng nguồn
- Ví dụ : $a = a + b \rightarrow \text{ADD } A, B$
- Giá trị cũ của 1 toán hạng nguồn bị mất vì phải chứa kết quả
- Rút gọn độ dài từ lệnh, được sử dụng phổ biến

➤ Số lượng địa chỉ toán hạng trong lệnh (tiếp)

○ Một địa chỉ toán hạng:

- Một toán hạng được chỉ ra trong lệnh
- Một toán hạng là ngầm định, thường là thanh ghi tích lũy (accumulator)
- Ví dụ : $a = b + c$

```
LOAD B  
ADD C  
STORE A
```

○ Không địa chỉ toán hạng:

- Các toán hạng đều được ngầm định: Sử dụng Stack
- Ví dụ: $a = b + c$

```
PUSH B  
PUSH C  
ADD  
POP A
```

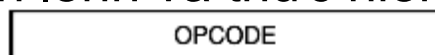
➤ Đánh giá về số địa chỉ toán hạng

○ Nhiều địa chỉ toán hạng

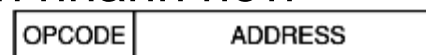
- Các lệnh phức tạp hơn
- Cần nhiều thanh ghi
- Chương trình có ít lệnh hơn
- Nhận lệnh và thực hiện lệnh chậm hơn

○ Ít địa chỉ toán hạng

- Các lệnh đơn giản hơn
- Cần ít thanh ghi
- Chương trình có nhiều lệnh hơn
- Nhận lệnh và thực hiện lệnh nhanh hơn



(a)



(b)



(c)



(d)

➤ Các kiểu toán hạng

- Địa chỉ
- Số
 - Số nguyên
 - Số dấu chấm động
 - Mã BCD
- Ký tự
 - Mã ASCII
- Dữ liệu logic
 - Các bit hoặc các cờ

Câu hỏi: Khi đọc trong 1 ô nhớ nhận được giá trị nhị phân 65, làm sao biết được đây là gì?

- Số nguyên 65
- Ký tự 'A'
- Lệnh CT 65
- Địa chỉ 65





4.3 Các kiểu thao tác của lệnh

➤ Phân loại lệnh:

- Di chuyển dữ liệu
- Xử lý số học với số nguyên
- Xử lý logic
- Điều khiển vào-ra (IO)
- Chuyển điều khiển (rẽ nhánh)
- Điều khiển hệ thống

➤ Các lệnh di chuyển dữ liệu

- MOVE Copy dữ liệu từ nguồn đến đích
- LOAD Nạp dữ liệu từ bộ nhớ đến bộ xử lý
- STORE Cất dữ liệu từ bộ xử lý đến bộ nhớ
- EXCHANGE Hoán đổi nội dung của nguồn và đích
- CLEAR Chuyển các bit 0 vào toán hạng đích
- SET Chuyển các bit 1 vào toán hạng đích
- PUSH Cất nội dung toán hạng nguồn vào ngăn xếp
- POP Lấy nội dung đỉnh ngăn xếp đưa đến toán hạng đích

➤ Các lệnh số học

- | | |
|-------------|---|
| ○ ADD | Cộng hai toán hạng |
| ○ SUBTRACT | Trừ hai toán hạng |
| ○ MULTIPLY | Nhân hai toán hạng |
| ○ DIVIDE | Chia hai toán hạng |
| ○ ABSOLUTE | Lấy trị tuyệt đối toán hạng |
| ○ NEGATE | Đổi dấu toán hạng (lấy 0 trừ toán hạng) |
| ○ INCREMENT | Tăng toán hạng thêm 1 |
| ○ DECREMENT | Giảm toán hạng đi 1 |
| ○ COMPARE | Trừ hai toán hạng để lập cờ |

➤ Các lệnh logic

- AND Thực hiện phép AND hai toán hạng
- OR Thực hiện phép OR hai toán hạng
- XOR Thực hiện phép XOR hai toán hạng
- NOT Đảo bit của toán hạng (lấy bù 1)
- TEST Thực hiện phép AND hai toán hạng để lập cờ

➤ Ví dụ các lệnh logic

- Giả sử có hai thanh ghi chứa dữ liệu như sau:
 - $(R1) = 1010\ 1010$
 - $(R2) = 0000\ 1111$
- $R1 \leftarrow (R1) \text{ AND } (R2) = 0000\ 1010$
- Phép toán AND dùng để xoá (Clear) một số bit và giữ nguyên một số bit còn lại của toán hạng.
- $R1 \leftarrow (R1) \text{ OR } (R2) = 1010\ 1111$
- Phép toán OR dùng để thiết lập (Set) một số bit và giữ nguyên một số bit còn lại của toán hạng.
- $R1 \leftarrow (R1) \text{ XOR } (R2) = 1010\ 0101$
- Phép toán XOR dùng để đảo một số bit và giữ nguyên một số bit còn lại của toán hạng.

➤ Các lệnh logic (tiếp)

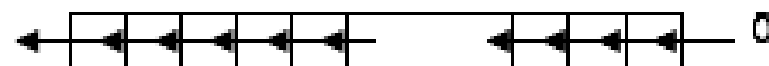
○ SHIFT

Dịch trái (phải) toán
hạng

○ ROTATE

Quay trái (phải)
toán hạng

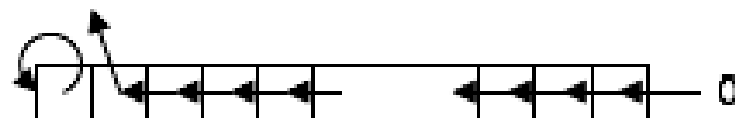
Dịch trái logic



Dịch phải logic



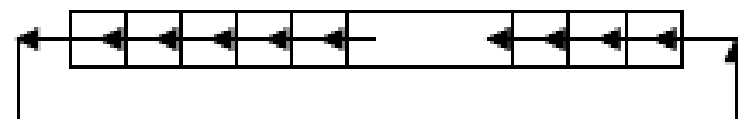
Dịch trái số học



Dịch phải số học



Quay trái logic



Quay phải logic



➤ Các lệnh nhập xuất chuyên dụng

- INPUT : Copy dữ liệu từ một cổng xác định đưa đến đích (thiết bị → bộ nhớ)
- OUTPUT: Copy dữ liệu từ nguồn đến một cổng xác định (bộ nhớ → thiết bị)

➤ Các lệnh chuyển điều khiển

- JUMP (BRANCH): Lệnh rẽ nhánh không điều kiện
- CONDITIONAL JUMP : Lệnh rẽ nhánh có điều kiện
- CALL : Lệnh gọi chương trình con
- RETURN : Lệnh trở về từ chương trình con

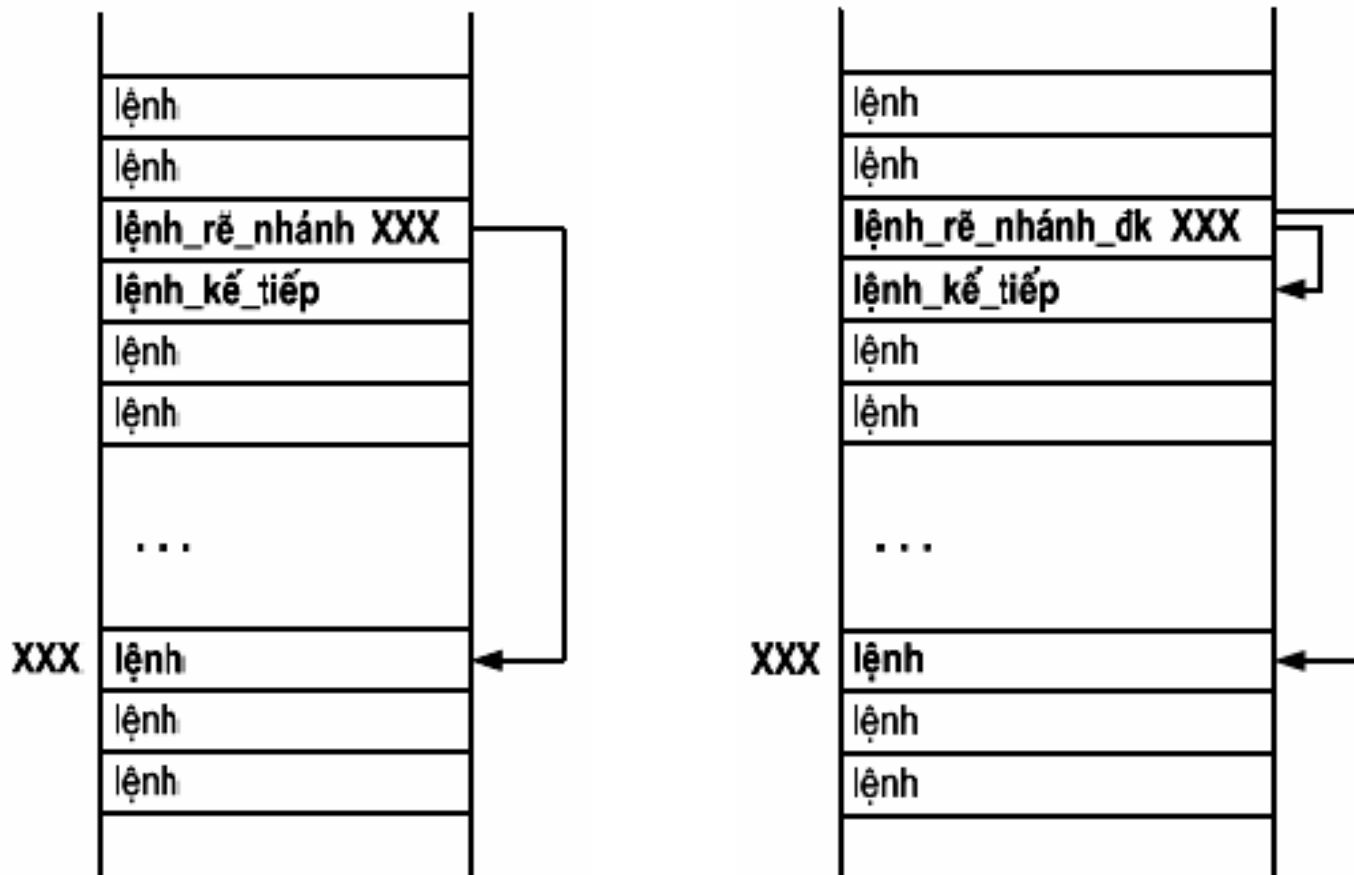
➤ Lệnh rẽ nhánh có điều kiện

- Trong lệnh có kèm theo điều kiện
- Kiểm tra điều kiện trong lệnh:
 - Nếu điều kiện đúng → chuyển tới thực hiện lệnh ở vị trí có địa chỉ XXX

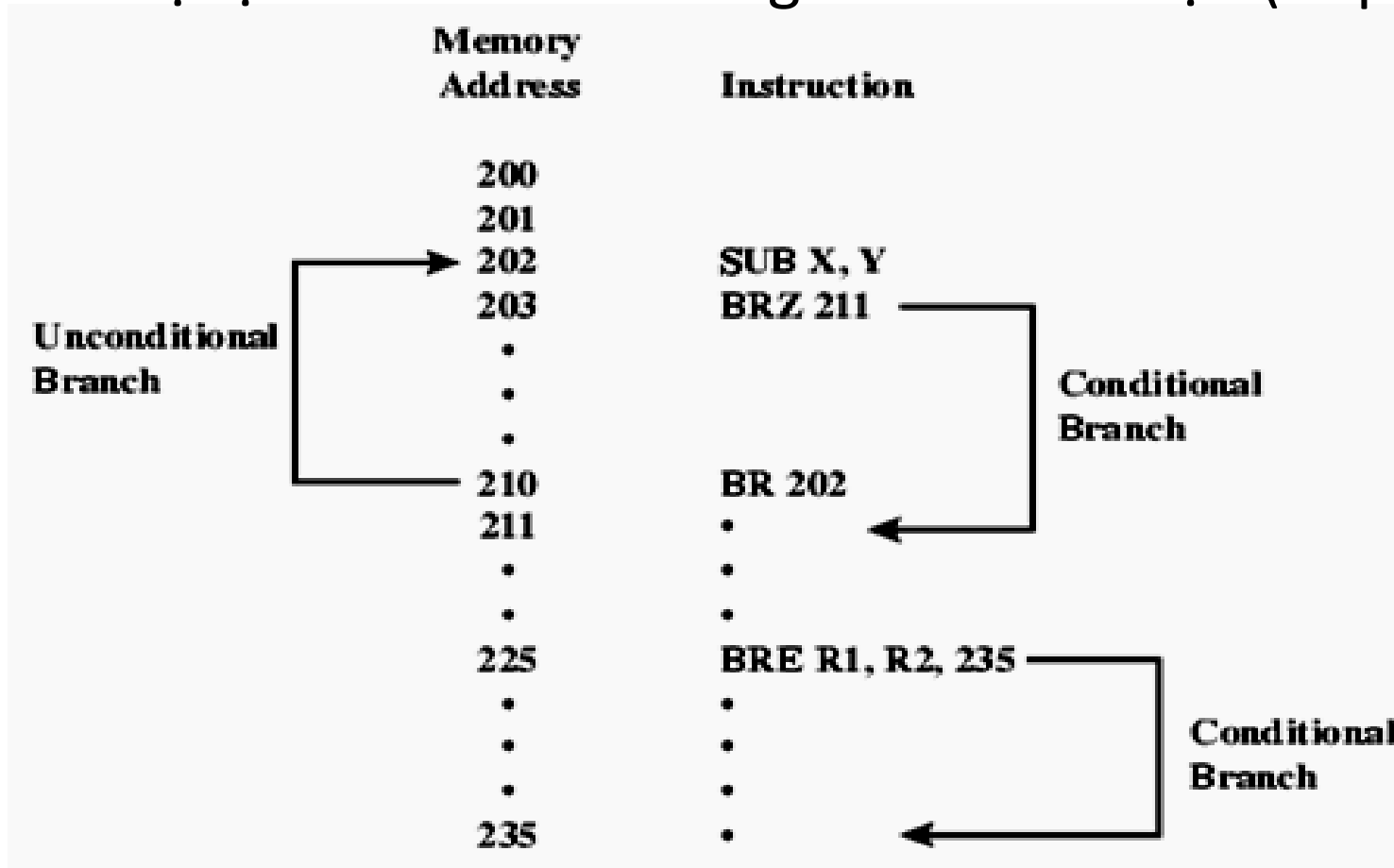
PC ← XXX

- Nếu điều kiện sai → chuyển sang thực hiện **lệnh_kế_tiếp**
- Điều kiện thường được kiểm tra thông qua các cờ
- Có nhiều lệnh rẽ nhánh theo các điều kiện khác nhau

➤ Minh họa lệnh rẽ nhánh không và có điều kiện



- Minh hoạ lệnh rẽ nhánh không và có điều kiện (tiếp)



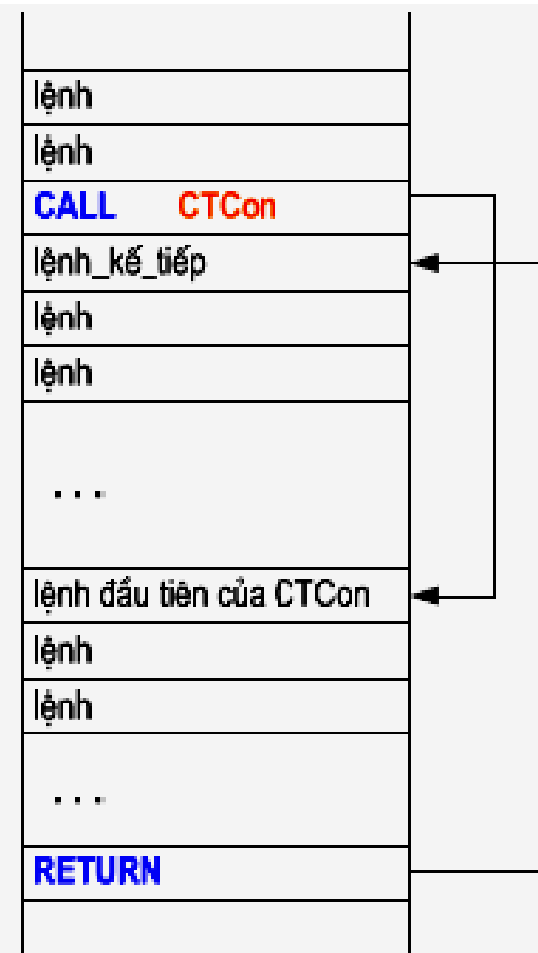
➤ Lệnh CALL và RETURN

○ CALL: Gọi chương trình con

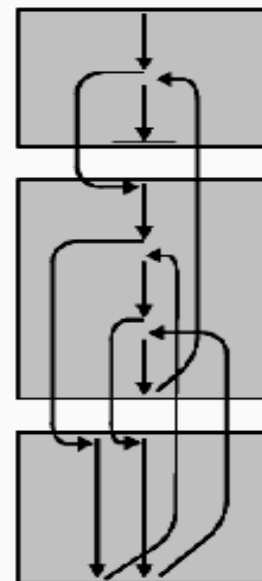
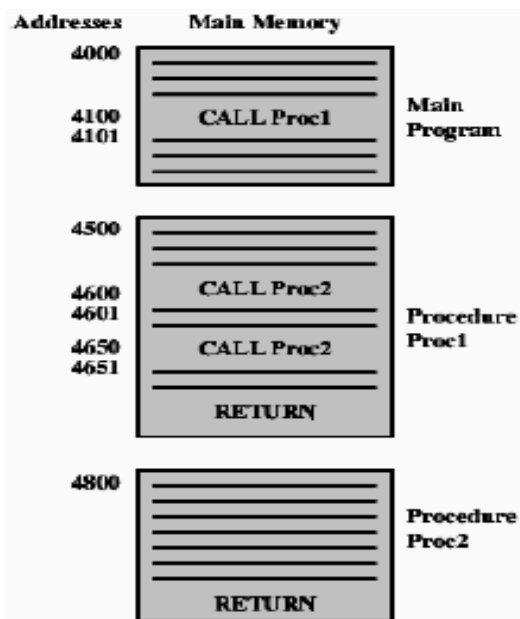
- Cất nội dung PC (chứa địa chỉ của **lệnh_kế_tiếp**) ra Stack
- Nạp vào PC địa chỉ lệnh đầu tiên của chương trình con được gọi
- Bộ xử lý được chuyển sang thực hiện chương trình con tương ứng

○ RETURN: Trở về từ chương trình con

- Lấy địa chỉ của **lệnh_kế_tiếp** được cất ở Stack nạp trả lại cho PC
- Bộ xử lý được điều khiển quay trở về thực hiện tiếp lệnh nằm sau lệnh CALL

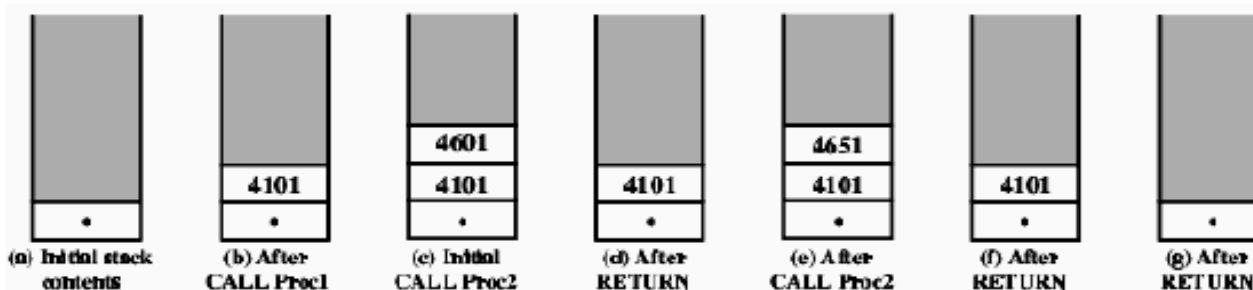


➤ Gọi các chương trình con lồng nhau



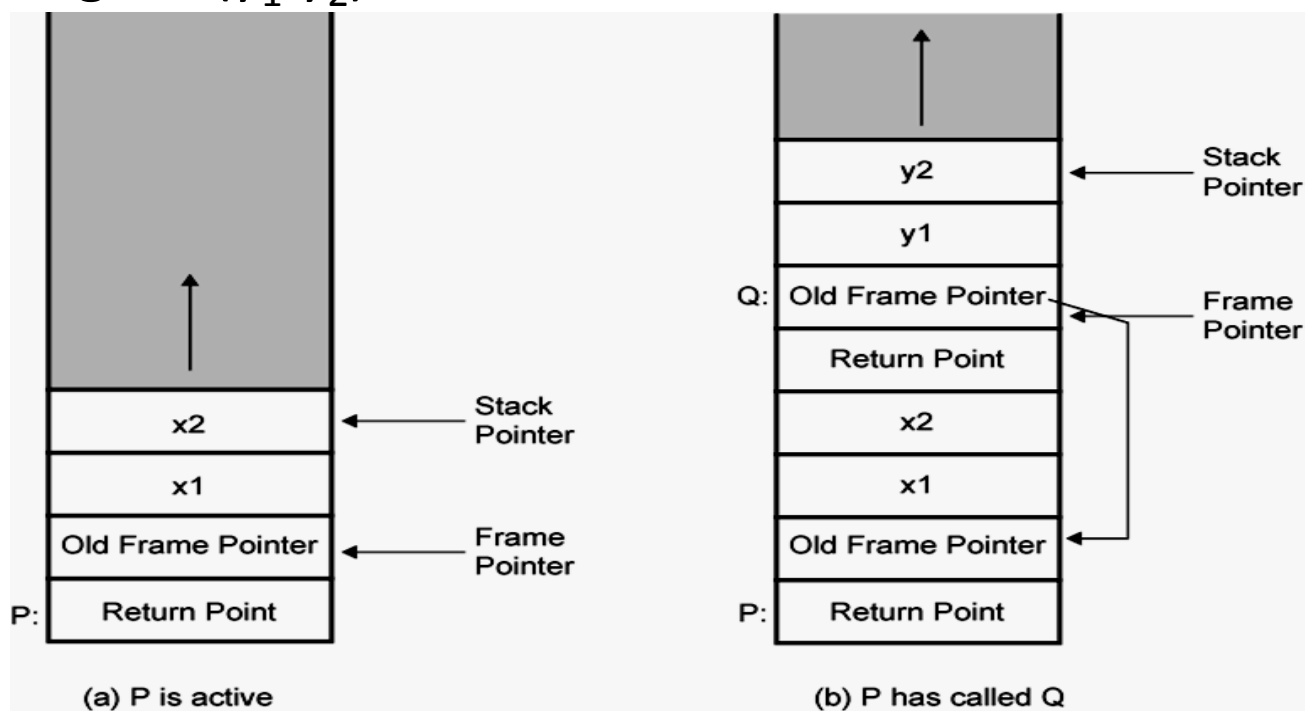
(a) Calls and returns

(b) Execution sequence



➤ Truyền tham số giữa các chương trình con

- Truyền qua Stack
- Ví dụ: P gọi Q(y_1, y_2) có 2 tham số.



➤ Các lệnh điều khiển hệ thống

- HALT : Dừng thực hiện chương trình
- WAIT : Tạm dừng thực hiện chương trình, lặp kiểm tra điều kiện cho đến khi thoả mãn thì tiếp tục thực hiện
- NO OPERATION : Không thực hiện gì cả
- LOCK : Cấm không cho xin chuyển nhượng bus
- UNLOCK : Cho phép xin chuyển nhượng bus



4.4 Các phương pháp định địa chỉ

- Khái niệm về định địa chỉ (addressing)
 - Toán hạng của lệnh có thể là:
 - Một giá trị cụ thể nằm ngay trong lệnh
 - Nội dung của thanh ghi
 - Nội dung của ngăn nhớ hoặc cổng IO
 - Phương pháp định địa chỉ (addressing modes) là cách thức địa chỉ hóa trong vùng địa chỉ của lệnh để xác định nơi chứa toán hạng
 - Định địa chỉ tức thì
 - Định địa chỉ thanh ghi
 - Định địa chỉ trực tiếp
 - Định địa chỉ gián tiếp qua thanh ghi
 - Định địa chỉ gián tiếp
 - Định địa chỉ dịch chuyển

➤ Định địa chỉ tức thì (Immediate Addressing)

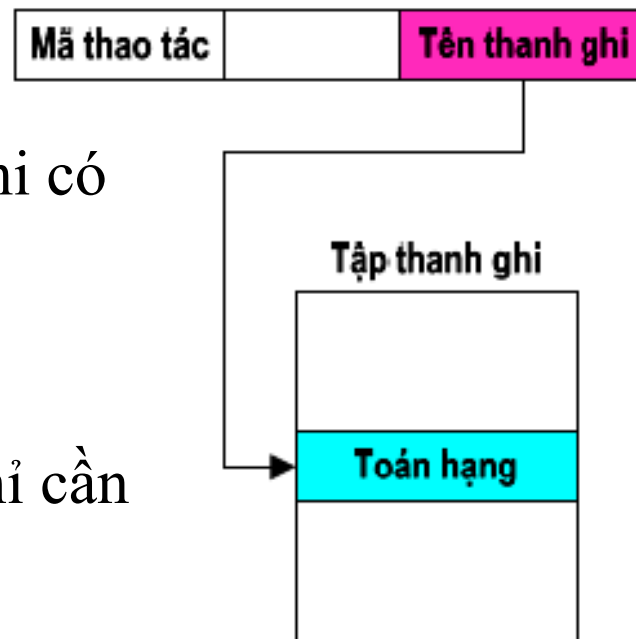
- Toán hạng nằm ngay trong vùng địa chỉ của lệnh
- Chỉ có thể là toán hạng nguồn
- Ví dụ: `ADD R1, 5 ; R1 ← R1+5`
- Không tham chiếu bộ nhớ
- Truy cập toán hạng rất nhanh
- Dải giá trị của toán hạng bị hạn chế

Mã thao tác		Toán hạng
-------------	--	-----------

➤ Định địa chỉ thanh ghi (Register Addressing)

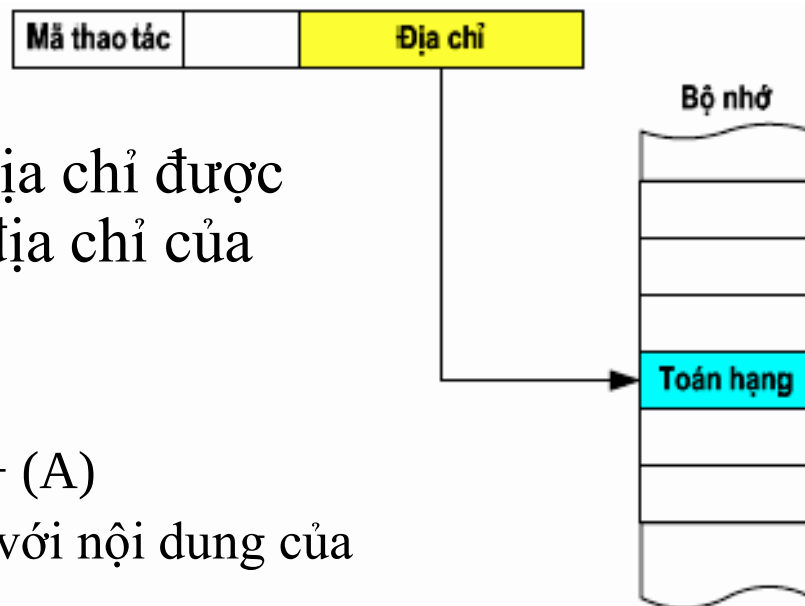
- Toán hạng được chứa trong thanh ghi có tên trong vùng địa chỉ
- Ví dụ:

$$\text{ADD R1, R2 ; R1} \leftarrow \text{R1} + \text{R2}$$
- Số lượng thanh ghi ít $\rightarrow$ vùng địa chỉ cần ít bit hơn
- Không tham chiếu bộ nhớ
- Truy cập toán hạng nhanh
- Tăng số lượng thanh ghi $\rightarrow$ hiệu quả hơn



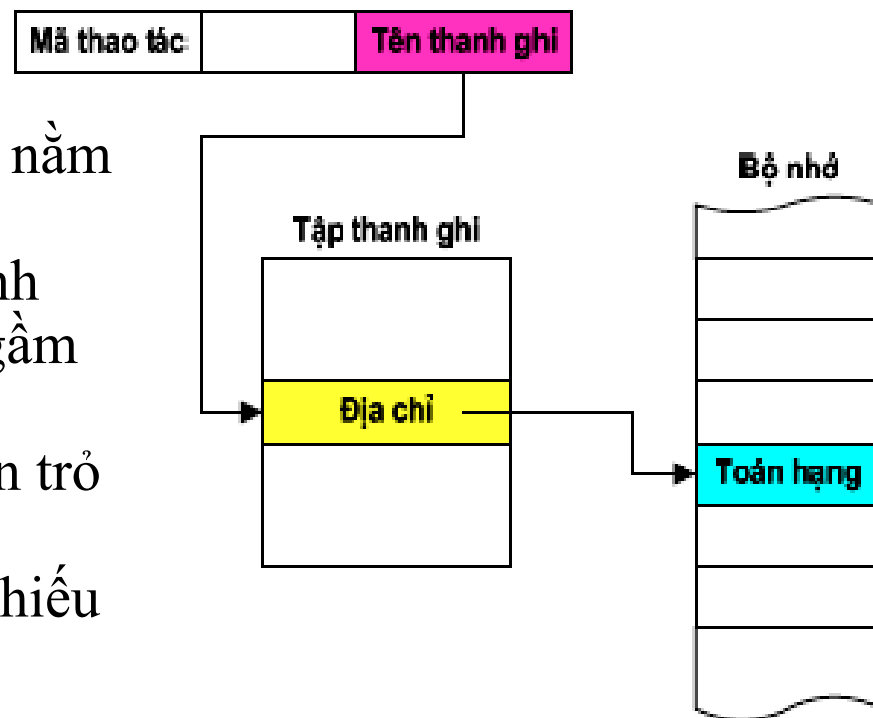
➤ Định địa chỉ trực tiếp (Direct Addressing)

- Toán hạng là ngăn nhớ có địa chỉ được chỉ ra trực tiếp trong vùng địa chỉ của lệnh
- Ví dụ:
 - $\text{ADD R1, A} \quad ; \text{R1} \leftarrow \text{R1} + (\text{A})$
 - Cộng nội dung thanh ghi R1 với nội dung của ô nhớ có địa chỉ là A
 - Tìm toán hạng trong bộ nhớ ở địa chỉ A
- CPU tham chiếu bộ nhớ một lần để truy cập dữ liệu



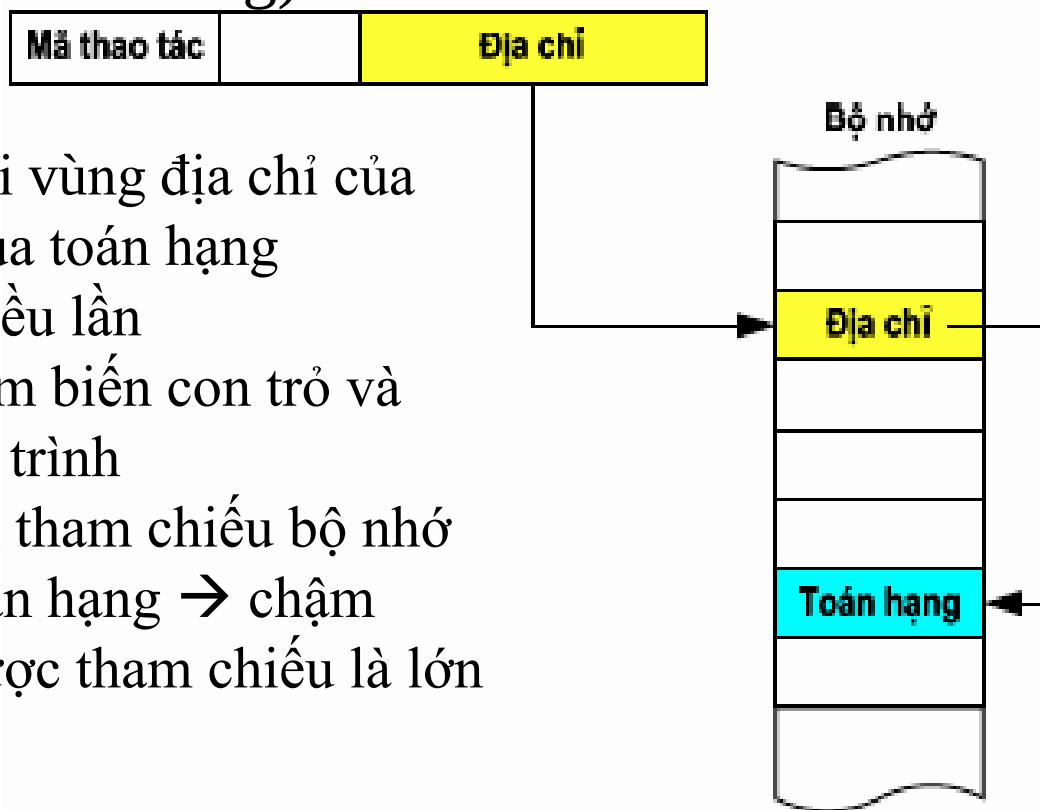
➤ Định địa chỉ gián tiếp qua thanh ghi (Register Indirect Addressing)

- Toán hạng là ô nhớ có địa chỉ nằm trong thanh ghi
- Vùng địa chỉ cho biết tên thanh ghi đó. Thanh ghi có thể là ngầm định
- Thanh ghi này được gọi là con trỏ (pointer)
- Vùng nhớ có thể được tham chiếu là lớn (2^n , với n là độ dài của thanh ghi)



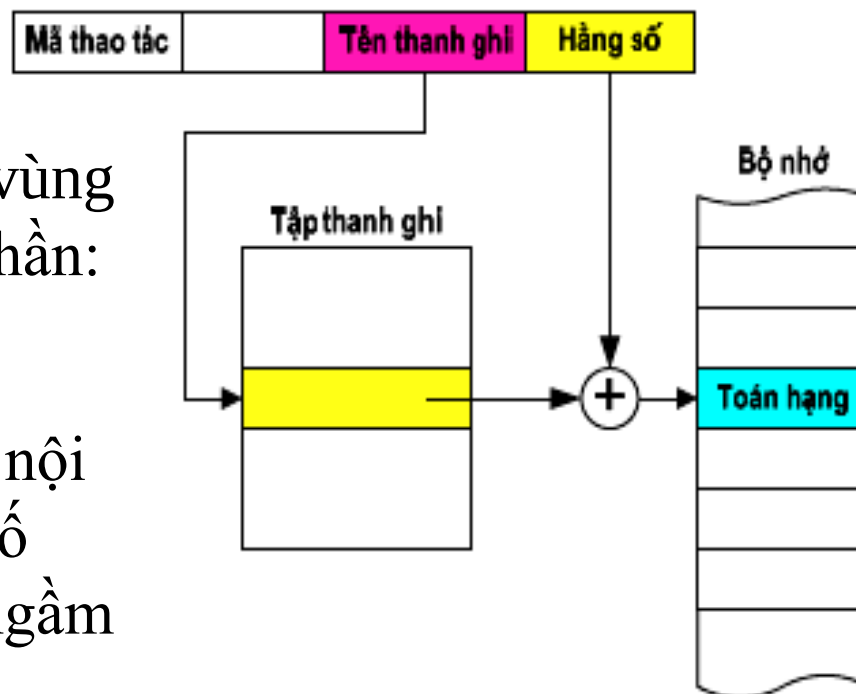
➤ Định địa chỉ gián tiếp qua bộ nhớ (Indirect Memory Addressing)

- Bộ nhớ được trỏ bởi vùng địa chỉ của lệnh chứa địa chỉ của toán hạng
- Có thể gián tiếp nhiều lần
- Giống như khái niệm biến con trỏ và biến động trong lập trình
- CPU phải thực hiện tham chiếu bộ nhớ nhiều lần để tìm toán hạng → chậm
- Vùng nhớ có thể được tham chiếu là lớn



➤ Định địa chỉ dịch chuyển (Displacement Addressing)

- Để xác định toán hạng, vùng địa chỉ chứa hai thành phần:
 - Tên thanh ghi
 - Hằng số
- Địa chỉ của toán hạng = nội dung thanh ghi + hằng số
- Thanh ghi có thể được ngầm định



➤ Định địa chỉ dịch chuyển (tiếp)

○ Các dạng địa chỉ dịch chuyển

- Địa chỉ hoá tương đối với PC
 - Thanh ghi là Bộ đếm chương trình PC
 - Toán hạng có địa chỉ cách ô nhớ được trỏ bởi PC một độ lệch xác định
- Định địa chỉ cơ sở (base)
 - Thanh ghi chứa địa chỉ cơ sở
 - Hằng số là chỉ số
- Định địa chỉ chỉ số (index)
 - Hằng số là địa chỉ cơ sở
 - Thanh ghi chứa chỉ số



4.5 Phân loại tập lệnh

➤ CISC và RISC

- CISC: Complex Instruction Set Computer:
 - Máy tính với tập lệnh đầy đủ
 - Ví dụ: Intel x86, Motorola 680x0
- RISC: Reduced Instruction Set Computer:
 - Máy tính với tập lệnh thu gọn
 - Ví dụ: SunSPARC, Power PC, MIPS, ARM ...
- RISC đối nghịch với CISC

➤ Các đặc trưng của CISC

- Số lượng lệnh nhiều (vài trăm lệnh) → Dễ lập trình, chương trình ngắn hơn (chiếm ít bộ nhớ)
- Truy cập toán hạng ở các thanh ghi lẫn bộ nhớ
- Cấu trúc CPU phức tạp
- Thời gian thực hiện lệnh cần nhiều chu kỳ máy
- Số lượng khuôn dạng lệnh lớn
- CPU có tập thanh ghi nhỏ
- Có nhiều mode địa chỉ
- Một số lệnh không có mạch phần cứng riêng (cần có vi chương trình để thực hiện)

➤ Các đặc trưng của RISC

- Số lượng lệnh ít (vài chục lệnh) và cơ bản nhất → Khó lập trình, chương trình dài hơn
- Hầu hết các lệnh truy cập toán hạng ở các thanh ghi
- Cấu trúc CPU đơn giản
- Thời gian thực hiện lệnh là một chu kỳ máy
- Số lượng khuôn dạng lệnh ít (≤ 4)
- CPU có tập thanh ghi lớn
- Có ít mode địa chỉ (≤ 4)
- Mỗi lệnh có mạch phần cứng riêng (không cần vi chương trình)



➤ So sánh CISC và RISC

Loại	CISC			RISC	
	IBM	DEC VAX	Intel	Motorola	MIPS
Hãng SX					
Hệ thống MT	370/168	11/780	486	88000	R4000
Năm SX	1973	1978	1989	1988	1991
Số lượng lệnh	208	303	235	51	94
Kích thước lệnh (B)	2-6	2-57	1-11	4	32
Addressing modes	4	22	11	3	1
Số lượng thanh ghi	16	16	8	32	32
Vi ChươngTrình (KB)	420	480	246	0	0

➤ Thống kê 10 lệnh Intel x86 sử dụng nhiều nhất

<u>TT</u>	<u>Lệnh</u>	<u>Tỷ lệ (%)</u>
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-register	4%
9	call	1%
10	return	1%
<u>Total</u>		<u>96%</u>

- Tại sao kiến trúc CISC của Intel vẫn sử dụng nhiều?
- Vấn đề tương thích
 - Dễ xây dựng trình dịch (compiler) hơn
 - Phù hợp với nhiều NNLT cấp cao (HLL)
 - Phần mềm có sẵn đang sử dụng nhiều
 - Thực tế hiện nay sử dụng hệ thống tập lệnh lai giữa RISC và CISC
 - Tổ chức bên trong theo RISC
 - Kiến trúc lập trình bên ngoài theo CISC
 - Sử dụng vi chương trình làm trung gian

➤ Ưu nhược điểm của CISC

○ Ưu điểm

- Chương trình ít lệnh hơn, ít tốn bộ nhớ để lưu trữ
- Truy cập bộ nhớ với ít lệnh hơn
- Chương trình dễ viết, dễ đọc và dễ hiểu hơn

○ Nhược điểm

- Dạng lệnh phức tạp, giải mã lệnh chậm
- Lệnh phức tạp nên không uyển chuyển, không áp dụng cho nhiều trường hợp khác nhau
- Xử lý ngắt chậm hơn (do lệnh chiếm nhiều chu kỳ máy) nên thời gian đáp ứng kém



4.6 Kiến trúc tập lệnh Intel x86

Intel Processor	Date of Product Introduction	Performance in MIPS ¹	Max. CPU Frequency at Introduction	No. of Transistors on the Die	Main CPU Register Size ²	Extern. Data Bus Size ²	Max. Extern. Addr. Space	Caches in CPU Package ³
8086	1978	0.8	8 MHz	29 K	16	16	1 MB	None
Intel 286	1982	2.7	12.5 MHz	134 K	16	16	16 MB	Note 3
Intel 386™ DX	1985	6.0	20 MHz	275 K	32	32	4 GB	Note 3
Intel 486™ DX	1989	20	25 MHz	1.2 M	32	32	4 GB	8KB L1
Pentium®	1993	100	60 MHz	3.1 M	32	64	4 GB	16KB L1
Pentium® Pro	1995	440	200 MHz	5.5 M	32	64	64 GB	16KB L1; 256KB or 512KB L2
Pentium II®	1997	466	<u>266</u>	7 M	32	64	64 GB	32KB L1; 256KB or 512KB L2
<u>Pentium® III</u>	<u>1999</u>	<u>1000</u>	<u>500</u>	<u>8.2 M</u>	<u>32 GP</u> <u>128</u> <u>SIMD-FP</u>	<u>64</u>	<u>64 GB</u>	<u>32KB L1;</u> <u>512KB L2</u>



Intel Processor	Date Introduced	Microarchitecture	Clock Frequency at Introduction	Transistors Per Die	Register Sizes ¹	System Bus Bandwidth	Max. Extern. Addr. Space	On-Die Caches ²
Pentium 4 Processor	2000	Intel NetBurst Microarchitecture	1.50 GHz	42 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	64 GB	12K μ op Execution Trace Cache; 8KB L1; 256-KB L2
Intel Xeon Processor	2001	Intel NetBurst Microarchitecture	1.70 GHz	42 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	64 GB	12K μ op Trace Cache; 8-KB L1; 256-KB L2
Intel Xeon Processor	2002	Intel NetBurst Microarchitecture; Hyper-Threading Technology	2.20 GHz	55 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	64 GB	12K μ op Trace Cache; 8-KB L1; 512-KB L2



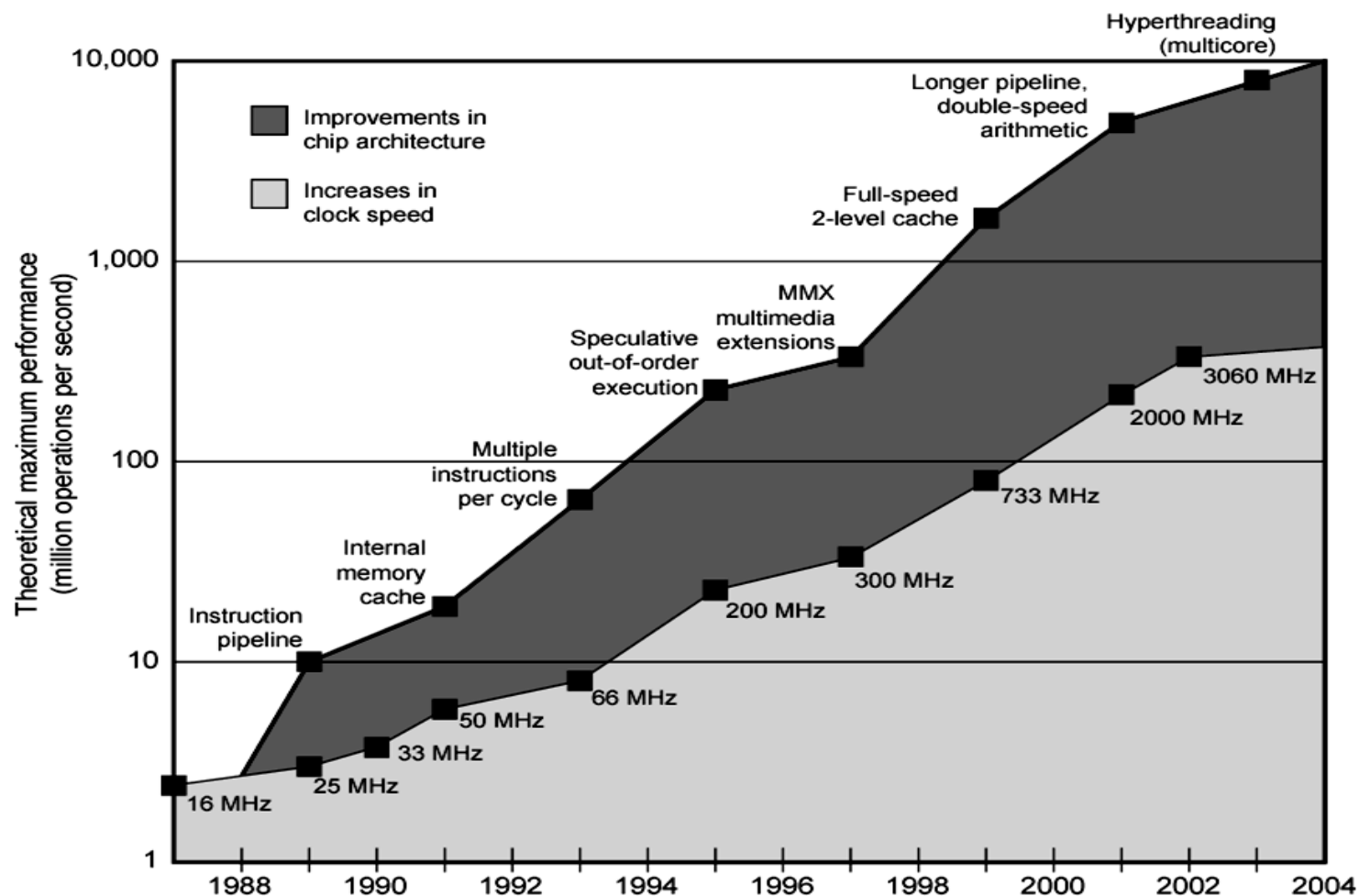
Intel Processor	Date Introduced	Microarchitecture	Clock Frequency at Introduction	Transistors Per Die	Register Sizes ¹	System Bus Bandwidth	Max. Extern. Addr. Space	On-Die Caches ²
Intel Pentium 4 Processor Supporting Hyper-Threading Technology at 90 nm process	2004	Intel NetBurst Microarchitecture; Hyper-Threading Technology	3.40 GHz	125 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	6.4 GB/s	64 GB	12K μ op Execution Trace Cache; 16 KB L1; 1 MB L2
Intel Pentium M Processor 755 ³	2004	Intel Pentium M Processor	2.00 GHz	140 M	GP: 32 FPU: 80 MMX: 64 XMM: 128	3.2 GB/s	4 GB	L1: 64 KB L2: 2MB



Intel Processor	Date Introduced	Micro-architec-ture	Top-Bin Fre-quency at Intro-duction	Tran-sistor s	Register Sizes	System Bus/QP I Link Speed	Max. Extern . Addr. Space	On-Die Caches
Quad-core Intel Xeon Processor 5355	2006	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.66 GHz	582 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	256 GB	L1: 64 KB L2: 4MB (8 MB Total)
Intel Core 2 Duo Processor E6850	2007	Intel Core Microarchitecture; Dual Core; Intel 64 Architecture; Intel Virtualization Technology; Intel Trusted Execution Technology	3.00 GHz	291 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	10.6 GB/s	64 GB	L1: 64 KB L2: 4MB (4MB Total)
Intel Xeon Processor 7350	2007	Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	2.93 GHz	582 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	8.5 GB/s	1024 GB	L1: 64 KB L2: 4MB (8MB Total)
Intel Xeon Processor 5472	2007	Enhanced Intel Core Microarchitecture; Quad Core; Intel 64 Architecture; Intel Virtualization Technology.	3.00 GHz	820 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	12.8 GB/s	256 GB	L1: 64 KB L2: 6MB (12MB Total)



Intel Processor	Date Introduced	Micro-architec-ture	Top-Bin Fre-quency at Intro-duction	Tran-sistor s	Register Sizes	System Bus/QP I Link Speed	Max. Extern . Addr. Space	On-Die Caches
Intel Core i7-620M Processor	2010	Intel Turbo Boost Technology, Intel microarchitecture codename Westmere; Dualcore; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology., Integrated graphics	2.66 GHz	383 M	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128		64 GB	L1: 64 KB L2: 256KB L3: 4MB
Intel Xeon-Processor 7560	2010	Intel Turbo Boost Technology, Intel microarchitecture codename Nehalem; Eight core; HyperThreading Technology; Intel 64 Architecture; Intel Virtualization Technology.	2.26 GHz	2.3B	GP: 32, 64 FPU: 80 MMX: 64 XMM: 128	QPI: 6.4 GT/s; Memory: 50 GB/s	16 TB	L1: 64 KB L2: 256KB L3: 24MB



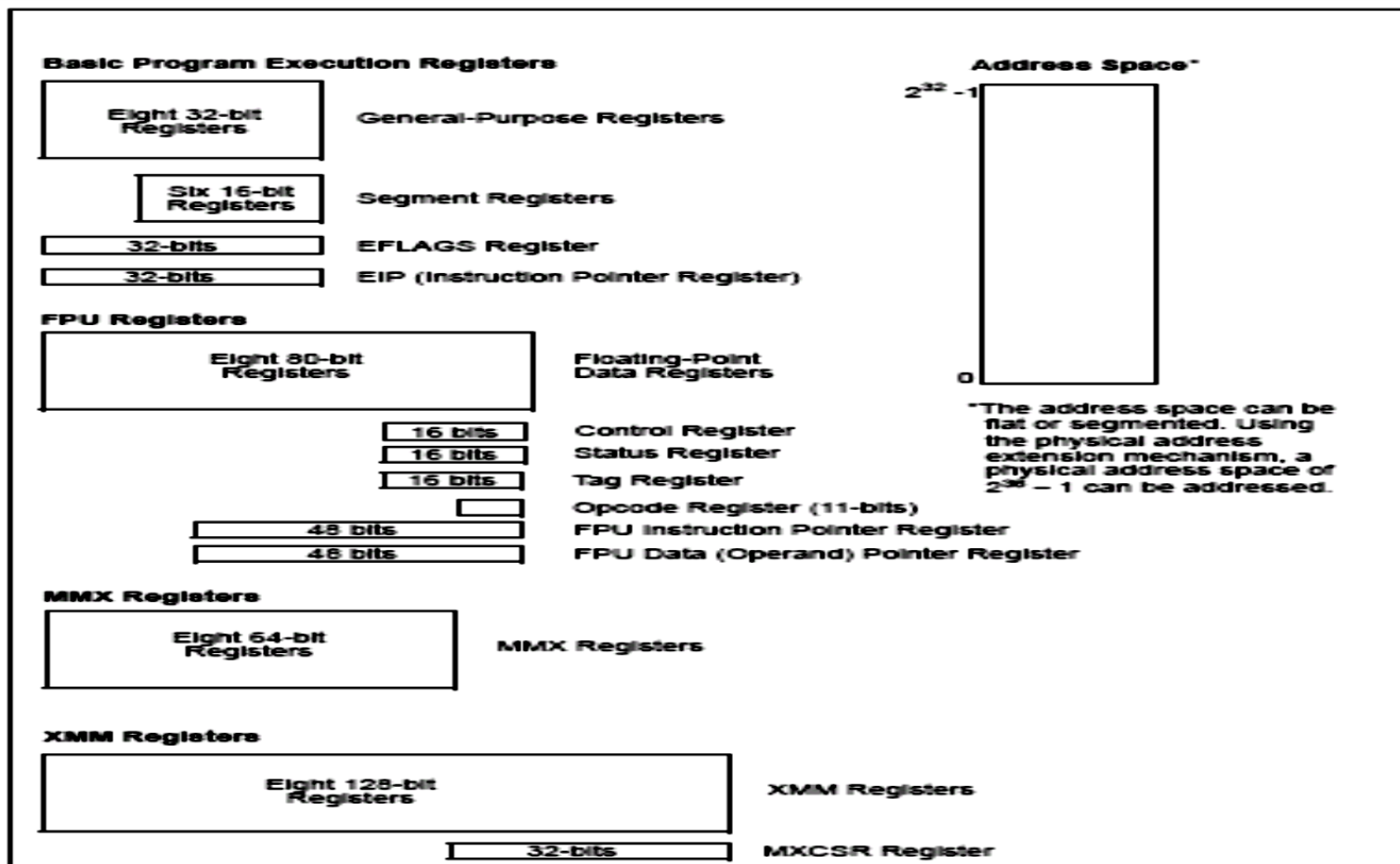


Figure 3-1. IA-32 Basic Execution Environment



Câu hỏi









Chương 5 Bộ xử lý trung tâm (Central Processing Unit)

Tổ chức của CPU
Hoạt động của chu trình lệnh
Đơn vị điều khiển
Kỹ thuật đường ống lệnh
Cấu trúc bộ xử lý tiên tiến



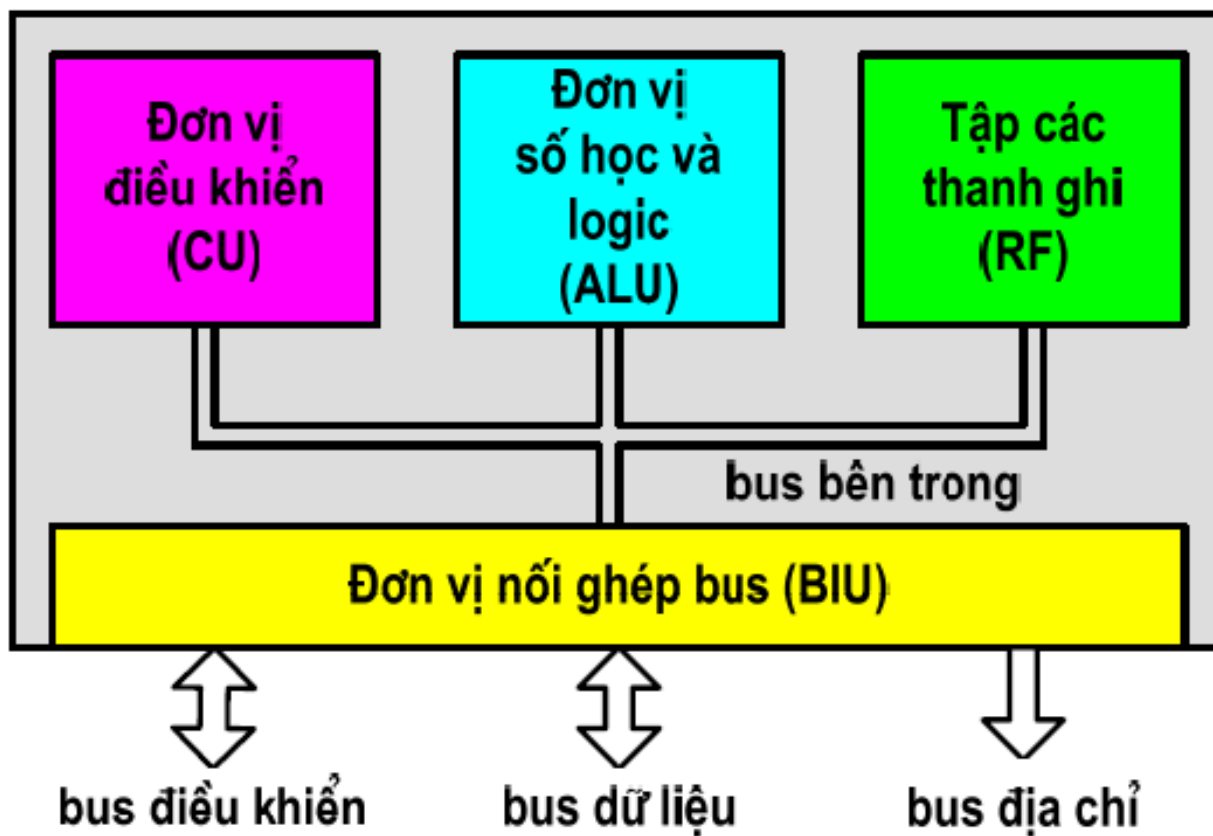
Chương 5 Bộ xử lý trung tâm (Central Processing Unit)

1. Tổ chức của CPU
2. Hoạt động của chu trình lệnh
3. Đơn vị điều khiển
4. Kỹ thuật đường ống lệnh
5. Cấu trúc bộ xử lý tiên tiến



5.1 Tổ chức của CPU

➤ Cấu trúc cơ bản của CPU

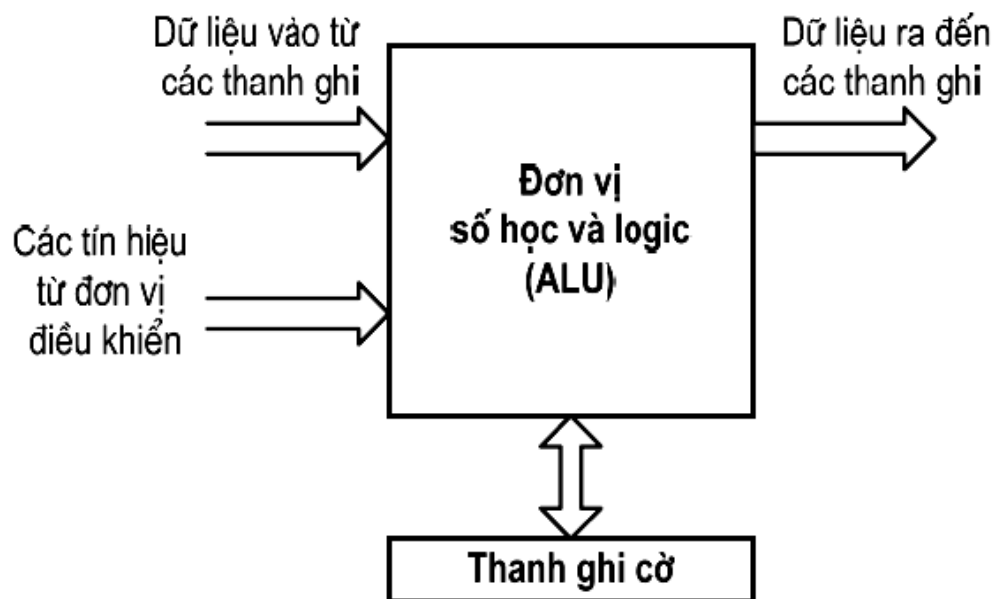


➤ Cấu trúc cơ bản của CPU (tiếp)

- Đơn vị điều khiển (Control Unit - CU): điều khiển hoạt động của máy tính theo chương trình đã định sẵn.
- Đơn vị số học và logic (Arithmetic and Logic Unit - ALU): thực hiện các phép toán số học và phép toán logic.
- Tập thanh ghi (Register File - RF): lưu giữ các thông tin tạm thời phục vụ cho hoạt động của CPU.
- Đơn vị nối ghép bus (Bus Interface Unit - BIU): kết nối và trao đổi thông tin giữa bus bên trong (internal bus) và bus bên ngoài (external bus).

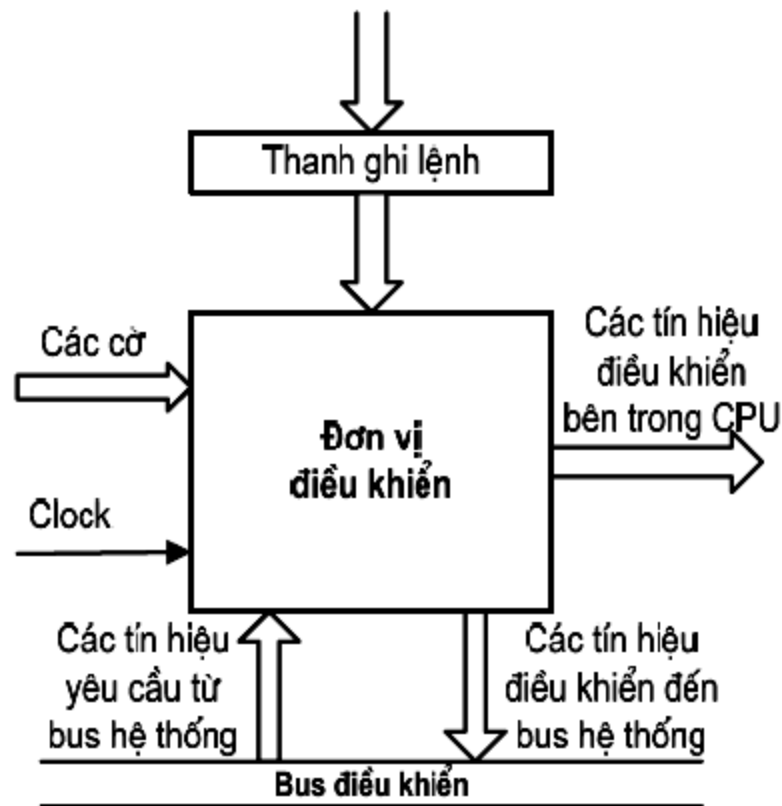
➤ Đơn vị số học và luận lý ALU

- Thực hiện các phép toán số học và phép toán luận lý:
 - Số học: Cộng, trừ, nhân, chia, tăng, giảm, đảo dấu,...
 - Luận lý: AND, OR, XOR, NOT, phép dịch bit,...



➤ Đơn vị điều khiển CU

- Điều khiển nhận lệnh từ bộ nhớ đưa vào thanh ghi lệnh
- Tăng nội dung của PC để trỏ sang lệnh kế tiếp
- Giải mã lệnh đã được nhận để xác định thao tác mà lệnh yêu cầu
- Phát ra các tín hiệu điều khiển thực hiện lệnh
- Nhận các tín hiệu yêu cầu từ bus hệ thống và đáp ứng với các yêu cầu đó.

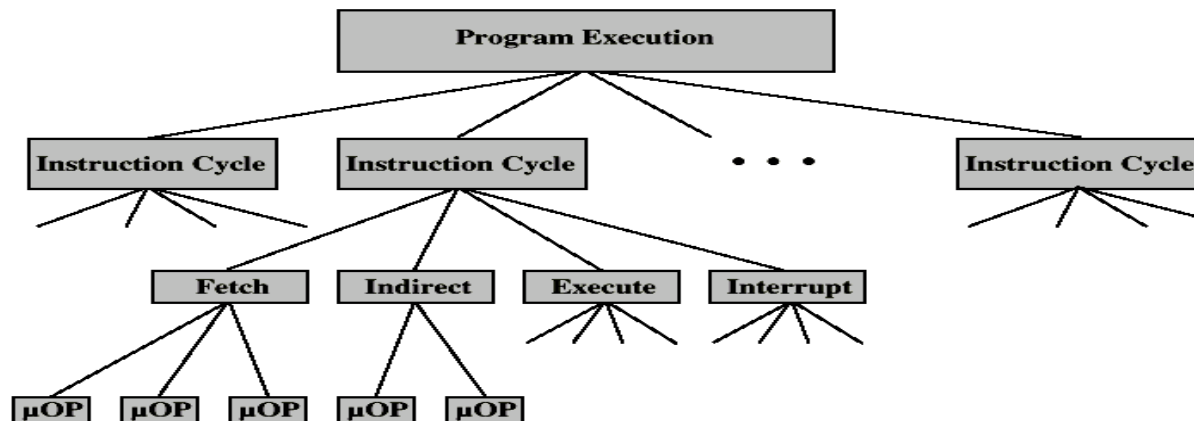


- Các tín hiệu đưa đến đơn vị điều khiển
 - Clock: tín hiệu xung nhịp từ mạch tạo dao động bên ngoài.
 - Mã lệnh từ thanh ghi lệnh đưa đến để giải mã.
 - Các cờ từ thanh ghi cờ cho biết trạng thái của CPU.
 - Các tín hiệu yêu cầu từ bus điều khiển
- Các tín hiệu phát ra từ đơn vị điều khiển
 - Các tín hiệu điều khiển bên trong CPU:
 - Điều khiển các thanh ghi
 - Điều khiển ALU
 - Các tín hiệu điều khiển bên ngoài CPU:
 - Điều khiển bộ nhớ
 - Điều khiển các mô-đun nhập xuất

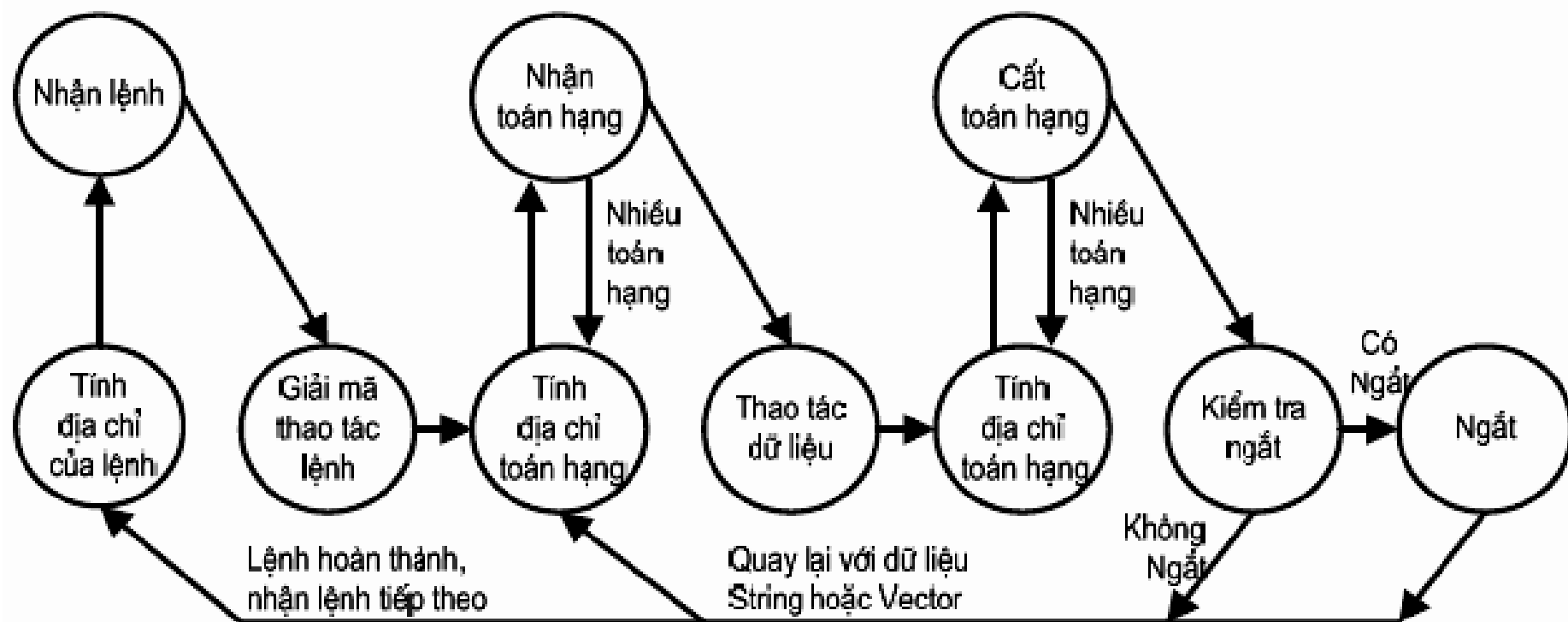
5.2 Hoạt động của chu trình lệnh

➤ Chu trình lệnh

- Nhận lệnh (Fetch Instruction - FI)
- Giải mã lệnh (Decode Instruction - DI)
- Nhận toán hạng (Fetch Operands - FO)
- Thực hiện lệnh (Execute Instruction - EI)
- Cài toán hạng (Write Operands - WO)
- Ngắt (Interrupt Instruction - II)

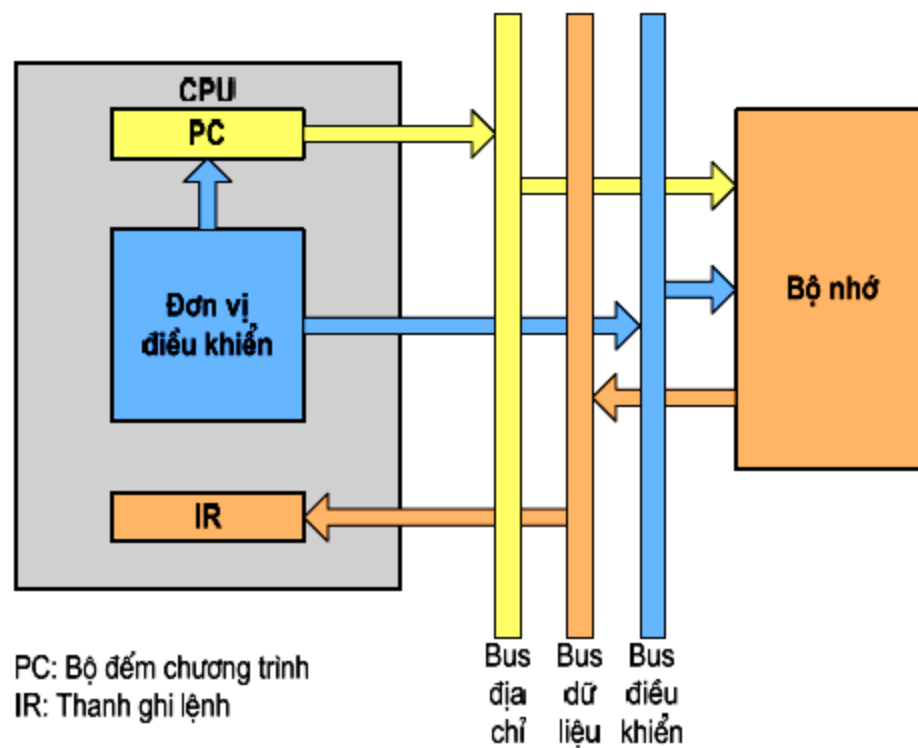


➤ chu trình lệnh (tiếp)



➤ Nhận lệnh (Fetch)

- CPU đưa địa chỉ của lệnh cần nhận từ bộ đếm chương trình PC ra bus địa chỉ
- CPU phát tín hiệu điều khiển đọc bộ nhớ
- Lệnh từ bộ nhớ được đặt lên bus dữ liệu và được CPU chép vào thanh ghi lệnh IR
- CPU tăng nội dung PC để trở sang lệnh kế tiếp



➤ Giải mã lệnh (Decode)

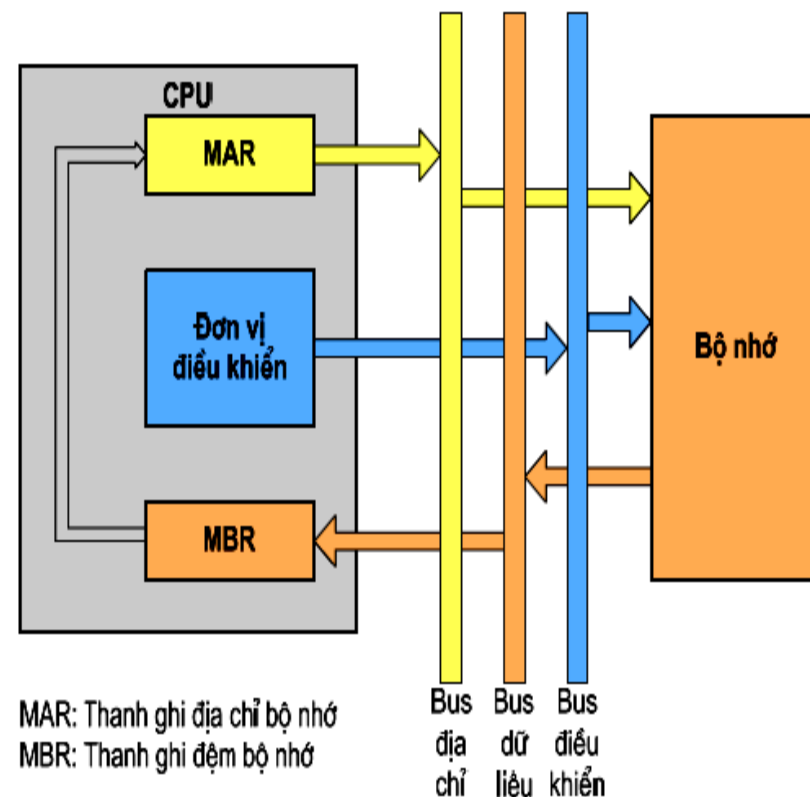
- Lệnh từ thanh ghi lệnh IR được đưa đến đơn vị điều khiển
- Đơn vị điều khiển tiến hành giải mã lệnh để xác định thao tác phải thực hiện
- Giải mã lệnh xảy ra bên trong CPU

➤ Nhận dữ liệu (Fetch Operand)

- CPU đưa địa chỉ của toán hạng ra bus địa chỉ
- CPU phát tín hiệu điều khiển đọc
- Toán hạng được đọc vào CPU
- Tương tự như nhận lệnh

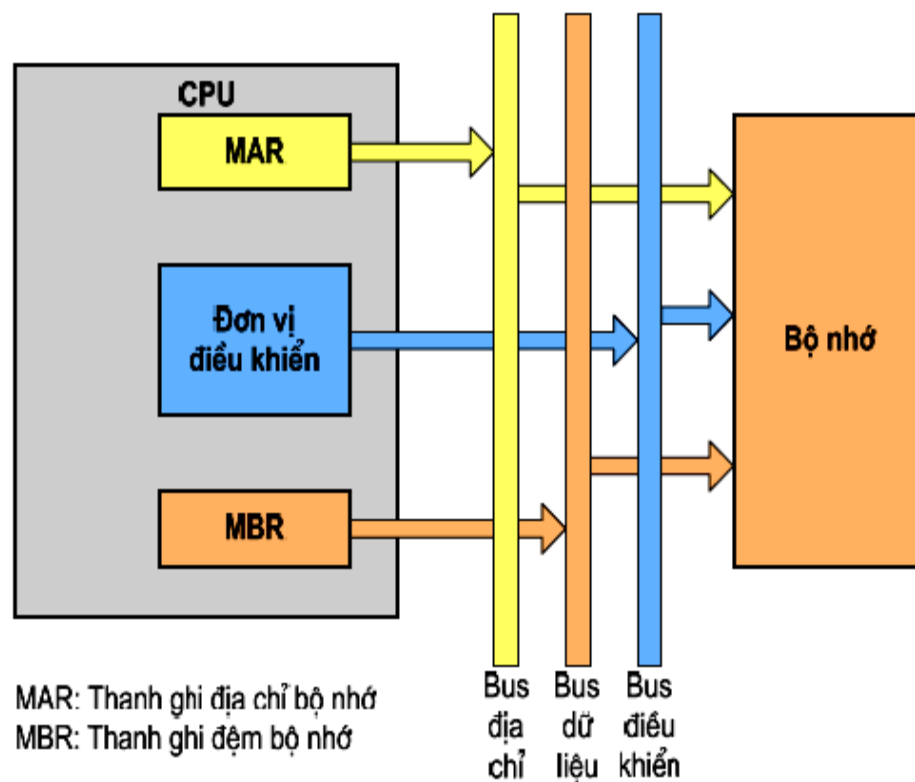
➤ Nhận dữ liệu gián tiếp

- CPU đưa địa chỉ ra bus địa chỉ
- CPU phát tín hiệu điều khiển đọc
- Nội dung ngăn nhớ được đọc vào CPU, đó chính là địa chỉ của toán hạng
- Địa chỉ này được CPU phát ra bus địa chỉ để tìm ra toán hạng
- CPU phát tín hiệu điều khiển đọc
- Toán hạng được đọc vào CPU



➤ Thực hiện lệnh (Execute)

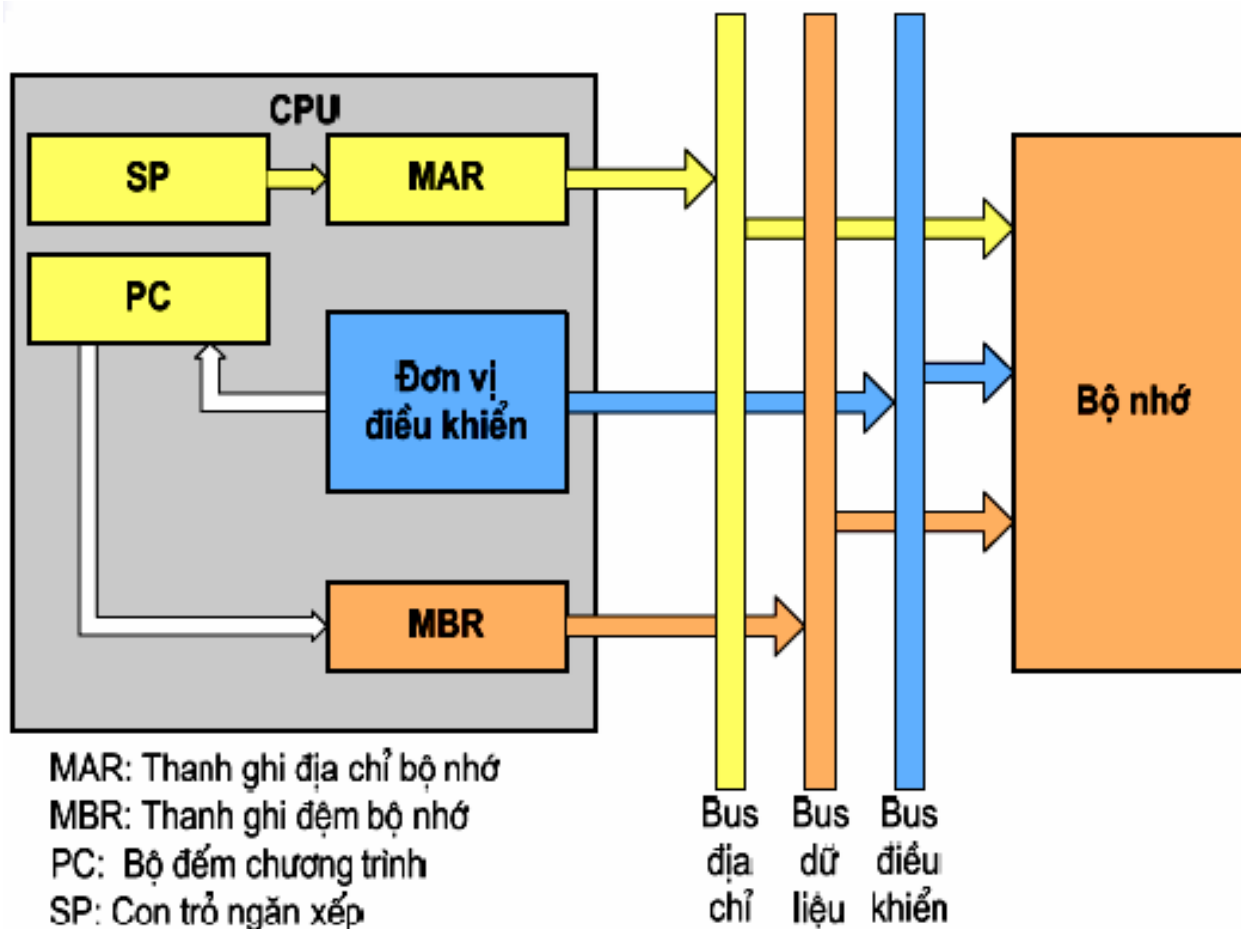
- Có nhiều dạng tùy thuộc vào lệnh
- Có thể là:
 - Đọc/Ghi bộ nhớ
 - Nhập/ xuất
 - Chuyển dữ liệu giữa các thanh ghi với nhau
 - Chuyển dữ liệu giữa thanh ghi và bộ nhớ
 - Thao tác số học/logic
 - Chuyển điều khiển (rẽ nhánh)
 - Ngắt
 - ...



➤ Ngắt (Interrupt)

- Nội dung của bộ đếm chương trình PC (địa chỉ trở về sau khi ngắt) được đưa ra bus dữ liệu
- CPU đưa địa chỉ (thường được lấy từ con trỏ ngăn xếp SP) ra bus địa chỉ
- CPU phát tín hiệu điều khiển ghi bộ nhớ
- Địa chỉ trở về trên bus dữ liệu được ghi ra vị trí xác định (ở ngăn xếp)
- Địa chỉ lệnh đầu tiên của chương trình con điều khiển ngắt được nạp vào PC

➤ Ngắt (tiếp)





5.3 Đơn vị điều khiển

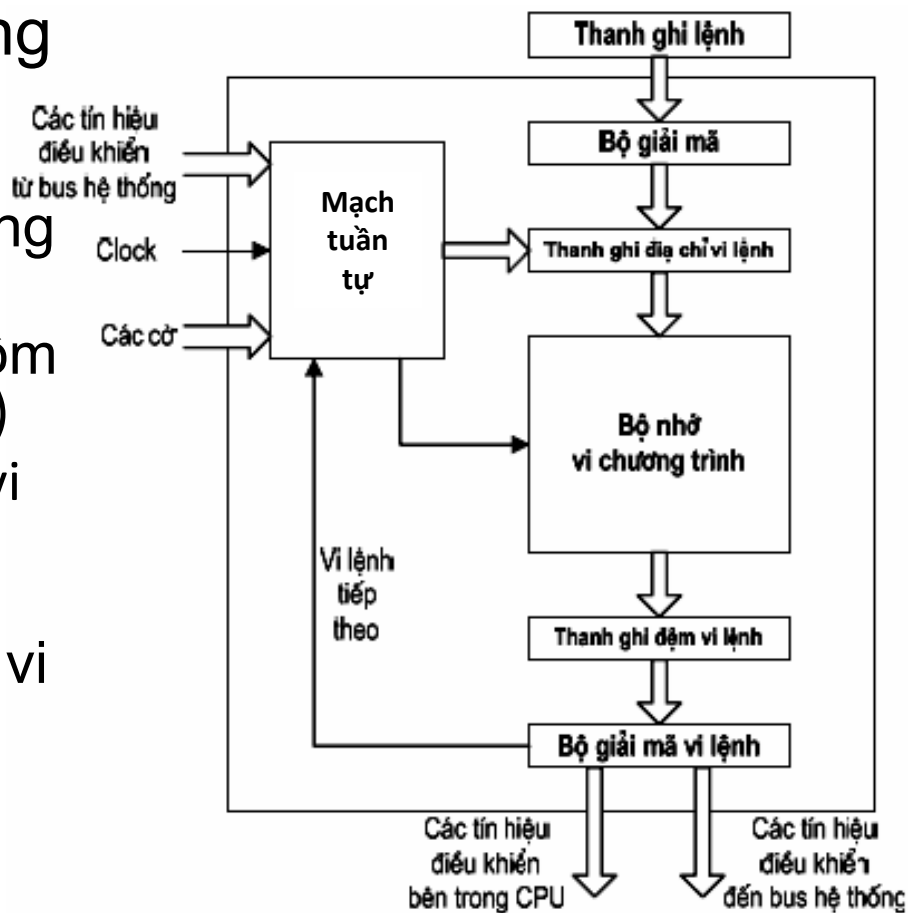
➤ Gồm 2 loại:

- Đơn vị điều khiển vi chương trình
(Microprogrammed Control Unit)
- Đơn vị điều khiển phần cứng
(Hardwired Control Unit)



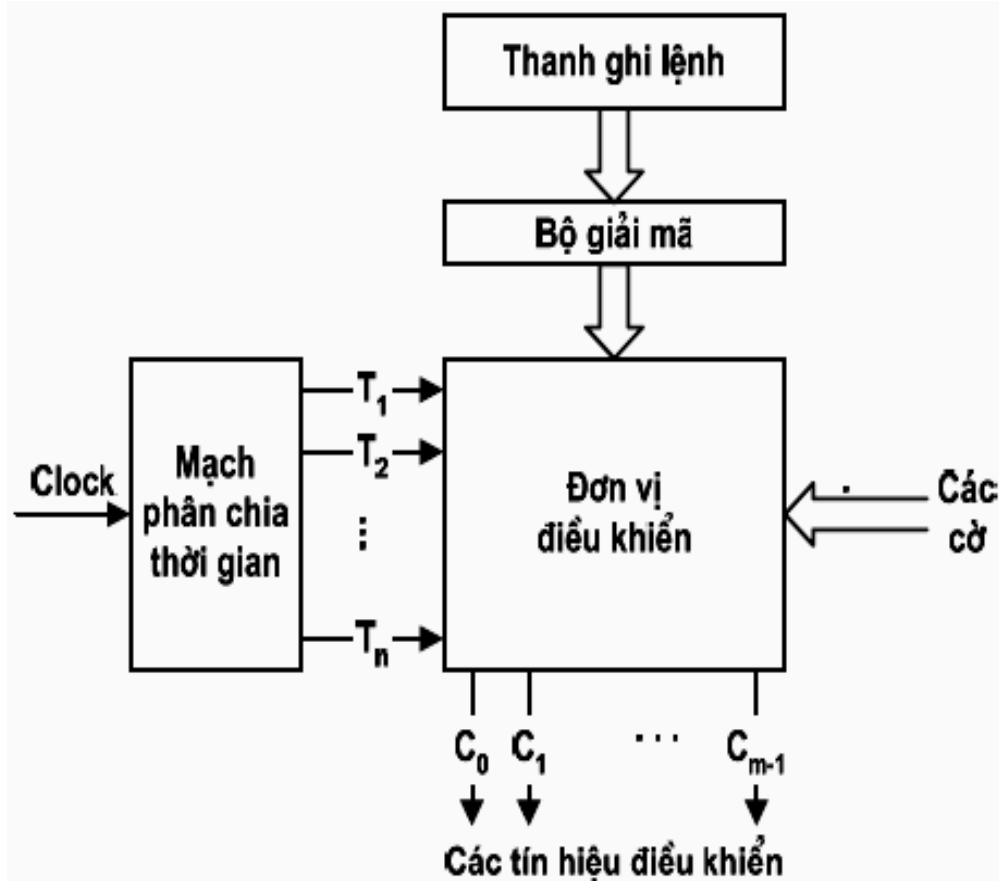
Đơn vị điều khiển vi chương trình

- Bộ nhớ vi chương trình (ROM) lưu trữ các vi chương trình (microprogram)
- Một vi chương trình bao gồm các vi lệnh (microinstruction)
- Mỗi vi lệnh mã hoá cho một vi thao tác (microoperation)
- Để hoàn thành một lệnh cần thực hiện một hoặc một vài vi chương trình
- Tốc độ chậm



➤ Đơn vị điều khiển phần cứng

- Sử dụng vi mạch phần cứng để giải mã và tạo các tín hiệu điều khiển thực hiện lệnh
- Tốc độ nhanh
- Đơn vị điều khiển phức tạp





5.4 Kỹ thuật đường ống lệnh

➤ Khái niệm

- Mỗi chu trình lệnh cần thực hiện bằng nhiều thao tác
- Kỹ thuật đơn hướng (Scalar): Thực hiện tuần tự từng thao tác cho mỗi lệnh → chậm
- Kỹ thuật đường ống (Pipeline): Thực hiện song song các thao tác cho nhiều lệnh đồng thời → nhanh hơn
- Ví dụ chu trình 1 lệnh gồm 5 bước:
 - Nhận lệnh (I)
 - Giải mã lệnh (D)
 - Nhận toán hạng (F)
 - Thực hiện lệnh (E)
 - Cát toán hạng (W)



So Sánh scala và pipeliner

➤ Scalar

Nhiều chu kỳ máy
cho 1 lệnh

Chu kỳ	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Lệnh 1	I	D	F	E	W										
Lệnh 2						I	D	F	E	W					
Lệnh 3											I	D	F	E	W

➤ Pipeline

Mỗi chu kỳ
máy thực hiện
xong 1 lệnh

Chu kỳ	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Lệnh 1	I	D	F	E	W										
Lệnh 2		I	D	F	E	W									
Lệnh 3			I	D	F	E	W								
Lệnh 4				I	D	F	E	W							
Lệnh 5					I	D	F	E	W						
Lệnh 6						I	D	F	E	W					
Lệnh 7							I	D	F	E	W				
Lệnh 8								I	D	F	E	W			
Lệnh 9									I	D	F	E	W		
Lệnh 10										I	D	F	E	W	
Lệnh 11											I	D	F	E	W

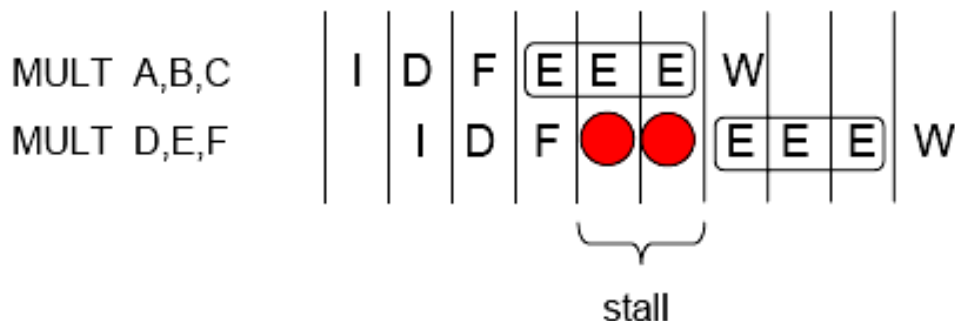
Các trở ngại của đường ống lệnh

- Thực tế không thể luôn đạt 1 chu kỳ máy/lệnh do các trở ngại dẫn đến sự gián đoạn của ống lệnh
- Trở ngại cấu trúc: do nhiều công đoạn dùng chung một tài nguyên
- Trở ngại dữ liệu: lệnh sau sử dụng dữ liệu kết quả của lệnh trước
- Trở ngại điều khiển: do các lệnh rẽ nhánh gây ra



Trở ngại về cấu trúc

- Nguyên nhân: Dùng chung tài nguyên
- Khắc phục:
 - Nhân tài nguyên để tránh xung đột
 - Làm trễ
- Ví dụ 1: Bus dữ liệu truyền lệnh và dữ liệu → Bus lệnh riêng, bus dữ liệu riêng (cache lệnh và cache dữ liệu)
- Ví dụ 2: Lệnh nhân cần nhiều chu kỳ thực thi (E)

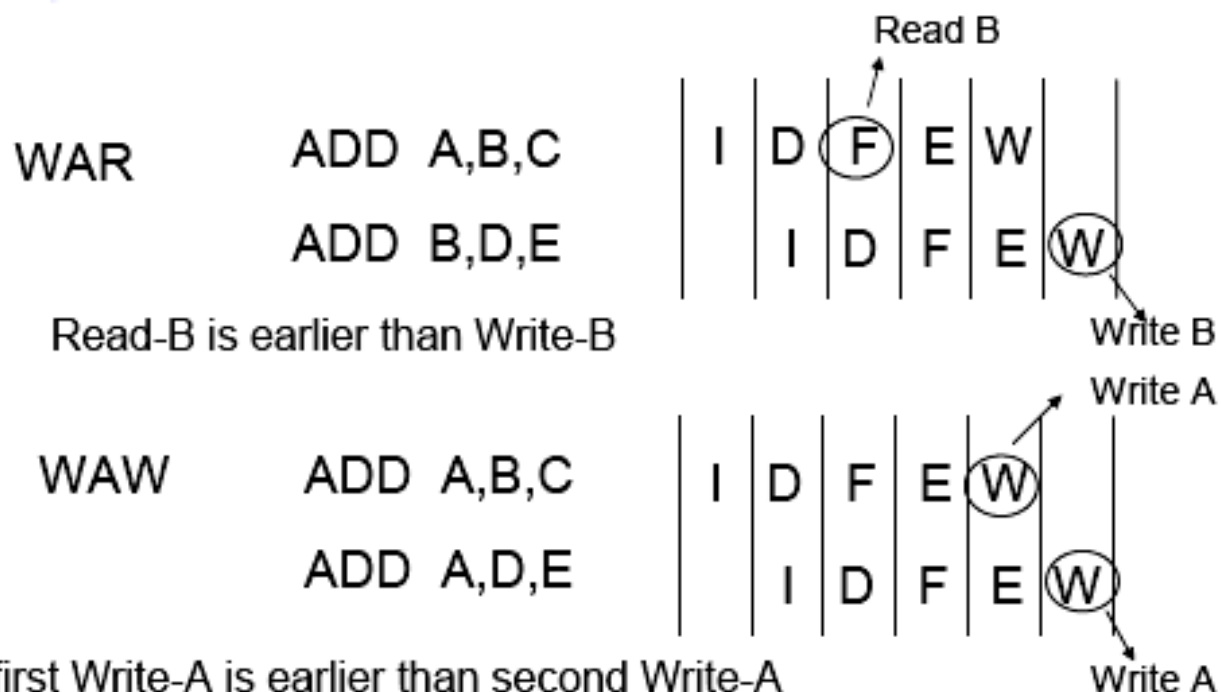




Trở ngại về dữ liệu

- Nguyên nhân: lệnh sau sử dụng dữ liệu kết quả của lệnh trước
- Các dạng:

RAW	ADD A,B,C ADD E,A,D	Write-A must be earlier than Read-A
WAR	ADD A,B,C ADD B,D,E	Read-B must be earlier than Write-B
WAW	ADD A,B,C ADD A,D,E	First Write-A must be earlier Than second Write-A



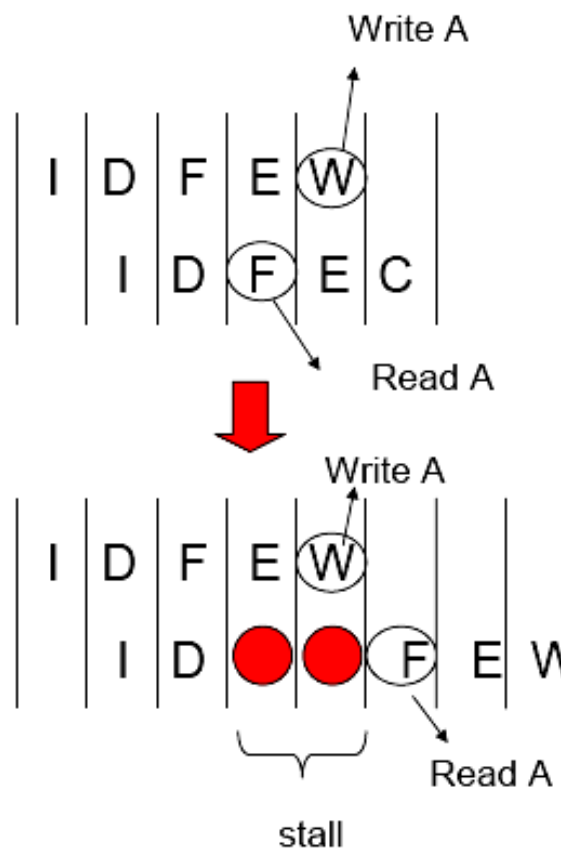
no conflict at in-order pipeline
conflict at out-of-order pipeline

○ RAW

ADD A,B,C

ADD E,A,D

Write-A must be earlier
Than Read-A





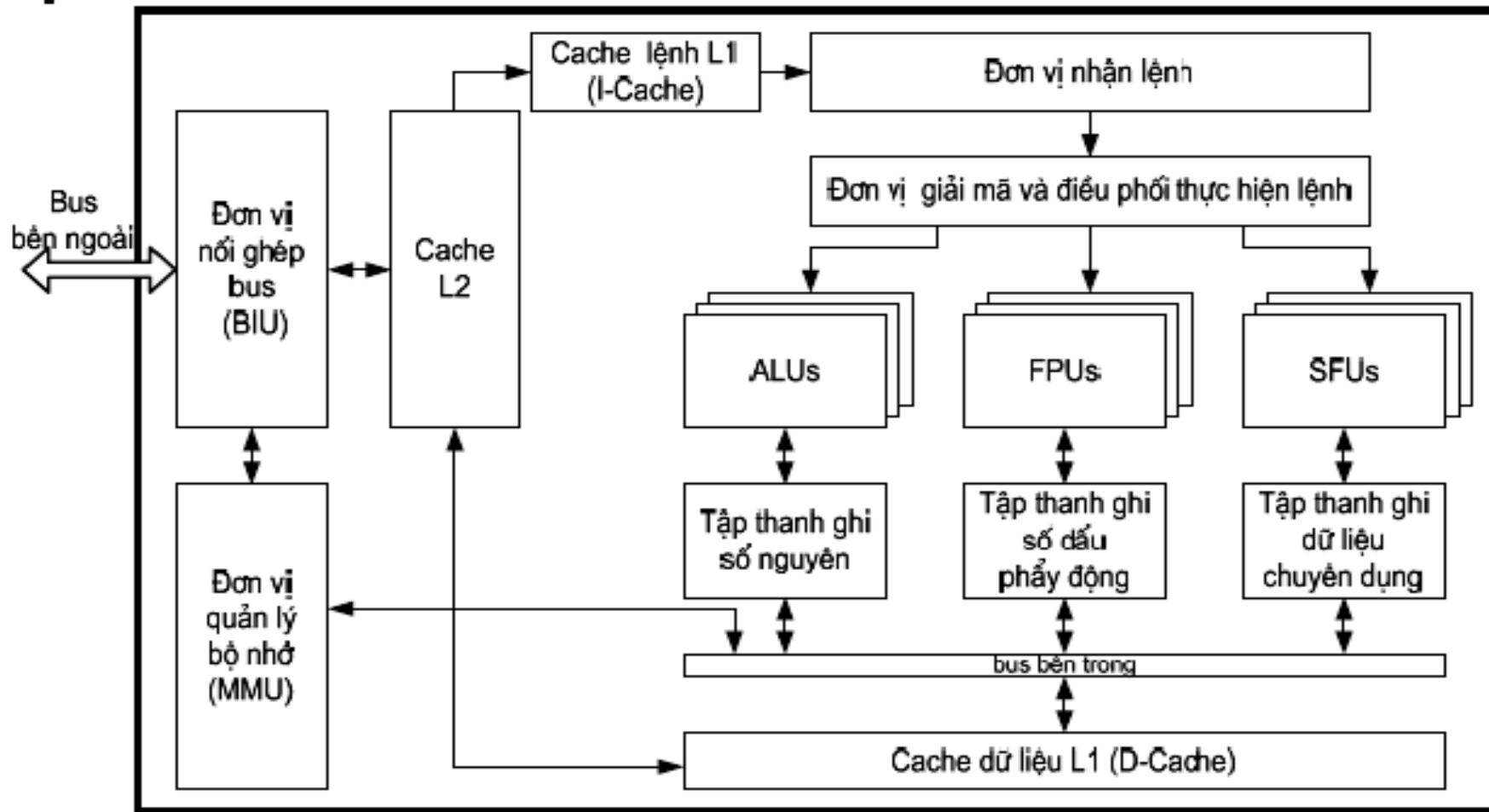
Trở ngại về điều khiển

- Do lệnh rẽ nhánh gây ra
- Đây là dạng trở ngại gây thiệt hại nhiều nhất cho ống lệnh: toàn bộ các lệnh đang thực thi trong ống phải huỷ

Chu kỳ	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Lệnh 1	I	D	F	E	W										
Lệnh 2		I	D	F	E				BRA 25 IF Zero						
Lệnh 3			I	D	F										
Lệnh 4				I	D										
Lệnh 5					I										
Lệnh 25						I	D	F	E	W					
Lệnh 26							I	D	F	E	W				
Lệnh 27								I	D	F	E	W			



5.5 Cấu trúc bộ xử lý tiên tiến



- Các đơn vị xử lý dữ liệu chuyên dụng
 - Các đơn vị số nguyên (ALU)
 - Các đơn vị số dấu chấm động (FPU)
 - Các đơn vị chức năng đặc biệt (SFU)
 - Đơn vị xử lý dữ liệu âm thanh
 - Đơn vị xử lý dữ liệu hình ảnh
 - Đơn vị xử lý dữ liệu vector
- Mục đích: Tăng khả năng xử lý các chức năng chuyên biệt

➤ Bộ nhớ cache

- Được tích hợp trên chip vi xử lý
- Bao gồm hai đến ba mức cache
- Cache L1 gồm hai phần tách rời:
 - Cache lệnh (Instruction cache)
 - Cache dữ liệu (Data cache)

→ Giải quyết xung đột khi nhận lệnh và dữ liệu
- Cache L2 và L3: chung cho lệnh và dữ liệu

➤ Mục đích: Tăng hiệu suất truy cập bộ nhớ chính

- Đơn vị quản lý bộ nhớ
 - Thường gọi là đơn vị MMU (Memory Management Unit) dùng để quản lý bộ nhớ ảo
 - Chuyển đổi địa chỉ ảo (trong chương trình) thành địa chỉ vật lý (trong bộ nhớ)
 - Cung cấp cơ chế phân trang/phân đoạn
 - Cung cấp chế độ bảo vệ bộ nhớ
- Mục đích : Tăng dung lượng bộ nhớ chính bằng cách sử dụng bộ nhớ phụ

➤ Các kiến trúc máy tính song song

- Nhu cầu giải các bài toán lớn ngày càng nhiều, cần những máy tính cực mạnh có khả năng xử lý tốc độ cao
- Kiến trúc máy tính tuần tự (Von-Neumann) tiến đến giới hạn tốc độ, một bộ xử lý duy nhất khó nâng cao hơn nữa khả năng xử lý
- Các kiến trúc máy tính song song giúp tăng hiệu suất tính toán cho máy tính:
 - Kiến trúc song song mức lệnh IPL (Instruction-level parallelism) : Tăng số lượng lệnh thi hành được trên cùng 1 đơn vị thời gian
 - Kiến trúc song song mức xử lý (Machine parallelism) : Tăng số lượng đơn vị xử lý phần cứng
- Cần kết hợp cả 2 kiến trúc song song để tạo ra các máy tính có hiệu suất cao

➤ Kiến trúc song song mức lệnh

○ Siêu đường ống (Superpipeline)

- Chia mỗi thao tác trong chu trình lệnh ra n bước nhỏ → ống lệnh dài hơn
- Cần $1/n$ chu kỳ máy cho mỗi thao tác

○ Siêu hướng (Superscalar)

- Sử dụng nhiều ống lệnh → CPU gồm nhiều đơn vị chức năng, cho phép thi hành nhiều lệnh đồng thời
- Mỗi chu kỳ máy thực hiện được nhiều lệnh

○ VLIW (Very Long Instruction Word)

- Ghép nhiều lệnh đơn vào 1 từ máy để thực hiện đồng thời
- Ví dụ : CPU Itanium họ IA-64 của Intel cho phép ghép 3 lệnh/từ máy gọi là bundle gồm 128 bit

➤ Superpipeline

Chu kỳ	1		2		3		4		5		6		7	
Lệnh 1	I1	I2	D1	D2	F1	F2	E1	E2	W1	W2				
Lệnh 2		I1	I2	D1	D2	F1	F2	E1	E2	W1	W2			
Lệnh 3			I1	I2	D1	D2	F1	F2	E1	E2	W1	W2		
Lệnh 4				I1	I2	D1	D2	F1	F2	E1	E2	W1	W2	
Lệnh 5					I1	I2	D1	D2	F1	F2	E1	E2	W1	W2

➤ Superscalar

Chu kỳ	1	2	3	4	5	6	7	8	9
Lệnh 1	I	D	F	E	W				
Lệnh 2	I	D	F	E	W				
Lệnh 3		I	D	F	E	W			
Lệnh 4		I	D	F	E	W			
Lệnh 5			I	D	F	E	W		
Lệnh 6			I	D	F	E	W		
Lệnh 7				I	D	F	E	W	
Lệnh 8				I	D	F	E	W	
Lệnh 9					I	D	F	E	W
Lệnh 10					I	D	F	E	W

○ VLIW

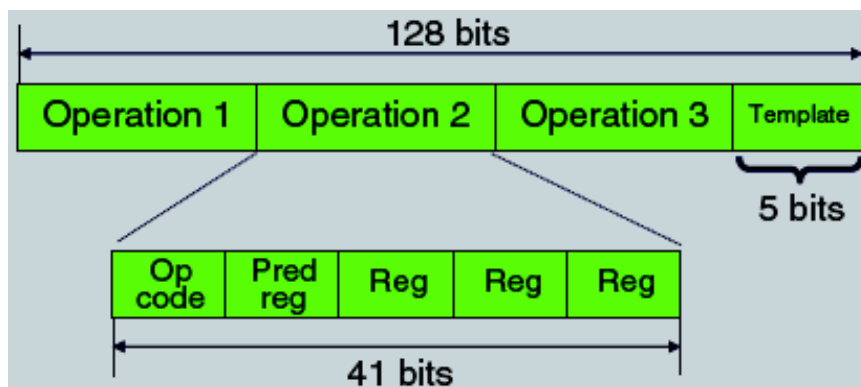
Từ lệnh thông thường



Từ lệnh dài



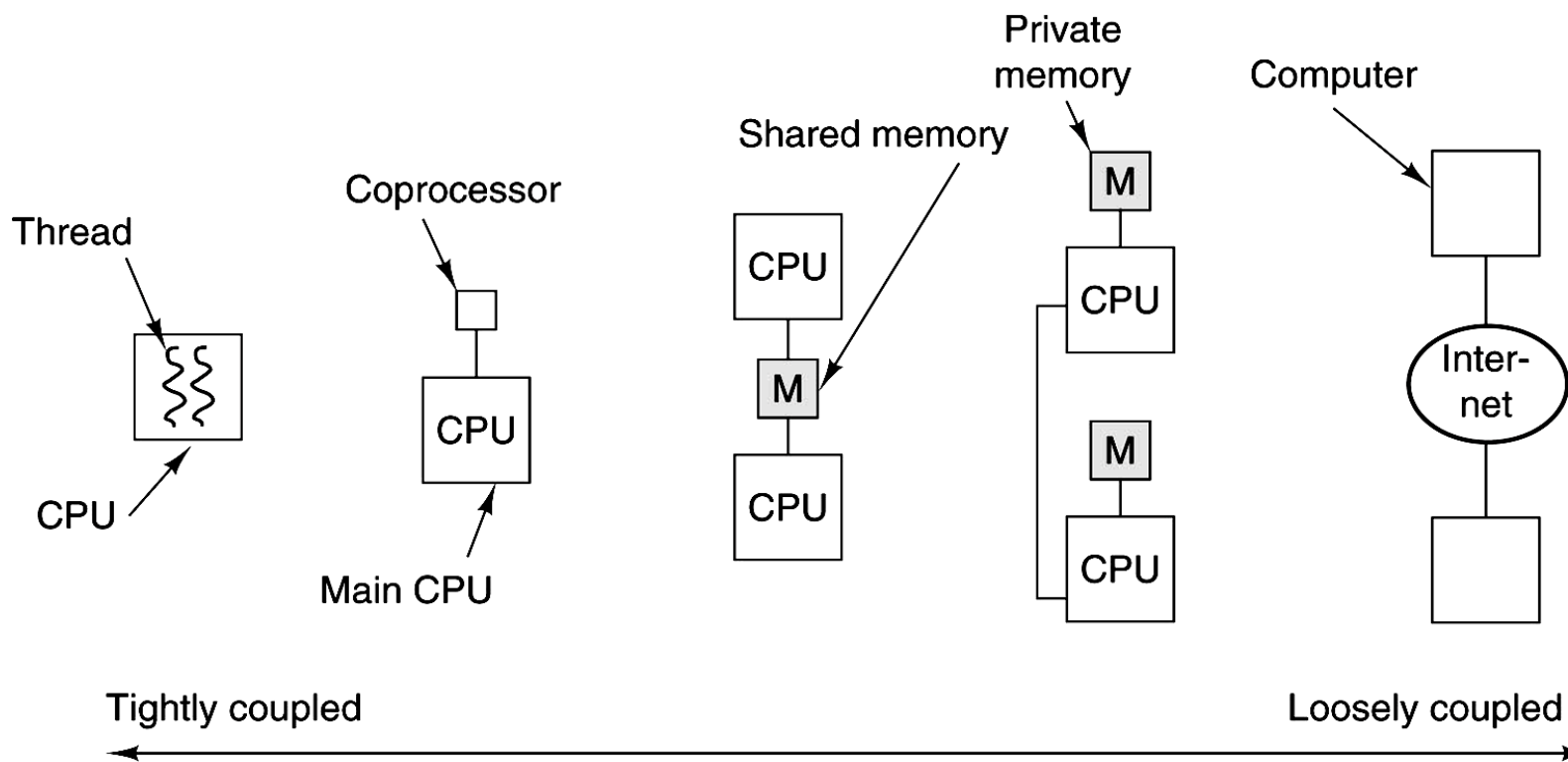
○ Ví dụ: Khuôn dạng lệnh của CPU Intel Itanium



➤ Kiến trúc song song mức xử lý

- Tích hợp nhiều bộ xử lý đồng thời để tăng khả năng thi hành chương trình
- Các xu hướng phát triển:
 - Đa chương (multi-programming)
 - Đa luồng (multi-threading)
 - Đa nhân (multi-core)
 - Đa xử lý (multi-processing)
 - Đa máy tính (multi-computer)

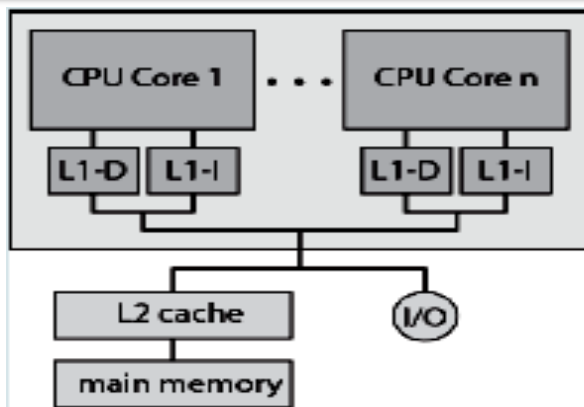
○ Kiến trúc song song mức xử lý (tiếp)



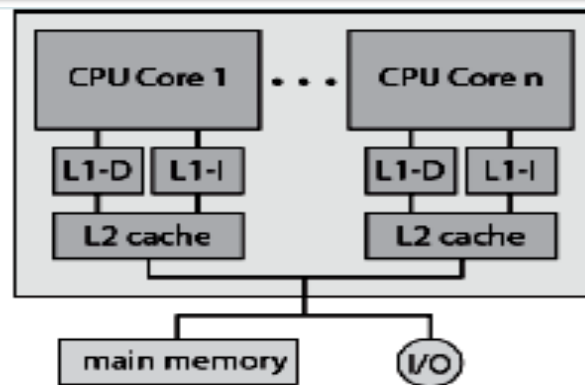
(a) On-chip parallelism (b) Coprocessor (c) Multiprocessor (d) Multicomputer (e) Grid



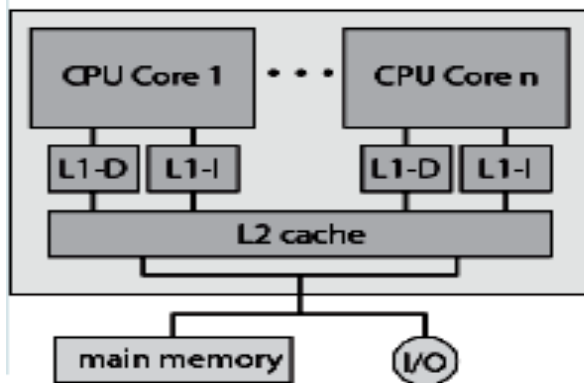
Multi-core



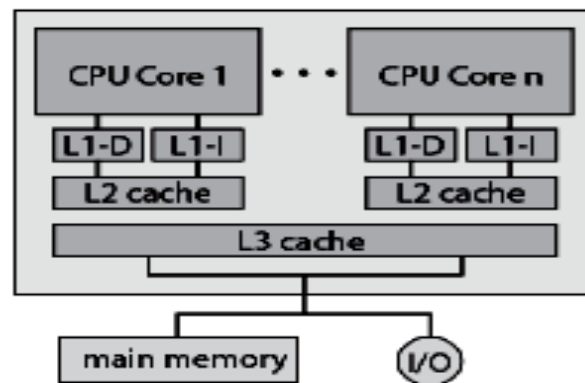
(a) Dedicated L1 cache



(b) Dedicated L2 cache



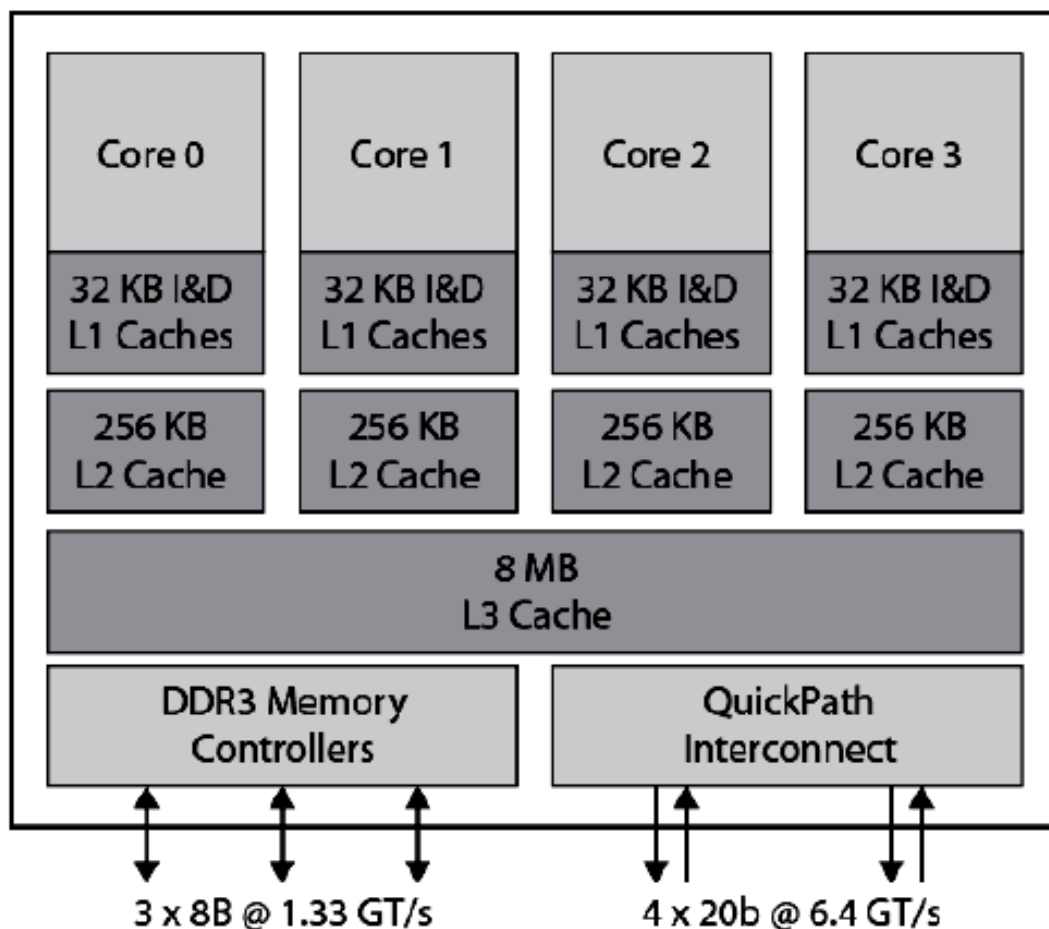
(c) Shared L2 cache



(d) Shared L3 cache



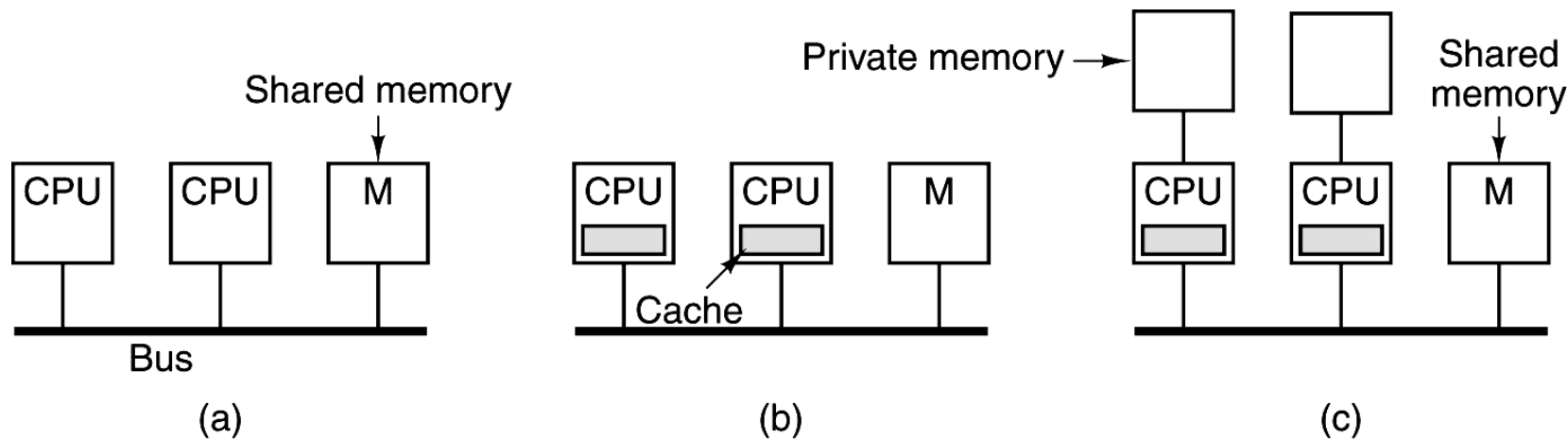
Ví dụ : CPU Intel Core i7 gồm 4 nhân





Multi-processor

- Sử dụng bus chung hoặc switch
- Sử dụng bộ nhớ chung hoặc riêng biệt

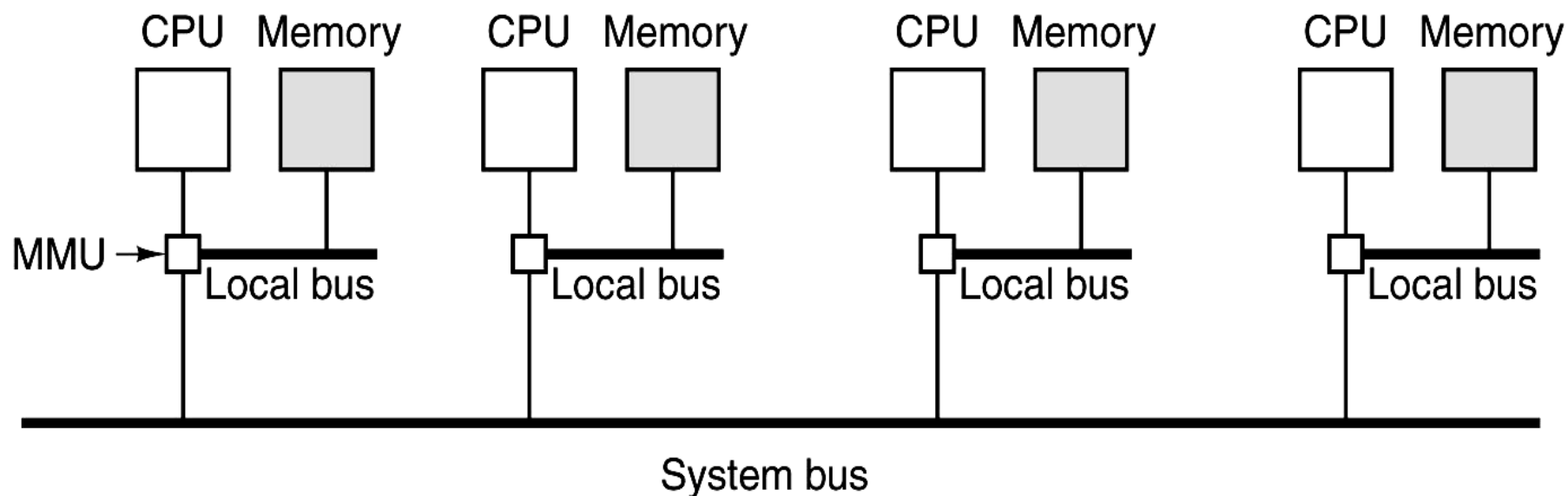


Sơ đồ UMA (Uniform Memory Access) dùng bus chung và bộ nhớ chung

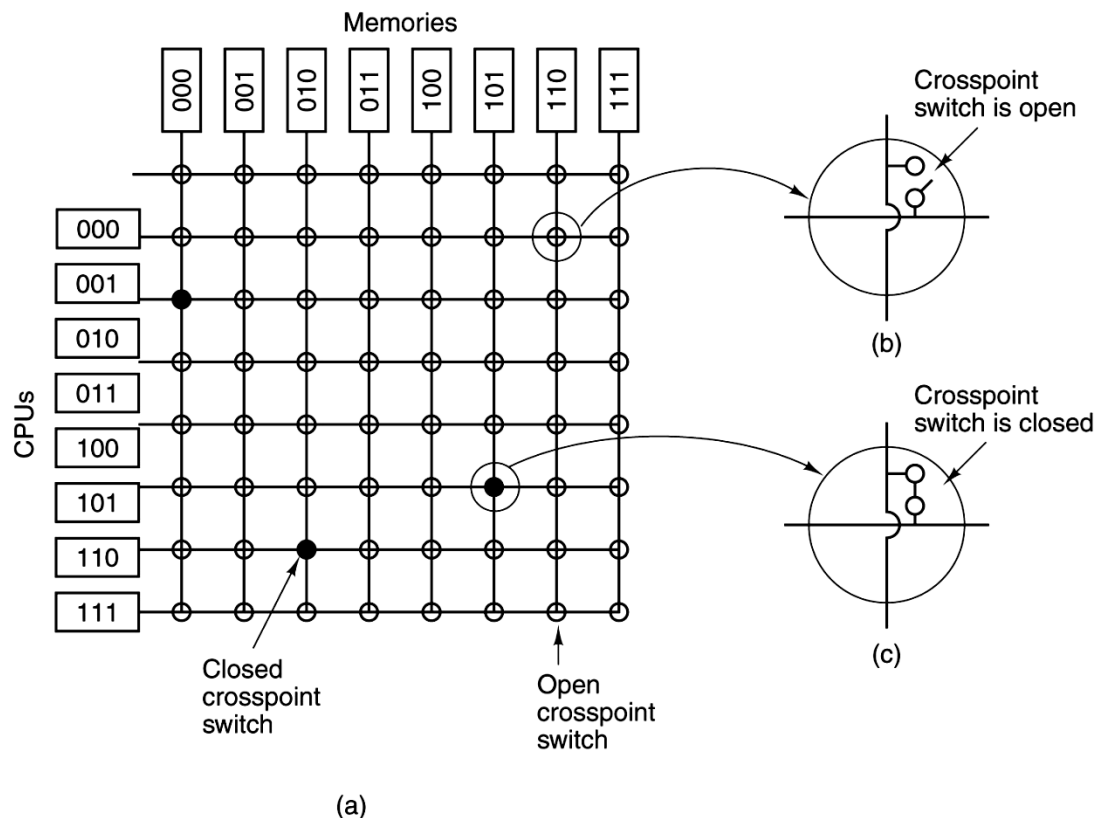


Multi-processor (tiếp)

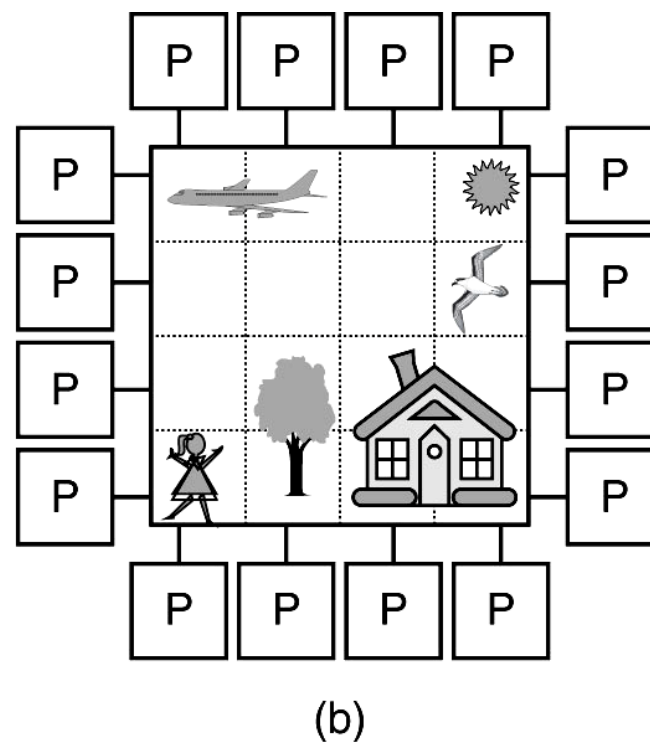
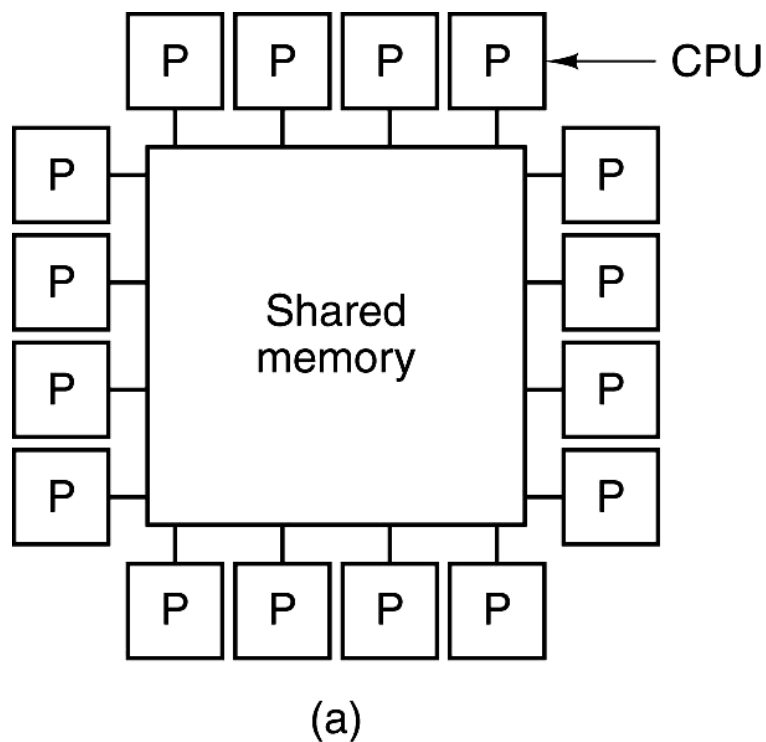
- Sơ đồ NUMA (Non-Uniform Memory Access) dùng bus chung và bộ nhớ riêng



- Sơ đồ UMA (Uniform Memory Access) dùng switch và bộ nhớ riêng
- Còn gọi là hệ thống đa xử lý đối xứng SMP (Symmetric Multi-Processors)

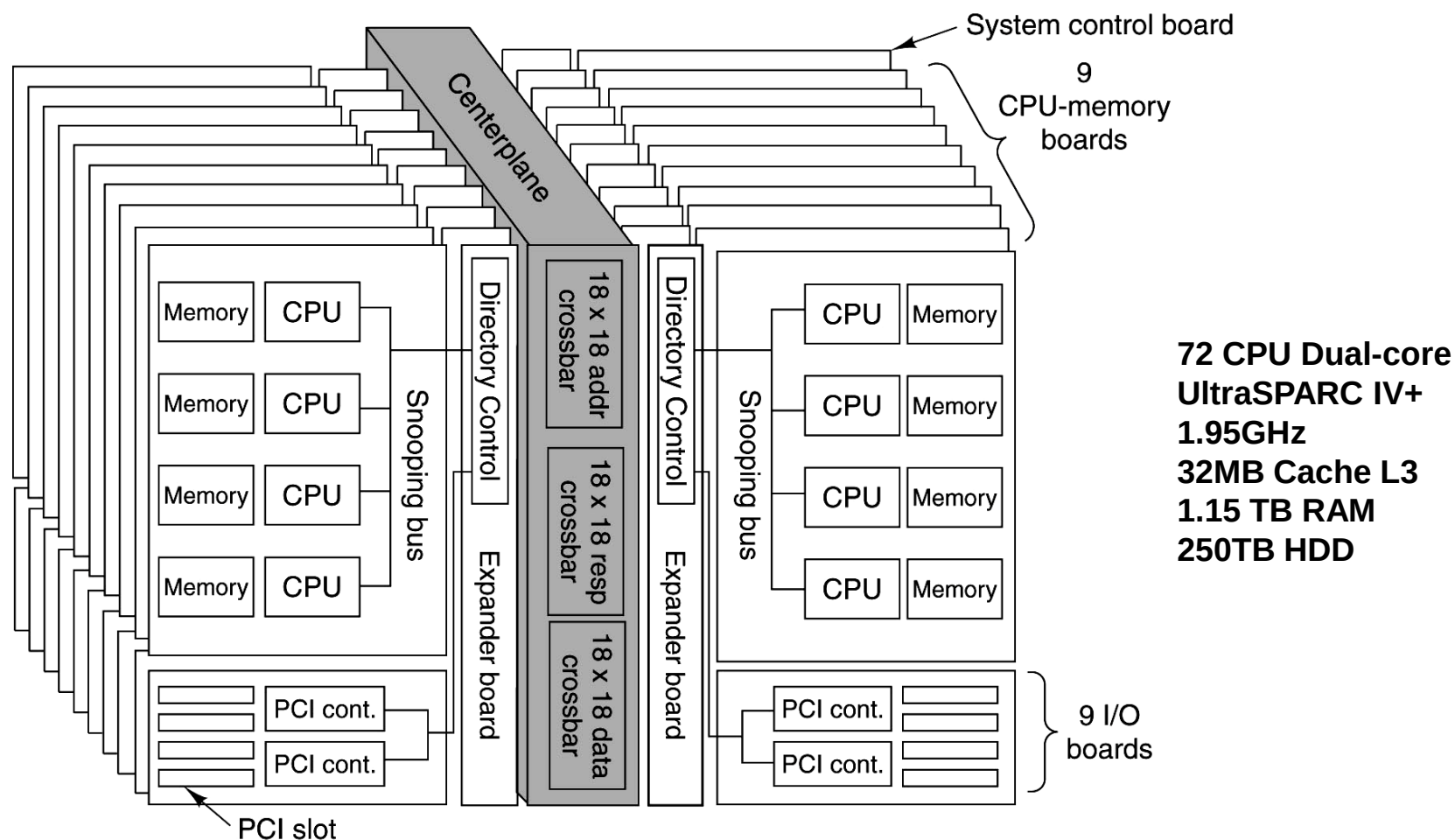


➤ Sơ đồ multi-processor dùng bộ nhớ chung





VD: Hệ thống SUN E25K (NUMA multi-processor)



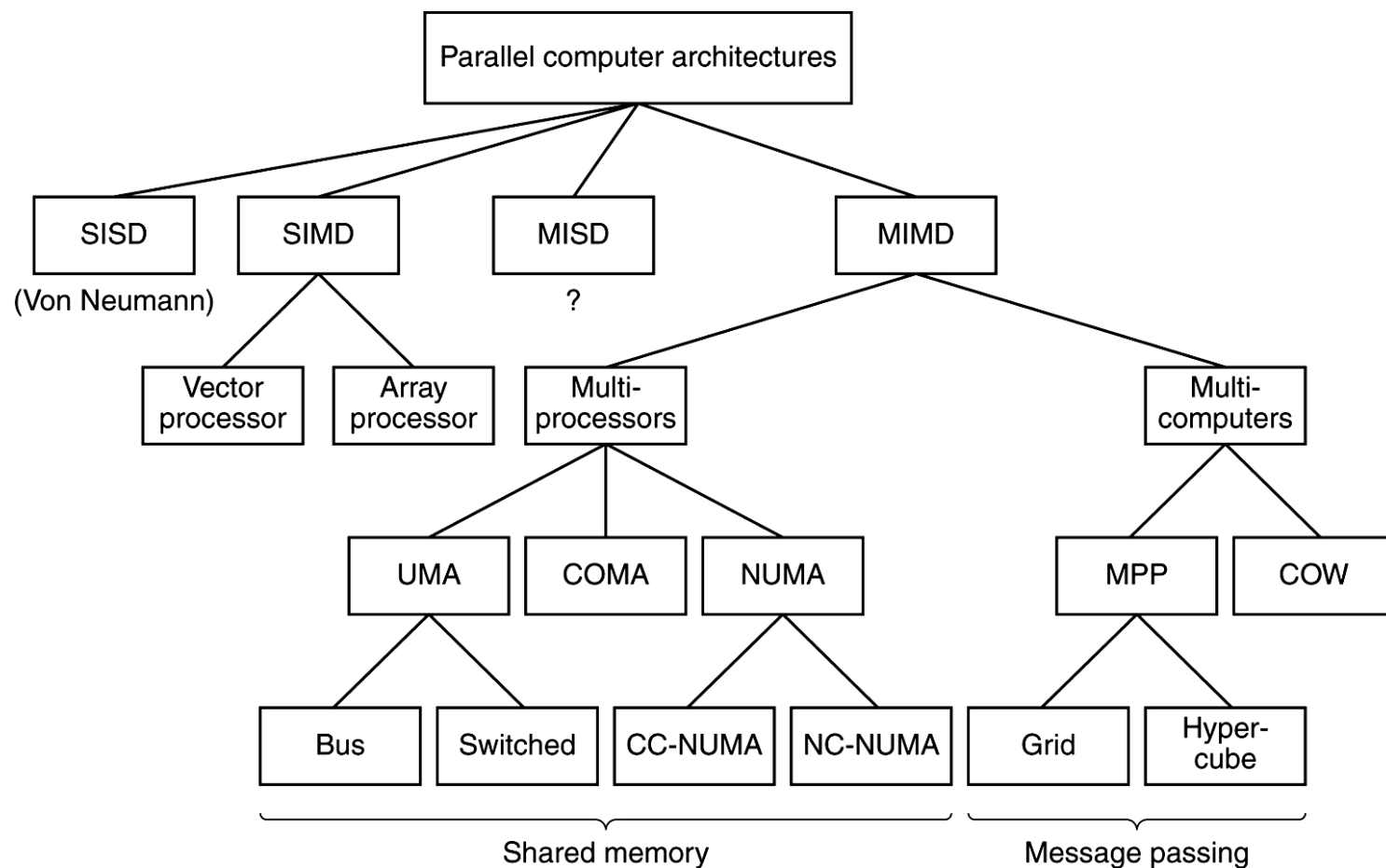


Multi-computer

- Phân loại theo Flynn (1966): Căn cứ vào số lượng lệnh và số lượng dữ liệu có thể xử lý là 1 hay nhiều
- Single instruction, single data stream – **SISD**
 - Single instruction, multiple data stream – **SIMD**
 - Multiple instruction, single data stream – **MISD**
 - Multiple instruction, multiple data stream- **MIMD**



Sơ đồ phân loại Flynn





Ví dụ về SIMD

ADD R3 $\leftarrow$ R1, R2

R1	a7	a6	a5	a4	a3	a2	a1	a0
	+	+	+	+	+	+	+	+
R2	b7	b6	b5	b4	b3	b2	b1	b0
	=	=	=	=	=	=	=	=
R3	a7+b7	a6+b6	a5+b5	a4+b4	a3+b3	a2+b2	a1+b1	a0+b0

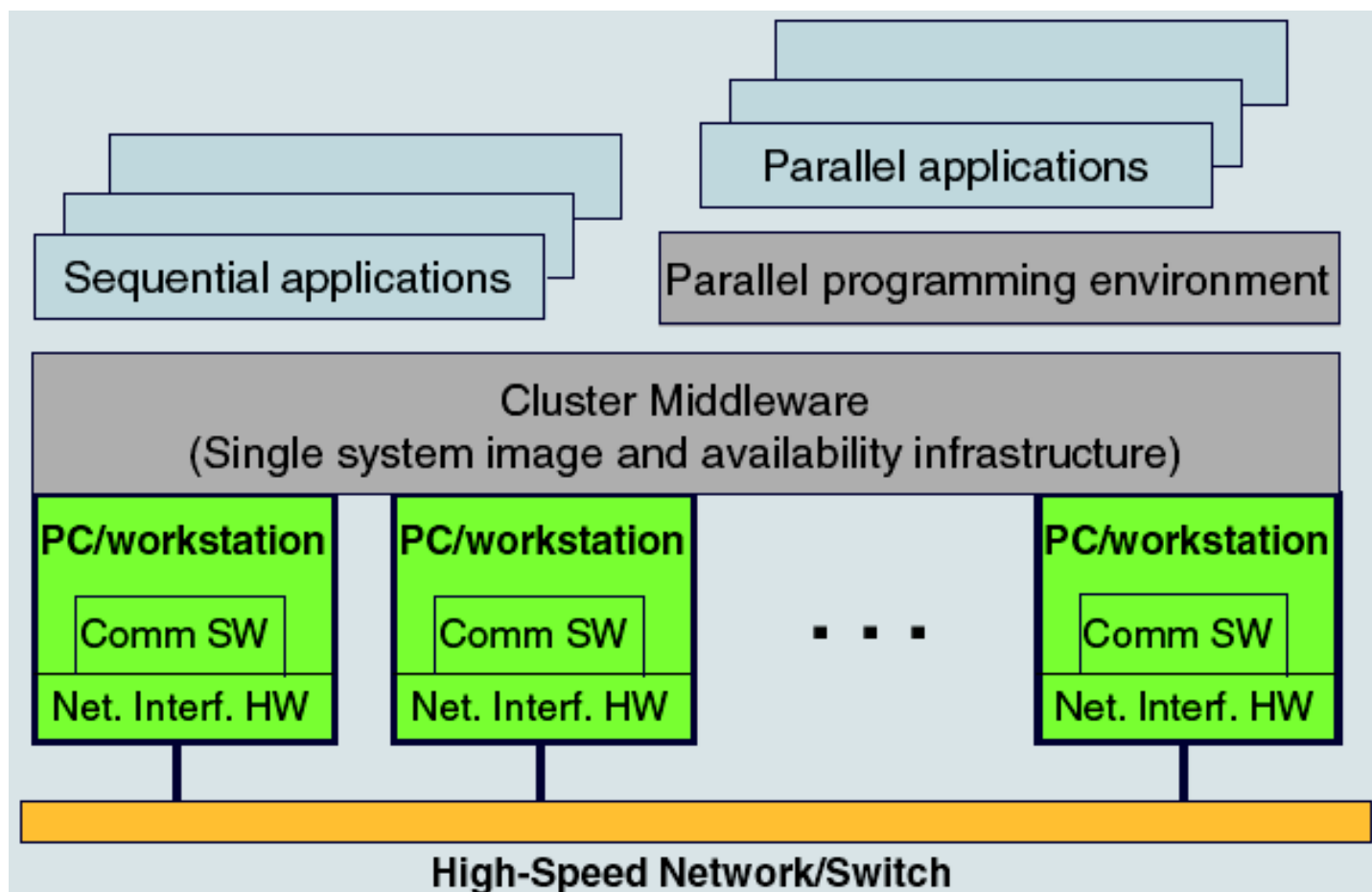
MULADD R3 $\leftarrow$ R1, R2

R1	a7	a6	a5	a4	a3	a2	a1	a0
	x&+	x&+	x&+	x&+	x&+	x&+	x&+	x&+
R2	b7	b6	b5	b4	b3	b2	b1	b0
	=	=	=	=	=	=	=	=
R3	(a6 x b6)+(a7 x b7)	(a4 x b4)+(a5 x b5)	(a2 x b2)+(a3 x b3)	(a0 x b0)+(a1 x b1)				



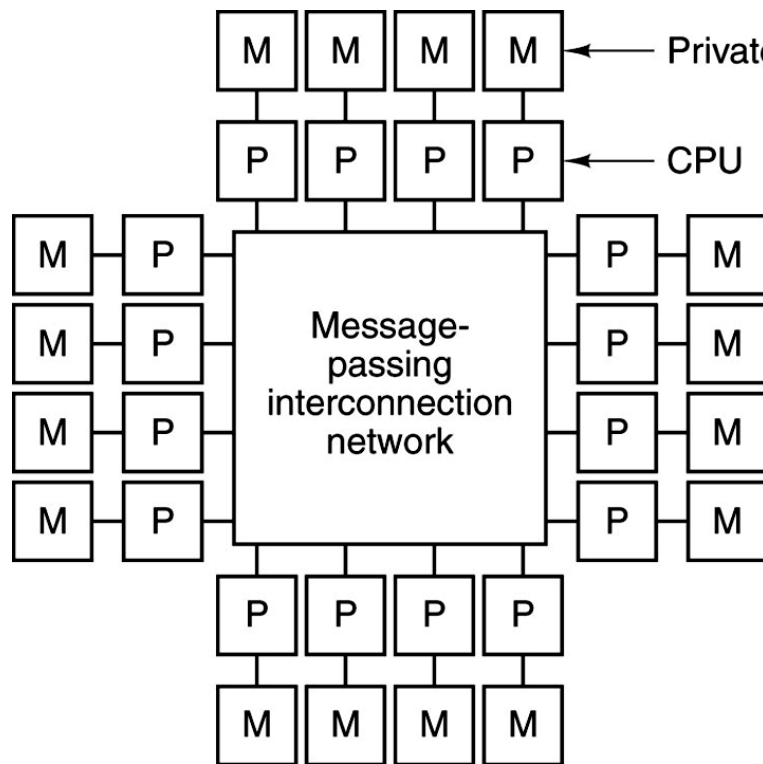
Cluster

- Là 1 dạng máy tính loại MIMD gồm nhiều máy tính độc lập kết nối qua mạng tốc độ cao, mỗi máy có CPU, BN và IO riêng
- Dùng phương pháp truyền thông báo (Message Passing) để trao đổi thông tin (bằng phần mềm)
 - MPI (Message Passing Interface)
 - PVM (Parallel Virtual Machine)
- Gồm 2 loại
 - NOW (Network of Workstations) hoặc COW (Cluster of Workstations) : Kết nối qua LAN
 - Grid : Kết nối qua Internet

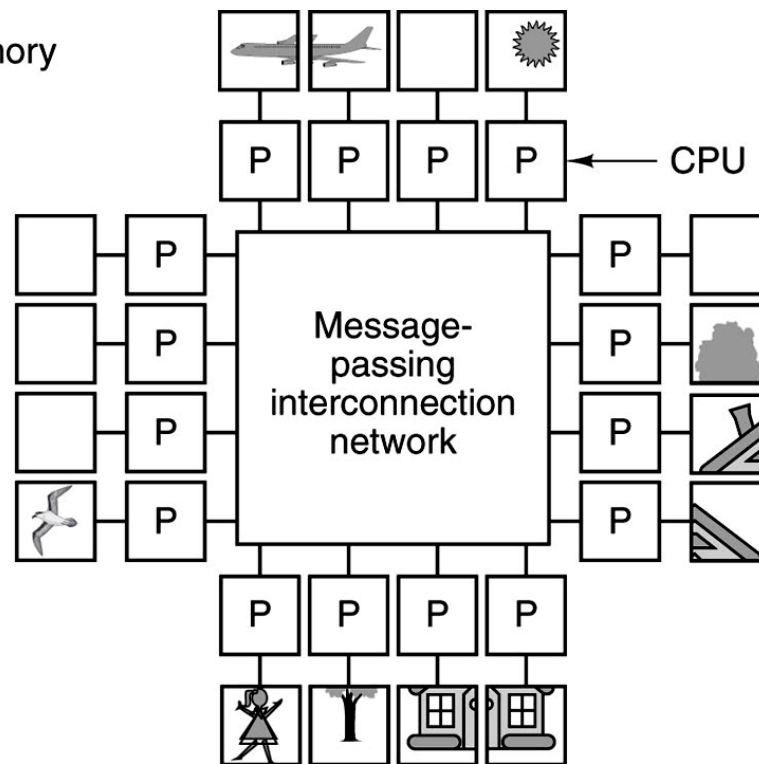




Message-passing multi-computer



(a)

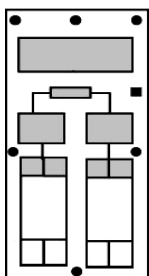


(b)



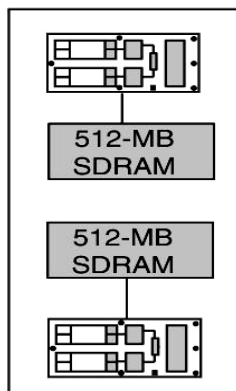
Ví dụ: Siêu máy tính Bluegen của IBM

512-MB



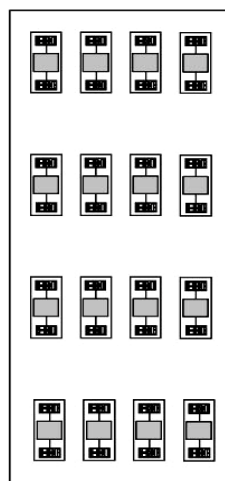
Chip:
2 core
PowerPC 440
700 MHz
4MB L3

(a)



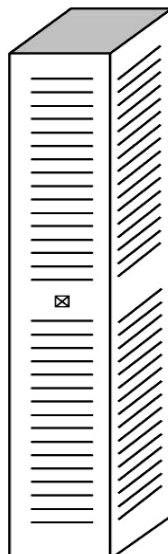
Card:
2 Chips
1 GB

(b)



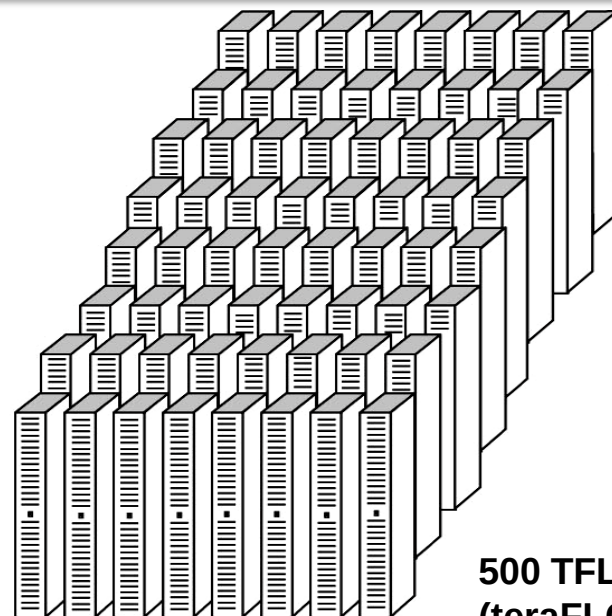
Board
16 Cards
32 Chips
16 GB

(c)



Cabinet
32 Boards
512 Cards
1024 Chips
512 GB

(d)



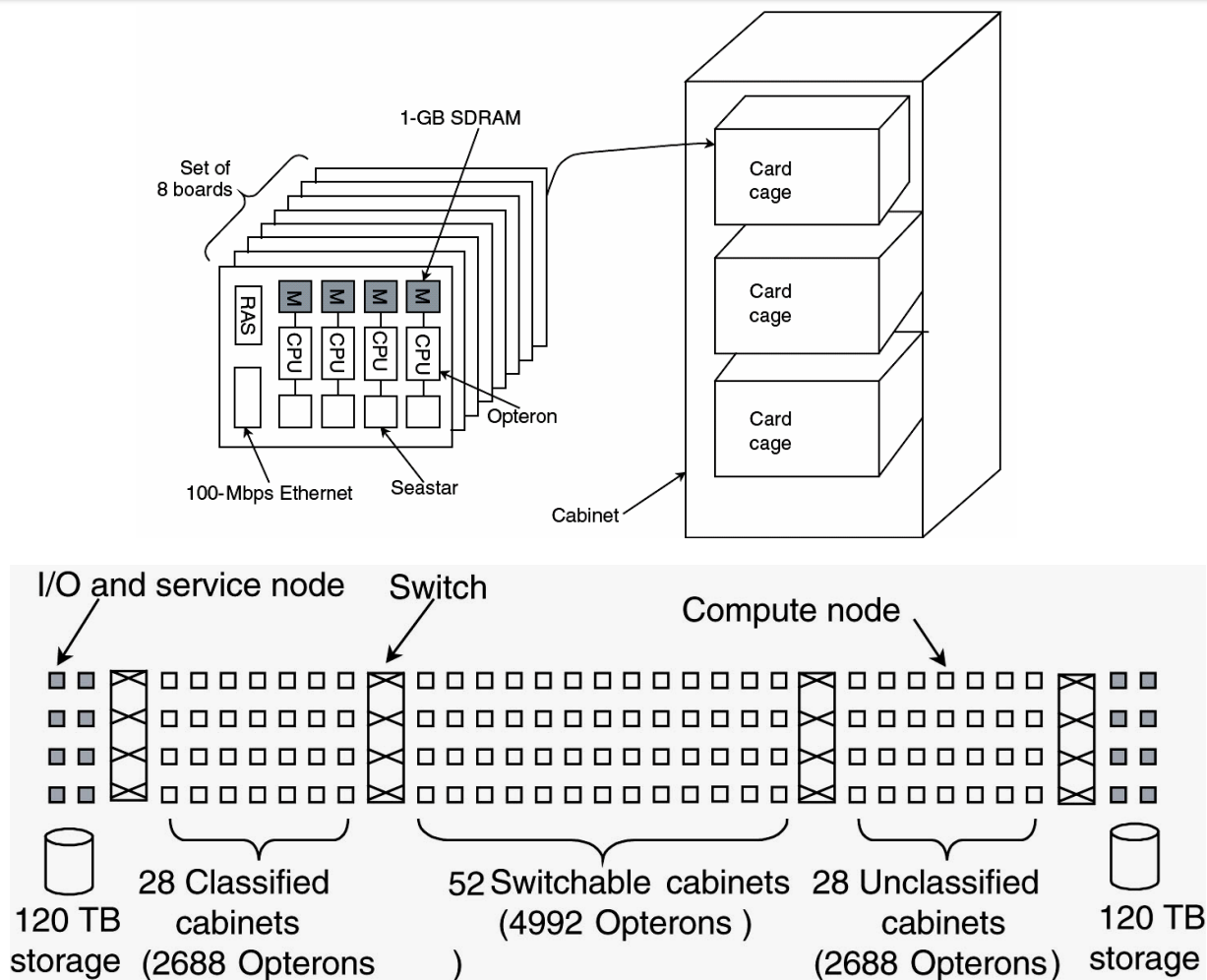
System
64 Cabinets
2048 Boards
32,768 Cards
65,536 Chips
32 TB

(e)

500 TFLOPS
(teraFLOPS)



Ví dụ: Siêu máy tính Red Storm của Cray





So sánh 2 siêu máy tính Bluegen & Red Storm

Item	BlueGene/L	Red Storm
CPU	32-Bit PowerPC	64-Bit Opteron
Clock	700 MHz	2 GHz
Compute CPUs	65,536	10,368
CPUs/board	32	4
CPUs/cabinet	1024	96
Compute cabinets	64	108
Teraflops/sec	71	41
Memory/CPU	512 MB	2–4 GB
Total memory	32 TB	10 TB
Router	PowerPC	Seastar
Number of routers	65,536	10,368
Interconnect	3D torus $64 \times 32 \times 32$	3D torus $27 \times 16 \times 24$
Other networks	Gigabit Ethernet	Fast Ethernet
Partitionable	No	Yes
Compute OS	Custom	Custom
I/O OS	Linux	Linux
Vendor	IBM	Cray Research
Expensive	Yes	Yes



Top 10 siêu máy tính 06/2010 trên trang top500.org

Rank	Site	Computer
1	Oak Ridge National Laboratory United States	Jaguar - Cray XT5-HE Opteron Six Core 2.6 GHz Cray Inc.
2	National Supercomputing Centre in Shenzhen China (Thâm Quyến)	Nebulae (Tinh Vân) - Dawning TC3600 Blade, Intel X5650 Dawning
3	DOE/NNSA/LANL United States	Roadrunner - BladeCenter QS22/LS21 Cluster, PowerXCell 8i 3.2 Ghz / Opteron DC 1.8 GHz, Voltaire Infiniband IBM
4	National Institute for Computational Sciences/University of Tennessee United States	Kraken XT5 - Cray XT5-HE Opteron Six Core 2.6 GHz Cray Inc.
5	Forschungszentrum Juelich (FZJ) Germany	JUGENE - Blue Gene/P Solution IBM
6	NASA/Ames Research Center/NAS United States	Pleiades - SGI Altix ICE 8200EX/8400EX, Xeon HT QC 3.0 Ghz SGI
7	National SuperComputer Center in Tianjin/NUDT China (Thiên Tân)	Tianhe-1 (Tinh Hà) - NUDT TH-1 Cluster, Xeon E5540/E5450 NUDT
8	DOE/NNSA/LLNL United States	BlueGene/L - eServer Blue Gene Solution IBM
9	Argonne National Laboratory United States	Intrepid - Blue Gene/P Solution IBM
10	National Renewable Energy Laboratory United States	Red Sky - Sun Blade x6275, Xeon X55xx 2.93 Ghz, Infiniband Sun



Top 10 siêu máy tính 06/2011 trên trang top500.org

Rank	Site	Computer
1	RIKEN Advanced Institute for Computational Science - Japan	K computer, SPARC64 VIIIfx 2.0GHz Fujitsu
2	National Supercomputing Center in Tianjin (Thiên Tân) – China	Tianhe-1A (Tinh Hà) X5670 2.93Ghz 6C, NVIDIA GPU NUDT
3	DOE/SC/Oak Ridge National Laboratory United States	Jaguar - Cray XT5-HE Opteron 6-core 2.6 GHz Cray Inc.
4	National Supercomputing Centre in Shenzhen (Thâm Quyến) – China	Nebulae (Tinh Vân) Intel X5650, NVidia Tesla C2050 GPU Dawning
5	GSIC Center, Tokyo Institute of Technology Japan	TSUBAME 2.0 G7 Xeon 6C X5670, Nvidia GPU, NEC/HP
6	DOE/NNSA/LANL/SNL United States	Cielo - Cray XE6 8-core 2.4 GHz Cray Inc.
7	NASA/Ames Research Center/NAS United States	Pleiades Xeon HT QC 3.0/Xeon 5570/5670 2.93 Ghz SGI
8	DOE/SC/LBNL/NERSC United States	Hopper - Cray XE6 12-core 2.1 GHz Cray Inc.
9	Commissariat a l'Energie Atomique (CEA) France	Tera-100 - Bull bullx super-node S6010/S6030 Bull SA
10	DOE/NNSA/LANL United States	Roadrunner - PowerXCell 8i 3.2 Ghz / Opteron DC 1.8 GHz IBM



Top 10 siêu máy tính 06/2012 trên trang top500.org

Rank	Site	Computer
1	DOE/NNSA/LLNL United States	Sequoia - BlueGene/Q , Power BQC 16C 1.60 GHz, Custom IBM
2	RIKEN Advanced Institute for Computational Science Japan	K computer , SPARC64 VIIIfx 2.0GHz, Tofu interconnect Fujitsu
3	DOE/SC/Argonne National Laboratory United States	Mira - BlueGene/Q , Power BQC 16C 1.60GHz, Custom IBM
4	Leibniz Rechenzentrum Germany	SuperMUC - iDataPlex DX360M4 , Xeon E5-2680 8C 2.70GHz, Infiniband FDR IBM
5	National Supercomputing Center in Tianjin China	Tianhe-1A - NUDT YH MPP , Xeon X5670 6C 2.93 GHz, NVIDIA 2050 NUDT
6	DOE/SC/Oak Ridge National Laboratory United States	Jaguar - Cray XK6 , Opteron 6274 16C 2.200GHz, Cray Gemini interconnect , NVIDIA 2090 Cray Inc.
7	CINECA Italy	Fermi - BlueGene/Q , Power BQC 16C 1.60GHz, Custom IBM
8	Forschungszentrum Juelich (FZJ) Germany	JuQUEEN - BlueGene/Q , Power BQC 16C 1.60GHz, Custom IBM
9	CEA/TGCC-GENCI France	Curie thin nodes - Bullx B510 , Xeon E5-2680 8C 2.700GHz, Infiniband QDR Bull
10	National Supercomputing Centre in Shenzhen (NSCS) China	Nebulae - Dawning TC3600 Blade System , Xeon X5650 6C 2.66GHz, Infiniband QDR , NVIDIA 2050 Dawning



Top 10 siêu máy tính 11/2012 trên trang top500.org

Rank	Site	System	Cores
1	DOE/SC/Oak Ridge National Laboratory United States	Titan - Cray XK7 , Opteron 6274 16C 2.200GHz, Cray Inc.	560.640
2	DOE/NNSA/LLNL United States	Sequoia - BlueGene/Q, Power BQC 16C 1.60 Hz, IBM	1.572.864
3	RIKEN Advanced Institute for Computational Science Japan	K computer , SPARC64 VIIIfx 2.0GHz, Fujitsu	705.024
4	DOE/SC/Argonne National Laboratory United States	Mira - BlueGene/Q, Power BQC 16C 1.60GHz, IBM	786.432
5	Forschungszentrum Juelich (FZJ) Germany	JUQUEEN - BlueGene/Q, Power BQC 16C 1.60GHz, IBM	393.216
6	Leibniz Rechenzentrum Germany	SuperMUC - iDataPlex DX360M4, Xeon E5-2680 8C 2.70GHz, IBM	147.456
7	Texas Advanced Computing Center/Univ. of Texas United States	Stampede - PowerEdge C8220, Xeon E5-2680 8C 2.700GHz, Intel Xeon Phi Dell	204.900
8	National Supercomputing Center in Tianjin China	Tianhe-1A - NUDT YH MPP, Xeon X5670 6C 2.93 GHz, NVIDIA 2050 NUDT	186.368
9	CINECA Italy	Fermi - BlueGene/Q, Power BQC 16C 1.60GHz, IBM	163.840
10	IBM Development Engineering United States	DARPA Trial Subset - Power 775, POWER7 8C 3.836GHz IBM	63.360



Câu hỏi





Chương 6 Bộ nhớ(Memory)

1. Tổng quan về hệ thống nhớ
2. Bộ nhớ bán dẫn
3. Bộ nhớ cache
4. Bộ nhớ ngoài
5. Bộ nhớ ảo



6.1 Tổng quan về hệ thống nhớ

Các đặc trưng của hệ thống nhớ

○ Vị trí

- Bên trong CPU: Tập thanh ghi, cache
- Bộ nhớ trong: Bộ nhớ chính, cache
- Bộ nhớ ngoài: các thiết bị lưu trữ, RAID

○ Dung lượng

- Độ dài từ nhớ (tính bằng bit)
- Số lượng từ nhớ

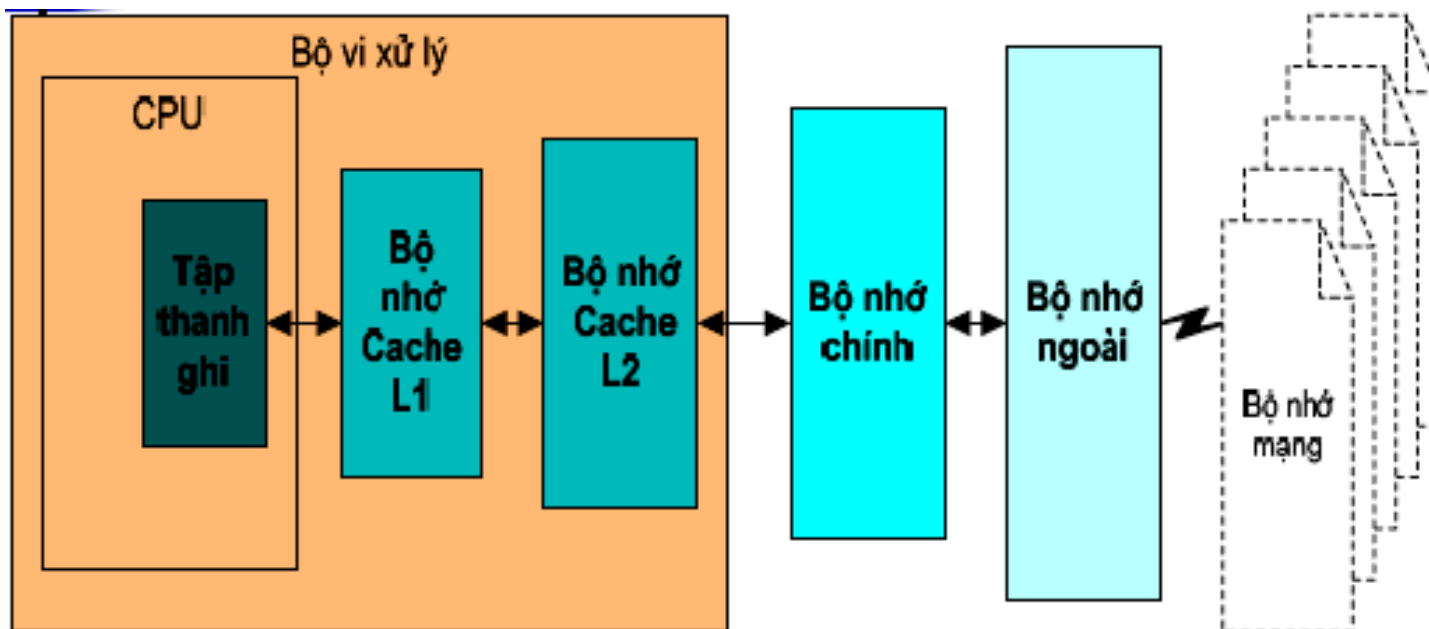
○ Đơn vị truyền

- Từ nhớ (word)
- Khối nhớ (block)

- Phương pháp truy cập
 - Truy cập tuần tự (băng từ)
 - Truy cập trực tiếp (các loại đĩa)
 - Truy cập ngẫu nhiên (bộ nhớ bán dẫn)
 - Truy cập kết hợp (cache)
- Hiệu năng (performance)
 - Thời gian truy cập
 - Tốc độ truyền
- Kiểu vật lý
 - Bộ nhớ bán dẫn
 - Bộ nhớ từ
 - Bộ nhớ quang
- Các đặc tính vật lý
 - Tự mất/ Không tự mất (volatile/ nonvolatile)
 - Xoá được/ không xoá được



Phân cấp hệ thống nhớ



Từ trái sang phải:

- dung lượng tăng dần
- tốc độ giảm dần
- giá thành/1bit giảm dần



6.2 Bộ nhớ bán dẫn

➤ Phân loại

○ ROM (Read Only Memory)

- Bộ nhớ chỉ đọc
- Không tự mất dữ liệu khi cắt nguồn điện

○ RAM (Random Access Memory)

- Bộ nhớ đọc/ ghi
- Tự mất dữ liệu khi cắt nguồn điện

○ Cache

- Bộ nhớ có tốc độ cao nhưng dung lượng thấp
- Trung gian giữa bộ nhớ chính và thanh ghi trong CPU
- Ngày nay thường được tích hợp sẵn trong CPU



ROM(Read Only Memory)

- Thông tin được ghi khi sản xuất
- Không xoá/ sửa được nội dung khi sử dụng
- Ứng dụng:
 - Thư viện các chương trình con
 - Các chương trình điều khiển hệ thống nhập xuất cơ bản BIOS (Basic Input Output System)
 - Phần mềm kiểm tra khi bật máy POST (Power On Self Test)
 - Phần mềm khởi động máy tính (OS loader)
 - Vi chương trình



Phân loại ROM

- Mask ROM
 - Thông tin được ghi khi sản xuất
 - Không xoá/ sửa được nội dung
 - Giá thành rất đắt
- PROM (Programmable ROM)
 - Khi sản xuất chưa có nội dung (ROM trắng)
 - Cần thiết bị chuyên dụng để ghi
 - Cho phép ghi được một lần, gọi là OTP (One Time Programmable) hoặc WORM (Write-Once-Read-Many)
- EPROM (Erasable PROM)
 - Có thể xóa bằng tia cực tím UV (Ultra Violet)
 - Cần thiết bị chuyên dụng để ghi
 - Ghi/ xoá được nhiều lần

○ EEPROM (Electrically EPROM)

- Xóa bằng mạch điện, không cần tia UV → Không cần tháo chip ROM ra khỏi máy tính
- Có thể ghi theo từng byte
- 2 chế độ điện áp:
 - Điện áp cao : Ghi + Xoá
 - Điện áp thấp : Chỉ đọc

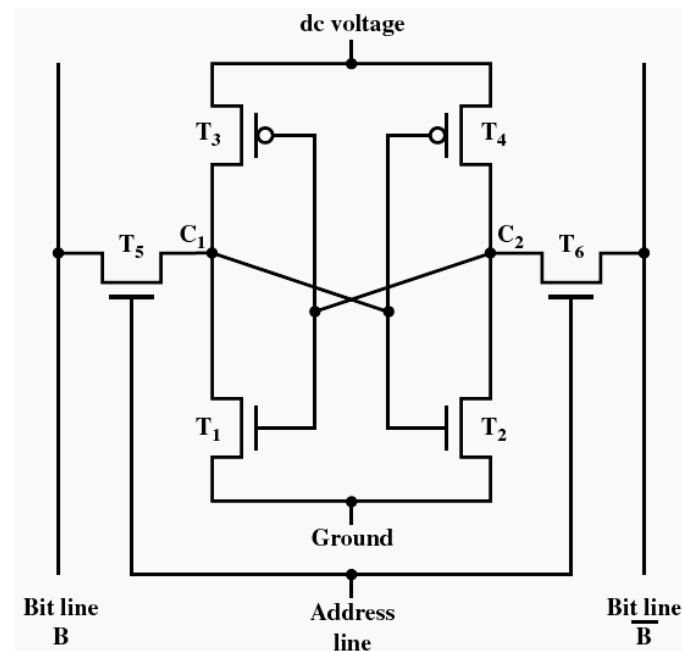
○ Flash memory (Bộ nhớ cực nhanh)

- EEPROM sản xuất bằng công nghệ NAND, tốc độ truy cập nhanh, mật độ cao
- Xóa bằng mạch điện; Ghi theo từng block
- Ngày nay được sử dụng rộng rãi dưới dạng thẻ nhớ (CF, SD,...) , thanh USB, ổ SSD (thay thế cho ổ đĩa cứng)

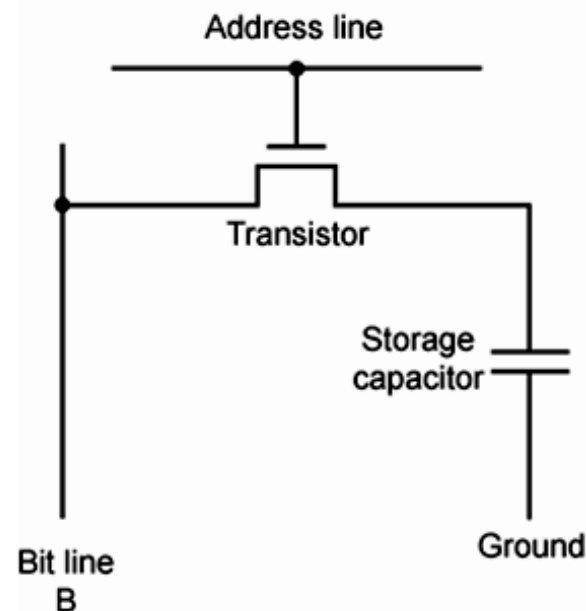
RAM (Random Access Memory)

- Bộ nhớ đọc-ghi (Read/Write Memory)
- Có thể ghi/ xoá trong quá trình sử dụng → Làm bộ nhớ chính trong máy tính
- Tự mất dữ liệu khi cắt nguồn điện. Chỉ lưu trữ thông tin tạm thời khi chạy chương trình, khi kết thúc chương trình cần lưu trữ dữ liệu ra bộ nhớ ngoài
- Có hai loại:
 - SRAM (Static RAM): RAM tĩnh
 - DRAM (Dynamic RAM): RAM động

- Các bit được lưu trữ bằng các Flip-Flop
- Thông tin ổn định, không tự mất dữ liệu theo thời gian
- Cấu trúc phức tạp
- Dung lượng chip nhỏ
- Tốc độ truy cập nhanh
- Đắt tiền
- Dùng làm bộ nhớ cache



- Các bit được lưu trữ trên mạch tụ điện
- Tự mất dữ liệu theo thời gian → cần phải có mạch làm tươi (refresh)
- Cấu trúc đơn giản
- Dung lượng lớn
- Tốc độ chậm hơn
- Rẻ tiền hơn
- Dùng làm bộ nhớ chính

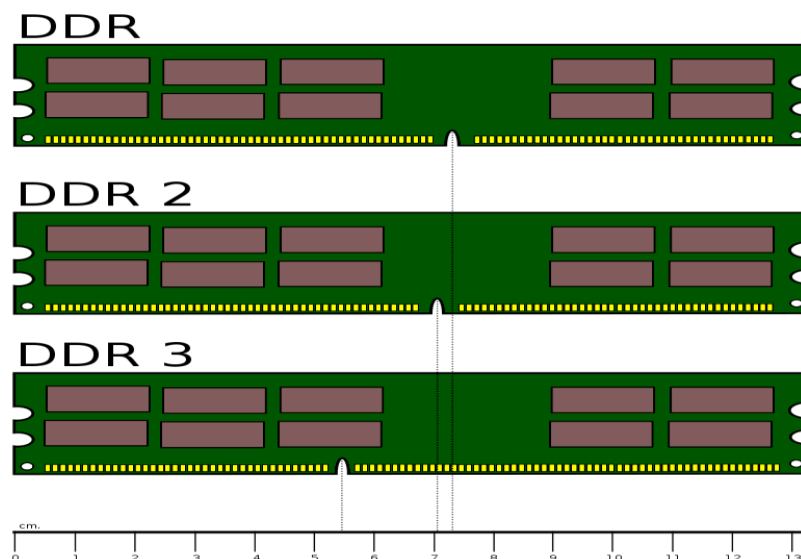




Phân loại DRAM theo cơ chế hoạt động

- FPM (Fast Page Mode)
 - Truy cập theo từng trang bộ nhớ (cùng hàng khác cột)
- EDO (Enhanced Data Out)
 - Khi xuất dữ liệu có thể đồng thời đọc địa chỉ của ô nhớ kế tiếp
 - Cho phép đọc nhanh gấp đôi so với RAM thường
- SDRAM (Synchronous DRAM)
 - Đồng bộ với system clock → CPU không cần chu kỳ chờ
 - Truyền dữ liệu theo block
- RDRAM (Rambus DRAM)
 - Bộ nhớ tốc độ cao, truyền dữ liệu theo block
 - Do công ty Rambus và Intel sản xuất để sử dụng cho CPU Pentium 4 khi mới xuất hiện năm 2000
 - Giá thành đắt nên ngày nay ít sử dụng

- **DDR-SDRAM (Double Data Rate-SDRAM)**
 - Phiên bản cải tiến của SDRAM nhằm nâng cao tốc độ truy cập nhưng có giá thành rẻ hơn RDRAM
 - Gửi dữ liệu 2 lần trong 1 chu kỳ clock
- **DDR2/ DDR3: Gửi dữ liệu 4 hoặc 8 lần trong 1 chu kỳ clock**





Phân loại DRAM theo hình thức đóng gói

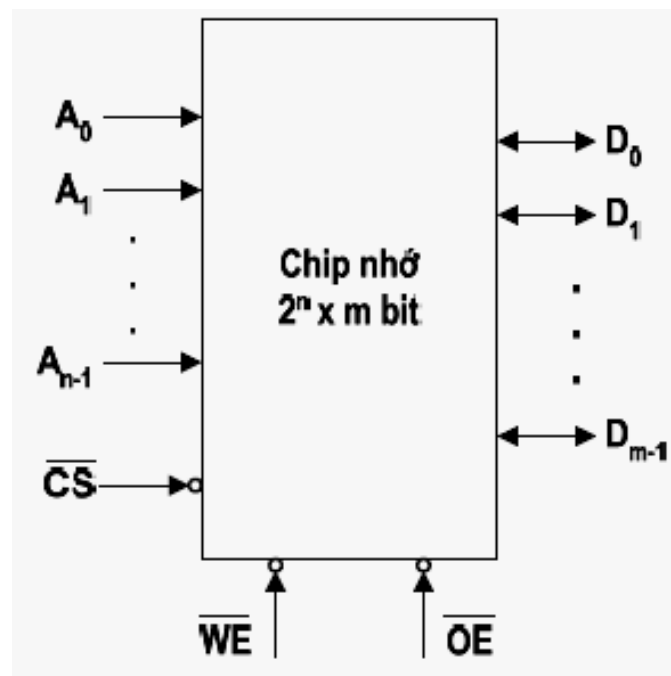
- SIMM (Single Inline Memory Module)
- DIMM (Dual Inline Memory Module)
- RIMM (Rambus Inline Memory Module)
- SO-DIMM (Small Outline DIMM)
- SO-RIMM (Small Outline RIMM)





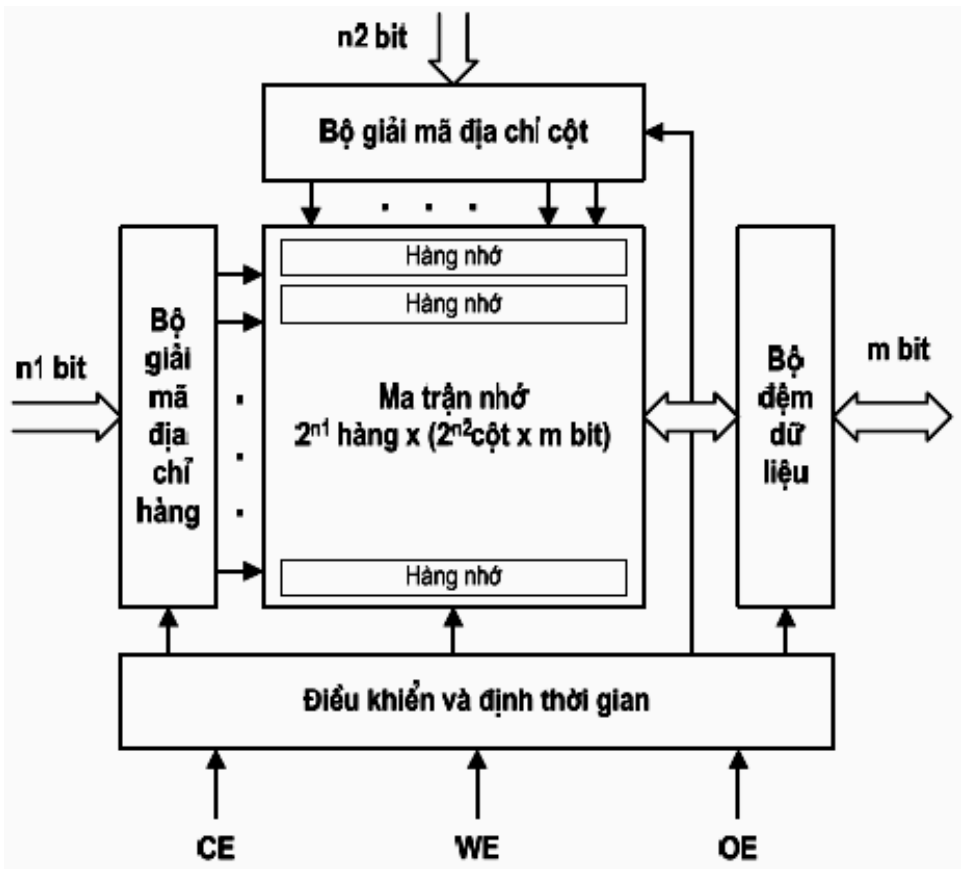
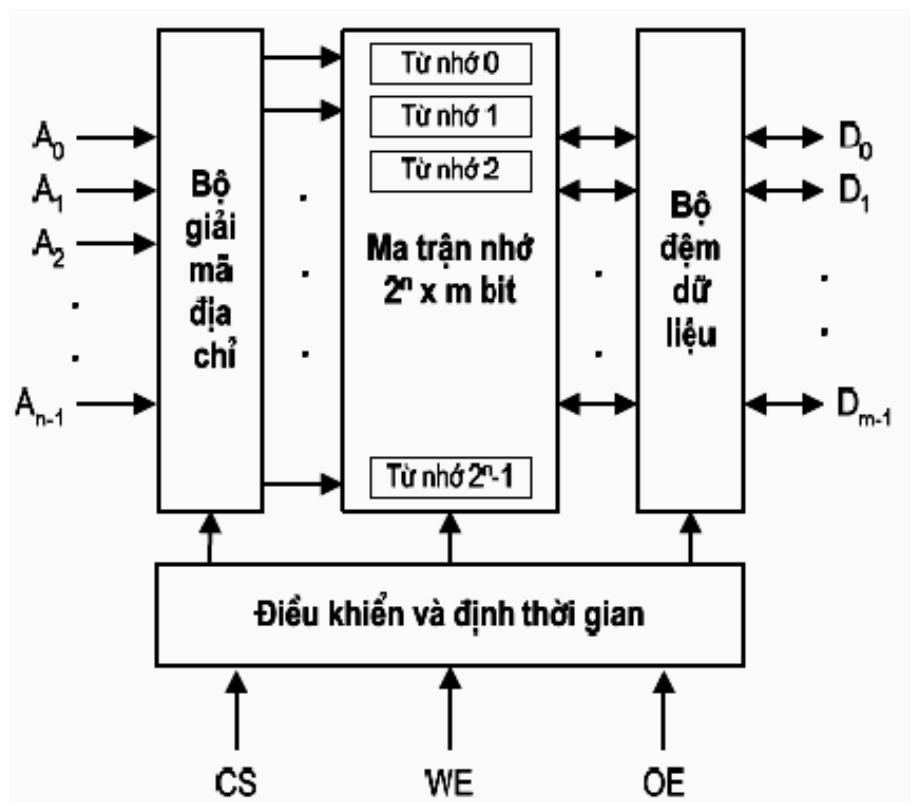
Tổ chức của chip nhớ

- Các đường địa chỉ: $A_{n-1} \div A_0 \rightarrow$ có 2^n từ nhớ
- Các đường dữ liệu: $D_{m-1} \div D_0 \rightarrow$ độ dài từ nhớ = m bit
- Dung lượng chip nhớ = $2^n * m$ bit
- Các đường điều khiển:
 - Tín hiệu chọn chip CS (Chip Select)
 - Tín hiệu điều khiển đọc OE (Output Enable)
 - Tín hiệu điều khiển ghi WE (Write Enable)
 - Các tín hiệu điều khiển thường tích cực với mức 0





Tổ chức bộ nhớ 1 chiều và 2 chiều





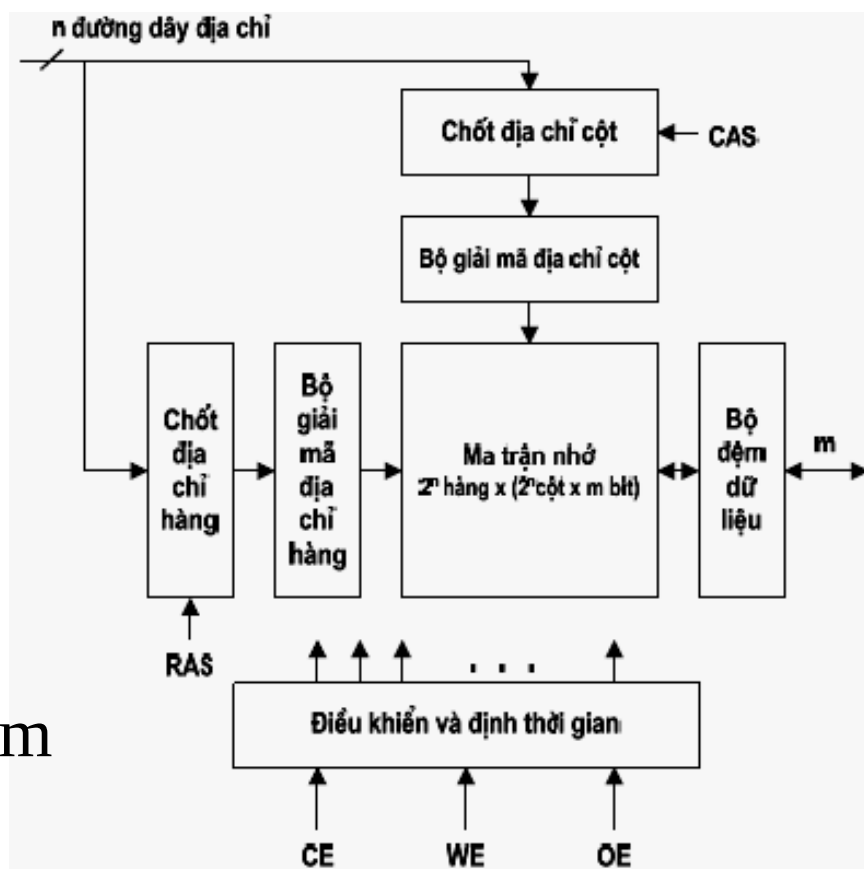
Tổ chức bộ nhớ hai chiều

- Có n đường địa chỉ: $n = n_1 + n_2$
 - 2^{n_1} hàng,
 - mỗi hàng có 2^{n_2} từ nhớ,
- Có m đường dữ liệu:
 - mỗi từ nhớ có độ dài m -bit.
- Dung lượng của chip nhớ:
 - $[2^{n_1} \times (2^{n_2} \times m)] \text{ bit} = (2^{n_1+n_2} \times m) \text{ bit} = (2^n \times m) \text{ bit}$.
- Hoạt động giải mã địa chỉ:
 - Bước 1: bộ giải mã hàng chọn 1 trong 2^{n_1} hàng.
 - Bước 2: bộ giải mã cột chọn 1 trong 2^{n_2} từ nhớ (cột) của hàng đã được chọn.



Tổ chức của DRAM

- Dùng n đường địa chỉ dòng kênh $\rightarrow$ cho phép truyền 2^n bit địa chỉ
- Tín hiệu chọn địa chỉ hàng RAS (Row Address Strobe)
- Tín hiệu chọn địa chỉ cột CAS (Column Address Strobe)
- Dung lượng DRAM = $2^{2n} \times m$ bit



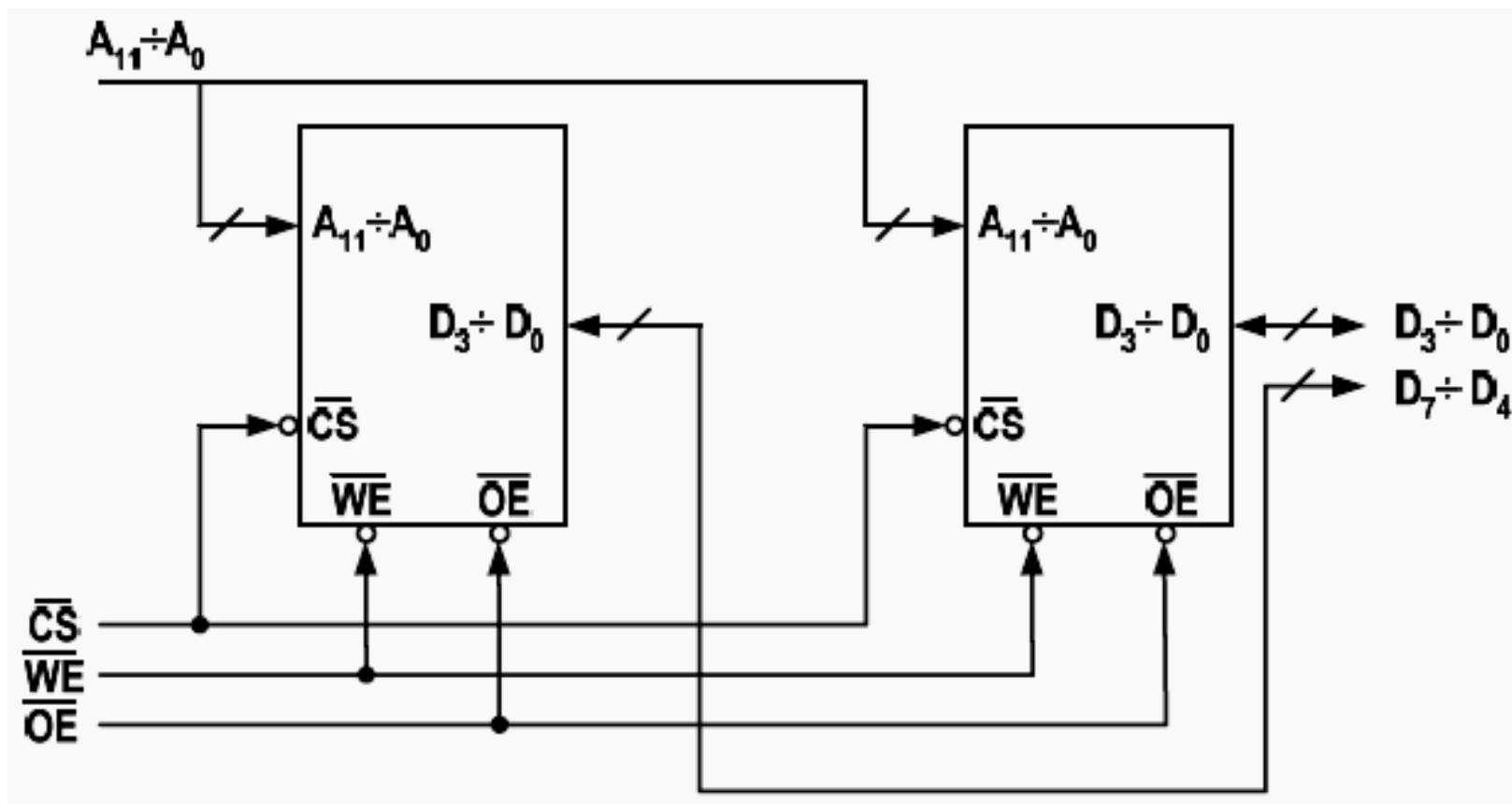


Thiết kế mô-đun nhớ bán dẫn

- Dung lượng chip nhớ $2^n \times m$ bit
- Cần thiết kế để tăng dung lượng:
 - Thiết kế tăng độ dài từ nhớ
 - Thiết kế tăng số lượng từ nhớ
 - Thiết kế kết hợp: tăng cả độ dài và số lượng từ nhớ
- Quy tắc: ghép nối các chip nhớ song song (tăng độ dài) hoặc nối tiếp bằng mạch giải mã (tăng số lượng)

- Tăng độ dài từ nhớ
 - VD:
 - Cho chip nhớ SRAM 4K x 4 bit
 - Thiết kế mô-đun nhớ 4K x 8 bit
 - Giải:
 - Dung lượng chip nhớ = $2^{12} \times 4$ bit
 - chip nhớ có:
 - 12 chân địa chỉ
 - 4 chân dữ liệu
 - mô-đun nhớ cần có:
 - 12 chân địa chỉ
 - 8 chân dữ liệu
 - Tổng quát
 - Cho chip nhớ $2^n \times m$ bit
 - Thiết kế mô-đun nhớ $2^n \times (k.m)$ bit
 - Dùng k chip nhớ

- Ví dụ: Tăng độ dài từ nhớ



➤ Tăng số lượng từ nhớ

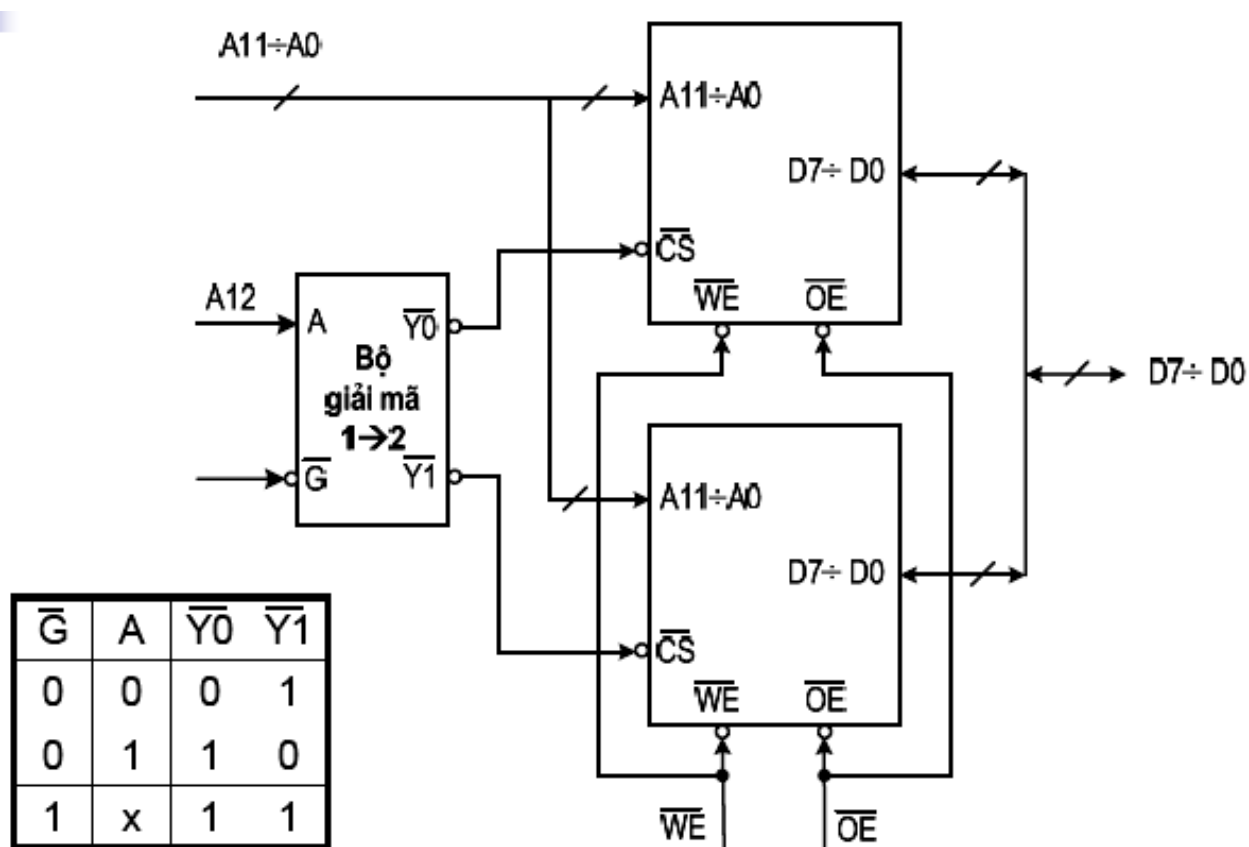
○ VD:

- Cho chip nhớ SRAM 4K x 8 bit
- Thiết kế mô-đun nhớ 8K x 8 bit

○ Giải:

- Dung lượng chip nhớ = 2^{12} x 8 bit
- Chip nhớ có:
 - 12 chân địa chỉ
 - 8 chân dữ liệu
- Dung lượng mô-đun nhớ = 2^{13} x 8 bit
 - 13 chân địa chỉ
 - 8 chân dữ liệu

- Ví dụ: Tăng số lượng từ nhớ



➤ Bài tập

- Tăng số lượng từ gấp 4 lần:
 - Cho chip nhớ SRAM 4K x 8 bit
 - Gợi ý: Dùng mạch giải mã 2 → 4
- Tăng số lượng từ gấp 8 lần:
 - Cho chip nhớ SRAM 4K x 8 bit
 - Thiết kế mô-đun nhớ 32K x 8 bit
 - Gợi ý: Dùng mạch giải mã 3 → 8
- Thiết kế kết hợp:
 - Cho chip nhớ SRAM 4K x 4 bit
 - Thiết kế mô-đun nhớ 8K x 8 bit



Bộ nhớ chính

➤ Các đặc trưng cơ bản

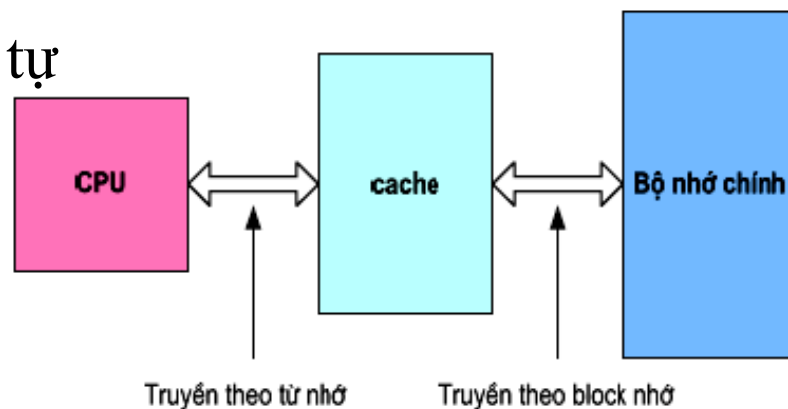
- Chứa các chương trình đang thực hiện và các dữ liệu đang được sử dụng
- Tồn tại trên mọi hệ thống máy tính
- Bao gồm các ô nhớ được đánh địa chỉ trực tiếp bởi CPU
- Dung lượng của bộ nhớ chính trên thực tế thường nhỏ hơn không gian địa chỉ bộ nhớ mà CPU có thể quản lý.
- Việc quản lý logic bộ nhớ chính tùy thuộc vào hệ điều hành → người lập trình chỉ sử dụng bộ nhớ logic.



6.3 Bộ nhớ cache

➤ Nguyên tắc chung của cache

- Nguyên lý cục bộ : Một chương trình thường sử dụng 90% thời gian chỉ để thi hành 10% câu lệnh
- Cache được đặt giữa CPU và bộ nhớ chính nhằm tăng tốc độ truy cập bộ nhớ của CPU
- Ví dụ:
 - Cấu trúc chương trình tuần tự
 - Vòng lặp có thân nhỏ
 - Cấu trúc dữ liệu mảng



➤ Thao tác trên bộ nhớ cache

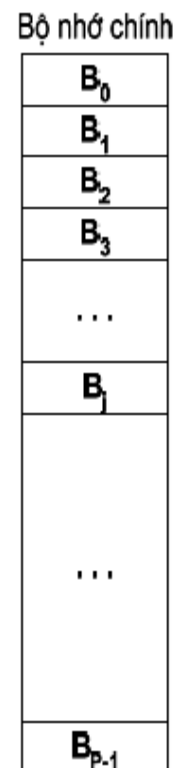
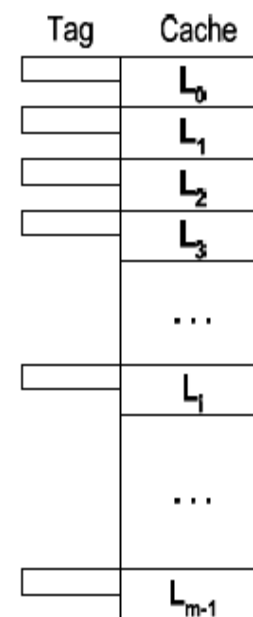
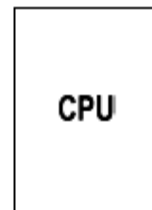
- CPU yêu cầu nội dung của ô nhớ
- CPU kiểm tra trên cache với dữ liệu này
- Nếu có, CPU nhận dữ liệu từ cache (nhanh)
- Nếu không có thực hiện 2 bước sau:
 - Đọc Block chứa dữ liệu từ bộ nhớ chính vào cache (chậm hơn)
 - Chuyển dữ liệu từ cache vào CPU



Cấu trúc chung của cache

○ Bộ nhớ chính có 2^N byte nhớ

- Bộ nhớ chính và cache được chia thành các khối có kích thước bằng nhau
- Bộ nhớ chính: $B_0, B_1, B_2, \dots, B_{p-1}$ (p Blocks)
- Bộ nhớ cache: $L_0, L_1, L_2, \dots, L_{m-1}$ (m Lines)
- Kích thước của Block = 8,16,32,64,128 byte



- Một số Block của bộ nhớ chính được nạp vào các Line của cache.
- Nội dung Tag (thẻ nhớ) cho biết Block nào của bộ nhớ chính hiện đang được chứa ở Line đó.
- Khi CPU truy cập (đọc/ghi) một từ nhớ, có hai khả năng xảy ra:
 - Từ nhớ đó có trong cache (cache hit)
 - Từ nhớ đó không có trong cache (cache miss)



Tổ chức bộ nhớ cache

- Kích thước cache
 - Cache càng lớn càng hiệu quả nhưng đắt tiền
 - Cần nhiều thời gian để giải mã và truy cập
- Kỹ thuật ánh xạ : Phương pháp xác định vị trí dữ liệu trong cache
 - Ánh xạ trực tiếp (Direct mapping)
 - Ánh xạ kết hợp toàn phần (Fully associative mapping)
 - Ánh xạ kết hợp theo bộ (Set associative mapping)
- Giải thuật thay thế
 - Phương pháp chọn lựa vùng nhớ nào lưu trong cache để tăng hiệu suất sử dụng cache



Ảnh xạ trực tiếp

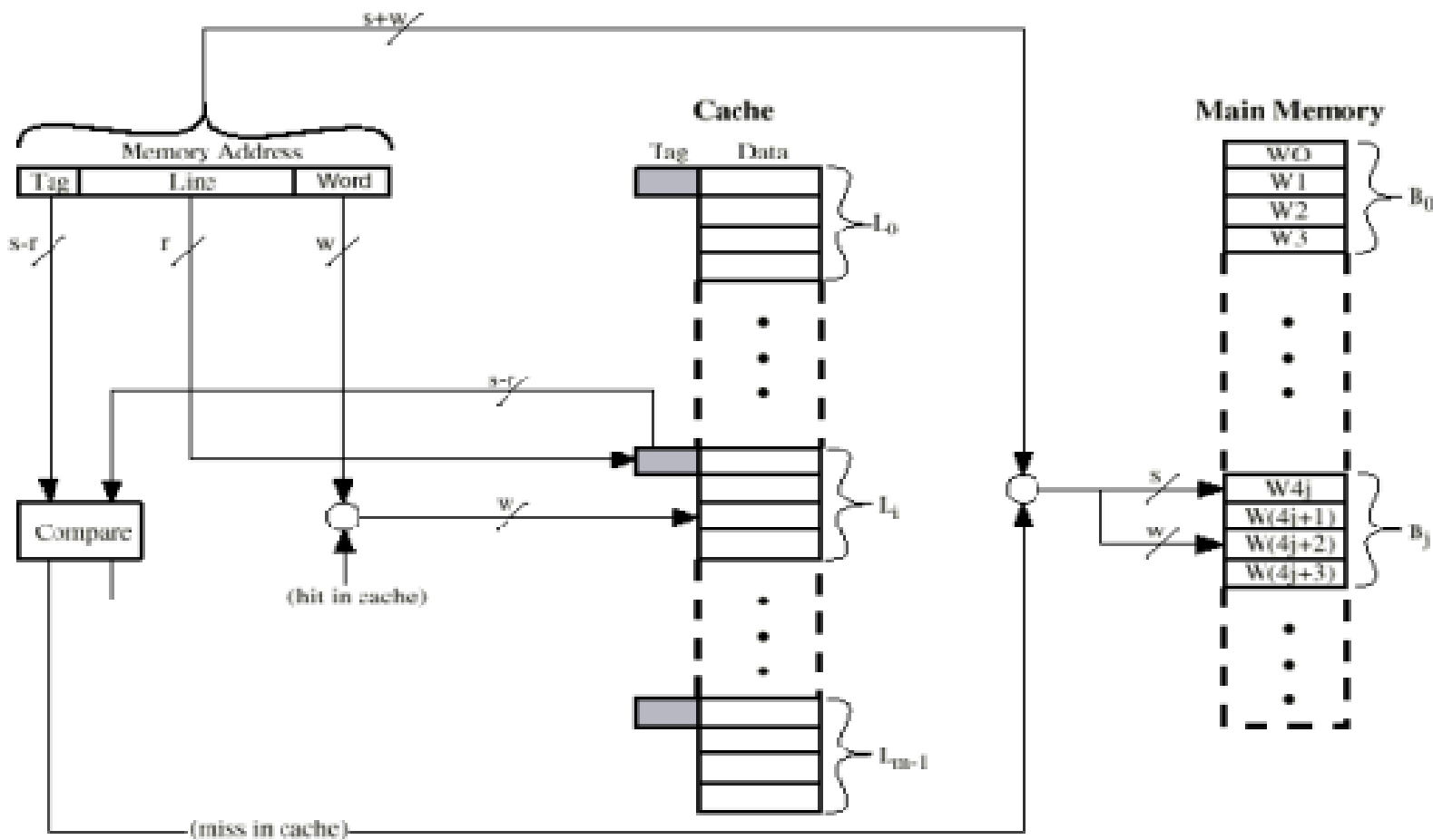
- Mỗi Block của bộ nhớ chính chỉ có thể được nạp vào một Line của cache:

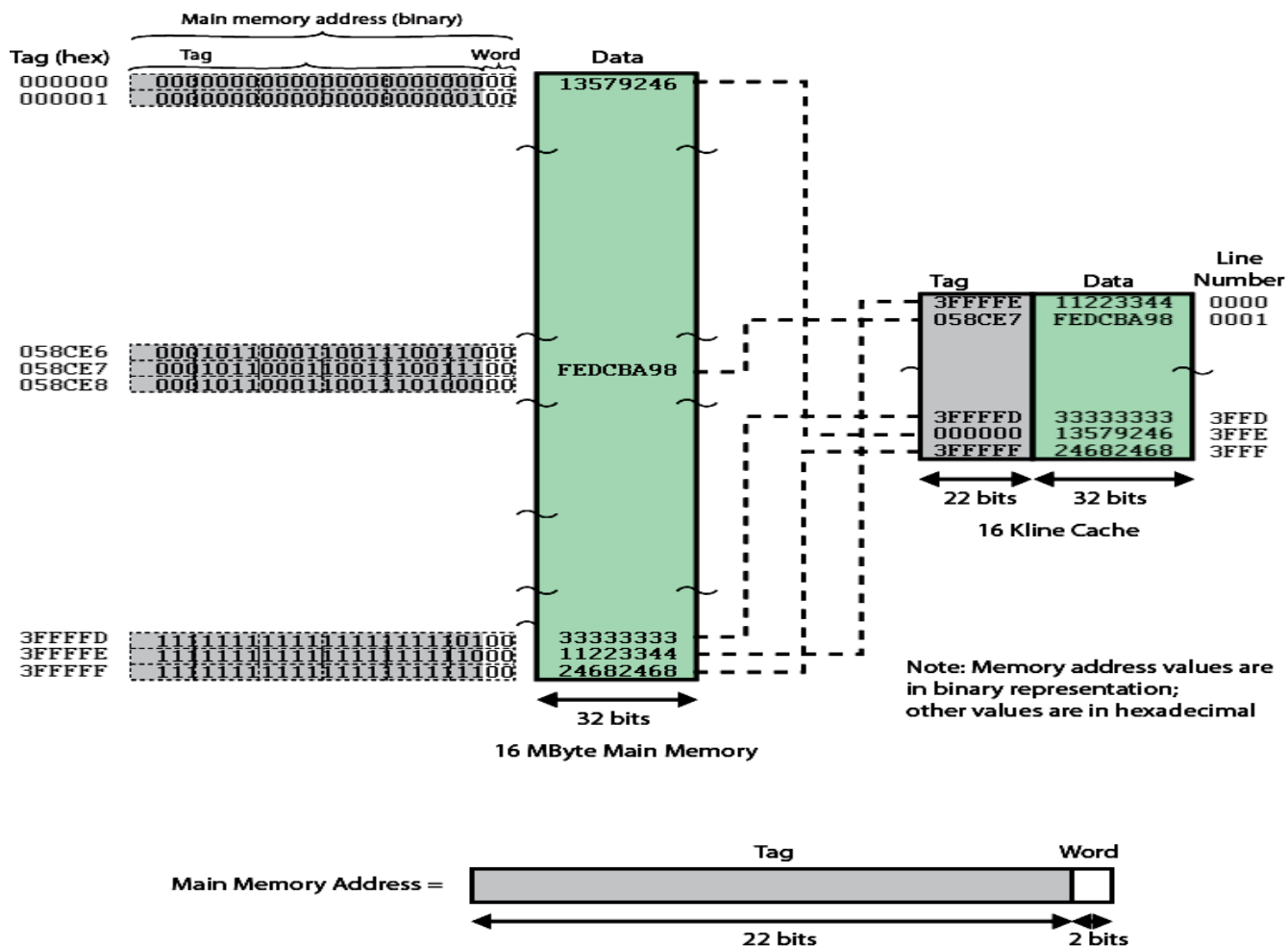
- $B_0 \rightarrow L_0$
- $B_1 \rightarrow L_1$
-
- $B_{m-1} \rightarrow L_{m-1}$
- $B_m \rightarrow L_0$
- $B_{m+1} \rightarrow L_1$
-

Cache line	Main Memory blocks s
0	0, m, 2m, 3m,..., 2^S-m
1	1, m+1, 2m+1,..., 2^S-m+1
m-1	m-1, 2m-1, 3m-1,... 2^S-1

- Tổng quát

- B_j chỉ có thể nạp vào $L_{(j \bmod m)}$
- m là số Line của cache.







Đặc điểm của ánh xạ trực tiếp

- Mỗi một địa chỉ N bit của bộ nhớ chính gồm ba vùng:
 - Vùng *Word* gồm W bit xác định một từ nhớ trong *Block* hay *Line*:
 - $2^W =$ kích thước của *Block* hay *Line*
 - Vùng *Line* gồm L bit xác định một trong số các *Line* trong cache:
 - $2^L =$ số *Line* trong cache = m
 - Vùng *Tag* gồm T bit:
 - $T = N - (W+L)$
- Bộ so sánh đơn giản
- Giá thành rẻ
- Tỷ lệ cache hit thấp (vd: 2 block liên tục cùng map vào 1 line thì khả năng Cache miss rất cao.)

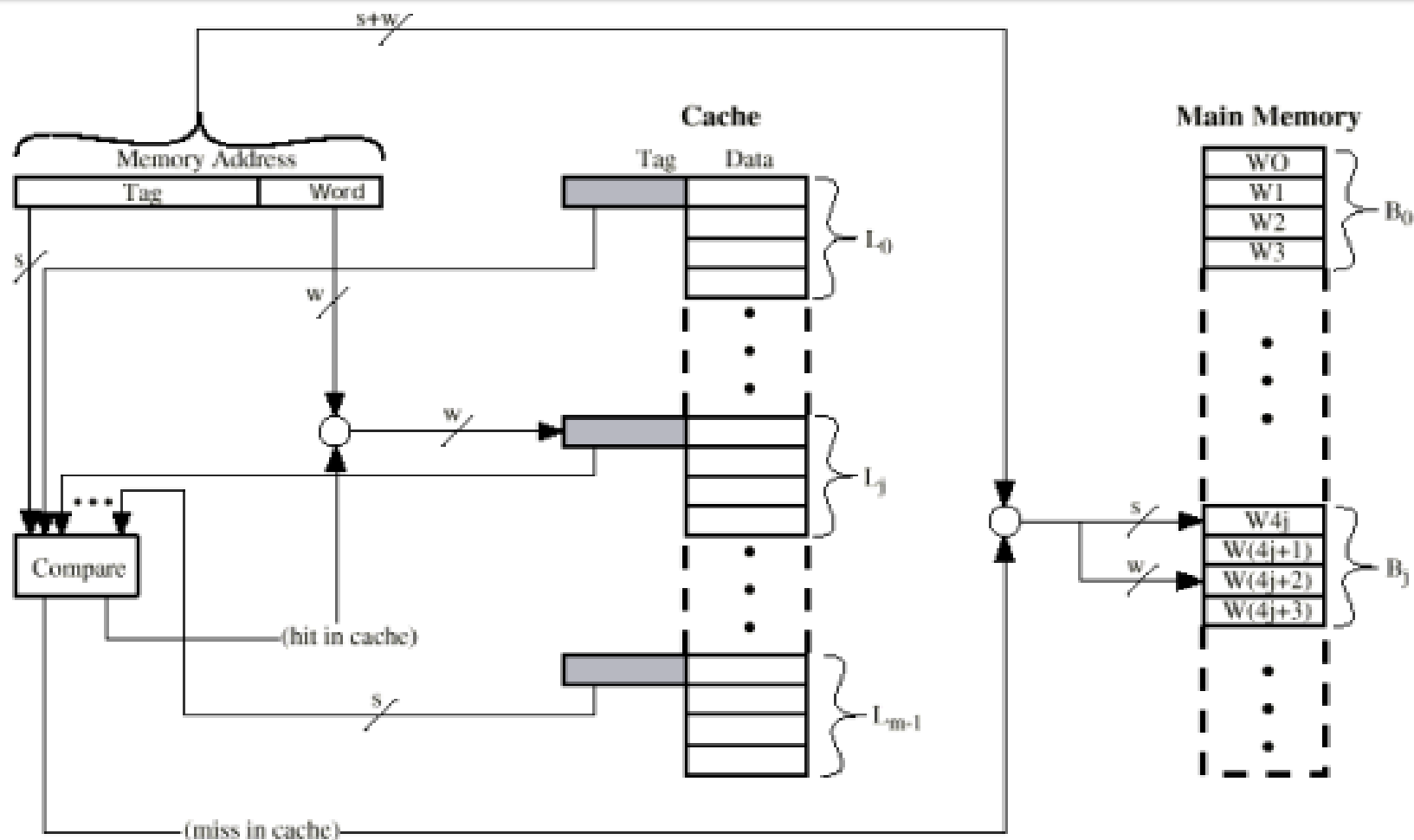


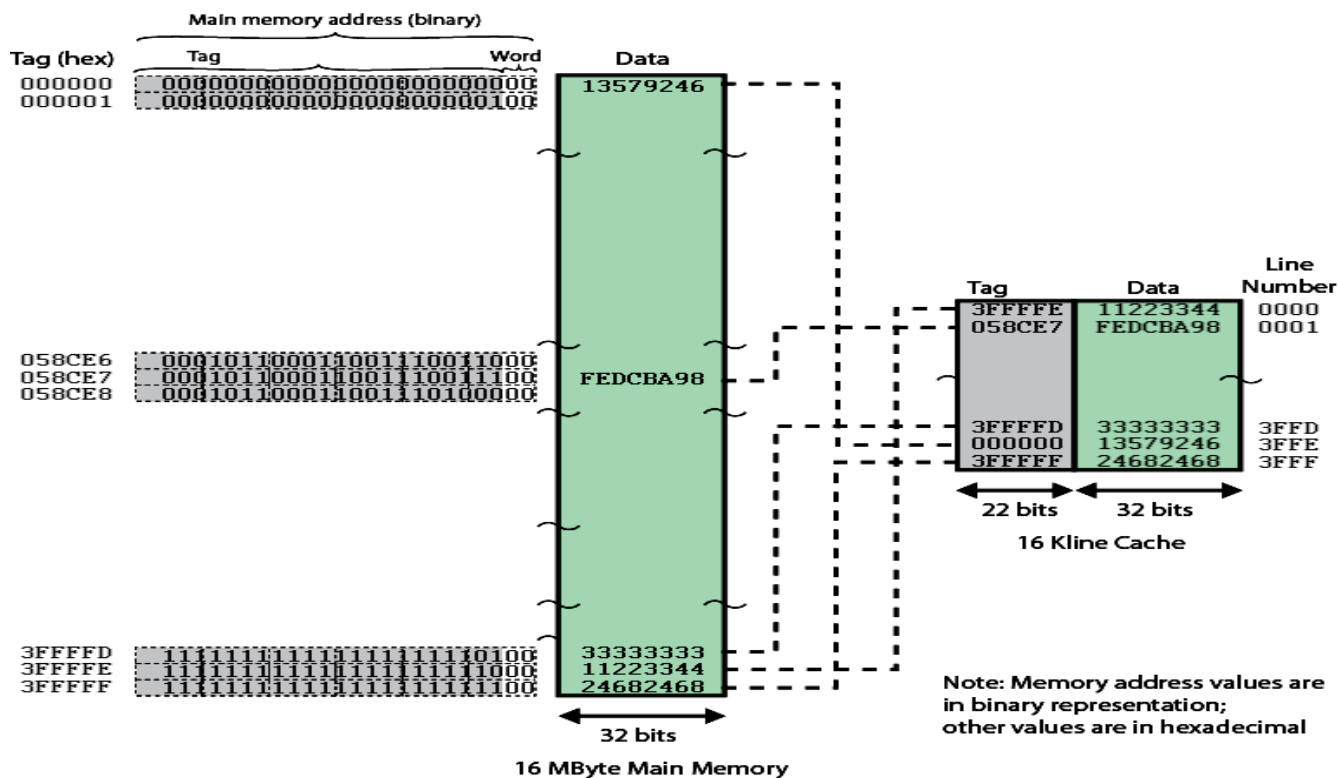
Ánh xạ kết hợp (Associative mapping)

- Mỗi Block có thể nạp vào bất kỳ Line nào của cache.
- Địa chỉ của CPU phát ra bao gồm hai vùng:
 - Vùng Word giống như trường hợp ánh xạ trực tiếp.
 - Vùng Tag dùng để xác định Block của bộ nhớ chính.
- Tag xác định Block đang nằm ở Line đó

➤ Đặc điểm

- So sánh đồng thời Block nhớ với tất cả các Tag → mất nhiều thời gian
- Tỷ lệ cache hit cao.
- Bộ so sánh phức tạp

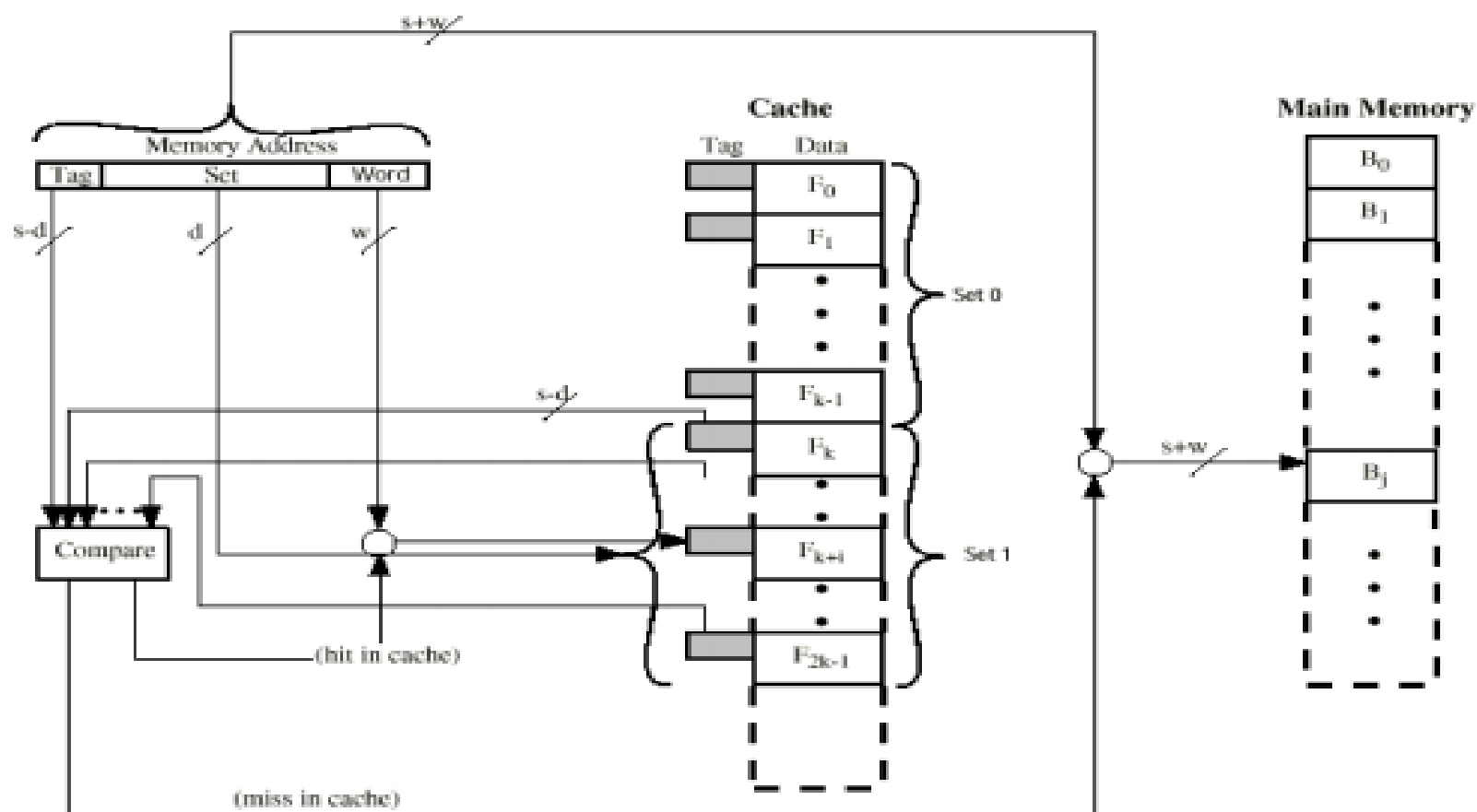


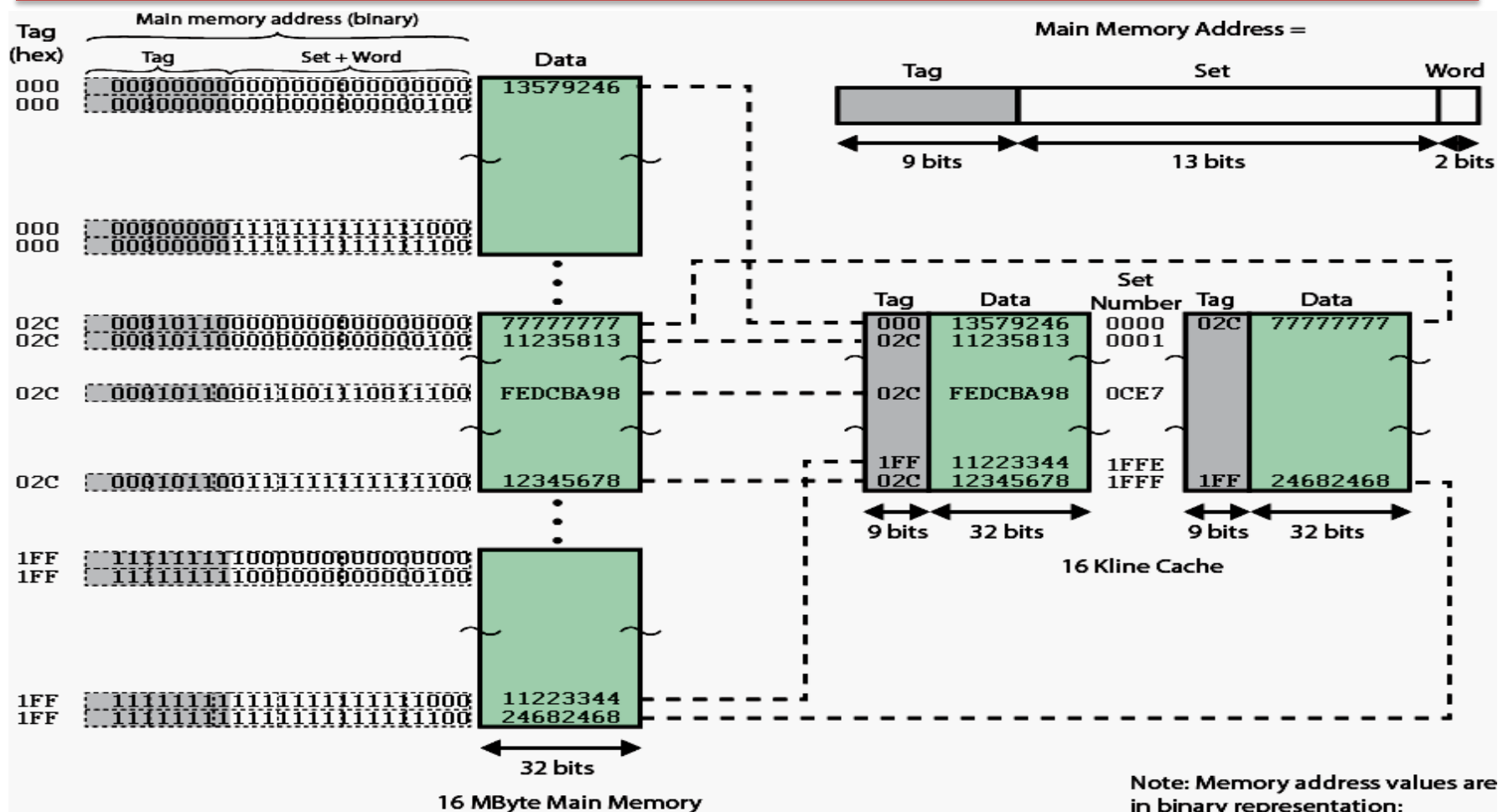




Ảnh xạ kết hợp theo bộ (Set Associative mapping)

- Cache được chia thành các bộ (Set)
- Mỗi một Set chứa một số Line
- Ví dụ:
 - 4 Line/Set $\rightarrow$ 4-way associative mapping
- Ảnh xạ theo nguyên tắc sau:
 - $B_0 \rightarrow S_0$
 - $B_1 \rightarrow S_1$
 - $B_2 \rightarrow S_2$





Đặc điểm ánh xạ kết hợp theo bộ

- Kích thước Block = 2^W Word
- Vùng Set có S bit dùng để xác định một trong số $V = 2^S$ Set (dùng để xác định Set nào trong cache)
- Vùng Tag có T bit: $T = N - (W+S)$ dùng để xác định Line nào có trong Set
- Mỗi Block bộ nhớ có thể ánh xạ vào Line bất kỳ trong 1 Set tương ứng
- Là phương pháp tổng hợp từ hai phương pháp trên
- Thông thường sử dụng 2, 4, 8, 16 Lines/Set



Ví dụ về ánh xạ địa chỉ

- Không gian địa chỉ bộ nhớ chính = 4GB
- Dung lượng bộ nhớ cache là 256KB
- Kích thước Line (Block) = 32byte.
- Xác định số bit của các trường địa chỉ cho ba trường hợp tổ chức:
 - Direct mapping
 - Fully associative mapping
 - 4-way set associative mapping



Ví dụ: Direct mapping

- Bộ nhớ chính : $4\text{GB} = 2^{32} \text{ byte} \rightarrow N = 32 \text{ bit}$
- Cache : $256 \text{ KB} = 2^{18} \text{ byte}$.
- Line : $32 \text{ byte} = 2^5 \text{ byte} \rightarrow W = 5 \text{ bit}$
- Line trong cache : $2^{18} / 2^5 = 2^{13} \text{ Line} \rightarrow L = 13 \text{ bit}$
- $T = 32 - (13 + 5) = 14 \text{ bit}$

Tag	Line	Word
14 bit	13 bit	5 bit

Ví dụ: Fully associative mapping

- Bộ nhớ chính : $4\text{GB} = 2^{32} \text{ byte} \rightarrow N = 32 \text{ bit}$
- Line : $32 \text{ byte} = 2^5 \text{ byte} \rightarrow W = 5 \text{ bit}$
- Số bit của vùng Tag : $T = 32 - 5 = 27 \text{ bit}$

Tag	Word
27 bit	5 bit

Ví dụ: 4-way set associative mapping

- Bộ nhớ chính : $4\text{GB} = 2^{32} \text{ byte} \rightarrow N = 32 \text{ bit}$
- Line : $32 \text{ byte} = 2^5 \text{ byte} \rightarrow W = 5 \text{ bit}$
- Line trong cache = $2^{18} / 2^5 = 2^{13} \text{ Line}$
- Một Set có 4 Line = 2^2 Line
- Set trong cache = $2^{13} / 2^2 = 2^{11} \text{ Set} \rightarrow S = 11 \text{ bit}$
- Số bit của vùng Tag : $T = 32 - (11 + 5) = 16 \text{ bit}$

Tag	Set	Word
16 bit	11 bit	5 bit



Giải thuật thay thế cache

- Mục đích: Giải thuật thay thế dùng để xác định Line nào trong cache sẽ được chọn để đưa dữ liệu vào cache khi không còn chỗ trống
- Thường được cài đặt bằng phần cứng để tăng tốc độ xử lý
- Đối với ánh xạ trực tiếp:
 - Không phải lựa chọn
 - Mỗi Block chỉ ánh xạ vào một Line xác định

○ Giải thuật thay thế với ánh xạ kết hợp

- FIFO (First In First Out): Thay thế Block nào nằm lâu nhất ở trong Set đó
- LFU (Least Frequently Used): Thay thế Block nào trong Set có số lần truy cập ít nhất trong cùng một khoảng thời gian
- LRU (Least Recently Used): Thay thế Block ở trong Set tương ứng có thời gian lâu nhất không được tham chiếu tới.
- Giải thuật tối ưu nhất: LRU



Đặc điểm ghi dữ liệu ra cache

- Chỉ ghi vào 1 block trong cache khi nội dung trong bộ nhớ chính thay đổi
- Nếu CPU ghi ra cache, ô nhớ tương ứng bị lạc hậu (invalid) → cần update ra BN chính. Ngược lại, nếu ghi vào BN chính, nội dung trong cache sẽ bị invalid → cần update lại cache
- Trong hệ đa xử lý có nhiều CPU với cache riêng, khi ghi vào 1 cache, các cache khác sẽ bị invalid



Phương pháp ghi dữ liệu khi cache hit

- Write-through:
 - Ghi cả cache và cả bộ nhớ chính
 - Tốc độ chậm
 - Cho phép CPU khác hoặc IO truy cập dữ liệu đã ghi từ BN
- Write-back:
 - Chỉ ghi ra cache
 - Tốc độ nhanh
 - CPU khác hoặc IO không đọc được dữ liệu mới trong BN
 - Khi Block trong cache bị thay thế cần phải ghi Block này về bộ nhớ chính

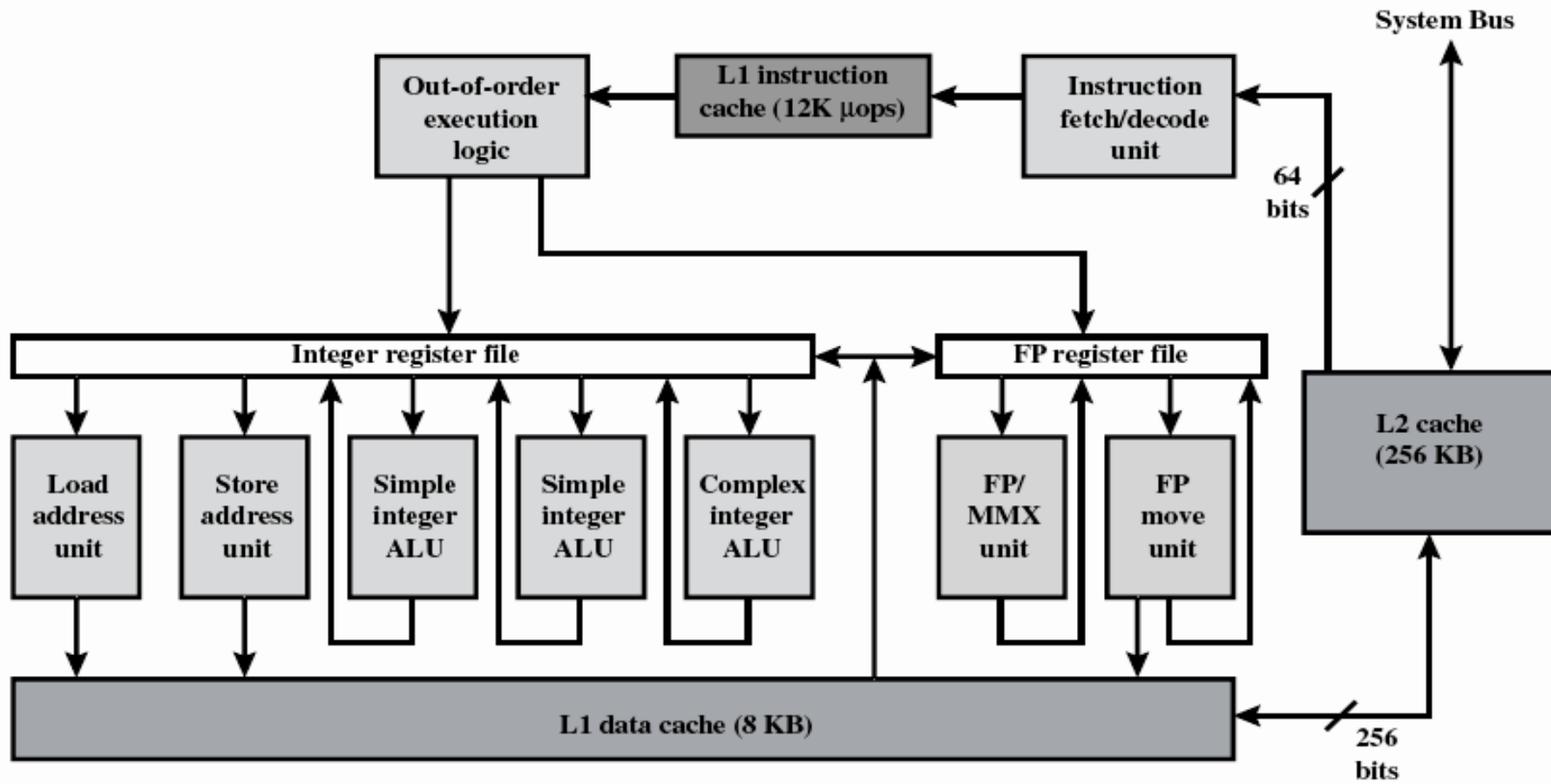


Cache tách biệt và cache đồng nhất

- **Cache tách biệt** (Split Cache): Tổ chức cache riêng cho dữ liệu và cache riêng cho lệnh chương trình
 - Tối ưu cho từng loại cache
 - Hỗ trợ kiến trúc pipeline
 - Phần cứng phức tạp
- **Cache đồng nhất** (Unified Cache): Sử dụng 1 cache chung cho cả dữ liệu lẫn lệnh chương trình
 - Cân bằng về tỷ lệ cache hit
 - Phần cứng đơn giản



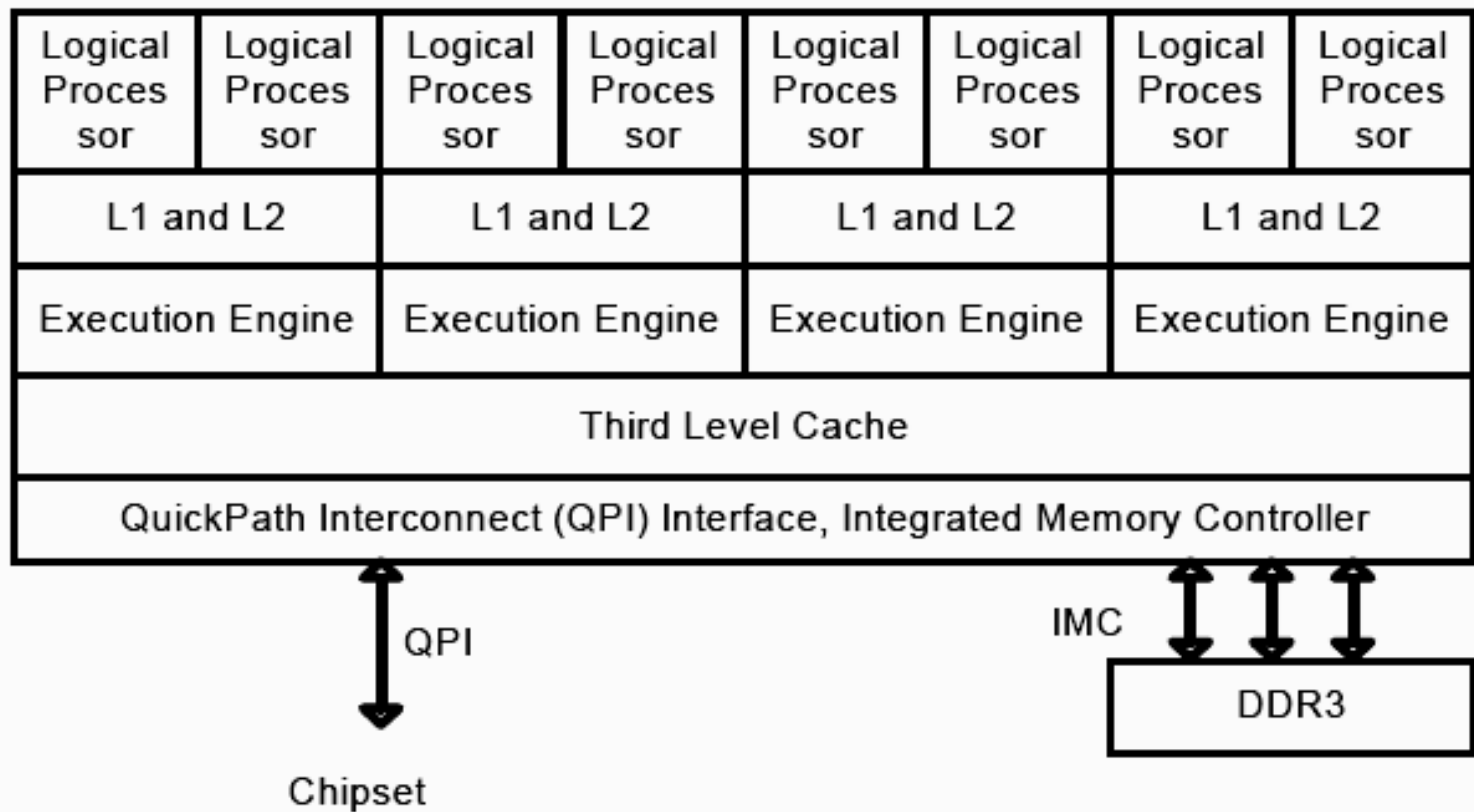
Ví dụ: Hệ thống cache trong CPU Intel Pentium 4





Ví dụ: Hệ thống cache trong CPU Intel Core i7

Intel Core i7 Processor





6.4 Bộ nhớ ngoài

➤ Các kiểu bộ nhớ ngoài

- Trống từ (Drum): Ngày nay không còn sử dụng
- Băng từ (Tape): Chuyên dùng cho backup dữ liệu
- Đĩa từ: Đang sử dụng rộng rãi nhất
- Đĩa quang: Dùng để trao đổi dữ liệu giữa các máy tính và phân phối phần mềm
- Flash Disk: Loại bộ nhớ bán dẫn gắn ngoài qua cổng USB, nhỏ gọn và thuận tiện để trao đổi dữ liệu
- SSD (Solid State Disk): Cũng là bộ nhớ bán dẫn có dung lượng lớn giao tiếp với máy tính tương tự ổ đĩa cứng, tốc độ truy cập cao, ít tốn điện, không ồn, chống sốc tốt → rất phù hợp với máy xách tay. Nhược điểm giá thành đắt.



Đĩa từ

- Bao gồm đĩa mềm (floppy disk) và đĩa cứng (hard disk)
- Các đặc tính đĩa từ
 - Đầu từ cố định hay đầu từ di động
 - Đĩa cố định hay thay đổi được (removable)
 - Một mặt hay hai mặt
 - Một tấm đĩa (đĩa mềm) hay nhiều tấm đĩa (đĩa cứng)
 - Cơ chế đầu từ
 - Tiếp xúc (đĩa mềm)
 - Không tiếp xúc



○ Đĩa mềm

- 8", 5.25", 3.5"
- Dung lượng nhỏ: chỉ tới 1.44MB
- Tốc độ chậm
- Hiện nay không sản xuất nữa

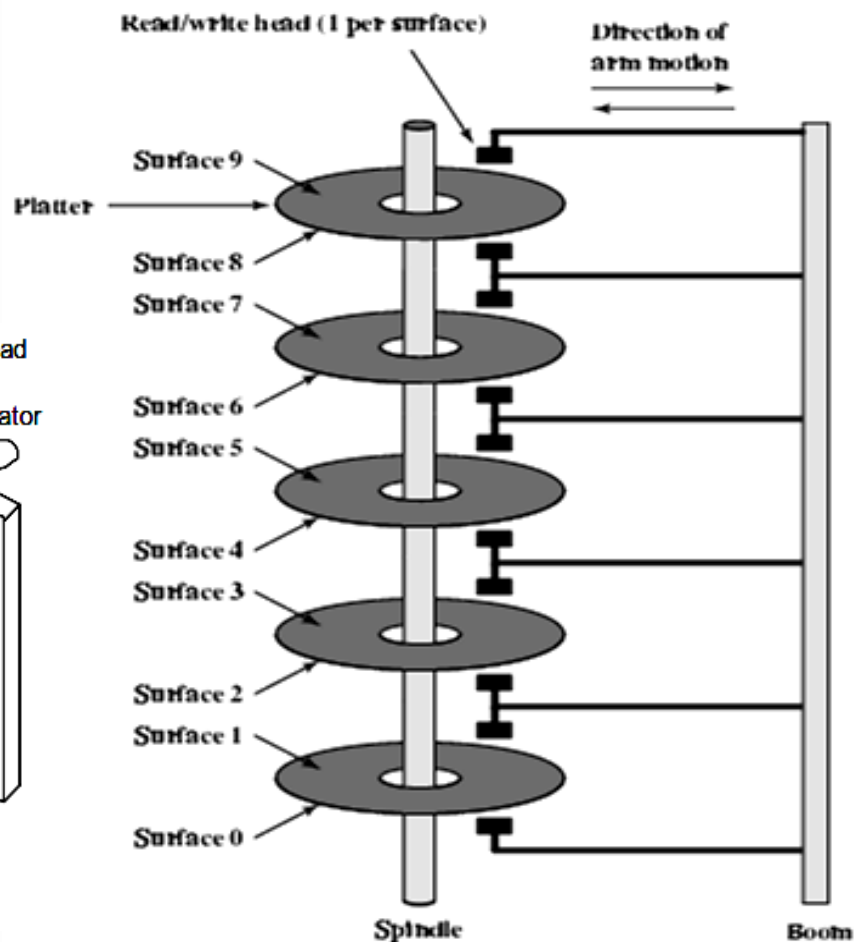
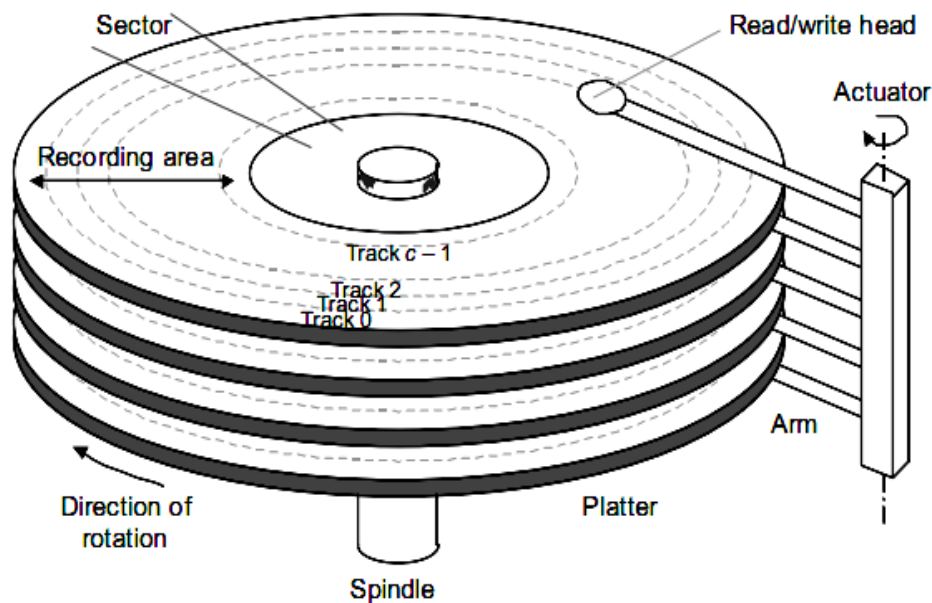
○ Đĩa cứng

- Thường có nhiều tấm đĩa
- Đang sử dụng rộng rãi
- Dung lượng lớn (hiện nay có ổ 3TB – 2011)
- Tốc độ đọc/ghi nhanh
- Rẻ tiền

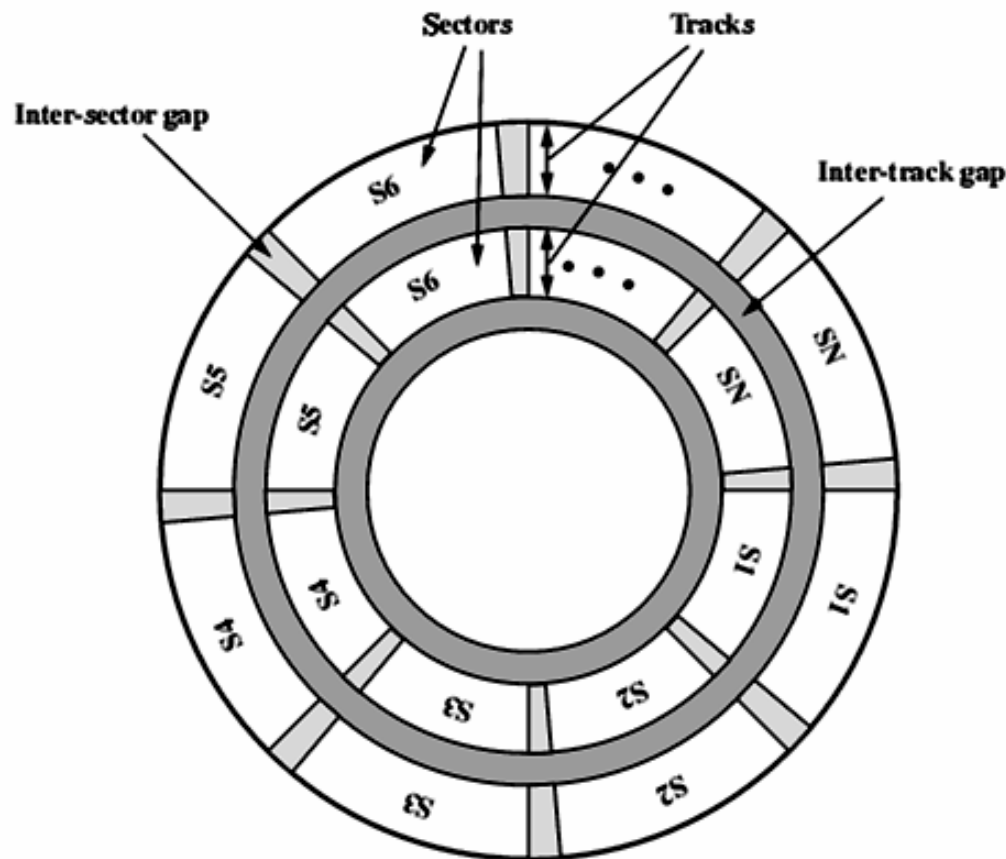


Cấu trúc vật lý đĩa cứng

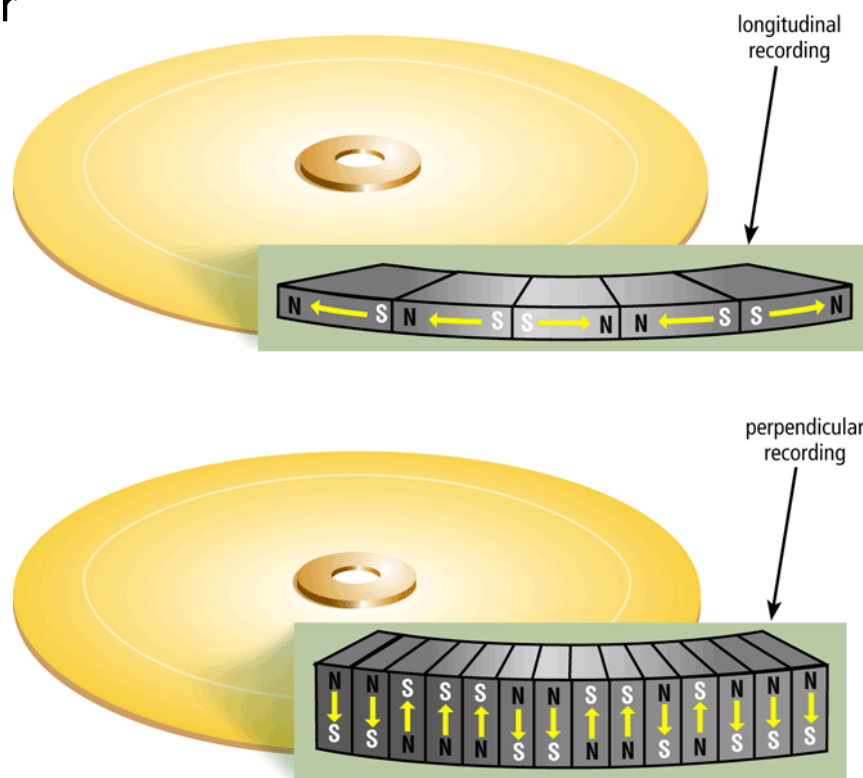
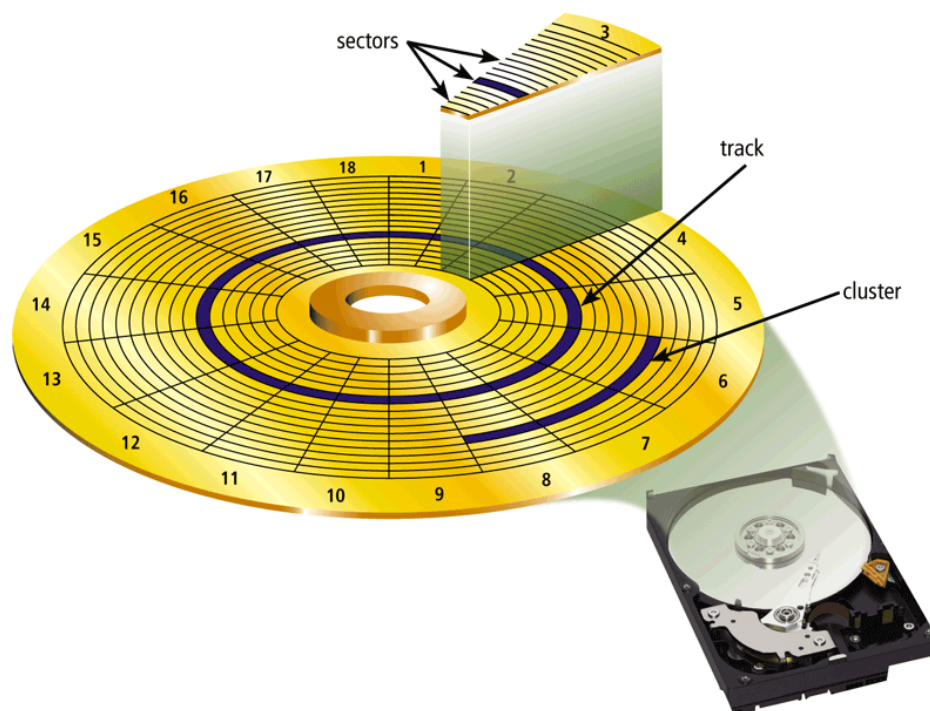
- Mặt đĩa
- Track (cylinder)
- Sector



- Track là các vòng tròn đồng tâm
- Đơn vị đọc ghi: từng sector (~ 512Byte), có thể đọc ghi theo block nhiều sector (cluster)
- Thời gian đọc ghi:
 - Seek time
 - Latency time
 - Transfer time
- Đĩa quay với vận tốc góc không đổi CAV (constant angular velocity)

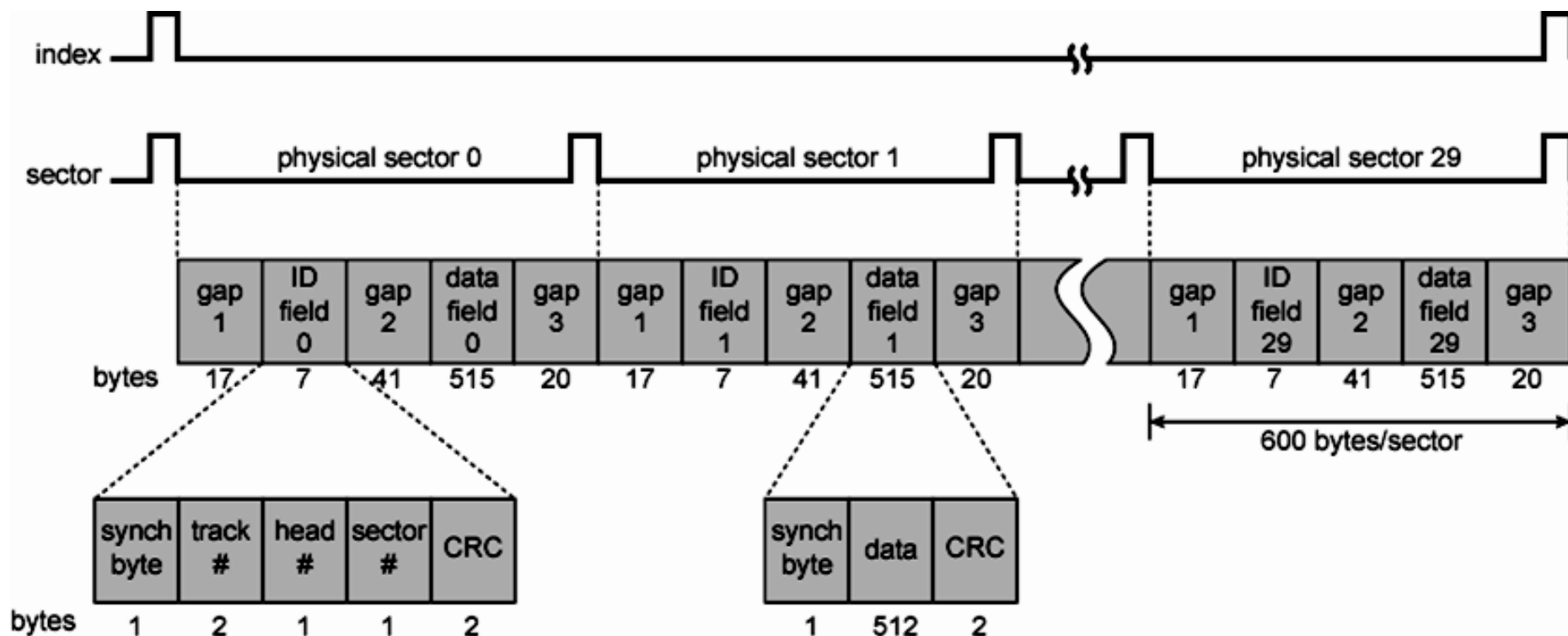


- Longitudinal recording: Ghi tuyến tính
- Perpendicular recording: Ghi trực giao
- Cluster: Một bộ gồm nhiều sector





Định dạng sector





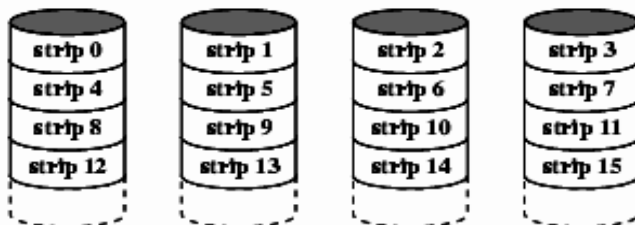
Kỹ thuật RAID

➤ Kỹ thuật RAID

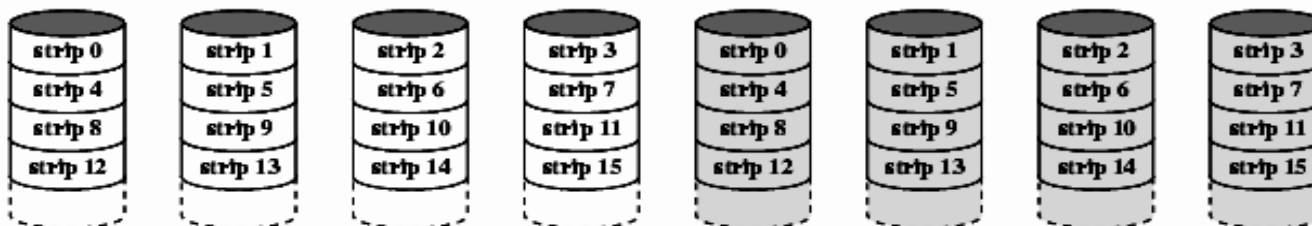
- RAID: Redundant Array of Independent Disks
- Ghép nhiều ổ đĩa vật lý để truy cập như 1 ổ luận lý
 - Tăng tốc độ truy cập (đọc ghi luân phiên và song song)
 - Tăng độ an toàn dữ liệu khi đĩa hư hỏng (ghi dư thừa hoặc ghi thêm thông tin ECC/parity)
 - Tăng dung lượng tối đa của đơn vị lưu trữ (nhiều đĩa)
- Hiện có 7 loại thông dụng: RAID0 – RAID6



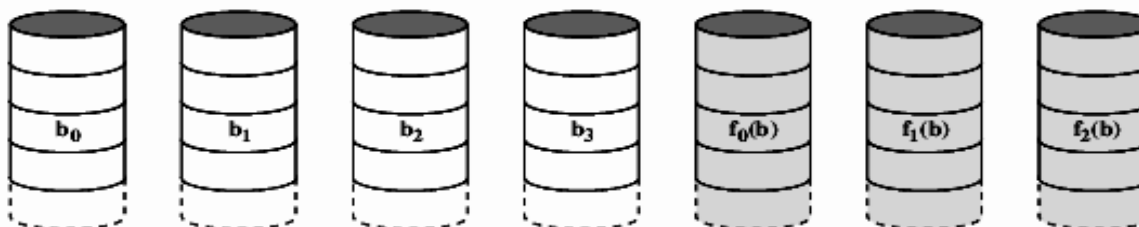
RAID 0, 1 và 2



(a) RAID 0 (non-redundant)



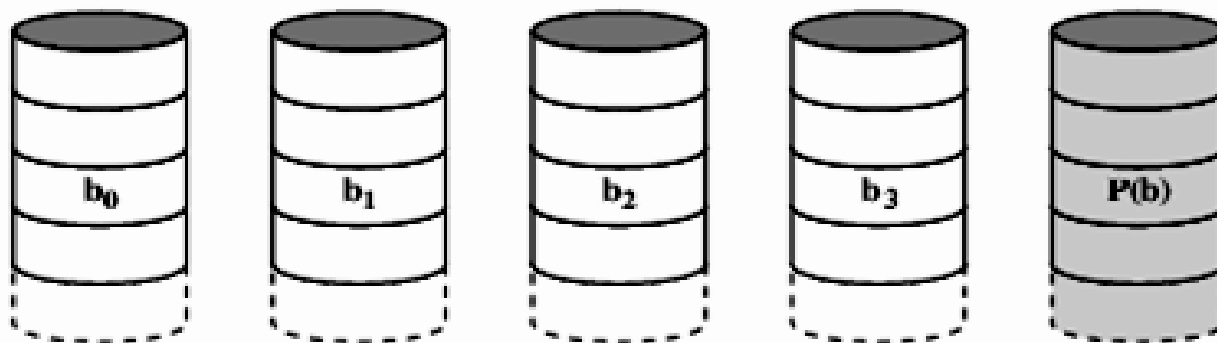
(b) RAID 1 (mirrored)



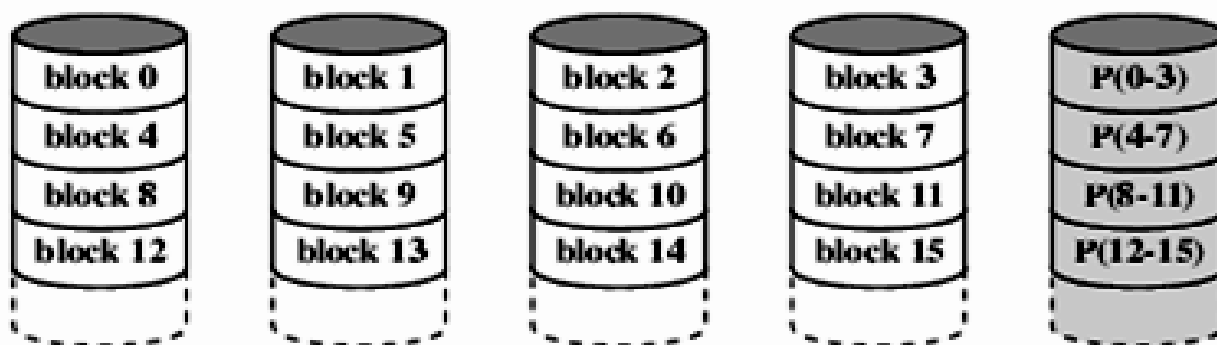
(c) RAID 2 (redundancy through Hamming code)



RAID 3 và 4



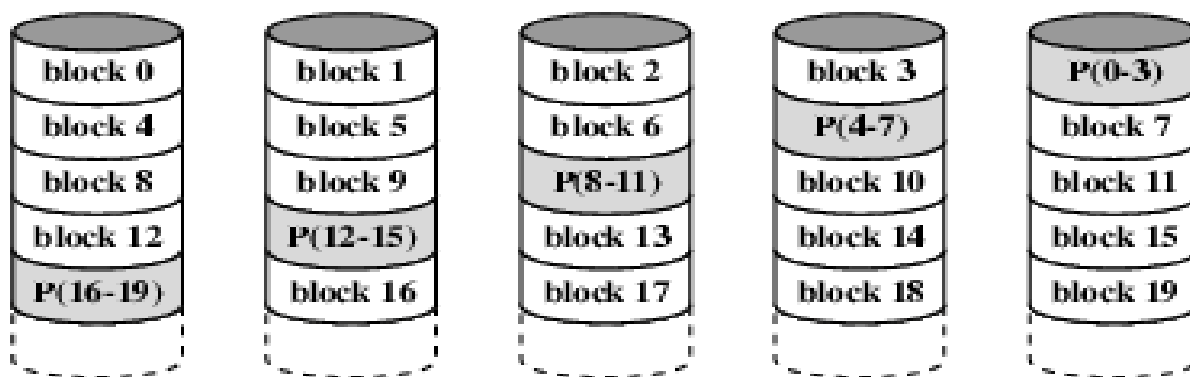
(d) RAID 3 (bit-interleaved parity)



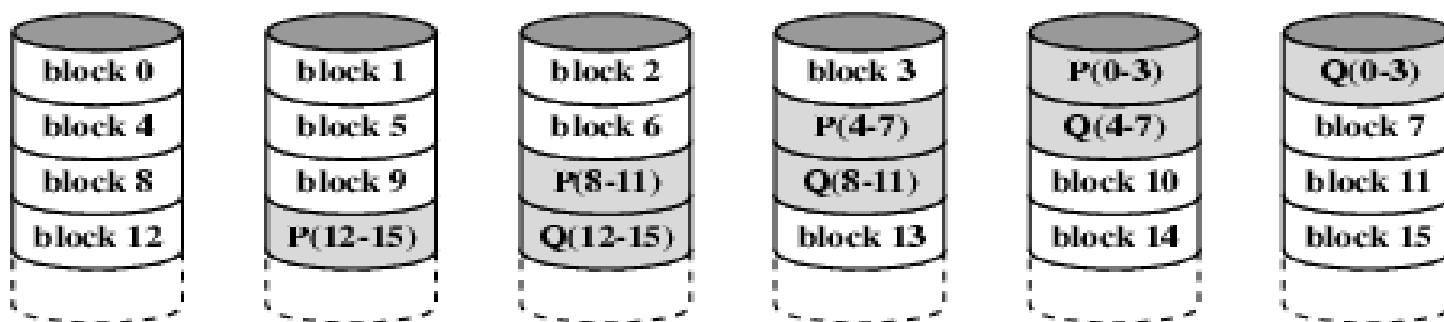
(e) RAID 4 (block-level parity)



RAID 5 và 6



(f) RAID 5 (block-level distributed parity)



(g) RAID 6 (dual redundancy)



Tóm tắt kỹ thuật RAID

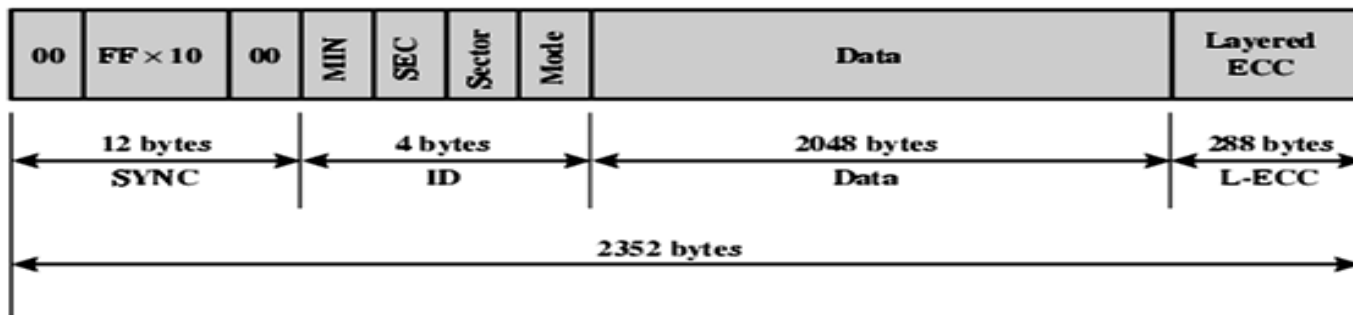
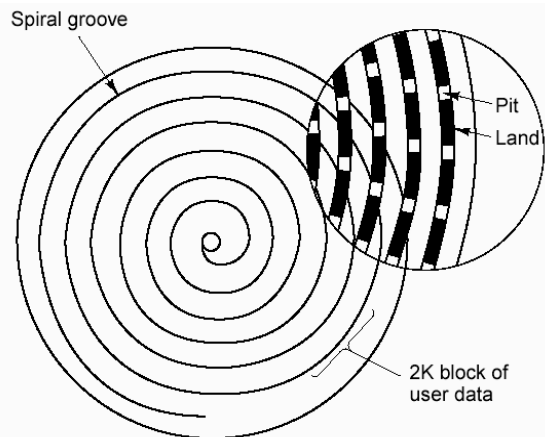
Category	Level	Description	I/O Request Rate (Read/Write)	Data Transfer Rate (Read/Write)	Typical Application
Striping	0	Non-redundant	Large strips: Excellent	Small strips: Excellent	Applications requiring high performance for non-critical data
Mirroring	1	Mirrored	Good/Fair	Fair/Fair	System drives; critical files
Parallel access	2	Redundant via Hamming code	Poor	Excellent	
	3	Bit-interleaved parity	Poor	Excellent	Large I/O request size applications, such as imaging, CAD
Independent access	4	Block-interleaved parity	Excellent/Fair	Fair/Poor	
	5	Block-interleaved distributed parity	Excellent/Fair	Fair/Poor	High request rate, read-intensive, data lookup
	6	Block-interleaved dual distributed parity	Excellent/Poor	Fair/Poor	Applications requiring extremely high availability



Đĩa quang (optical disk)

○ CD (compact disk)

- Khả năng đọc/ghi: CD-ROM, CD-R, CD-RW
- Đường kính: 12cm, 8cm
- Dung lượng: 700MB, 200MB
- Track: Ghi theo các vòng hướng tâm, tốc độ dài không đổi CLV (constant linear velocity)
- Tốc độ đọc ghi: 1x – 52x (1x= ??)
- Chuẩn định dạng: ISO 9660, UDF (Universal Disk Format)



- DVD (Digital Versatile Disk): Loại đĩa dung lượng cao (so với CD), xuất phát từ đĩa phim video (Digital Video Disk)
- Khả năng đọc ghi: DVD-ROM, DVD±R, DVD±RW, DVD-RAM
- Số mặt/ số lớp: 1-2 mặt, 1-2 lớp/mặt
- Đường kính: 12cm, 8cm
- Tốc độ: 1x – 24x (1x=?)
- Đĩa DVD dung lượng cao
 - HD-DVD (15-60GB)
 - Blue ray (25-50GB)

Sides	Layers	Diameter (cm)	Capacity (GB)
1	1	8	1.46
1	2	8	2.66
2	2	8	2.92
2	4	8	5.32
1	1	12	4.7
1	2	12	8.54
2	2	12	9.4
2	4	12	17.08



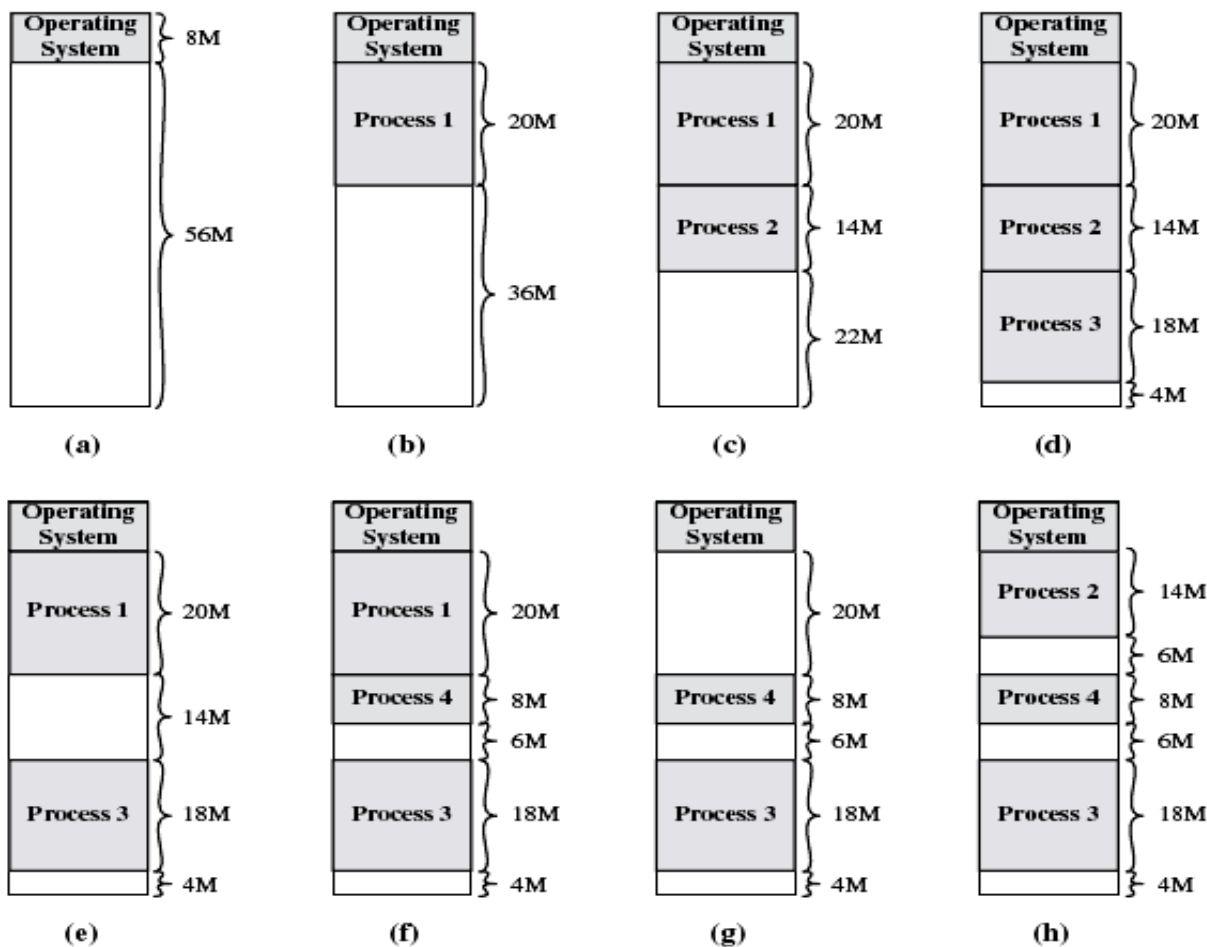
6.5 Bộ nhớ ảo

➤ Bộ nhớ thật

- Không gian địa chỉ trong chương trình trùng với không gian địa chỉ trong bộ nhớ. Cho phép người lập trình truy cập trực tiếp vào 1 ô nhớ → Khó bảo vệ bộ nhớ.
- Khi thi hành, hệ điều hành nạp toàn bộ chương trình vào bộ nhớ (nạp trước) → bộ nhớ máy tính phải đủ lớn để chạy các CT lớn
- Chương trình được cấp phát 1 vùng nhớ có địa chỉ liên tục (cấp phát liên tục). HĐH sẽ thu hồi vùng nhớ sau khi chương trình kết thúc
- Để thực hiện đa chương, HĐH cần chia BN ra nhiều vùng (partition), mỗi vùng cấp phát cho 1 CT
- Khi bộ nhớ đầy
 - HĐH không cấp tiếp, các CT phải chờ đến khi có 1 vùng nhớ trống
 - HĐH cấp tiếp: Cần kỹ thuật trao đổi (swapping) để ghi tạm vùng nhớ của 1 CT khác ra BN ngoài, lấy chỗ trống cấp cho CT mới



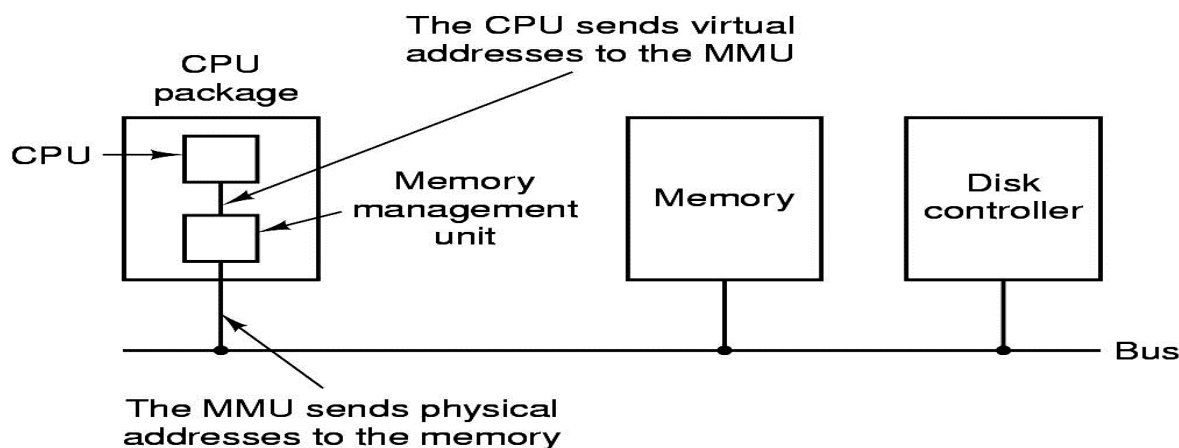
Bộ nhớ thật (tiếp)





Bộ nhớ ảo (Virtual Memory)

- Không gian địa chỉ trong CT (địa chỉ ảo) được tách biệt với không gian địa chỉ trong BN (địa chỉ thực) → CPU và HĐH sẽ phối hợp để ánh xạ (mapping) địa chỉ ảo trong CT thành địa chỉ thật trong BN
- Việc ánh xạ và quản lý BN ảo được thực hiện qua đơn vị MMU (Memory Management Unit)

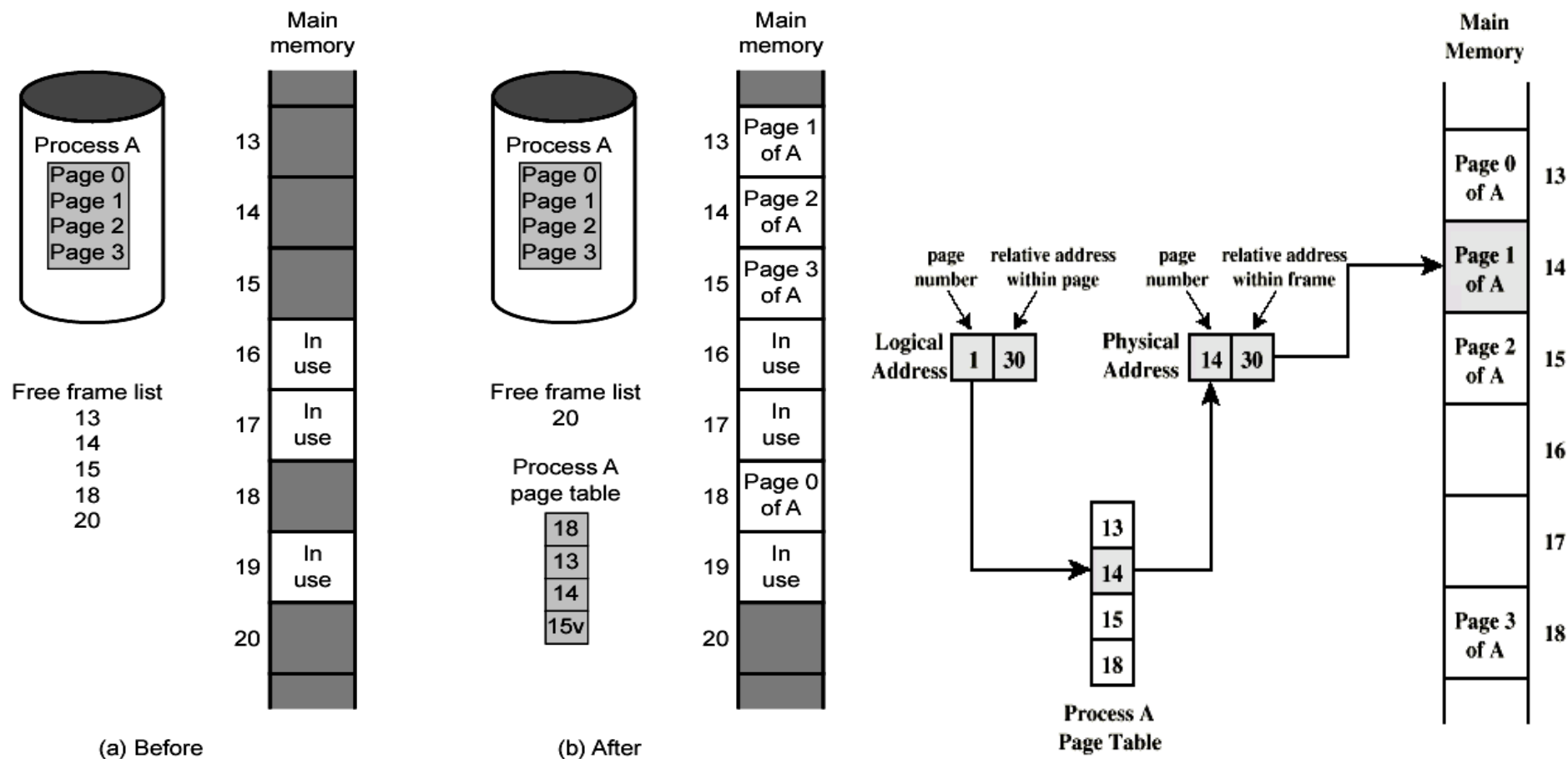


- Khi thi hành, hệ điều hành chỉ nạp các phần cần thiết của CT vào bộ nhớ (nạp theo yêu cầu), không cần nạp toàn bộ CT → tránh lãng phí BN
- Các CT được cấp phát nhiều vùng nhớ có địa chỉ tách biệt nhau (cấp phát không liên tục).
- Sử dụng kỹ thuật trao đổi (swapping) để ghi tạm thời các vùng nhớ chưa cần đến ra BN ngoài (swap-out) để lấy chỗ trống nạp thông tin cần thiết vào BN (swap-in) khi cần đến
- BN ngoài thông dụng là đĩa cứng
- Có 2 kỹ thuật BN ảo:
 - Kỹ thuật phân trang : Kích thước các vùng nhớ cố định
 - Kỹ thuật phân đoạn : Kích thước các vùng nhớ thay đổi



Kỹ thuật phân trang (paging)

- Không gian địa chỉ ảo trong CT được chia đều ra các trang ảo (virtual page, gọi tắt là page) có kích thước bằng nhau, mỗi trang là 1 đơn vị cấp phát BN của HĐH
- Không gian địa chỉ thật trong BN cũng được chia đều thành các khung trang (page frame, gọi tắt là frame) có kích thước bằng 1 trang (thường là 4KB)
- Khi có yêu cầu cấp phát BN, HĐH có thể nạp 1 trang theo yêu cầu vào bất cứ frame nào trong BN thật
- Khi CT truy cập vào 1 trang chưa được cấp phát sẽ gây ra lỗi trang (page fault) → HĐH phải xử lý bằng cách swapping với 1 trang khác chưa cần sử dụng đến (chậm)
- HĐH cần 1 bảng quản lý để theo dõi trang nào đang được nạp vào frame nào trong BN cho mỗi CT, gọi là bảng trang (page table)



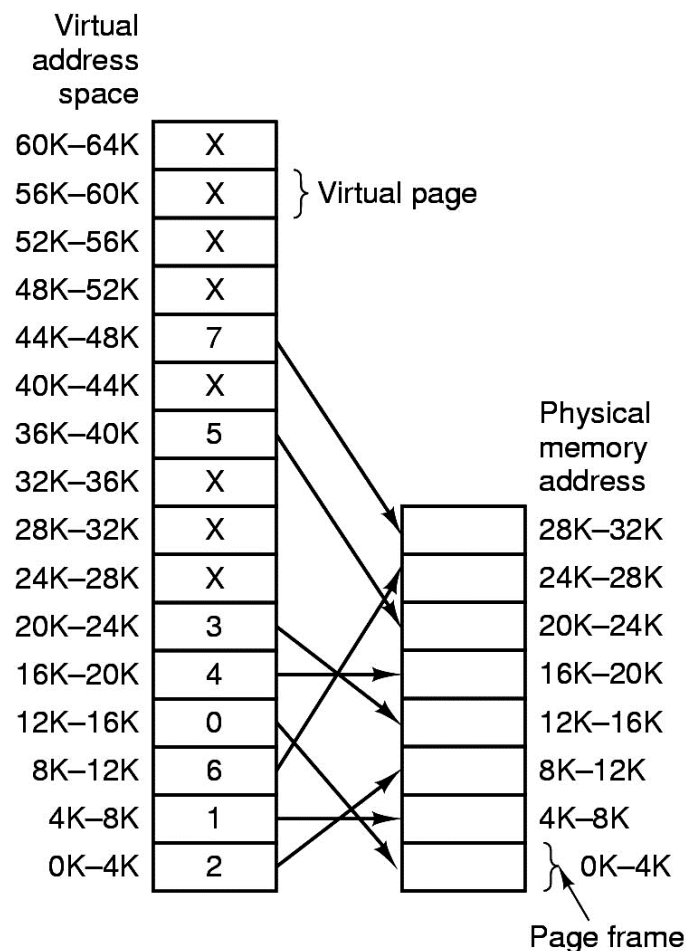


Ví dụ về BN phân trang

- BN ảo trong CT gồm 64KB được chia ra 16 trang, mỗi trang 4KB
- BN thực gồm 32KB được chia ra 8 frame
- BN đang được cấp phát như thể hiện trong bảng trang

Bài tập: Hãy tính địa chỉ thật từ các địa chỉ ảo

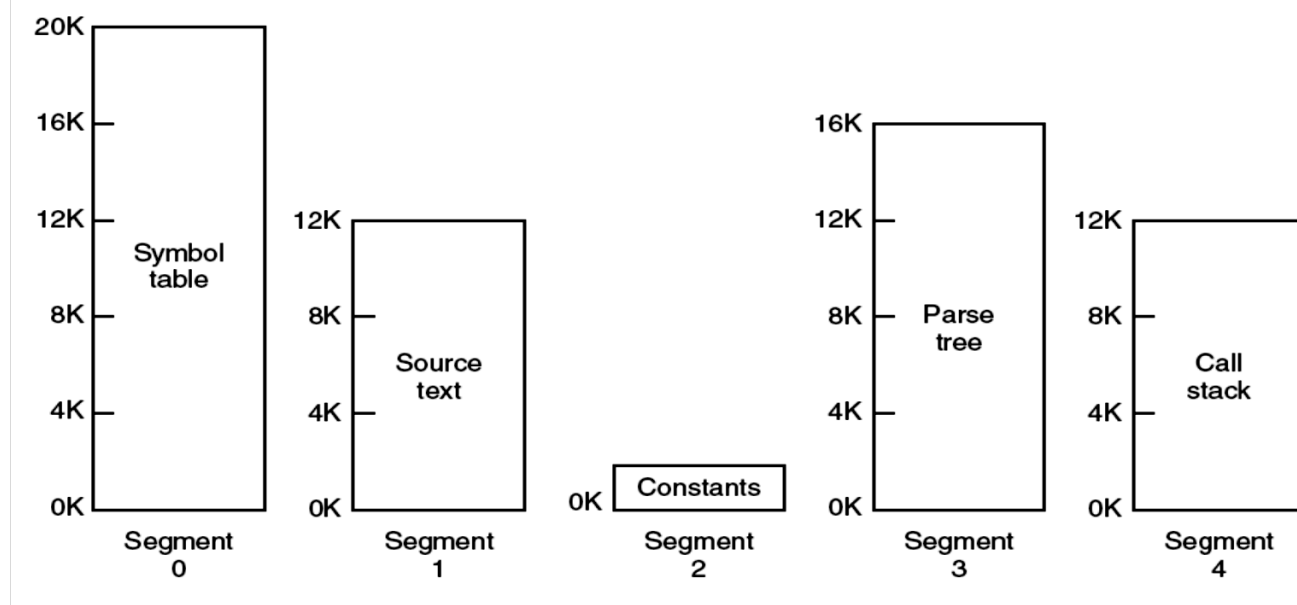
- 10.000
- 20.000
- 30.000





Kỹ thuật phân đoạn (segmentation)

- Quan điểm người lập trình về BN
 - Chương trình bao gồm nhiều module
 - Dữ liệu bao gồm nhiều array, chuỗi, ...
 - Khi truy cập sẽ căn cứ vào địa chỉ tương đối của module (lệnh thứ mấy) hay array (phần tử thứ mấy)

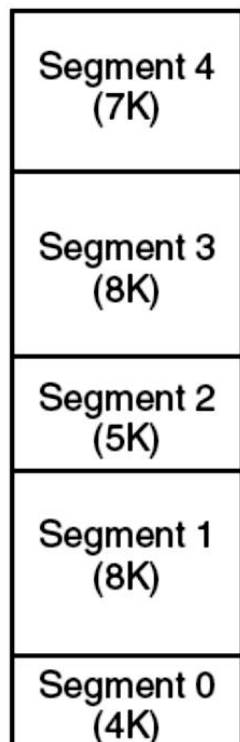




- HĐH sẽ cấp phát BN theo từng đoạn (segment) có kích thước theo yêu cầu lập trình, người lập trình truy cập BN theo offset trong từng segment
- Địa chỉ ảo có dạng (segment, offset)
- Khi có yêu cầu cấp phát BN, HĐH có thể nạp 1 segment theo yêu cầu vào vùng trống trong BN thật. Nếu không có vùng trống đủ lớn HĐH cần dời BN để tạo ra vùng trống đủ lớn.
- Khi CT truy cập vào 1 segment chưa được cấp phát sẽ gây ra lỗi segment (segment fault) → HĐH phải xử lý bằng cách swapping với 1 hoặc vài segment khác chưa cần sử dụng đến (chậm)
- HĐH cần 1 bảng quản lý để theo dõi segment nào đang được nạp vào vị trí nào trong BN cho mỗi CT, gọi là bảng segment (segment table)

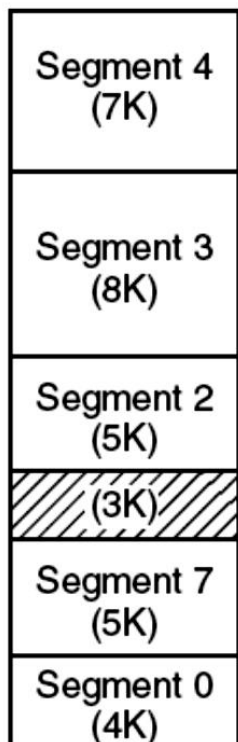


Ví dụ về BN phân đoạn



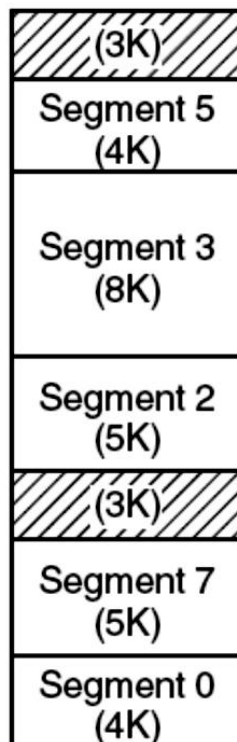
(a)

Ban đầu



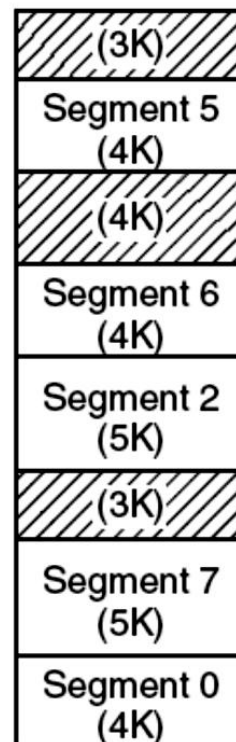
(b)

S1 swap-out
S7 swap-in



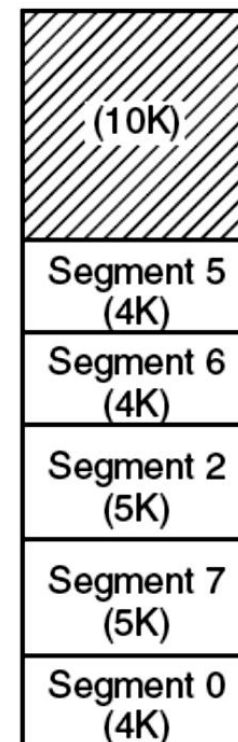
(c)

S4 swap-out
S5 swap-in



(d)

S3 swap-out
S6 swap-in

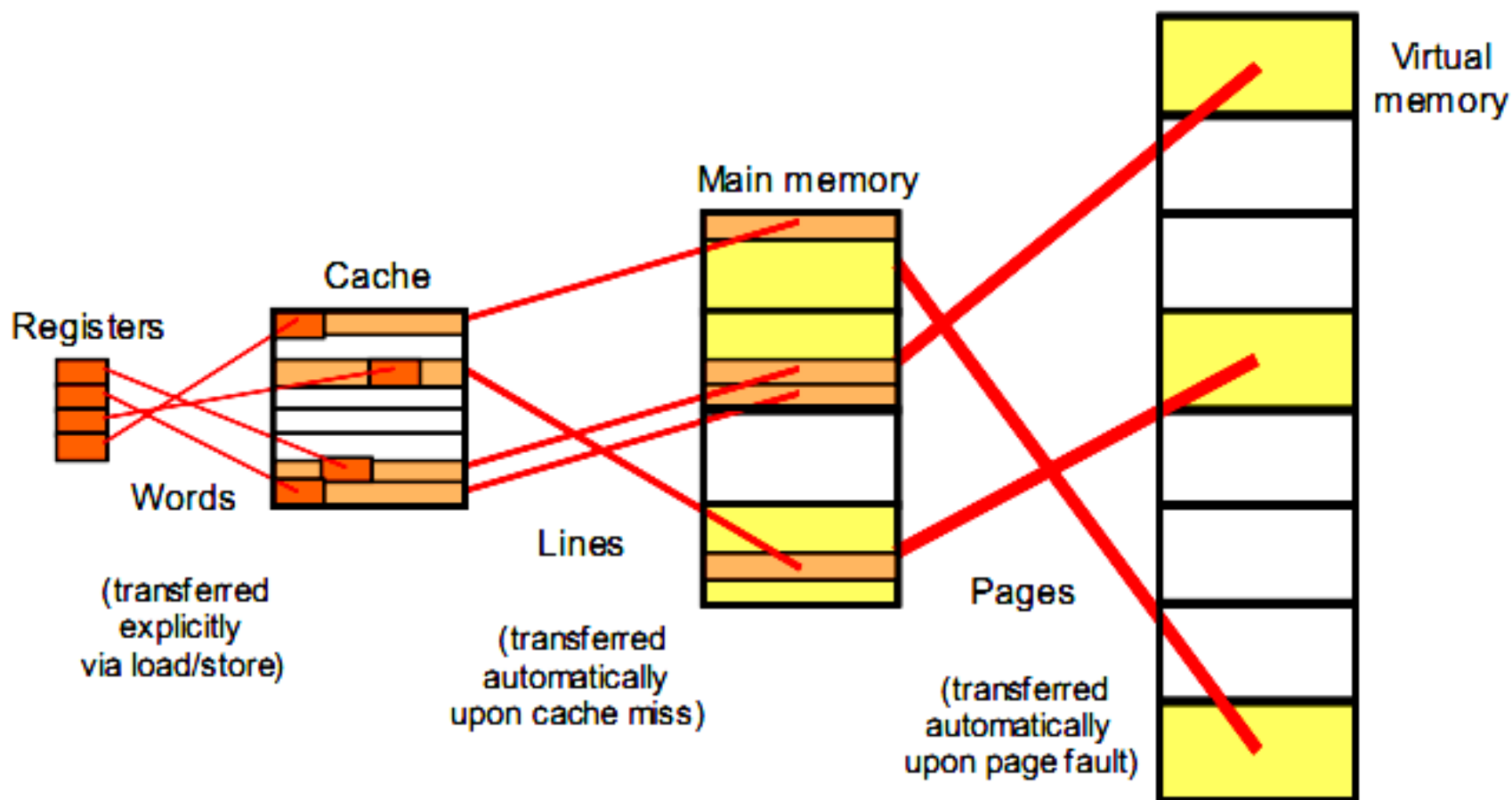


(e)

Dồn bộ nhớ



Tổng quát việc truy cập bộ nhớ





Đánh giá Bộ nhớ ảo

➤ Ưu điểm BN ảo

- Cho phép CT lớn hơn BN vẫn chạy được
- Chỉ nạp phần CT nào cần đến vào BN → tiết kiệm BN

➤ Nhược điểm BN ảo

- Tăng phí tổn hệ thống (overhead): Tốn thời gian tính toán địa chỉ ảo sang địa chỉ thật, tốn không gian BN chứa bảng trang/ segment
- Truy cập BN chậm hơn so với quản lý BN thực: Cần gấp đôi thời gian truy cập BN. Khi có page/ segment fault việc truy cập BN biến thành truy cập IO

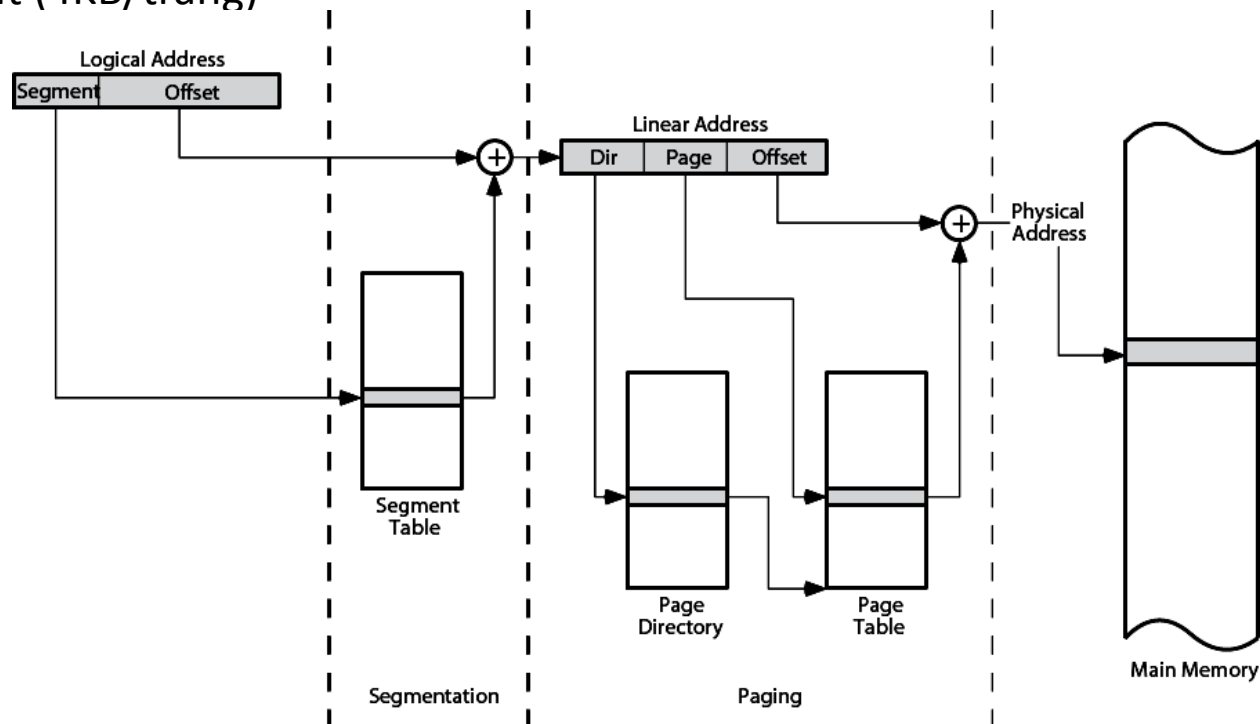
➤ Cách khắc phục

- Cần phần cứng đặc biệt hỗ trợ HĐH để quản lý BN
- Cần giải thuật thay trang/ segment tối ưu



Ví dụ: BN ảo trong CPU Intel Pentium 4

- Phân segment kết hợp phân trang 2 cấp
 - Phân segment: Segment 16 bit, Offset: 32 bit.
 - Phân trang: Địa chỉ tuyến tính 32 bit chia ra: Directory 10 bit, page 10 bit và offset 12 bit (4KB/trang)





Câu hỏi





Chương 7 Hệ thống IO (Input Output System)

1. Tổng quan về hệ thống IO
2. Điều khiển IO
3. Nối ghép thiết bị ngoại vi
4. Các thiết bị ngoại vi thông dụng



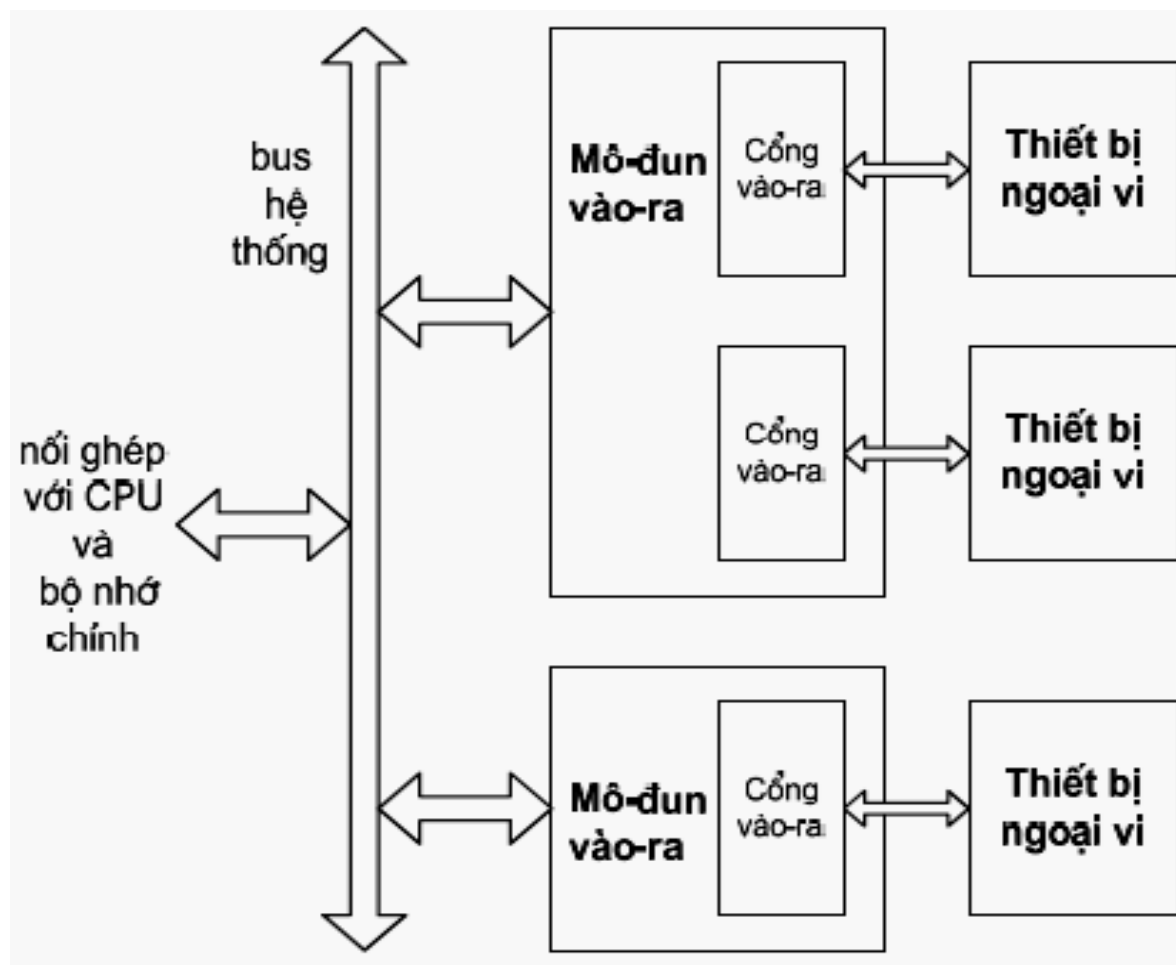
7.1 Tổng quan về hệ thống IO

➤ Giới thiệu chung

- Chức năng của hệ thống IO: Trao đổi thông tin giữa máy tính với thế giới bên ngoài
 - Các thao tác cơ bản:
 - Nhập dữ liệu (Input)
 - Xuất dữ liệu (Output)
 - Các thành phần chính:
 - Các thiết bị ngoại vi
 - Các mô-đun IO (IO module)
- ☞ Tất cả các thiết bị ngoại vi đều chậm hơn CPU và RAM → Cần có các mô-đun IO để nối ghép các thiết bị ngoại vi với CPU và bộ nhớ chính



Cấu trúc cơ bản của hệ thống IO



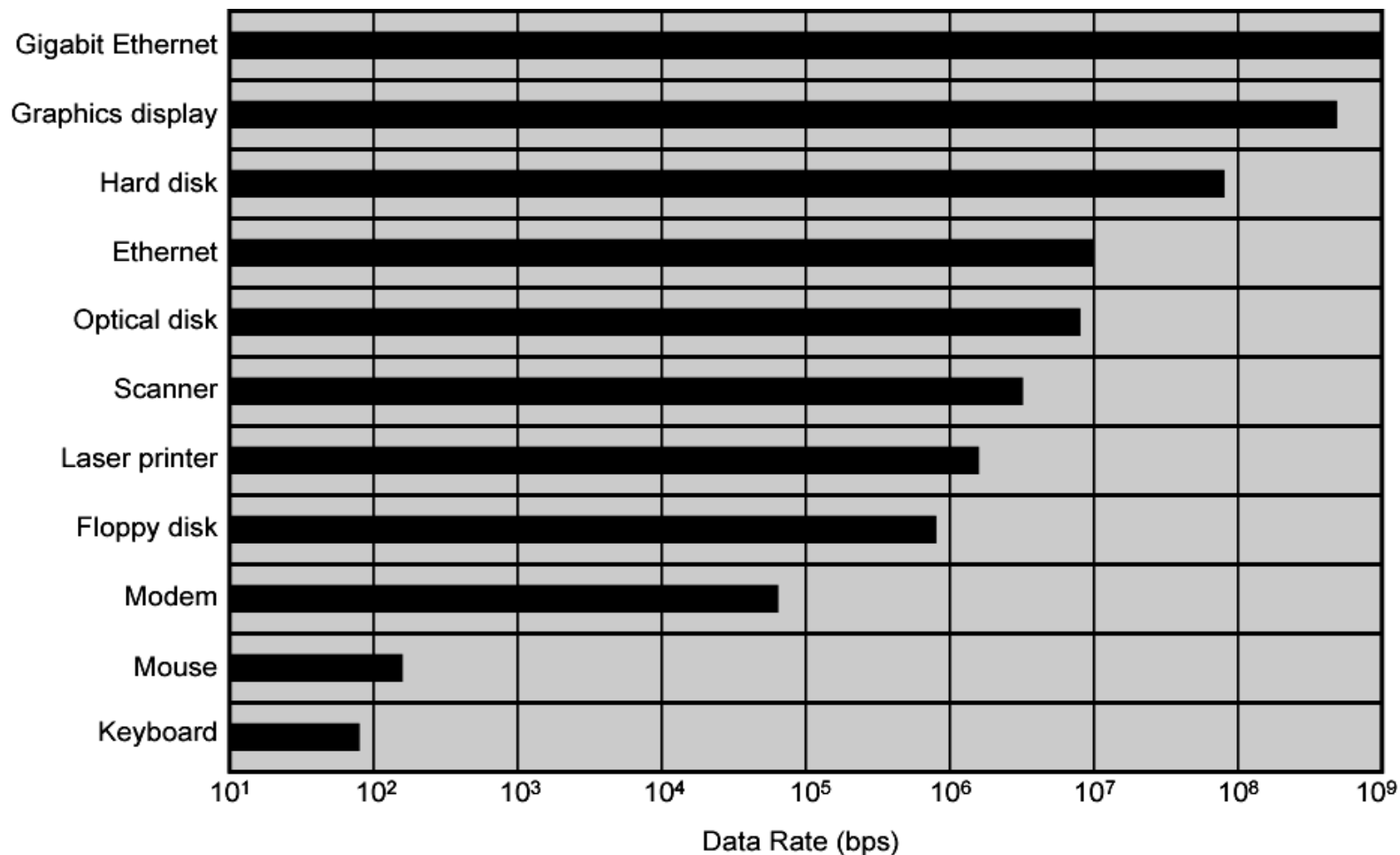


Các thiết bị ngoại vi

- Chức năng: chuyển đổi dữ liệu giữa bên trong và bên ngoài máy tính
- Phân loại:
 - Thiết bị ngoại vi giao tiếp người-máy (người đọc): Bàn phím, Màn hình, Máy in,...
 - Thiết bị ngoại vi giao tiếp máy-máy (máy đọc): Đĩa cứng, CDROM, USB,...
 - Thiết bị ngoại vi truyền thông: Modem, Network Interface Card (NIC)



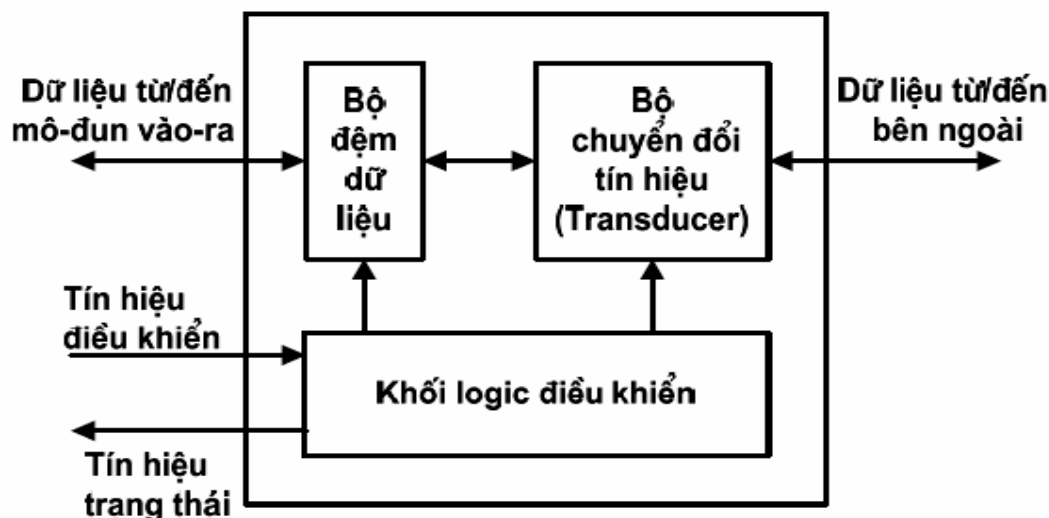
Tốc độ 1 số TBNV





Các thành phần của thiết bị ngoại vi

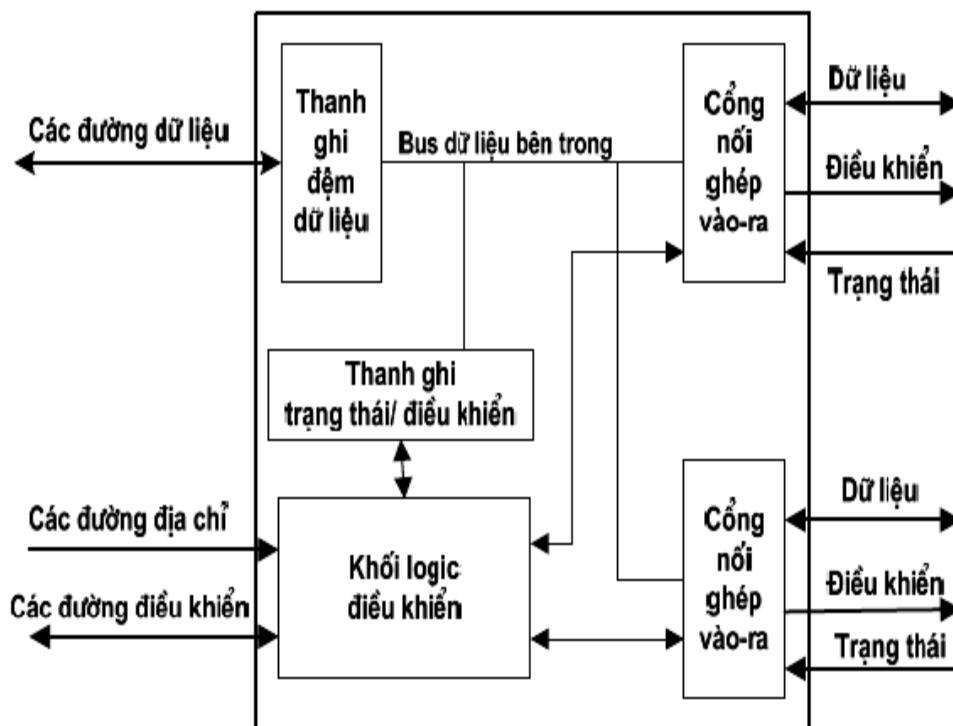
- Bộ chuyển đổi tín hiệu: chuyển đổi dữ liệu giữa bên ngoài và bên trong máy tính
- Bộ đệm dữ liệu: đệm dữ liệu khi truyền giữa mô-đun IO và thiết bị ngoại vi
- Khối logic điều khiển: điều khiển hoạt động của thiết bị ngoại vi đáp ứng theo yêu cầu từ mô-đun IO





Chức năng của mô-đun IO:

- Điều khiển và định thời
- Trao đổi thông tin với CPU hoặc bộ nhớ chính
- Trao đổi thông tin với thiết bị ngoại vi
- Đệm giữa bên trong máy tính với thiết bị ngoại vi
- Phát hiện lỗi của thiết bị ngoại vi





Không gian địa chỉ của CPU

- Một số CPU quản lý duy nhất một không gian địa chỉ:
 - Không gian địa chỉ bộ nhớ: 2^M địa chỉ
- Một số CPU quản lý hai không gian địa chỉ tách biệt:
 - Không gian địa chỉ bộ nhớ: 2^M địa chỉ
 - Không gian địa chỉ IO: 2^I địa chỉ
 - Có tín hiệu điều khiển phân biệt truy nhập không gian địa chỉ
 - Tập lệnh có các lệnh IO chuyên dụng
- Ví dụ: CPU Intel Pentium 4
 - Không gian địa chỉ bộ nhớ = 2^{36} byte = 64GB
 - Không gian địa chỉ IO = 2^{16} byte = 64KB
 - Lệnh IO chuyên dụng: IN, OUT



Các phương pháp địa chỉ hoá cổng IO

➤ IO riêng biệt (Isolated IO, IO mapped IO)

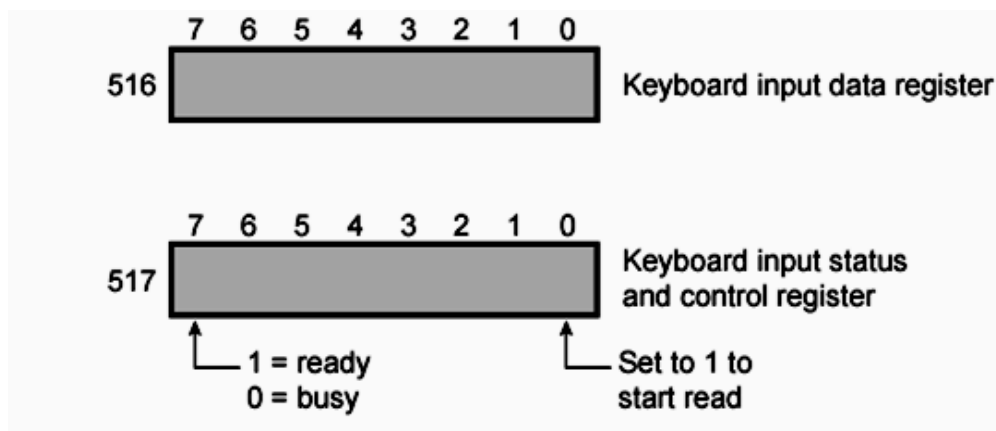
- Cổng IO được đánh địa chỉ theo không gian địa chỉ IO
- CPU trao đổi dữ liệu với cổng IO thông qua các lệnh IO chuyên dụng (IN, OUT)
- Chỉ có thể thực hiện trên các hệ thống có quản lý không gian địa chỉ IO riêng biệt

➤ IO theo bộ nhớ (Memory mapped IO)

- Cổng IO được đánh địa chỉ theo không gian địa chỉ bộ nhớ
- IO giống như đọc/ghi bộ nhớ
- CPU trao đổi dữ liệu với cổng IO thông qua các lệnh truy nhập dữ liệu bộ nhớ
- Có thể thực hiện trên mọi hệ thống



Ví dụ: So sánh 2 phương pháp IO



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

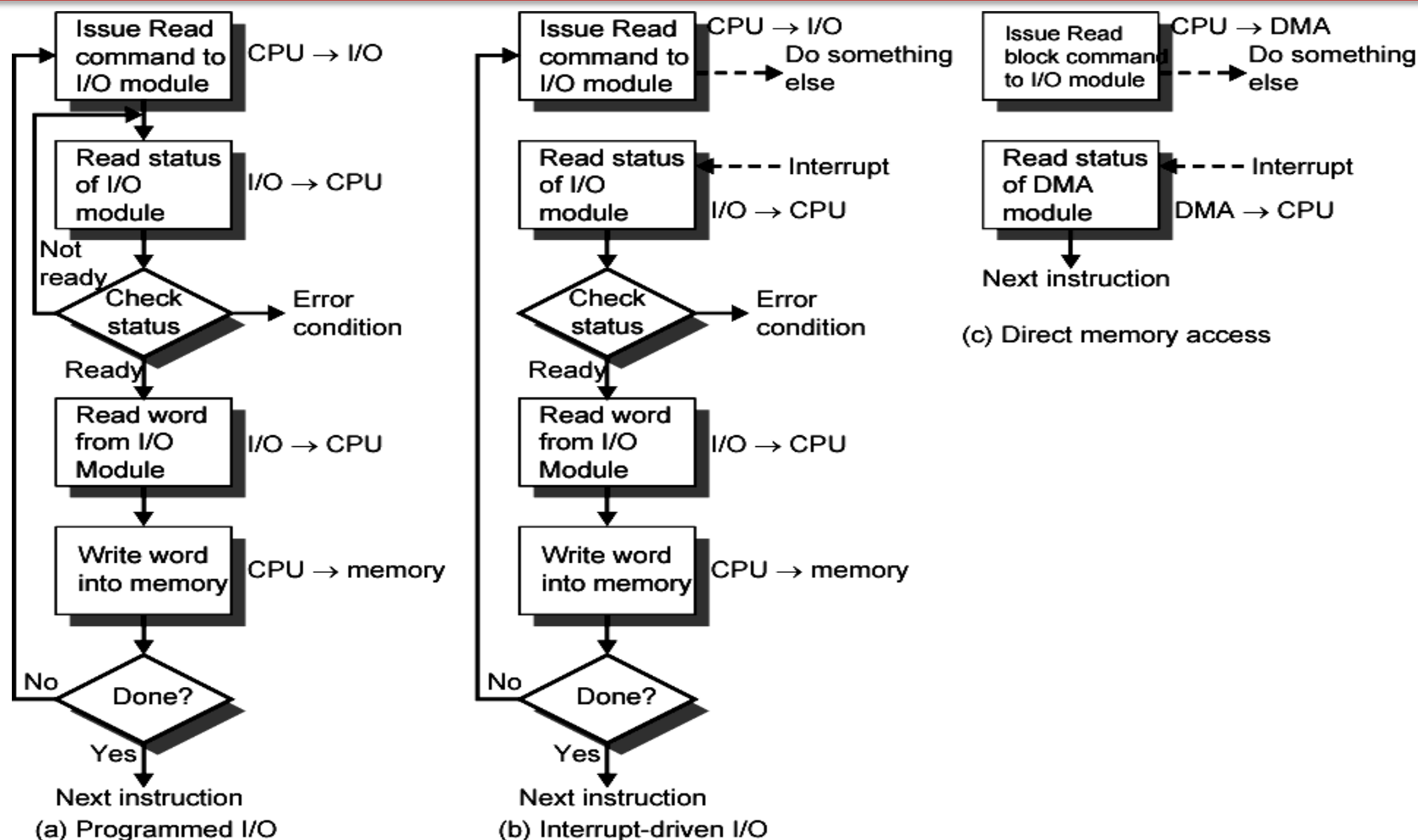
(b) Isolated I/O



7.2 Điều khiển IO

➤ Các phương pháp điều khiển IO

- a. IO bằng chương trình (Programmed IO)
- b. IO điều khiển bằng ngắt (Interrupt Driven IO)
- c. Truy nhập bộ nhớ trực tiếp DMA (Direct Memory Access)





a. Điều khiển IO bằng chương trình

- Nguyên tắc chung: CPU điều khiển trực tiếp IO bằng chương trình → cần phải lập trình IO.
- Với IO riêng biệt: sử dụng các lệnh IO chuyên dụng (IN, OUT).
- Với IO theo bản đồ bộ nhớ: sử dụng các lệnh trao đổi dữ liệu với bộ nhớ để trao đổi dữ liệu với cổng IO.



Các tín hiệu điều khiển IO

- Tín hiệu điều khiển (Control): kích hoạt & khởi động thiết bị ngoại vi
- Tín hiệu kiểm tra (Test): kiểm tra trạng thái của mô-đun IO và thiết bị ngoại vi
- Tín hiệu điều khiển đọc (Read): yêu cầu mô-đun IO nhận dữ liệu từ thiết bị ngoại vi và đưa vào thanh ghi đệm dữ liệu, rồi CPU nhận dữ liệu đó
- Tín hiệu điều khiển ghi (Write): yêu cầu mô-đun IO lấy dữ liệu trên bus dữ liệu đưa đến thanh ghi đệm dữ liệu rồi chuyển ra thiết bị ngoại vi



Hoạt động của IO bằng chương trình

➤ Hoạt động của IO bằng chương trình

- CPU yêu cầu thao tác IO
- Mô-đun IO thực hiện thao tác
- Mô-đun IO thiết lập các bit trạng thái
- CPU kiểm tra các bit trạng thái:
 - Nếu chưa sẵn sàng thì quay lại kiểm tra
 - Nếu sẵn sàng thì chuyển sang trao đổi dữ liệu với mô-đun IO

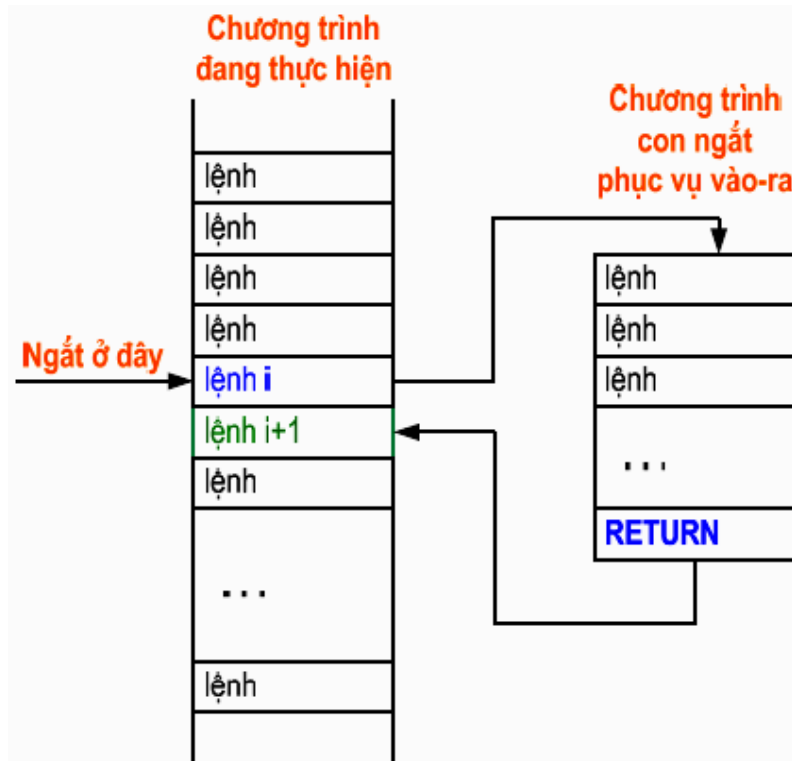
➤ Đặc điểm

- IO do ý muốn của người lập trình
- CPU trực tiếp điều khiển IO
- CPU đợi mô-đun IO → tiêu tốn thời gian của CPU



b. IO điều khiển bằng ngắt

- Sau khi gửi yêu cầu IO, CPU không phải đợi trạng thái sẵn sàng của mô-đun IO, CPU thực hiện một chương trình nào đó
- Khi mô-đun IO sẵn sàng thì nó phát tín hiệu ngắt CPU
- CPU thực hiện chương trình con IO tương ứng để trao đổi dữ liệu (trình xử lý ngắt)
- CPU trở lại tiếp tục thực hiện chương trình đang bị ngắt



-
- Hoạt động nhập dữ liệu nhìn từ mô-đun IO
- Mô-đun IO nhận tín hiệu điều khiển đọc từ CPU
 - Mô-đun IO nhận dữ liệu từ thiết bị ngoại vi, trong khi đó CPU làm việc khác
 - Khi đã có dữ liệu → mô-đun IO phát tín hiệu ngắt CPU
 - CPU yêu cầu dữ liệu
 - Mô-đun IO chuyển dữ liệu đến CPU

➤ Hoạt động nhập dữ liệu: nhìn từ CPU

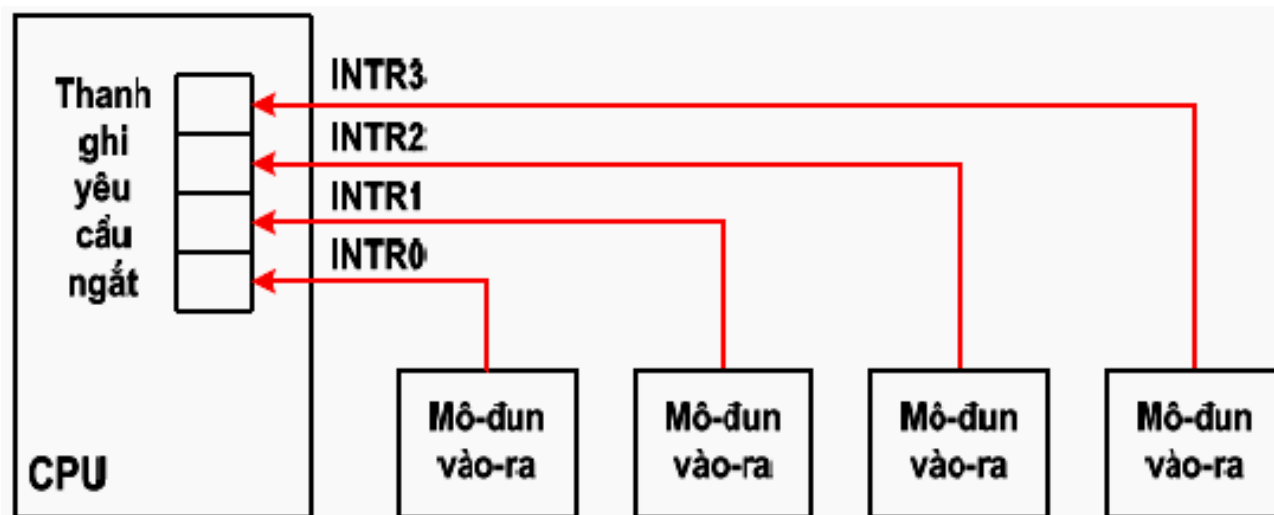
- Phát tín hiệu điều khiển đọc
- Làm việc khác
- Cuối mỗi chu trình lệnh, kiểm tra tín hiệu ngắt
- Nếu bị ngắt:
 - Cắt ngữ cảnh (nội dung các thanh ghi)
 - Thực hiện chương trình con ngắt để nhập dữ liệu
 - Khôi phục ngữ cảnh của chương trình đang thực hiện

- Các vấn đề nảy sinh khi có ngắt:
 - Xác định được mô-đun IO nào phát tín hiệu ngắt ?
 - Có nhiều yêu cầu ngắt cùng xảy ra ?
- Các phương pháp nối ghép ngắt
 - Sử dụng nhiều đường yêu cầu ngắt
 - Hỏi vòng bằng phần mềm (Software Poll)
 - Hỏi vòng bằng phần cứng (Daisy Chain or Hardware Poll)
 - Sử dụng bộ điều khiển ngắt lập trình được PIC (Programmable Interrupt Controller)



Nhiều đường yêu cầu ngắt

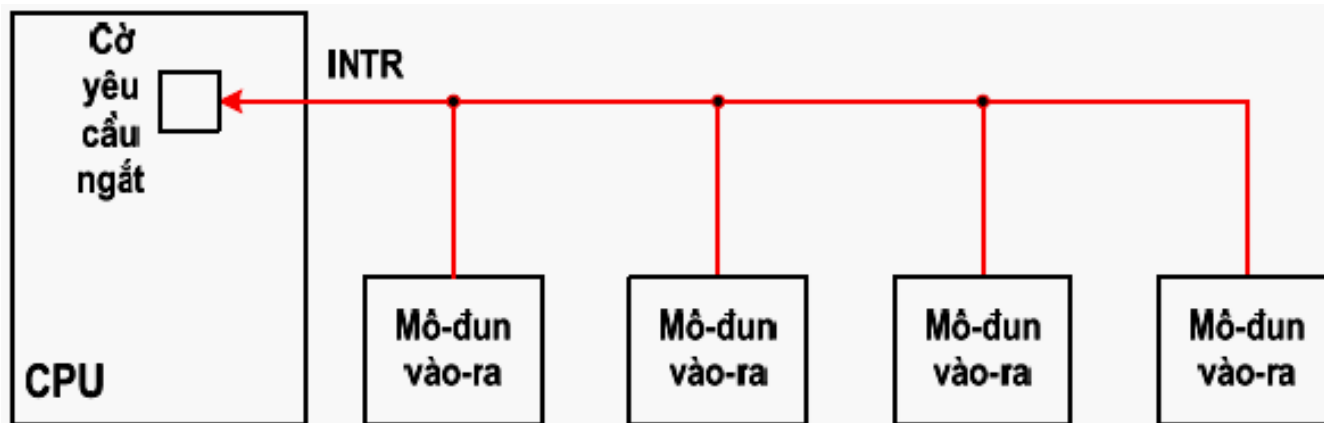
- Mỗi mô-đun IO được nối với một đường yêu cầu ngắt
- CPU phải có nhiều đường tín hiệu yêu cầu ngắt
- Hạn chế số lượng mô-đun IO
- Các đường ngắt được quy định mức ưu tiên





Hỏi vòng bằng phần mềm

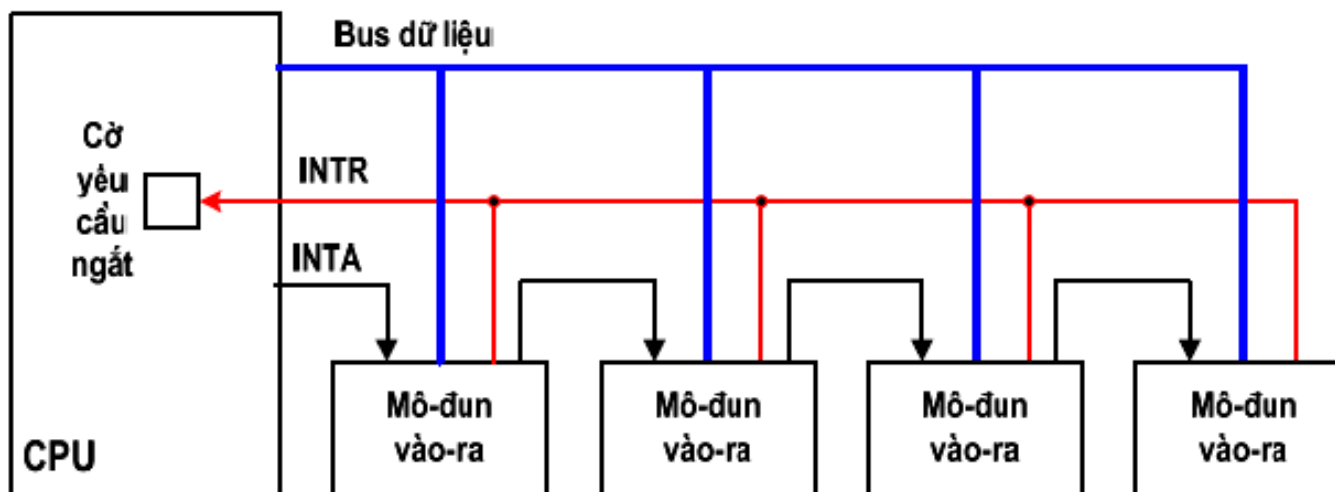
- CPU thực hiện phần mềm hỏi lần lượt từng mô-đun IO
- Chậm
- Thứ tự các mô-đun được hỏi vòng chính là thứ tự ưu tiên





Hỏi vòng bằng phần cứng

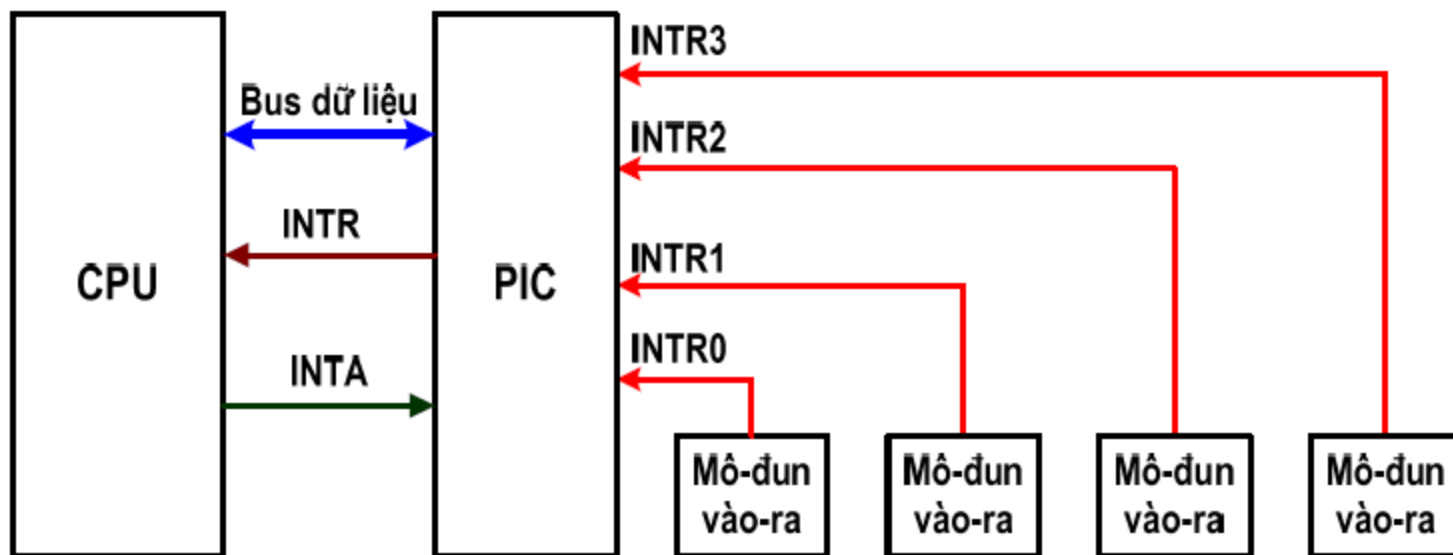
- CPU phát tín hiệu chấp nhận ngắt (INTA) đến mô-đun IO đầu tiên
- Nếu mô-đun IO đó không gây ra ngắt thì nó gửi tín hiệu đến mô-đun kế tiếp cho đến khi xác định được mô-đun gây ngắt
- Thứ tự các mô-đun IO kết nối trong chuỗi xác định thứ tự ưu tiên





Bộ điều khiển ngắt lập trình được PIC

- PIC có nhiều đường vào yêu cầu ngắt có qui định mức ưu tiên
- PIC chọn một yêu cầu ngắt không bị cấm có mức ưu tiên cao nhất gửi tới CPU





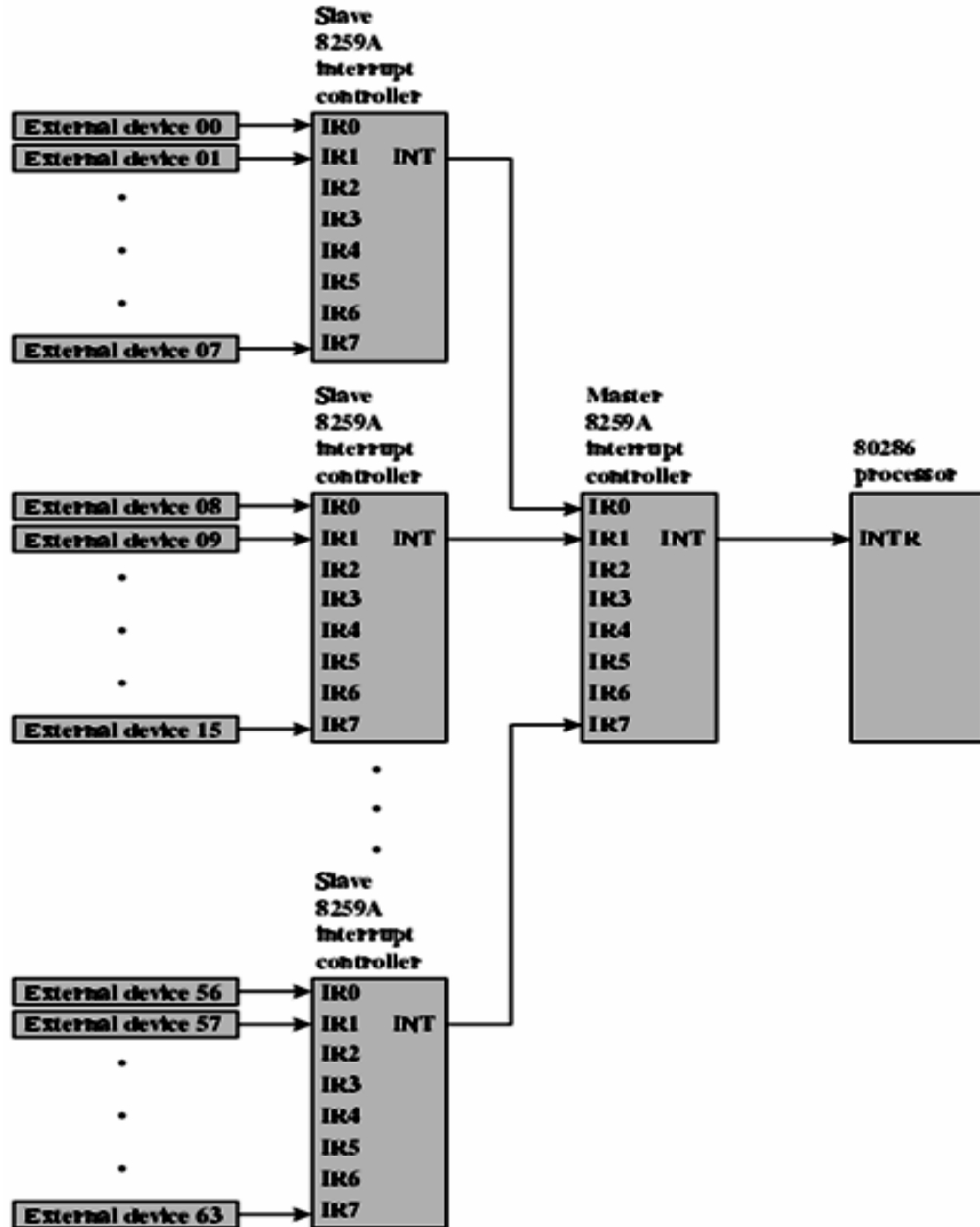
Đặc điểm của IO điều khiển bằng ngắt

- Có sự kết hợp giữa phần cứng và phần mềm
- Phần cứng: gây ngắt CPU
- Phần mềm: trao đổi dữ liệu
- CPU trực tiếp điều khiển IO
- CPU không phải đợi mô-đun IO → hiệu quả sử dụng CPU tốt hơn



Ví dụ: Hệ thống ngắt trên máy PC

- CPU Intel x86 có 1 chân tín hiệu ngắt
- PIC 8259A có 8 đường ngắt
- Có thể đấu nối nhiều PIC theo chế độ master/slaver để tăng số lượng đường ngắt phục vụ cho nhiều thiết bị





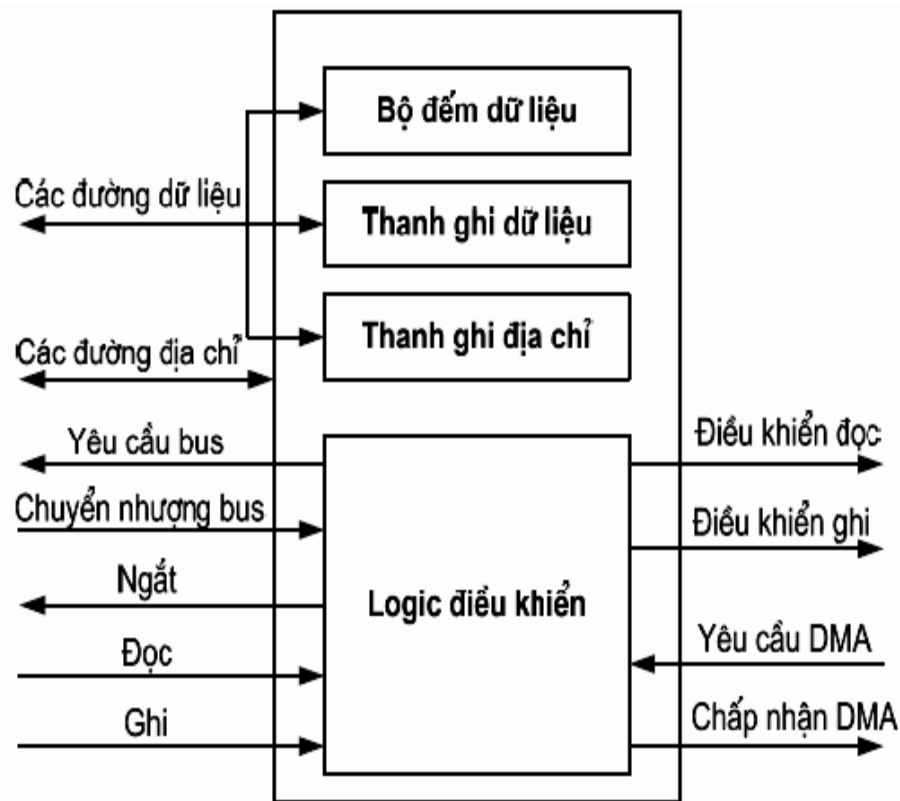
c. DMA (Direct Memory Access)

- IO bằng chương trình và bằng ngắt do CPU trực tiếp điều khiển:
 - Chiếm thời gian của CPU
 - Tốc độ truyền bị hạn chế vì phải chuyển dữ liệu qua CPU (thanh ghi có dung lượng nhỏ)
- Để khắc phục dùng DMA
 - Thêm mô-đun phần cứng trên bus → DMAC (DMA Controller)
 - DMAC điều khiển trao đổi dữ liệu giữa môđun IO với bộ nhớ chính



Sơ đồ cấu trúc của DMAC

- Thanh ghi dữ liệu: chứa dữ liệu trao đổi
- Thanh ghi địa chỉ: chứa địa chỉ ô nhớ dữ liệu
- Bộ đếm dữ liệu: chứa số từ dữ liệu cần trao đổi
- Logic điều khiển: điều khiển hoạt động của DMAC





Hoạt động DMA

- CPU gửi tín hiệu cho DMAC
 - Vào hay Ra dữ liệu
 - Địa chỉ thiết bị IO (cổng IO tương ứng)
 - Địa chỉ đầu của mảng nhớ chứa dữ liệu → nạp vào thanh ghi địa chỉ
 - Số từ dữ liệu cần truyền → nạp vào bộ đếm dữ liệu
- CPU làm việc khác
- DMAC điều khiển trao đổi dữ liệu
- Sau khi truyền được một từ dữ liệu thì:
 - nội dung thanh ghi địa chỉ tăng
 - nội dung bộ đếm dữ liệu giảm
- Khi bộ đếm dữ liệu = 0, DMAC gửi tín hiệu ngắt CPU để báo kết thúc DMA



Các kiểu thực hiện DMA

1. DMA truyền theo khối (Block-transfer DMA): DMAC sử dụng bus để truyền xong cả khối dữ liệu
2. DMA lấy lén chu kỳ (Cycle Stealing DMA): DMAC cưỡng bức CPU treo tạm thời từng chu kỳ bus, DMAC chiếm bus thực hiện truyền một từ dữ liệu.
3. DMA trong suốt (Transparent DMA): DMAC nhận biết những chu kỳ nào CPU không sử dụng bus thì chiếm bus để trao đổi một từ dữ liệu.



Đặc điểm của DMA

- CPU không tham gia trong quá trình trao đổi dữ liệu
- DMAC điều khiển trao đổi dữ liệu giữa bộ nhớ chính với mô-đun IO (hoàn toàn bằng phần cứng)
→ tốc độ nhanh
- Phù hợp với các yêu cầu trao đổi mảng dữ liệu có kích thước lớn (Block devices)

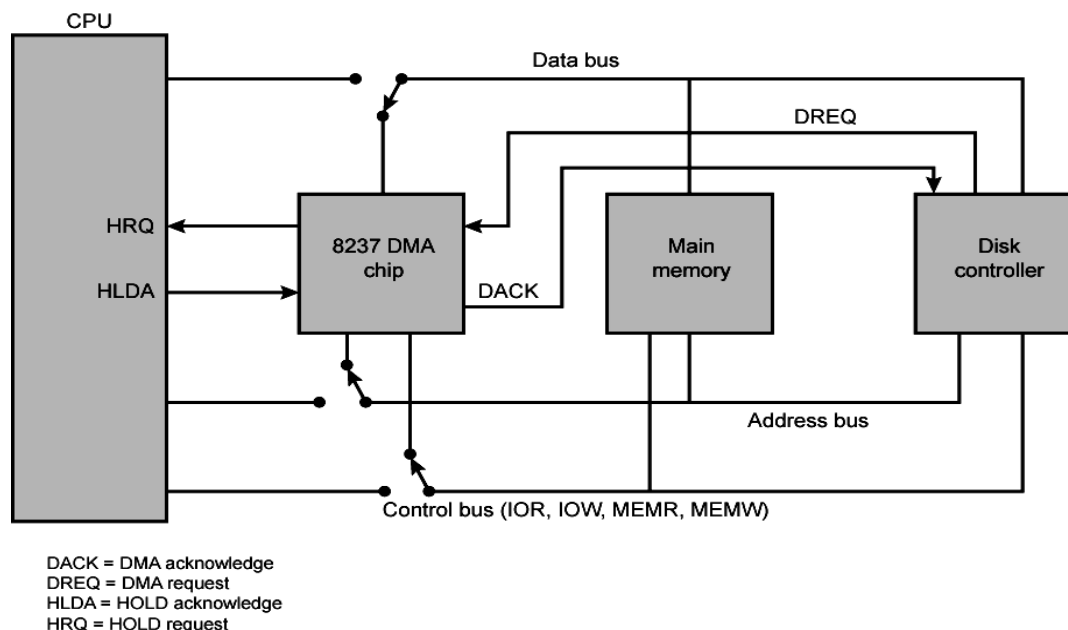
Phân loại TBNV

- Character devices
- Block devices



Ví dụ: Chip DMA trong máy PC

- Intel 8237A DMA Controller
- Giao tiếp với CPU Intel x86 và DRAM
- Khi DMA cần bus, nó gửi tín hiệu HRQ cho CPU
- CPU trả lời bằng tín hiệu HLDA
- DMA bắt đầu sử dụng bus





Kênh IO (IO channel)

- Việc điều khiển IO được thực hiện bởi một bộ xử lý IO chuyên dụng
- Bộ xử lý IO hoạt động theo chương trình của riêng nó
- Chương trình của bộ xử lý IO có thể nằm trong bộ nhớ chính hoặc nằm trong một bộ nhớ riêng
- Hoạt động theo kiến trúc đa xử lý
 - CPU gửi yêu cầu IO cho kênh IO
 - Kênh IO tự thực hiện việc truyền dữ liệu



Cấu hình DMA 1: Bus chung, DMA tách biệt

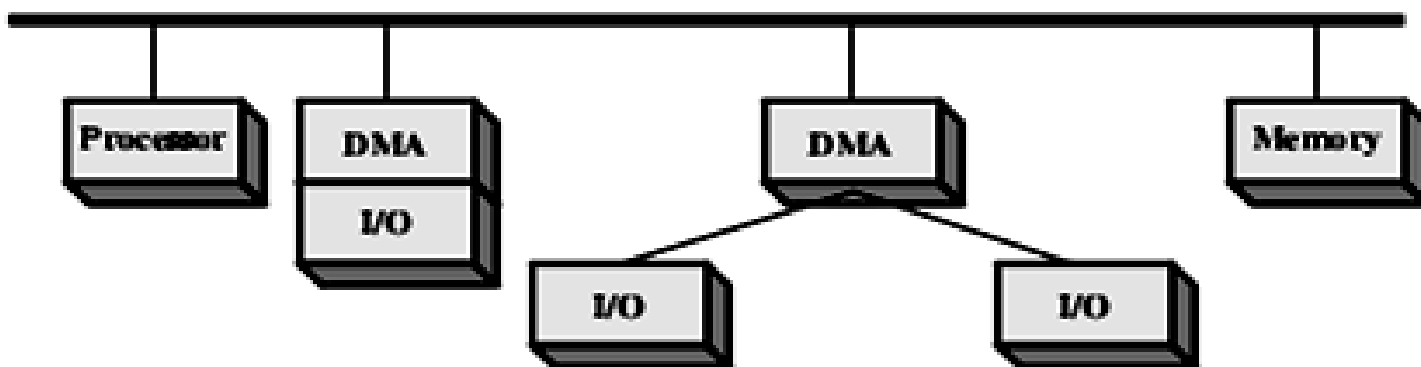
- Mỗi lần trao đổi một dữ liệu, DMAC sử dụng bus hai lần
 - Giữa mô-đun IO với DMAC
 - Giữa DMAC với bộ nhớ
- CPU bị treo khỏi bus 2 lần





Cấu hình DMA 2: Bus chung, DMA tích hợp

- DMAC điều khiển một hoặc vài mô-đun IO
- Mỗi lần trao đổi một dữ liệu, DMAC sử dụng bus một lần
 - Giữa DMAC với bộ nhớ
- CPU bị treo khỏi bus 1 lần

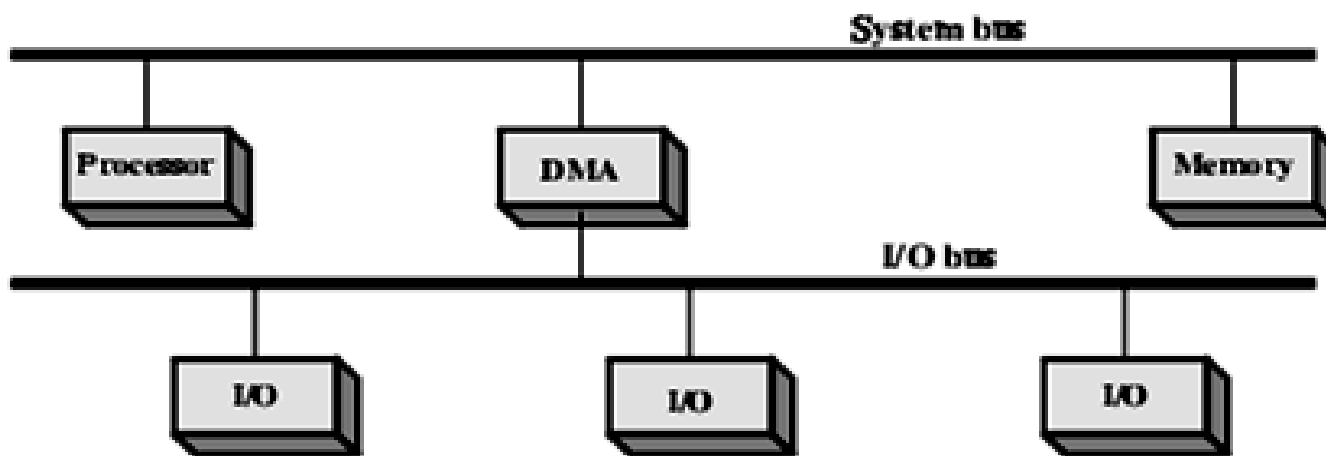


(b) Single-bus, Integrated DMA-I/O



Cấu hình DMA 3: Bus IO riêng

- Bus IO tách rời hỗ trợ tất cả các thiết bị cho phép DMA
- Mỗi lần trao đổi một dữ liệu, DMAC sử dụng bus một lần
 - Giữa DMAC với bộ nhớ
- CPU bị treo khỏi bus 1 lần



(c) I/O bus



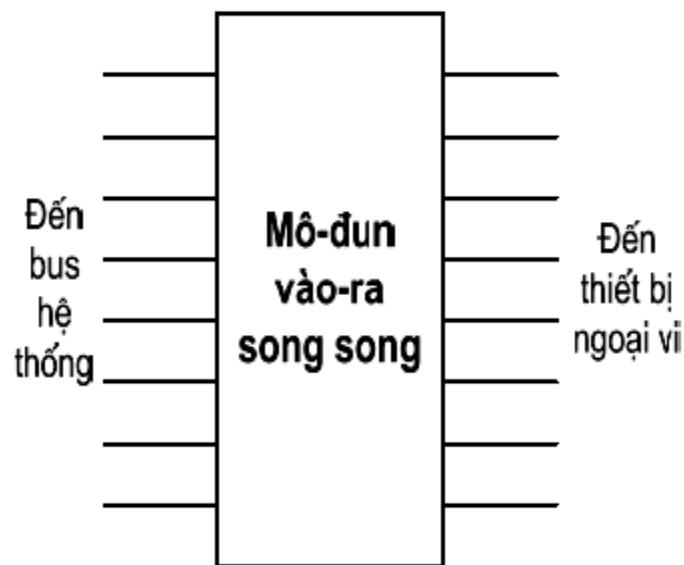
7.3 Nối ghép thiết bị ngoại vi

○ Các kiểu nối ghép

- Nối ghép song song (parallel)
- Nối ghép nối tiếp (serial)

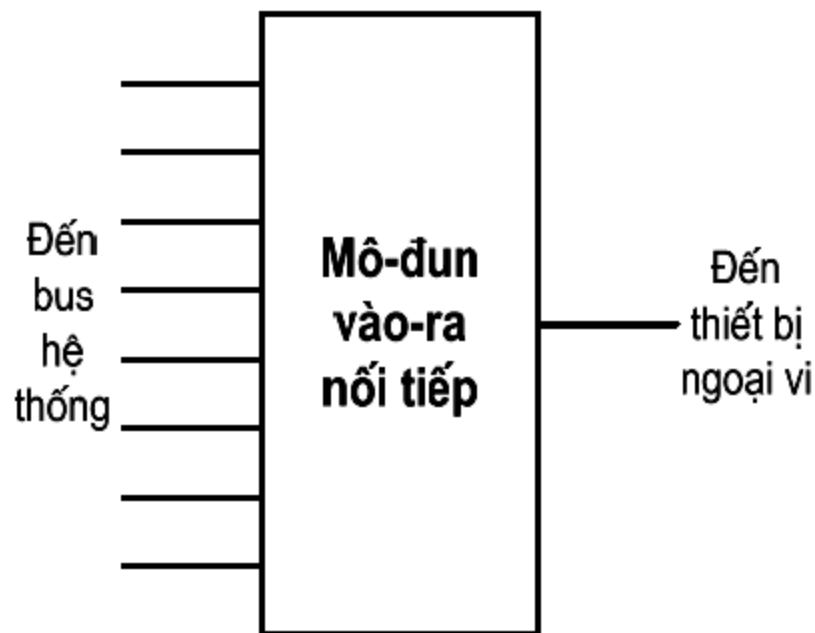
○ Nối ghép song song

- Truyền nhiều bit song song
- Cần nhiều đường truyền dữ liệu
- Tốc độ nhanh
- Dễ bị nhiễu giữa các tín hiệu



○ Nối ghép nối tiếp

- Truyền lần lượt từng bit
- Cần có bộ chuyển đổi từ dữ liệu song song sang nối tiếp hoặc/và ngược lại
- Cần ít đường truyền dữ liệu
- Tốc độ chậm hơn





Các cấu hình nối ghép

➤ Điểm tới điểm (Point to Point)

- Mỗi cổng IO nối ghép với một thiết bị ngoại vi
- Ví dụ:
 - SATA (Serial ATA)
 - SAS (Serial Attached SCSI)

➤ Điểm tới đa điểm (Point to Multipoint)

- Mỗi cổng IO cho phép nối ghép với nhiều thiết bị ngoại vi
- Ví dụ:
 - SCSI (Small Computer System Interface): 7 hoặc 15 thiết bị
 - USB (Universal Serial Bus): 127 thiết bị
 - IEEE 1394 (FireWire): 63 thiết bị



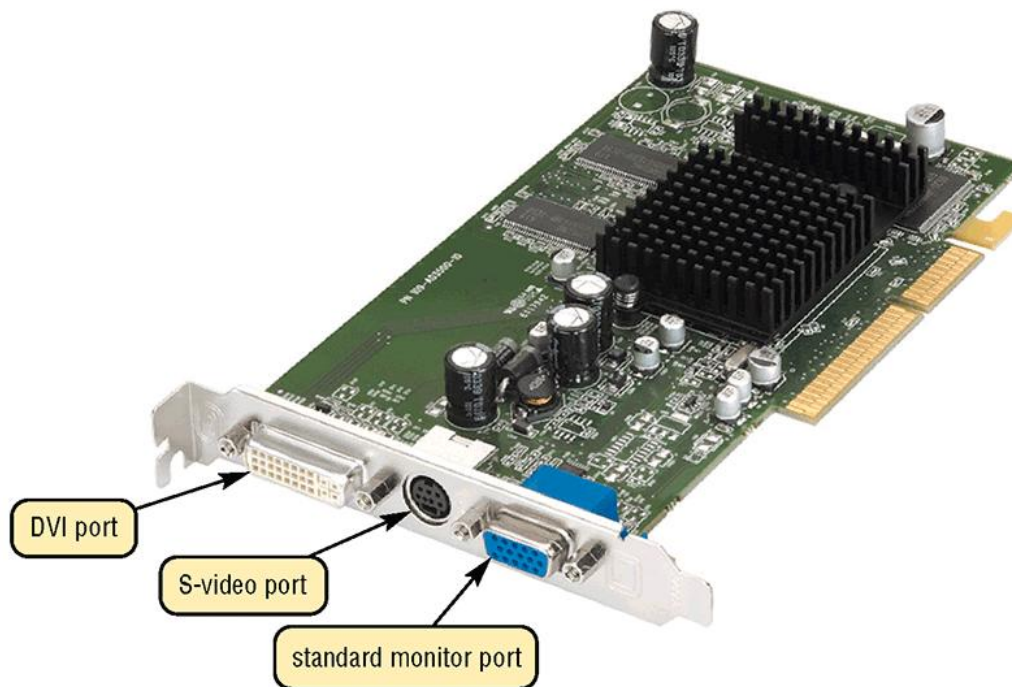
Các cổng vào ra thông dụng

- PS/2: nối ghép bàn phím và chuột – MiniDIN 6 chân
- RJ45: nối ghép mạng
- LPT (Line Printer): nối ghép với máy in, là cổng song song (Parallel Port) – 25 chân
- COM (Communication): nối ghép với Modem, là cổng nối tiếp (Serial Port) - 9 hoặc 25 chân
- USB (Universal Serial Bus): Cổng nối tiếp đa năng, cho phép nối ghép tối đa 127 thiết bị



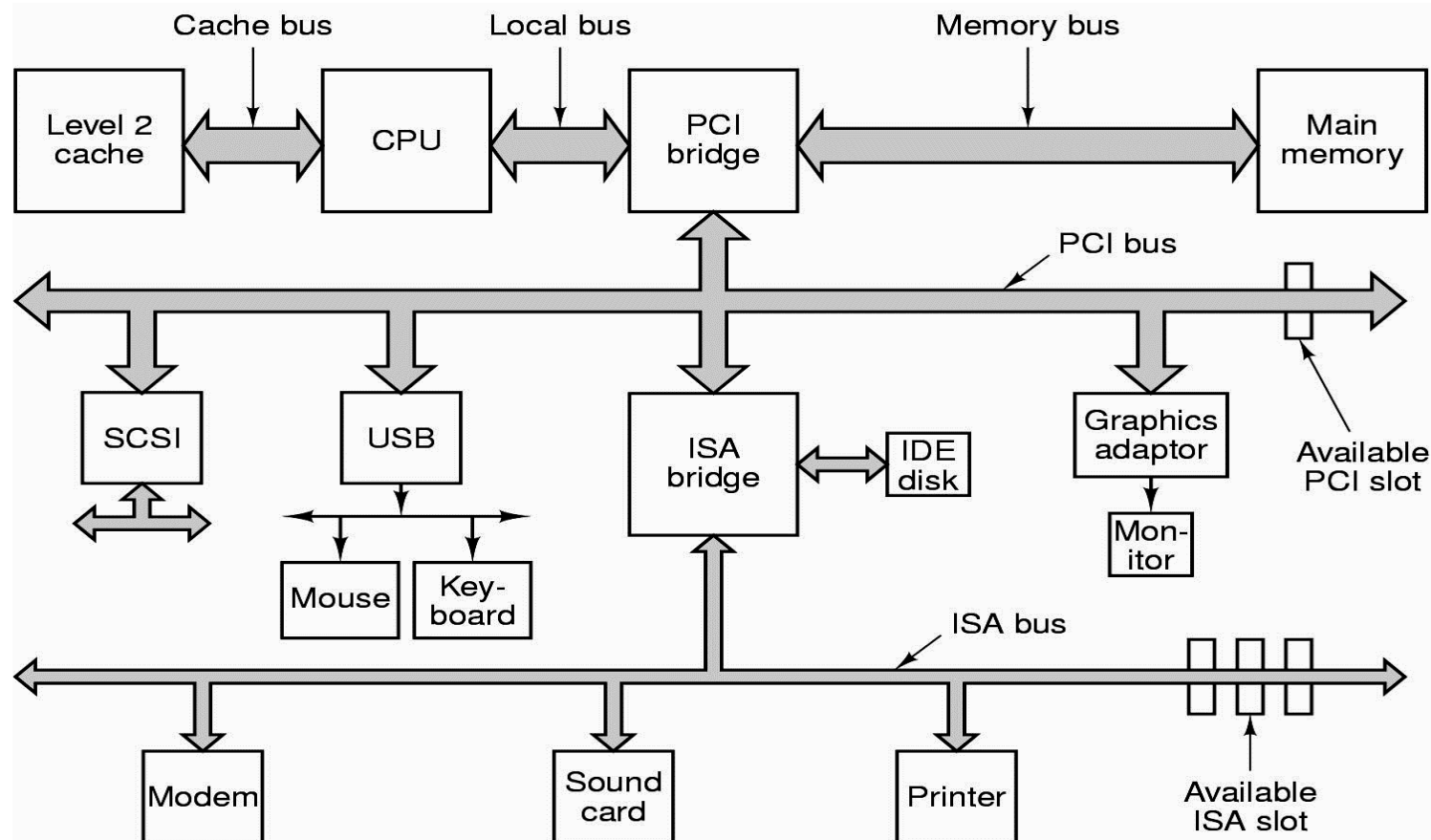
Ví dụ: Các cổng nối ghép trên card màn hình

- VGA: Cổng nối ghép màn hình Analog– 15 chân
- DVI: Cổng nối ghép màn hình Digital
- S-Video
- HDMI





Ví dụ: Hệ thống bus ngoại vi trên máy PC





- ISA (Industry Standard Architecture): Sử dụng trên máy PC 8086 (8 bit) và AT 80286 (16 bit)
- MCA (Micro Channel Architecture): Sử dụng trên máy 80386 của IBM (32 bit)
- EISA (Extended ISA) Sử dụng trên các máy 80386 tương thích (32 bit)
- VL bus (VESA Local bus): Sử dụng trên các máy 80486 (32 bit)

- AGP (Accelerated Graphics Port): Bus dành riêng cho card màn hình trên máy Pentium. Bao gồm các mức tốc độ 1x, 2x, 4x và 8x (1x=266MB/s).
- PCI (Peripheral Component Interconnect): Sử dụng trên các máy Pentium (32 & 64 bit)
 - PCI-X: Sử dụng tần số xung nhịp cao hơn (66-133 MHz) so với PCI 33 MHz
 - PCI-E (PCI-Express): Cho phép truyền dữ liệu tốc độ cao, được sử dụng trong các máy PC đời mới. Gồm nhiều mức tốc độ: 1x, 2x, ..., 32x (1x: 1 Lane có 4 đường truyền nối tiếp 250 MB/s)



Các cổng điều khiển đĩa

- Đĩa mềm : Dùng cáp 34 chân kết nối tối đa 2 ổ mềm
- Đĩa cứng/CD/DVD/SSD :
 - Chuẩn ST506
 - Chuẩn ESDI
 - Chuẩn IDE/UDMA/PATA
 - Chuẩn SCSI
 - Chuẩn SATA
 - Chuẩn SAS



Các thiết bị ngoại vi thông dụng

➤ Thiết bị nhập

- Bàn phím, chuột, scanner, digitizer, micro, đọc vân tay, đọc bar-code, camera, ...

➤ Thiết bị xuất

- Màn hình, máy in, máy vẽ, loa, projector, ...

➤ Thiết bị mạng & truyền thông

- Modem, Router, ...

➤ Thiết bị lưu trữ

- Đĩa mềm, đĩa cứng, SSD, CD, DVD, thẻ nhớ, ...



Câu hỏi

